

01 Storage, Unity Catalog and Data Placement

Where data lives, who can reach it and from which workspace: managed and external tables, external locations and regions. Most are decided before the first job runs.

ENGINES THIS PART TOUCHES



NO.	PATTERN	COMPUTE	STORAGE	DELTA
1.01	Every table is external so the data stays in your own storage. None of them get optimised automatically.			
1.02	You dropped a migrated table the way you always did. A week later its files are gone.			
1.03	Finance wants its own metastore. The account will not create a second one in the region.			
1.04	A developer dropped a production table from the dev workspace. Both workspaces share one metastore.			
1.05	The migration moved every table to Unity Catalog. A nightly job still reads last month's data.			
1.06	Your notebook reads abfss:// paths on the old cluster. On serverless the same cell fails.			
1.07	Another team points its tool at the folder behind a managed table. Its numbers stop matching yours.			
1.08	Jobs that read one storage account run far slower than the rest. The Azure bill shows data transfer.			

1.03 Finance wants its own metastore. The account will not create a second one in the region.

Unity Catalog allows **one metastore per region**, and every workspace in that region attaches to it. Isolation comes from **catalogs**, **workspace bindings** and catalog-level storage.

1

Metastore per region. All workspaces in that region share it, and a workspace attaches to the metastore in its own region.

Source: Databricks docs, Unity Catalog best practices, Oct 2026

HOW YOU WILL NOTICE

A plan that gives each business unit its own metastore stops at the second one. Check which metastore a workspace uses from any notebook or SQL warehouse.

```
SELECT current_metastore();
-- azure:westeurope-kuudb, one per region
```

WORKAROUNDS

Separate admins with catalog ownership and bindings, and separate storage with catalog locations. Data that must stay in another region is read through OpenSharing from the

NEIGHBOURING LIMITS

	1,000	10,000	1,000
	Catalogs per metastore. Can be raised	Schemas per catalog. Can be raised	Tables per catalog
RELATED	1.04 A developer dropped a production table from the dev workspace		
	1.08 Jobs that read one storage account run far slower than the rest		

Sanjay Chandra - Global CoE Community

27

1.07 Another team points its tool at the folder behind a managed table. Its numbers stop matching yours.

Path access to managed tables is **not supported**, and their folders are named by **UUID**. A reader that lists Parquet files without the Delta log counts superseded files too.



FIG. 1.07 Databricks reads through the Delta log. The outside tool reads the folder, so every rewrite changes its answer.

SEEN IN

The outside tool: a report, a pipeline or another engine's query

CAUSE

A Delta table is its log plus the files the log lists. After **MERGE**, **UPDATE** or **OPTIMIZE**, superseded files stay in the folder until **VACUUM**. With deletion vectors, a file still holds rows that were deleted. Unity Catalog does not support path access to managed tables, so nothing warns the reader.

FIX

Give each outside reader a governed route to the table, not the folder. Engines use the Unity REST API or the Iceberg REST catalog with credential vending. Another organisation or region uses OpenSharing. Fabric uses a mirrored Azure Databricks catalog. Writes from outside Delta clients to managed tables go through the Unity Databricks.

CONFIRM IN BOTH ENGINES

```
-- Metastore: External data access on
GRANT EXTERNAL USE SCHEMA
ON SCHEMA sales_prod.core TO 'bi-reader';

-- Same version, same answer, both sides
SELECT COUNT(*) FROM sales_prod.core.orders
VERSION AS OF 412;
```

EXTERNAL USE SCHEMA is not part of ALL PRIVILEGES, and only the catalog owner can grant it.

RELATED 1.01 Every table is external so the data stays in your own storage

1.06 Your notebook reads abfss:// paths on the old cluster

Sanjay Chandra - Global CoE Community

34

Azure Databricks Practitioner's Handbook 4.0

What each Azure Databricks feature is for, what breaks, and how to design systems that don't.

FREE

www.ssanjaychandra.com
github.com/ssanjaychandra123
linkedin.com/in/ssanjaychandra

Sanjay Chandra
GLOBAL COE COMMUNITY

23
COMPARISONS

134
PATTERNS

26
SCENARIOS

7
PATTERN TYPES

Pieces, failures and designs.

ENGINE

JOBS

PIPE

COMPUTE

STORAGE

DELTA

UC

WH

SEC

COST

PATTERNS

PAGE

SECTION 1 Know the pieces

C Commonly Confused 23 groups of features people mix up, one question each 04

SECTION 2 What breaks

WHERE THE DATA LIVES

1 Storage, Unity Catalog and Data Placement 1 3 7 3 7 2 3 2 8 28

HOW IT ARRIVES

2 Ingestion and Streaming 13 1 8 10 8 2 2 14 37

HOW TABLES ARE KEPT

3 Delta Lake and Table Layout 7 2 4 7 12 1 1 4 12 53

WHAT RUNS THE WORK

4 Compute and Clusters 7 1 12 1 1 3 9 12 67

5 Jobs and Pipelines 14 4 7 1 5 3 4 5 17 81

HOW IT IS SERVED

6 Databricks SQL and BI 3 1 8 2 3 12 2 6 12 100

GOVERNING AND RUNNING IT

7 Security and Governance 8 3 4 5 15 3 15 17 114

8 Platform, Networking and Cost 9 14 7 1 6 2 10 9 19 133

9 CI/CD and Operations 12 5 2 1 3 2 6 1 12 154

MACHINE LEARNING AND AI

10 ML and AI Workloads 6 6 1 5 9 3 3 6 11 168

SECTION 3 Design it

S System Design Scenarios 26 design briefs in six themes, each linked to the patterns 181

	PATTERN TYPE	SILENT FAILURE	SLOWDOWN	WORKS HERE, FAILS THERE	BETWEEN ENGINES	DESIGN DECISION	LIMIT	HABIT
1	Storage, Unity Catalog and Data Placement	1	1	1	1	2	1	1
2	Ingestion and Streaming	4	3	3		1	1	2
3	Delta Lake and Table Layout	2	2	1	3	2	1	1
4	Compute and Clusters	1	1	5		2		3
5	Jobs and Pipelines	5		6		1	2	3
6	Databricks SQL and BI	2	3	4		2	1	
7	Security and Governance	6	1	3		1	3	3
8	Platform, Networking and Cost	4		3	1	6	2	3
9	CI/CD and Operations	3		3		4		2
10	ML and AI Workloads	4		1		5		1

Cell number = patterns in the part that involve the engine (top) or follow the pattern type (bottom); darker = a larger share of the part.

BEFORE YOU START

How to use this book.

The book has three sections. Start with the one that matches what you are doing today.

SECTION 1

Know the pieces.

23 comparisons of features people mix up. Start here when you are learning Azure Databricks or choosing between two features.

Page 04

SECTION 2

What breaks.

134 patterns in ten parts, by layer. Start here when something fails, runs slow or shows the wrong numbers.

Page 28

SECTION 3

Design it.

26 design briefs, answered step by step. Start here when you plan a platform, a migration or a load.

Page 181

A COMPARISON GROUP

The **question** that separates the features, then one card per feature: what it is, its **answer** to the question, the usual **mix-up** and when to pick it. Numbered C.01 to C.23.

A PATTERN

Number in the part colour, then the **title**: the symptom as you meet it. **Engines**: the strip highlights every part of the platform involved. **Short answer** in one or two sentences, with **cost of failure**, status and the date the facts were verified. The body follows one of seven types, and **Related** patterns close it.

TYPE	LAYOUT	PATTERNS
1 Silent failure	Diagnostic path	32
2 Slowdown	Measure first	11
3 Works here, fails there	Diff	30
4 Between engines	Swim lanes	5
5 Design decision	Decision sheet	26
6 Limit	Number led	11
7 Habit that does not carry over	Myth cards	19

A DESIGN SCENARIO

The brief is the title. Then a **short answer**, the questions to ask first, the data path at a glance, the **build steps** in order with the trap at each step, security, checks and follow-ups. Numbered S.01 to S.26.

TAB COLOURS

 C Compare  1 Storage  2 Ingestion  3 Delta  4 Compute  5 Jobs  6 SQL  7 Security
 8 Platform  9 CI/CD  10 AI  S Scenarios

SCOPE

Azure Databricks as a platform, including running ML and AI workloads. General Spark and SQL behaviour is in the PySpark and SQL Handbooks 4.0; model building is out of scope. Terms are in the glossary at the back.

FACTS AND SOURCES

Every default, limit and behaviour was checked against the Azure Databricks documentation on Microsoft Learn and docs.databricks.com in October 2026. Only generally available behaviour is stated as fact; preview features are named as preview. Azure Databricks changes monthly, so check the date on each pattern.



Commonly Confused.

23 groups of Azure Databricks features that people mix up. Each group starts with the one question that tells them apart and answers it for every feature. Then it names the usual mix-up and says when to pick which.

NO.	THE GROUP, AND THE QUESTION THAT SEPARATES IT	PAGE
C.01	All-purpose compute, jobs compute, serverless compute and SQL warehouses Who starts it, who shares it, and how is it billed?	05
C.02	Standard access mode, dedicated access mode and the legacy no-isolation mode Who can run code on it, and what does Unity Catalog enforce?	06
C.03	Metastore, catalog, schema and workspace What does each one hold, and what boundary does it draw?	07
C.04	Managed tables, external tables, foreign tables and volumes Who owns the files, and what happens on DROP?	08
C.05	Storage credentials, external locations, service credentials and the access connector What does each one point at, and whose identity reaches Azure?	09
C.06	Lakeflow Jobs, Lakeflow Spark Declarative Pipelines and Lakeflow Connect Who decides the order of the work, and who writes the code?	10
C.07	Views, materialized views, streaming tables and metric views What stores the result, and what keeps it up to date?	11
C.08	Auto Loader, COPY INTO, read_files and Lakeflow Connect Where does the data come from, and who remembers what was loaded?	12
C.09	Partitioning, Z-order, liquid clustering and automatic liquid clustering Who picks the layout columns, and when are files rewritten?	13
C.10	OPTIMIZE, VACUUM, Auto compaction, Optimized writes and Predictive Optimization Does it change file size, remove files, or decide when the others run?	14
C.11	Time travel, RESTORE, UNDROP, shallow clone and deep clone How far back can you go, and what do you get back?	15
C.12	Row filters, column masks, dynamic views and ABAC policies Where is the rule attached, and who maintains it?	16
C.13	OpenSharing, Lakehouse Federation, external data access and Fabric mirroring Which side runs the query, and whose storage holds the data?	17
C.14	Personal access tokens, OAuth for users, OAuth for service principals and managed identities Who does the token stand for, and when does it expire?	18
C.15	Databricks-backed scopes, Key Vault-backed scopes, Unity Catalog secrets and service credentials Where is the secret stored, and who can read it?	19
C.16	Declarative Automation Bundles, Git folders, Terraform and the workspace UI What moves between environments, and who can change it by hand?	20
C.17	Billing system tables, tags, budgets and serverless usage policies Does it record spend, label it, warn about it, or attribute it?	21
C.18	Geo-redundant storage, deep clone replication, OpenSharing to another region and managed disaster recovery What survives a region outage, and how long until you are back?	22
C.19	DBFS root, mounts, workspace files and volumes Who governs the path, and should new work use it?	23
C.20	SYNC, SET MANAGED, DEEP CLONE and Hive metastore federation Does the data move, and can the old Hive jobs keep running?	24
C.21	Expectations, CHECK constraints and data quality monitoring Does it stop bad data, record it, or notice what nobody wrote a rule for?	25
C.22	SQL warehouses, Lakebase, synced tables and Model Serving How fast must one answer come back, and how many at once?	26
C.23	IP access lists, Private Link, VNet injection, NCC and serverless network policies Which direction is the traffic going, and which compute started it?	27

C.01 All-purpose compute, jobs compute, serverless compute and SQL warehouses.

All four run work on the same Unity Catalog tables, and a job task can point at any of them, so teams pick by habit. The real split is who starts the compute, who shares it, and which rate the bill uses.

THE QUESTION THAT SEPARATES THEM

Who starts it, who shares it, and how is it billed?

All-purpose compute

Classic compute you start yourself for interactive work. It runs until auto termination stops it.

Answer. You start it, people share it, and it bills at the all-purpose DBU rate plus the Azure VMs.

The mix-up. Teams run scheduled jobs on it to skip cold starts. Microsoft's guidance says the same work costs significantly less on jobs compute. **4.02 · 4.08**

Use it for interactive work that serverless cannot run.

Jobs compute

Classic compute a job run creates from its cluster spec, and terminates after the last task that uses it.

Answer. The job starts and stops it, only that run shares it, and it bills at the jobs rate plus the VMs.

The mix-up. People expect a job cluster to stay up for the next run. It is scoped to one run, so every run pays a cold start. **4.01 · 5.05**

Pick it when the code needs init scripts, a custom VM type, Maven libraries or RDDs, or reaches a network an NCC cannot.

Serverless compute

Compute Azure Databricks runs for notebooks, jobs, JAR tasks included, and pipelines. A serverless workspace offers no other kind.

Answer. Databricks starts and scales it per workload. The DBU rate includes the VMs. Storage behind a firewall needs a network security perimeter rule or an NCC private endpoint first.

The mix-up. People compare its DBU rate with the classic rate and leave out the VMs. **8.02 · 8.11**

Pick it when work is interactive or bursty, and the storage firewall already admits it. Steady long jobs stay classic.

SQL warehouse

Compute for SQL only. Serverless starts in seconds; pro and classic run in your subscription and take minutes.

Answer. Its owner sets size and scaling, everyone with CAN USE shares it, and it bills at its type's SQL rate.

The mix-up. People size it up to cure queueing. Size helps heavy queries; more clusters help many users. **6.01 · 6.02**

Use it for interactive SQL and BI, where it is the most cost-efficient engine.

WHICH ONE, WHEN

- If a job runs notebooks, Python, wheels or JARs, use serverless jobs, in standard mode when it has slack (8.06); keep steady long jobs and what serverless cannot run on jobs compute.
- If the work is SQL for people, dashboards or BI tools, use a serverless SQL warehouse.
- If a production job points at an all-purpose cluster, move it: the rate is higher and people share the cluster.

C.02 Standard access mode, dedicated access mode and the legacy no-isolation mode.

All three are one setting on the same classic compute form, and most notebooks run on any of them. Shared became standard and single user became dedicated, so old runbooks use both sets of names. The real split is whose identity runs the code and whether Unity Catalog takes part at all.

THE QUESTION THAT SEPARATES THEM

Who can run code on it, and what does Unity Catalog enforce?

Standard

Multi-user classic compute. Lakeguard isolates each user's code. Auto picks it unless you choose an ML runtime, a GPU or a runtime below 14.3.

Answer. Many users, each under their own identity. Unity Catalog enforces grants, row filters and column masks for every one of them.

The mix-up. Code that uses RDD APIs, R, Hive UDFs or sparkContext in Scala fails here after running for years on an old cluster. Teams then move everything to dedicated instead of fixing a few calls. **4.12**

Pick it when you have no reason not to. Databricks recommends standard for all workloads it supports.

Dedicated

Classic compute assigned to one user or one group. It runs R, the ML runtime, GPUs and RDD APIs.

Answer. Only the assigned user or group. Unity Catalog enforces grants; reading tables with row filters or column masks, or dynamic views, needs Databricks Runtime 15.4 LTS or above and serverless enabled, which does the filtering.

The mix-up. A job on a dedicated cluster with runtime 15.3 or below fails on a filtered table and the team blames the grant. The runtime is the cause. **6.10 · 7.04**

Pick it when standard cannot run the work: R, the ML runtime, GPUs or RDDs.

No isolation shared

A legacy mode where many users share one cluster with no isolation between them. It cannot reach Unity Catalog data.

Answer. Everyone attached runs code in one shared environment. Unity Catalog enforces nothing because it is not involved.

The mix-up. Admins turn on Disable legacy access and think the Hive metastore is closed. These clusters do not respect that setting. Enforce User Isolation stops them from starting. **7.03 · 1.06**

Use it for nothing new. List these clusters, test each move to standard in staging, then turn on Enforce User Isolation. Turn it off to revert.

WHICH ONE, WHEN

- If several people or tasks share a cluster, use standard.
- If the code needs R, the ML runtime, GPUs or RDDs, use dedicated, on 15.4 LTS or above if it reads filtered tables.
- If a job fails only on standard, check the standard mode limitations before switching mode.
- If any cluster still shows no isolation shared, agree the move with its owner and test it in staging. Then turn on Enforce User Isolation; running clusters fail only when they restart.

C.03 Metastore, catalog, schema and workspace.

Each looks like the top level of something, so teams draw environment and business unit lines on the wrong one. The real split is that data boundaries live in catalogs, and a workspace decides where people work.

THE QUESTION THAT SEPARATES THEM

What does each one hold, and what boundary does it draw?

Metastore

The top container of Unity Catalog, one per region. Every workspace in the region attaches to it.

Answer. It holds catalogs, credentials, external locations and shares. Its boundary is the region, not a team.

The mix-up. Teams plan one metastore per business unit. The region already has one, so the second cannot be created. **1.03**

Use it for regional placement. Data in another region is read through OpenSharing.

Catalog

The first part of catalog.schema.table. It can have its own managed storage location and be bound to chosen workspaces.

Answer. It holds schemas. Docs call it the primary unit of data isolation: storage, owner and binding are set here.

The mix-up. By default every workspace on the metastore can use every catalog, so a dev workspace reaches prod tables by name. **1.04**

Pick it when you separate environments or business units: one catalog each, with its own binding and storage.

Schema

The second level, inside a catalog. It can override the catalog's managed location.

Answer. It holds the objects. It groups them for privileges and ownership, but it has no workspace binding.

The mix-up. People grant SELECT on a schema and nobody can query. Readers also need USE CATALOG and USE SCHEMA. **7.01**

Use it for a domain or layer inside one environment, such as core or gold.

Workspace

The Azure resource where people work: notebooks, jobs, compute and dashboards. A serverless workspace adds Databricks-managed default storage.

Answer. It holds code, compute and jobs, not tables. Data access still comes from grants and bindings.

The mix-up. Teams build a separate prod workspace and think prod data is safe. Without a binding, the dev workspace reads the same catalogs. **1.04**

Use it for separate people, compute and admins per environment.

WHICH ONE, WHEN

- If two groups need data in different regions, they use different metastores and share through OpenSharing.
- If two environments or business units need separate data, give each a catalog with its own storage and binding.
- If a team needs its own compute, jobs and admins, give it a workspace.
- If dev must read prod but never write it, bind the prod catalog to the dev workspace read-only.

C.04 Managed tables, external tables, foreign tables and volumes.

All four sit under a schema in Catalog Explorer, and Unity Catalog grants govern all four. Hive habits treat them all as named folders. The real split is who owns the files, and so what DROP does to them.

THE QUESTION THAT SEPARATES THEM

Who owns the files, and what happens on DROP?

Managed table

A Delta or Iceberg table whose files Unity Catalog places under the managed location of its catalog or schema: your storage account, or default storage in a serverless workspace.

Answer. Unity Catalog does. DROP starts a recovery period, 7 days by default, for UNDROP TABLE; then the files are deleted.

The mix-up. People drop a migrated table expecting the Hive behaviour that kept the files, and come back after the purge. **1.02 · 1.01**

Pick it when Databricks writes the data. Docs recommend managed tables for new work.

External table

A table registered at a path you choose inside an external location. Tools outside Databricks can read and write the folder.

Answer. You own the files. DROP removes the table from Unity Catalog and leaves the folder alone.

The mix-up. Teams make every table external to be safe and lose Predictive Optimization, so nobody compacts or vacuums them. **1.01 · 1.07**

Pick it when a named system outside Databricks must write the files or read them by path.

Foreign table

A table in a foreign catalog from Lakehouse Federation, standing for a table in SQL Server, Snowflake or a Hive metastore.

Answer. The source owns the data. Foreign tables are read-only, except in a federated Hive metastore, so queries never change a source database.

The mix-up. People treat it as loaded data and build dashboards on it. Every query reads the source, at the source's speed. **6.04**

Use it for ad hoc reads, proofs of concept and a step-by-step move from a Hive metastore.

Volume

A Unity Catalog object for files that are not tables, such as CSV drops, images and JARs.

Answer. Managed: Unity Catalog, deleted 7 days after DROP VOLUME. External: you, and the files stay.

The mix-up. Teams register raw files as external tables to govern them. A volume governs files directly. **1.06**

Use it for landing files, libraries and exports that need governance but are not tables.

WHICH ONE, WHEN

- If Databricks writes the table, make it managed.
- If another system must write the files, make it external and name its owner.
- If the data must stay in the source database, use a foreign table and expect source speed.
- If it is files rather than rows, use a volume.

C.05 Storage credentials, external locations, service credentials and the access connector.

All four sit between Unity Catalog and Azure and often end at one managed identity, so people grant the wrong object. The real split is what each one points at, and where Azure checks the role.

THE QUESTION THAT SEPARATES THEM

What does each one point at, and whose identity reaches Azure?

Access connector

An Azure resource, Access Connector for Azure Databricks, that holds a system-assigned or user-assigned managed identity. You grant that identity roles such as Storage Blob Data Contributor.

Answer. It points at nothing by itself; it is the identity Azure sees. There is no secret to rotate, and storage behind network rules can admit it.

The mix-up. Users keep their own Azure roles on the same storage account, so their reads skip Unity Catalog entirely. **7.05**

Use it for every storage and service credential, one per environment.

Storage credential

A Unity Catalog object that wraps the connector's managed identity, or a service principal, for storage access.

Answer. It points at an identity, not a path. Unity Catalog uses it for users through external locations.

The mix-up. Admins grant users on the credential so they can read files. Grant only CREATE EXTERNAL LOCATION on it. **7.05**

Pick it when a new storage account or identity comes into use.

External location

A storage path in ADLS paired with one storage credential.

Answer. It points at a path. READ FILES, WRITE FILES, CREATE EXTERNAL TABLE and CREATE MANAGED STORAGE are granted here.

The mix-up. Teams create one location at the account root and grant READ FILES widely. Grant it sparingly and keep locations below the root. **1.06**

Pick it when a folder holds managed storage, external tables or external volumes.

Service credential

A Unity Catalog object that wraps a managed identity for Azure services other than storage, such as Key Vault.

Answer. It points at a service. Users with ACCESS on it call the service as the connector's identity from code.

The mix-up. Teams keep a service principal secret in a secret scope only to call an Azure service. A service credential needs no secret. **7.11**

Pick it when code calls an Azure service that accepts Microsoft Entra ID.

WHICH ONE, WHEN

- If Azure needs an identity to check, create an access connector and grant its managed identity the role.
- If Unity Catalog needs that storage, wrap the identity in a storage credential.
- If people need a folder, create an external location and grant on it.
- If code calls an Azure service, use a service credential instead of a stored key.
- If users hold Azure roles on the storage account, save the list, then remove them group by group with the storage owner's approval. The list is the revert.

C.06 Lakeflow Jobs, Lakeflow Spark Declarative Pipelines and Lakeflow Connect.

All three carry the Lakeflow name, and all three can move data from a source into a table. A pipeline runs as a task in a job, and a Lakeflow Connect connector produces a pipeline, so people describe one in the other's words. The real split is who decides the order of the work and who writes the code.

THE QUESTION THAT SEPARATES THEM

Who decides the order of the work, and who writes the code?

Lakeflow Jobs

Workflow automation for Databricks. A job is a set of tasks with dependencies, a trigger and compute: notebooks, Python, SQL, pipelines, other jobs, If/else and For each.

Answer. You decide the order, task by task, and you write the code in every task.

The mix-up. Teams chain twenty notebook tasks to build Silver and Gold, each with its own MERGE and checkpoint. A pipeline would work out the order and the incremental logic. **5.10 · 5.15**

Use it for schedules, triggers, retries, and every step that is not a table definition.

Lakeflow Spark Declarative Pipelines

Formerly DLT. They extend Apache Spark Declarative Pipelines: you declare streaming tables and materialized views in SQL or Python.

Answer. The pipeline decides. It reads the dependencies between datasets and runs the flows in order, in parallel where it can. You write the transformations and expectations.

The mix-up. People put API calls, file moves and emails inside a pipeline. A pipeline defines tables; the job around it does everything else. **5.06 · 5.08**

Pick it when the work is a chain of tables that should update incrementally.

Lakeflow Connect

Managed connectors for databases such as SQL Server and SaaS apps such as Salesforce. Each produces a serverless ingestion pipeline that writes streaming tables. Database connectors such as SQL Server also run a gateway continuously on classic compute.

Answer. Databricks writes the code and decides the order. You choose the source, the tables and the schedule.

The mix-up. Teams hand-build JDBC notebooks for a source a connector supports, then own the change capture logic. Connectors have costs too: the gateway bills while idle, and the source needs CDC or change tracking. Release status varies, so check it. **2.14 · S.10**

Pick it when a supported source must land in Bronze with little code, and a DBA approves change capture on the source and someone owns the gateway.

WHICH ONE, WHEN

- If work needs ordering, scheduling or retrying, use a job.
- If the work is tables built from other tables, use a pipeline and let a job start it.
- If the source is a database or SaaS app with a managed connector, use Lakeflow Connect: agree the source setup with the DBA, budget the gateway and name an owner.
- If you are about to write change capture by hand, check the connector list first.

C.07 Views, materialized views, streaming tables and metric views.

All four are defined by a query and read like tables, so people expect the same freshness from all four. The real split is whether a result is stored, and what brings it up to date.

THE QUESTION THAT SEPARATES THEM

What stores the result, and what keeps it up to date?

View

A saved query in Unity Catalog. Each read runs it on current data.

Answer. Nothing is stored, so it is current. The owner's rights reach the tables beneath; readers need SELECT on the view only.

The mix-up. People are surprised that a user reads a table through a view they could not query directly. **7.04**

Use it for renaming, light logic and security over tables fast enough to query directly.

Materialized view

A Unity Catalog managed table that stores a query result. A serverless pipeline does every refresh, not your warehouse.

Answer. The result is stored. It changes on REFRESH, a SCHEDULE or TRIGGER ON UPDATE, incrementally when it can.

The mix-up. It does not stay current by itself. Without TRIGGER ON UPDATE it waits for its schedule, and a failed refresh leaves old data. **6.05 · 5.08**

Pick it when a join or aggregate is read far more often than its sources change.

Streaming table

A Unity Catalog table that a pipeline or Databricks SQL updates incrementally from Auto Loader, Kafka or another stream. AUTO CDC targets are streaming tables.

Answer. Rows are stored; each refresh processes only new input. Appends add rows, AUTO CDC applies updates and deletes. A full refresh reprocesses what the source still holds.

The mix-up. People build aggregates on it and expect updated source rows to flow through. They are not reprocessed. **5.08 · 2.06**

Use it for landing files and events, and CDC targets, in Bronze and Silver.

Metric view

A view defined in YAML that separates measures from the dimensions that group them. Queries call MEASURE().

Answer. Nothing is stored by default; each measure is computed at query time. Optional materialization pre-computes aggregates.

The mix-up. Teams build one materialized view per report grain, and the totals drift. A metric view defines each measure once. **10.07**

Pick it when dashboards, Genie and BI tools need one measure at many grains.

WHICH ONE, WHEN

- If the query is fast on the base tables, use a view.
- If an expensive result is read often, use a materialized view and set its schedule or trigger.
- If rows arrive as new input or as a change feed, use a streaming table.
- If several reports must agree on a measure, define it in a metric view.

C.08 Auto Loader, COPY INTO, read_files and Lakeflow Connect.

All four load data into Delta tables, and three read the same files in cloud storage. The real split is where the data comes from and where the record of what was loaded lives.

THE QUESTION THAT SEPARATES THEM

Where does the data come from, and who remembers what was loaded?

Auto Loader

The cloudFiles streaming source. It picks up new files in ADLS or a volume, scales to billions of files, and runs in a job, a notebook or a pipeline.

Answer. Files in cloud storage. It records loaded files in RocksDB inside the checkpoint, which gives exactly-once processing.

The mix-up. People delete the checkpoint to unstick a stream. The checkpoint is the record of loaded files, so the next run loads everything again. **2.06 · 2.02**

Pick it when files keep arriving and each must load once, at any scale.

COPY INTO

A SQL command that loads files from a path or volume into a Delta table. It runs on a SQL warehouse or a cluster, and you schedule it.

Answer. Files in cloud storage. It records loaded files in the target table's metadata, so a rerun skips them.

The mix-up. Teams point it at folders with millions of files. For that scale, docs point SQL users at streaming tables.

Pick it when a SQL user loads thousands of files into one table on a schedule.

read_files

A table-valued SQL function that reads the files under a path and infers one schema. Inside a streaming table, STREAM read_files uses Auto Loader.

Answer. Files in cloud storage. A batch read_files remembers nothing and reads every matching file on each run.

The mix-up. People put batch read_files in a nightly INSERT and load every file again each night. **2.01**

Use it for exploring files, one-off loads, or with STREAM as a streaming table source.

Lakeflow Connect

Managed connectors for databases and SaaS apps, such as SQL Server and Salesforce. Each builds an ingestion pipeline into streaming tables.

Answer. Databases and applications, not files. The connector keeps its own progress, through change data capture where the source supports it.

The mix-up. Teams export database tables to files with Azure Data Factory, then load the files with Auto Loader. A connector reads the source directly. **2.14**

Pick it when the source is a supported database or SaaS app.

WHICH ONE, WHEN

- If files keep arriving, use Auto Loader, or STREAM read_files in a streaming table.
- If a SQL user loads a modest folder on a schedule, COPY INTO is enough.
- If you only want to look at the files, use read_files.
- If the source is a database or SaaS app, use a Lakeflow Connect connector where one exists.

C.09 Partitioning, Z-order, liquid clustering and automatic liquid clustering.

All four place rows so queries can skip files, and old code often uses two of them at once. The real split is who picks the layout columns and when the files are rewritten.

THE QUESTION THAT SEPARATES THEM

Who picks the layout columns, and when are files rewritten?

Partitioning

PARTITIONED BY splits the table into one folder per value of the partition columns. It is fixed when the table is created.

Answer. You pick the columns at CREATE, and every write lands in its partition. Changing them means rewriting the table.

The mix-up. Teams partition small tables by date and get thousands of small files. Docs say tables under 1 TB need no partitions, and most under 100 TB do not either. **3.01**

Use it for very large tables where a test shows it beats liquid clustering.

Z-order

ZORDER BY, an option of OPTIMIZE that sorts data in files by several columns. It works inside partitions.

Answer. You pick the columns on every OPTIMIZE run, and files change only when you run it. Predictive Optimization never runs ZORDER.

The mix-up. People add it to an hourly job on every table. It cannot combine with liquid clustering, which docs recommend for new tables. **3.03**

Use it for older tables you cannot move to liquid clustering yet.

Liquid clustering

CLUSTER BY with up to 4 keys. It replaces partitioning and ZORDER, and keys can change without a rewrite.

Answer. You pick the keys. OPTIMIZE or Predictive Optimization clusters the files; writes cluster only above a size, 64 MB for one key on a Unity Catalog table.

The mix-up. People set CLUSTER BY on an external table and never run OPTIMIZE, so nothing gets clustered. **3.02**

Pick it when you create a table and know the columns queries filter on.

Automatic liquid clustering

CLUSTER BY AUTO on a Unity Catalog managed table. It requires Predictive Optimization.

Answer. Databricks picks the keys from the query patterns and changes them as those change. Predictive Optimization rewrites the files.

The mix-up. Teams expect it on external tables or with Predictive Optimization off. It needs a managed table and Predictive Optimization. **1.01**

Pick it when managed tables have query patterns you do not know, or that keep changing.

WHICH ONE, WHEN

- If the table is managed, start with CLUSTER BY AUTO.
- If you know the filter columns, or the table is external, use liquid clustering with explicit keys.
- If a table is partitioned and under 100 TB, pilot REPLACE PARTITIONED BY WITH CLUSTER BY on one table. It needs Runtime 18.1 or above; read its behaviour changes first.
- If a job still runs ZORDER, plan its move to liquid clustering.

C.10 OPTIMIZE, VACUUM, Auto compaction, Optimized writes and Predictive Optimization.

All five keep Delta tables in shape by touching files, so teams schedule them together and pay twice. The real split is whether each one changes file size, removes files, or decides when the others run.

THE QUESTION THAT SEPARATES THEM

Does it change file size, remove files, or decide when the others run?

OPTIMIZE

A command that rewrites small files into larger ones and, on a clustered table, clusters them. It runs on the compute that calls it.

Answer. Changes file size and layout. The old files stay on storage until VACUUM removes them.

The mix-up. Teams run OPTIMIZE hourly on every table and pay to rewrite data that has not changed. **3.03**

Use it for external tables, or right after a large load.

VACUUM

A command that deletes files the table no longer references, once older than the retention, 7 days by default.

Answer. Removes files. Afterwards, time travel cannot go back past the retention.

The mix-up. People delete rows, run VACUUM and see no saving. Deletion vectors keep old files referenced until REORG TABLE ... APPLY (PURGE). **3.05 · 3.04**

Use it for external tables. Managed tables get it from Predictive Optimization.

Auto compaction and Optimized writes

Write-time settings. Optimized writes shuffle data so fewer, larger files land; Auto compaction compacts small files afterwards.

Answer. Change file size at write time, on the writer's compute. Optimized writes are on by default for MERGE.

The mix-up. Teams add OPTIMIZE after every streaming batch. Auto compaction handles most of it. **2.11**

Use it for streaming and frequent small writes.

Predictive Optimization

A service that runs OPTIMIZE, VACUUM and ANALYZE on Unity Catalog managed tables when worth it, billed as serverless jobs.

Answer. Decides when the others run. On by default for new accounts since November 2024, and for older ones by August 2026, unless disabled on a catalog or schema.

The mix-up. Old OPTIMIZE and VACUUM jobs keep running beside it. It covers managed tables only and never runs ZORDER. **1.01 · 3.03**

Use it for every managed table. Its history system table is in Public Preview and keeps 180 days.

WHICH ONE, WHEN

- If the table is managed, check that Predictive Optimization is on, then drop the old jobs.
- If writes are small and frequent, turn on Auto compaction for the table.
- If the table is external, schedule OPTIMIZE and VACUUM yourself.
- If storage must shrink after deletes, purge deletion vectors before VACUUM.

C.11 Time travel, RESTORE, UNDROP, shallow clone and deep clone.

All five bring back an older state of a table. The real split is how far back each one reaches, and whether you get a query result, the table itself, or a separate copy.

THE QUESTION THAT SEPARATES THEM

How far back can you go, and what do you get back?

Time travel

VERSION AS OF or TIMESTAMP AS OF in a query reads an older version of the table.

Answer. Blocked past `delta.deletedFileRetentionDuration`, 7 days by default: on managed tables from Runtime 12.2, on others from 18.0. You get a query result; the table is unchanged.

The mix-up. Teams treat it as last month's backup. Rely on 7 days unless retention is raised; materialized views have no time travel. **3.04**

Use it for comparing, auditing and copying rows out from the last few days.

RESTORE

RESTORE TABLE ... TO VERSION AS OF rolls the table back by writing a new commit.

Answer. The same reach as time travel. You get the table back in place, as a commit that counts as a data change.

The mix-up. People expect downstream streams to ignore it. Streaming readers see the restored data as new and process it again. **3.11**

Pick it when a bad write must be undone for every reader.

UNDROP

UNDROP TABLE brings back a dropped managed or external table, or a materialized view, with its privileges and owner.

Answer. Within the recovery period, 7 days by default. Setting it per catalog or schema (0, or 7 to 30 days) is Public Preview, on Runtime 17.3 and above.

The mix-up. People leave it for later. After the recovery period a managed table's files are purged, and nothing brings them back. **1.02**

Pick it when a table was dropped recently. SHOW TABLES DROPPED lists the IDs.

Shallow and deep clone

SHALLOW CLONE copies metadata and points at the source's files. DEEP CLONE copies data and metadata. Neither copies the history.

Answer. From any version time travel still reaches. A deep clone stands alone; a shallow clone needs the source's files.

The mix-up. Teams keep a shallow clone as a backup. When the source is dropped and its files purged, the clone stops working. **3.12**

Use it for shallow for a test copy, deep with long retention for a backup.

WHICH ONE, WHEN

- If you need to see old rows, use time travel.
- If the table must go back for everyone, use RESTORE and warn the owners of downstream streams.
- If the table was dropped, run UNDROP within the recovery period.
- If you need history beyond a week or a month, keep deep clones with their own retention.
- If a table has row filters, column masks or ABAC policies, time travel, RESTORE and clones fail unless the running identity is exempt (7.17).

C.12 Row filters, column masks, dynamic views and ABAC policies.

All four hide rows or values from some users with the same functions. The real split is where the rule is attached (a table, a view or tags across a catalog) and who keeps it current.

THE QUESTION THAT SEPARATES THEM

Where is the rule attached, and who maintains it?

Row filter

A SQL UDF attached to one table with ALTER TABLE ... SET ROW FILTER. It returns true for the rows a user may see.

Answer. On the table, so every query on it is filtered. The table owner can change it; the data protection owner reviews it.

The mix-up. People copy the table with CTAS and the copy has no filter. The rule belongs to the table, not to the data. **7.06 · 7.08**

Pick it when a few tables need row rules.

Column mask

A SQL UDF attached to one column with ALTER COLUMN ... SET MASK. It returns the real or a masked value for each user.

Answer. On the column, so every query sees the mask. The owner maintains it; the data protection owner reviews it.

The mix-up. The mask checks current_user(), and the job runs as a service principal, so the job gets masked values. **7.07**

Pick it when a few known columns, such as tax IDs, must be hidden from most readers.

Dynamic view

A view whose query uses current_user() or is_account_group_member() to drop rows or blank columns.

Answer. On a separate view, so only reads through the view are protected. The view owner maintains it.

The mix-up. Teams leave SELECT on the base table for the same groups, so anyone can skip the view. **7.04 · 6.10**

Pick it when the rule spans several tables, or you share it through OpenSharing.

ABAC policy

A row filter or column mask policy from CREATE POLICY on a catalog, schema or table, applied wherever governed tags match. GA since April 2026; DENY policies are Beta.

Answer. On a level of the hierarchy, matched by governed tags, written once by people with MANAGE there. It needs runtime 16.4 or above or serverless.

The mix-up. A policy covers only tagged data, so untagged tables stay open and whoever can assign the tag controls the rule. Limit ASSIGN to the data protection owner. **7.08**

Pick it when many tables share one rule, such as hide PII.

WHICH ONE, WHEN

- If one or two tables need a rule, attach a row filter or column mask.
- If the same rule fits many tables, tag them with governed tags and write one ABAC policy.
- If the rule must span tables or travel in a share, use a dynamic view and revoke SELECT on the base tables.
- Test every rule as the job's identity, and give recovery and sharing identities exempt groups (7.17).

C.13 OpenSharing, Lakehouse Federation, external data access and Fabric mirroring.

All four let one platform read data that another governs, without a copy pipeline. The real split is which side runs the query and whose storage holds the data.

THE QUESTION THAT SEPARATES THEM

Which side runs the query, and whose storage holds the data?

OpenSharing

OpenSharing, formerly Delta Sharing, shares Unity Catalog data with another metastore, or tables with any platform.

Answer. The recipient runs the query, read-only, against data that stays in the provider's storage.

The mix-up. Teams share between workspaces on one metastore, which already see the same catalogs. **1.03**

Pick it when data goes to another region, another metastore or a partner.

Lakehouse Federation

Foreign catalogs over databases such as SQL Server (query federation) or a Hive metastore (catalog federation). Only a federated Hive metastore accepts writes.

Answer. Query federation pushes work down to the source database; catalog federation reads its files on Databricks compute.

The mix-up. People build dashboards on a federated SQL Server table and blame Databricks for the speed. **6.04**

Use it for ad hoc reads, proofs of concept and a gradual move into Unity Catalog.

External data access

Outside engines, such as Spark, Trino or Snowflake, read Unity Catalog tables through the Unity REST API or the Iceberg REST catalog, with vended credentials.

Answer. The outside engine runs it, reading your storage directly with short-lived credentials, so Unity Catalog cannot police it afterwards. Filtered or masked tables and views are unsupported.

The mix-up. Teams grant ALL PRIVILEGES and the engine still fails. The metastore setting is off by default, and EXTERNAL USE SCHEMA must be granted explicitly. **1.07**

Pick it when another engine must read your tables, with security's approval.

Fabric mirroring

A mirrored Azure Databricks catalog in Fabric, created there or, in Public Preview, published from Databricks. It reads tables through OneLake shortcuts.

Answer. Fabric runs the query; the data stays in your ADLS Gen2 account. It needs external data access and EXTERNAL USE SCHEMA.

The mix-up. People expect grants to carry over. They do not, and tables with filters or masks, views and streaming tables are left out. **1.07**

Pick it when a Fabric team builds Power BI reports on Databricks Gold.

WHICH ONE, WHEN

- If the reader is another Databricks metastore or company, use OpenSharing.
- If the data lives in another database and should stay there, use Lakehouse Federation.
- If another engine reads your tables, get security's approval, enable external data access and grant EXTERNAL USE SCHEMA. Turn it off to stop.
- If Fabric reads Databricks tables, mirror the catalog and rebuild row rules in Fabric.

C.14 Personal access tokens, OAuth for users, OAuth for service principals and managed identities.

All four put a bearer token in the same request header, so a script works the same with any of them. The real split is whose identity the token carries, and how long it lives when nobody is watching it.

THE QUESTION THAT SEPARATES THEM

Who does the token stand for, and when does it expire?

Personal access token

A token a user or service principal creates in one workspace. Docs recommend OAuth instead.

Answer. It stands for whoever created it, with all their rights. New tokens live at most 730 days by default, and unused ones are revoked after 90 days.

The mix-up. Production jobs and tools run on an engineer's token, and the engineer leaves. **7.13**

Use it for short tests only. Admins can cap the lifetime or turn tokens off.

OAuth for users (U2M)

A person signs in through the browser, and the CLI, SDK or IDE receives an OAuth token for them.

Answer. It stands for the signed-in person. Access tokens last one hour, and the tool refreshes them.

The mix-up. People schedule a script on a laptop with it. It needs a person to sign in, so unattended work needs M2M. **9.03**

Use it for people working from the CLI, an IDE or a BI tool.

OAuth for service principals (M2M)

A service principal signs in with its client ID and an OAuth secret, or through workload identity federation from GitHub Actions or Azure DevOps.

Answer. It stands for the service principal. Access tokens last one hour; secrets last up to 730 days, and federation needs no secret at all.

The mix-up. Teams create a personal access token for the service principal, which brings back a long-lived secret to rotate. **7.13 · 9.05**

Use it for CI/CD and any automation outside Databricks.

Managed identity

An Azure managed identity, on a VM or a Function for example, added to Databricks as a service principal.

Answer. It stands for the Azure resource. Azure issues its tokens, so there is no secret to store or expire.

The mix-up. People confuse it with the access connector's identity. That one reaches storage for Unity Catalog; this one calls Databricks. **7.05**

Use it for code on an Azure resource that calls Databricks APIs.

WHICH ONE, WHEN

- If a person works interactively, use OAuth U2M.
- If a pipeline in GitHub or Azure DevOps deploys, use a service principal with workload identity federation.
- If code on an Azure resource calls Databricks, use its managed identity.
- If a personal access token runs anything in production, replace it and cap token lifetime.

C.15 Databricks-backed scopes, Key Vault-backed scopes, Unity Catalog secrets and service credentials.

All four keep passwords out of notebook code, and three are read with `dbutils.secrets.get`. The real split is where the secret is stored, how finely access is granted, and whether there needs to be a secret at all.

THE QUESTION THAT SEPARATES THEM

Where is the secret stored, and who can read it?

Databricks-backed scope

A workspace secret scope kept in an encrypted database that Azure Databricks manages. Secrets are written with the CLI or API.

Answer. Stored in one workspace. READ is granted per scope, so it opens every secret in the scope.

The mix-up. People trust the [REDACTED] in notebook output. Redaction is best effort, and anyone with READ can still get the value out. **7.11**

Pick it when an init script, a dedicated cluster or an old runtime needs it.

Key Vault-backed scope

A read-only scope over one Azure Key Vault. The vault must use the access policy model; Azure RBAC is not supported.

Answer. Stored in Key Vault and read at access time. READ on the scope opens every secret in the vault.

The mix-up. The vault firewall blocks Databricks. It needs Allow trusted Microsoft services to bypass this firewall, which Secure by perimeter in a network security perimeter overrides. **7.12**

Pick it when rotation lives in Key Vault and policy allows access policies.

Unity Catalog secrets

Secrets as Unity Catalog objects, `catalog.schema.secret`, GA since August 2026, up to 100 per schema and 1,000 per metastore. Backing a schema with Key Vault is in Beta.

Answer. Stored in Unity Catalog, granted per secret with READ SECRET, WRITE SECRET or REFERENCE SECRET, and audited.

The mix-up. A moved job fails: it needs standard access mode on 17.3 LTS or above, or serverless environment 4 or above. SQL reads them only inside UDFs with a SECRETS clause. **7.11 · 7.12**

Pick it when new secrets need per-secret grants or several workspaces.

Service credential

A Unity Catalog object over a managed identity for Azure services. C.05 compares it with the other credentials.

Answer. No secret is stored. Code gets Entra ID tokens as the identity.

The mix-up. Teams store a service principal secret only to call an Azure service. **C.05**

Pick it when the target accepts Microsoft Entra ID.

WHICH ONE, WHEN

- If the target accepts Microsoft Entra ID, use a service credential and store nothing.
- If new secrets need per-secret grants, use Unity Catalog secrets on supported compute.
- If rotation lives in Key Vault, use a Key Vault-backed scope; check its permission model and firewall.
- If a secret rotates, read it at use time, not once at job start.

C.16 Declarative Automation Bundles, Git folders, Terraform and the workspace UI.

All four can create a job in a workspace, and many teams use all four at once. The real split is what each one moves between environments, and whether a person can still change production by hand.

THE QUESTION THAT SEPARATES THEM

What moves between environments, and who can change it by hand?

Declarative Automation Bundles

Formerly Databricks Asset Bundles. YAML for jobs, pipelines, dashboards and more, with targets for dev, test and prod, deployed by the CLI.

Answer. Moves code and resource definitions together. Deployed jobs are UI_LOCKED: a hand edit needs Disconnect from source, and the next deploy overwrites it.

The mix-up. Teams deploy production from a laptop as themselves. Deploy from CI as a service principal with mode: production and run_as. **9.01 · 9.05**

Use it for jobs, pipelines and their code on the way from dev to prod.

Git folders

A Git client inside the workspace for cloning, branching, committing and pulling notebooks and files.

Answer. Moves code only. Job and pipeline settings stay in the workspace, and anyone who can edit the folder can change the files.

The mix-up. Teams pull main into a production Git folder and call it CI/CD. Compute, schedules and permissions are not in Git, so environments drift. **9.06**

Use it for development in the workspace against a branch.

Terraform

The Databricks Terraform provider manages workspaces, Unity Catalog objects, groups, policies and grants beside the Azure resources.

Answer. Moves platform configuration. Hand changes show up as drift on the next plan.

The mix-up. Jobs are managed in both Terraform and a bundle, and each overwrites the other. Give every resource one owner. **9.06**

Use it for the platform: workspaces, catalogs, external locations, grants and policies.

Workspace UI

Creating and editing jobs, compute and grants by hand in the workspace.

Answer. Moves nothing. Anyone with CAN MANAGE changes production directly, and only the audit log records it.

The mix-up. Someone disconnects a bundle job for a quick fix. The fix never reaches the repo, and the next deploy removes it. **9.02**

Use it for exploring in dev and reading production.

WHICH ONE, WHEN

- If jobs and pipelines move from dev to prod, define them in a bundle.
- If engineers develop in the workspace, give them Git folders on branches.
- If you build workspaces, catalogs and grants, use Terraform.
- If production can be edited by hand, limit CAN MANAGE to the deploying service principal and look for jobs with edit_mode EDITABLE.

C.17 Billing system tables, tags, budgets and serverless usage policies.

All four come up when someone asks who spent the money. They are set up by different people and often not at all. The real split is whether each one records spend, labels it, warns about it, or attributes serverless work that has no cluster to tag.

THE QUESTION THAT SEPARATES THEM

Does it record spend, label it, warn about it, or attribute it?

Billing system tables

system.billing.usage holds every billable usage record with workspace, SKU, identity, job or warehouse IDs and custom tags. system.billing.list_prices holds list prices.

Answer. Records spend at list price, kept 365 days by default. Records usually arrive within 12 hours.

The mix-up. People expect a job_id on every job's usage. Jobs on all-purpose compute have none; use cluster_id for them. **8.01 · 8.05**

Use it for the source of truth for chargeback and trend reports.

Tags

Key-value labels on workspaces, pools, all-purpose and job compute, and SQL warehouses. They land in custom_tags and on the Azure VMs.

Answer. Labels spend. Workspace, pool and cluster tags reach Azure resources, but a cluster from a pool does not pass its own tags to the VMs.

The mix-up. Teams tag clusters and expect serverless work to be labelled too. Serverless has no cluster to tag. **8.01 · 8.07**

Use it for every classic resource, enforced through compute policies.

Budgets

Account-level targets in US dollars at list price, filtered by workspace or tags, with up to 4 thresholds that send email.

Answer. Warns about spend. Alerts can lag by up to 24 hours, and a budget does not stop compute. Block usage applies to Unity Gateway requests only.

The mix-up. Finance reads the budget as a cap. For compute it only sends email; limits come from policies on compute. **8.04**

Use it for monthly targets per team or workspace that someone reads.

Serverless usage policies

In Public Preview. Each policy holds tags added to the serverless usage of the users assigned to it.

Answer. Attributes serverless spend through custom_tags. It enforces no limit.

The mix-up. Admins create a policy and expect existing jobs to pick it up. Existing notebooks, jobs and pipelines are not assigned automatically. **8.04**

Use it for serverless notebooks, jobs and pipelines in chargeback by team.

WHICH ONE, WHEN

- If you need the numbers, query the billing system tables.
- If classic spend must split by team, enforce tags through compute policies.
- If serverless spend must split by team, assign serverless usage policies.
- If someone must hear before month end, set a budget with thresholds per team.

C.18 Geo-redundant storage, deep clone replication, OpenSharing to another region and managed disaster recovery.

All four put something in a second region, so each gets called the DR plan. The real split is what each carries across, and how much work stands between the outage and running jobs.

THE QUESTION THAT SEPARATES THEM

What survives a region outage, and how long until you are back?

Geo-redundant storage

An Azure Storage redundancy option, such as GRS or GZRS, that copies a storage account's blobs to the paired region in the background.

Answer. Files survive, minus the newest writes. No metastore, job or grant comes with them. Microsoft says not to rely on it for workspace root storage.

The mix-up. Teams tick GRS and write DR: covered. Afterwards nothing knows what the files are. **8.08**

Use it for durability of the files, never as the whole plan.

Deep clone replication

A scheduled job runs DEEP CLONE into an external location in the second region. Each rerun copies only what changed.

Answer. Tables survive as of the last run, so the schedule sets RPO. A clone known only to the primary metastore is unreadable in an outage: register it in the secondary metastore and measure RPO there.

The mix-up. Clones fail on tables with filters, masks or ABAC policies unless the job's identity is exempt (7.17). **8.09 · 3.12**

Pick it when you want a copy you control, at a cost you can see.

OpenSharing to another region

A share from the primary metastore to a recipient metastore in the second region.

Answer. Nothing on its own: the recipient reads the provider's storage live. Only local copies survive.

The mix-up. Teams count the share as a replica. In an outage every shared table goes down with the primary. **8.09**

Use it for feeding a local copy in the second region, with cross-region egress on the bill.

Managed disaster recovery

A gated Preview and paid Mission Critical add-on. It replicates Unity Catalog metadata, managed table data and workspace assets.

Answer. Most of the workspace, failing over in minutes. It skips managed volume data, materialized views, streaming tables, pipelines, secrets, models, shares, and tables with filters, masks or ABAC tags (8.09).

The mix-up. Teams expect external table data to be copied. Only metadata is; you replicate the storage and create the external locations. **8.09**

Pick it when RTO must be short and the account team accepts you.

WHICH ONE, WHEN

- If nobody has written RPO and RTO per tier, do that first.
- If tables must exist in the second region, deep clone them on a schedule and redeploy jobs from code.
- If RTO must be short, apply for managed disaster recovery and plan for what it skips.
- If the plan is GRS or a share alone, add a real copy and test a failover.

C.19 DBFS root, mounts, workspace files and volumes.

All four look like a file path in a notebook, and old code mixes dbfs:/, /mnt/, /Workspace/ and /Volumes/ freely. The real split is whether Unity Catalog checks who reads the file, and whether Databricks still wants new work there.

THE QUESTION THAT SEPARATES THEM

Who governs the path, and should new work use it?

DBFS root

The storage created with the workspace, read at dbfs:/ paths. Hive managed tables and /FileStore uploads landed here by default.

Answer. Unity Catalog does not govern it. It is deprecated, and new accounts are provisioned without it.

The mix-up. Production tables and libraries still sit in dbfs:/user/hive/warehouse, and nobody can say who reads them. **1.05 · 7.03**

Use it for nothing new. Move data to managed tables or volumes.

Mounts

dbutils.fs.mount links an ADLS container to a /mnt/ path through one stored service principal or key.

Answer. The mount's credential, for everyone: all users in the workspace reach the container. Mounts are deprecated, do not work with Unity Catalog and do not run on serverless.

The mix-up. A job moves to serverless and fails on /mnt/. The fix is an external location and a volume, not a new mount. **1.06**

Use it for nothing new. List them with dbutils.fs.mounts() and replace each one.

Workspace files

Files under /Workspace beside notebooks: Python modules, YAML, small CSVs. Each file is at most 500 MB.

Answer. Workspace permissions, as for notebooks. Unity Catalog does not govern them.

The mix-up. People land data files here because it is easy. They are meant for code and small files, not datasets. **5.15**

Use it for code and configuration that travel with Git folders.

Volumes

Unity Catalog objects for files, at /Volumes/catalog/schema/volume. Managed volumes use Unity Catalog storage; external ones sit on an external location.

Answer. Unity Catalog, through READ VOLUME and WRITE VOLUME grants. Databricks recommends them in place of DBFS root and mounts.

The mix-up. Teams create one volume over a whole container, so one grant covers everything in it. **1.06**

Use it for landing files, libraries and exports in new work.

WHICH ONE, WHEN

- If a path starts with dbfs:/ or /mnt/, plan its move: a volume for files, a managed table for tables.
- If it is code or configuration, keep it in workspace files under Git.
- If data files need grants, use a volume per audience.
- Before you remove a mount, search jobs and notebooks for its /mnt/ path and switch them to /Volumes/.

C.20 SYNC, SET MANAGED, DEEP CLONE and Hive metastore federation.

All four bring Hive metastore tables into Unity Catalog, and migration plans often name one for everything. The real split is whether the files move, and whether jobs that still use hive_metastore keep working during the move.

THE QUESTION THAT SEPARATES THEM

Does the data move, and can the old Hive jobs keep running?

SYNC

SYNC TABLE or SYNC SCHEMA registers Hive external tables, and with AS EXTERNAL managed ones outside DBFS root, as Unity Catalog external tables.

Answer. No data moves; both names point at the same files, so Hive jobs keep running. It does not keep syncing: rerun it after Hive changes.

The mix-up. Teams run SYNC on Hive managed tables in DBFS root. Those cannot be synced; clone them instead. **1.05 · 7.15**

Pick it when external tables must reach Unity Catalog quickly, with no copy.

SET MANAGED

ALTER TABLE ... SET MANAGED turns an external table, or a federated Hive table with MOVE or COPY, into a managed table. It needs Runtime 17.3 LTS or serverless.

Answer. The data moves; writers pause for minutes. MOVE turns off Hive access to the source; COPY leaves two tables that drift.

The mix-up. Streams are left running; they must restart. Roll back within 14 days with UNSET MANAGED, also on 17.3 LTS or serverless; commits made meanwhile time travel by version only. **1.01 · 1.05**

Pick it when the table should end up managed.

DEEP CLONE

CREATE TABLE ... DEEP CLONE copies a Hive Delta table's data into a new Unity Catalog table. History is not copied.

Answer. The data is copied. Hive jobs keep writing to the old table, so the two drift until cutover.

The mix-up. Teams clone, move some jobs, and leave others writing to Hive. Clone again just before cutover, then stop the Hive writers. **3.12**

Pick it when Hive managed tables in DBFS root must move.

Hive metastore federation

A foreign catalog that mirrors the workspace's Hive metastore in Unity Catalog, with reads and writes for the internal metastore.

Answer. Nothing moves. Old jobs keep using hive_metastore while new ones use the foreign catalog under Unity Catalog grants.

The mix-up. Fallback mode stays on after the move, so unmigrated jobs bypass Unity Catalog checks. Audit its use, then turn it off. **1.05**

Pick it when hundreds of jobs must move one team at a time.

WHICH ONE, WHEN

- If tables are external and must reach Unity Catalog now, SYNC them.
- If jobs move one team at a time, federate the Hive metastore first.
- If the end state is managed, pilot SET MANAGED on one table in a quiet window, with its owner's approval; on federated tables choose MOVE so Hive stops writing.
- If managed tables sit in DBFS root, DEEP CLONE them and cut writers over on one date.

C.21 Expectations, CHECK constraints and data quality monitoring.

All three are called data quality, and a team with one often believes it has the others. The real split is when each acts: on each row in a pipeline, on the whole write, or afterwards, across tables nobody wrote rules for.

THE QUESTION THAT SEPARATES THEM

Does it stop bad data, record it, or notice what nobody wrote a rule for?

Expectations

Named SQL conditions on a pipeline's streaming tables, materialized views or temporary views, written as EXPECT ... ON VIOLATION.

Answer. Your choice per rule. The default keeps the row and records it; DROP ROW removes it; FAIL UPDATE stops the update. Counts go to the event log.

The mix-up. Teams write warn rules, never read the event log, and call the data checked. A warn rule stops nothing. **5.06**

Pick it when rows flow through a pipeline and bad ones should be dropped or counted.

CHECK constraints

ALTER TABLE ... ADD CONSTRAINT ... CHECK, and NOT NULL, on a Delta table. Primary and foreign keys are informational.

Answer. Stops the write: one violating row fails the whole transaction, from any writer. Adding one first checks the existing rows.

The mix-up. Teams declare a primary key and expect duplicates to be rejected. Only NOT NULL and CHECK are enforced. **5.01**

Pick it when a rule must hold for every writer, and a failed write is acceptable.

Data quality monitoring

Anomaly detection, in Public Preview, learns when each table in a schema updates and how many rows it gets. It covers managed and external tables, materialized views and streaming tables. Data profiling, GA, tracks distributions per table.

Answer. Notices: stale or short tables are flagged in Catalog Explorer and in `system.data_quality_monitoring.table_results`. It stops nothing.

The mix-up. Teams expect it on views or foreign tables, which it skips. It judges commit timing, not event time, and bills as serverless. **5.17**

Pick it when hundreds of tables need a freshness and completeness net nobody has to write.

WHICH ONE, WHEN

- If a bad row should be dropped or counted in a pipeline, write an expectation.
- If no writer may ever break a rule, add a CHECK or NOT NULL constraint.
- If you need to hear that a table went stale, enable anomaly detection on its schema (MANAGE needed) and alert on the results.
- If a rule only warns, someone must read the event log; otherwise make it drop or fail.

C.22 SQL warehouses, Lakebase, synced tables and Model Serving.

All four answer requests from apps and dashboards over Unity Catalog data, so teams use whichever they already run. The real split is how fast one answer must come back and how many arrive at once.

THE QUESTION THAT SEPARATES THEM

How fast must one answer come back, and how many at once?

SQL warehouse

Compute for SQL over Delta tables, used by dashboards, BI tools and the SQL APIs.

Answer. Built for scans and aggregates in seconds. Pro and classic warehouses run 10 queries per cluster and queue the rest.

The mix-up. An app's per-customer lookups go to a warehouse, and clusters are added as traffic grows. Each lookup is still an analytical query.

6.01 · 10.05

Use it for BI, dashboards and ad hoc SQL.

Lakebase

Managed Postgres inside Azure Databricks, GA, with autoscaling and scale to zero.

Answer. Indexed reads and writes by key. Postgres roles govern its own tables. Scale to zero is on by default; keep it off where a wake-up delay hurts (S.25).

The mix-up. Teams build a separate Postgres server and a copy job, and own both. **10.05 · S.25**

Pick it when an app reads or writes single rows in milliseconds.

Synced tables

A managed pipeline copies a Unity Catalog table into a read-only Postgres table in Lakebase, in Snapshot, Triggered or Continuous mode.

Answer. Reads hit Lakebase. Continuous trails the source by seconds; Triggered and Continuous need change data feed.

The mix-up. The app writes to a synced table, and the next refresh overwrites it. Row filters and column masks on the source do not carry over, so sync only what every reader may see. **10.05**

Pick it when Gold rows must be looked up by key from an app.

Model Serving

Serverless REST endpoints for MLflow and foundation models, with autoscaling and optional scale to zero.

Answer. One prediction per request, with under 50 ms of overhead. Route optimization lifts a workspace past 200 queries per second.

The mix-up. Production endpoints scale to zero. The first request waits for a cold start, and docs advise against it. **10.03 · 10.04**

Pick it when the answer is a prediction, not a stored row.

WHICH ONE, WHEN

- If people query with SQL and seconds are fine, use a SQL warehouse.
- If an app reads or writes rows in milliseconds, use Lakebase.
- If those rows come from Gold, feed Lakebase with synced tables, not a copy job.
- If the answer is a prediction, serve the model and look features up in Lakebase.

C.23 IP access lists, Private Link, VNet injection, NCC and serverless network policies.

All five are called network security. The real split is the direction of the traffic and which compute started it: people coming in, classic compute going out, or serverless compute going out.

THE QUESTION THAT SEPARATES THEM

Which direction is the traffic going, and which compute started it?

IP access lists and front-end Private Link

Two ways to limit who reaches the workspace. A list allows or blocks public IP ranges; Private Link gives users a private endpoint.

Answer. Inbound, from people and tools, for any compute. A list cannot block private-endpoint traffic. Neither controls what compute sends out.

The mix-up. An allow list saved without the admins' own range locks them out. Private Link also needs private DNS. **8.16**

Pick it when users must reach the workspace only from known networks.

VNet injection

Classic compute runs in two subnets of your VNet, with no public node IPs.

Answer. Outbound from classic compute only: your NSGs, routes and firewall apply. A new VNet needs an explicit way out, such as a NAT gateway.

The mix-up. Teams expect subnet rules to cover serverless jobs, which do not run there, and get 403 from storage. **8.10 · 8.11**

Pick it when classic compute must reach on-premises or pass your firewall.

NCC private endpoints

Rules in a network connectivity configuration make Databricks create private endpoints from serverless compute to your storage, SQL or Key Vault.

Answer. Outbound from serverless only, to the resources you name. The resource owner must approve each endpoint, and each bills per hour.

The mix-up. Teams add the rule and leave it PENDING; it expires after 14 days. **8.12**

Pick it when serverless must reach a resource that has public access off.

Serverless network policies

An account-level policy on serverless egress: full access, or restricted to listed domains, storage and external locations, with a dry-run mode.

Answer. Outbound from serverless only. Restricted access denies by default, including direct storage paths in notebooks. It opens no private route.

The mix-up. Admins restrict egress, and packages or external model endpoints fail until their domains are listed. Dry-run first. **8.18**

Pick it when you must limit where serverless code can send data.

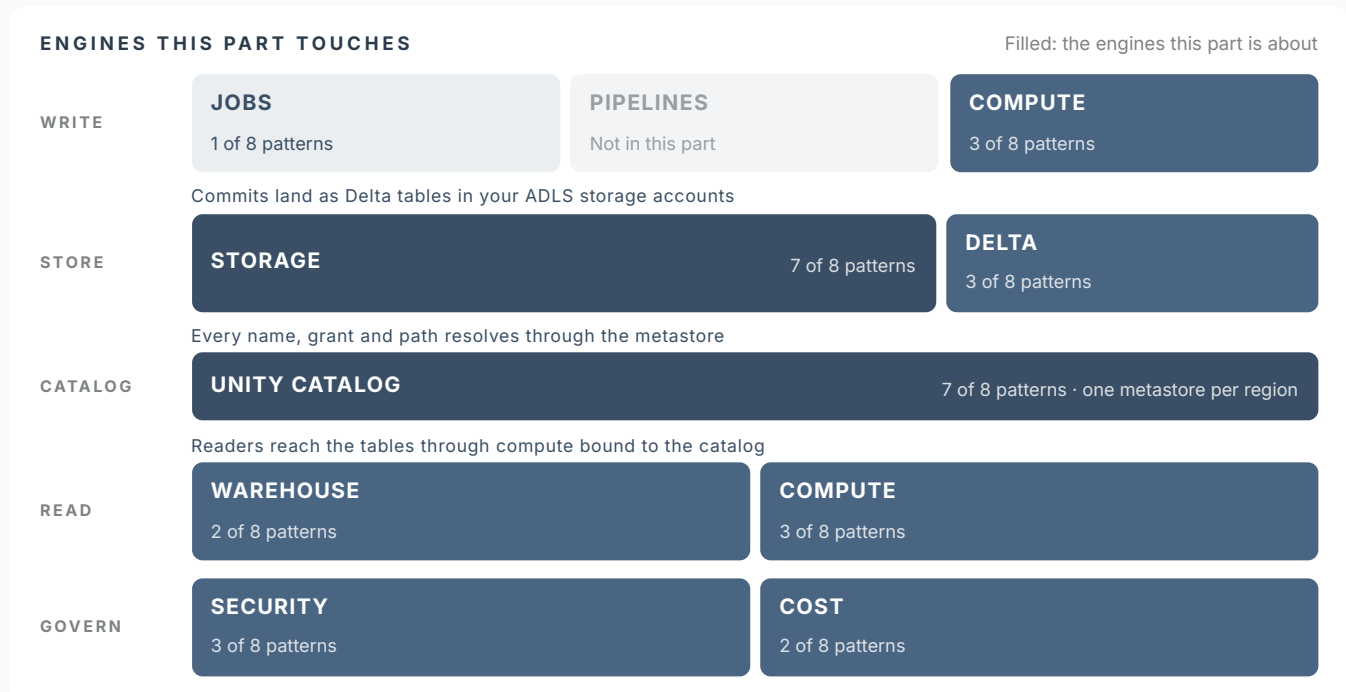
WHICH ONE, WHEN

- If the question is who may reach the workspace, use IP access lists, front-end Private Link, or both.
- If classic compute must sit in your network, use VNet injection and give it a route out.
- If serverless must reach a resource with public access off, use an NCC private endpoint and get it approved.
- If serverless must not send data anywhere unlisted, use a restricted network policy after a dry run.

01

Storage, Unity Catalog and Data Placement.

Where data lives, who can reach it and from which workspace: metastore, catalogs, managed and external tables, external locations and regions. Most of these failures are decided before the first job runs.



NO.	PATTERN	COMPUTE	STORAGE	DELTA	UC	WH	SEC	PAGE
1.01	Every table is external so the data stays in your own storage. None of them get optimised automatically.							29
1.02	You dropped a migrated table the way you always did. A week later its files are gone.							30
1.03	Finance wants its own metastore. The account will not create a second one in the region.							31
1.04	A developer dropped a production table from the dev workspace. Both workspaces share one metastore.							32
1.05	The migration moved every table to Unity Catalog. A nightly job still reads last month's data.							33
1.06	Your notebook reads abfss:// paths on the old cluster. On serverless the same cell fails.							34
1.07	Another team points its tool at the folder behind a managed table. Its numbers stop matching yours.							35
1.08	Jobs that read one storage account run far slower than the rest. The Azure bill shows data transfer.							36

1.01 Every table is external so the data stays in your own storage. None of them get optimised automatically.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

In a classic workspace, managed tables also live in **your storage account**, under the catalog's managed location. Only managed tables get **Predictive Optimization** and automatic liquid clustering. Make managed the default, then check that Predictive Optimization is on.

Cost of failure **Cost**

Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Everything external	You choose every path. Tools outside Databricks read and write the folders directly. <code>DROP TABLE</code> never deletes files.	No Predictive Optimization, so <code>OPTIMIZE</code> , <code>VACUUM</code> and statistics are your jobs. Other tools' schema changes reach the catalog only after <code>MSCK REPAIR TABLE ... SYNC METADATA</code> .
Everything managed	Files sit under the catalog's managed location, which is Databricks-managed default storage in a serverless workspace unless the catalog names your own. Predictive Optimization runs <code>OPTIMIZE</code> , <code>VACUUM</code> and <code>ANALYZE</code> .	No path access, so a tool that only reads folders cannot use them. JSON, CSV and Avro cannot be managed tables; they go to volumes.
Managed by default DEFAULT	Tables Databricks writes are managed. A table is external only when a named system outside Databricks writes it, or must read it by path.	Needs a written rule for exceptions and an owner for every external location.

FIG. 1.01 What changes between options is who maintains the files and who may read them by path.

DEFAULT	SIGNS YOU CHOSE WRONG
Managed by default, external by named exception. Raw files land in volumes, and tables are created by name, never by path.	A nightly job runs <code>OPTIMIZE</code> over hundreds of tables. Some tables never see <code>VACUUM</code> . Paths nobody can match to an owner.

CHECK IT

List the external tables. Predictive Optimization is on by default for accounts created since 11 November 2024, and existing accounts were enabled in a rollout due to finish in August 2026, so read the setting on each catalog. Then confirm it ran (history table in Public Preview).

```
SELECT table_catalog, table_schema, table_name
FROM system.information_schema.tables
WHERE table_type = 'EXTERNAL';

DESCRIBE CATALOG EXTENDED sales_prod;
-- Predictive Optimization row: enabled, disabled or inherited

SELECT catalog_name, schema_name, table_name, operation_type, start_time
FROM system.storage.predictive_optimization_operations_history
ORDER BY start_time DESC LIMIT 20;
```

RELATED **1.02** You dropped a migrated table the way you always did

1.07 Another team points its tool at the folder behind a managed table

1.02 You dropped a migrated table the way you always did. A week later its files are gone.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

In the Hive metastore, dropping an **external** table left the files alone. A Unity Catalog **managed** table keeps its files for a **recovery period**, 7 days by default, then deletes them.

Cost of failure **Data loss**Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (HIVE METASTORE)

"DROP TABLE removes the name. The data is still in the folder if I need it back."

IN A UNITY CATALOG MANAGED TABLE

DROP TABLE starts the recovery period, 7 days by default. Setting it per catalog or schema with ALTER SCHEMA ... RETAIN DROPPED TO 14 DAYS (7 to 30 days, or 0 hours to switch recovery off) is in Public Preview, needs Databricks Runtime 17.3 or above and MANAGE or ownership, and covers only tables dropped after you set it. Inside the period, UNDROP TABLE brings the table back. After it, the files are deleted from your storage account within 48 hours. External tables keep their files.

Do this. Check the type before you drop: DESCRIBE TABLE EXTENDED shows MANAGED or EXTERNAL. To recover, run SHOW TABLES DROPPED IN the schema, then UNDROP TABLE by name or WITH ID.

RELATED 1.01 Every table is external so the data stays in your own storage

1.05 The migration moved every table to Unity Catalog

COMPARE

STORAGE

INGESTION

DELTA

COMPUTE

JOBS

SQL

SECURITY

PLATFORM

CI/CD

AI

SCENARIOS

1.03 Finance wants its own metastore. The account will not create a second one in the region.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Unity Catalog allows **one metastore per region**, and every workspace in that region attaches to it. Isolation comes from **catalogs**, **workspace bindings** and catalog-level storage.

Cost of failure **Rework**Status **GA** · Verified **Oct 2026**

1

Metastore per region. All workspaces in that region share it, and a workspace can only attach to the metastore in its own region.

Source: Databricks docs, Unity Catalog best practices, Oct 2026

HOW YOU WILL NOTICE

A plan that gives each business unit its own metastore stops at the second one. Check which metastore a workspace uses from any notebook or SQL warehouse.

```
SELECT current_metastore();  
-- azure:westeurope:<uuid>, one per region
```

GUARD

Give finance isolation inside the shared metastore: its own catalog, managed storage in its own storage account, and a binding to its own workspaces.

```
CREATE CATALOG finance_prod  
MANAGED LOCATION  
'abfss://prod@stfin.dfs.core.windows.net/';  
-- path must sit inside an external location
```

WORKAROUNDS

Separate admins with catalog ownership and bindings, and separate storage with catalog-level managed locations. Data that must stay in another region is read through OpenSharing from that region's metastore.

NEIGHBOURING LIMITS

1,000

Catalogs per metastore. Can be raised

10,000

Schemas per catalog. Can be raised

1,000,000

Tables per metastore. Fixed

RELATED **1.04** A developer dropped a production table from the dev workspace

1.08 Jobs that read one storage account run far slower than the rest

1.04 A developer dropped a production table from the dev workspace. Both workspaces share one metastore.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Every catalog is reachable from every workspace on the metastore until you **bind** it. Grants decide who; bindings decide **from which workspace**. Bind each environment's catalogs to that environment's workspaces.

Cost of failure **Outage**

Status **Public Preview** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Catalog per team	One clear owner per catalog. Simple grants while the platform is small.	All environments sit inside one catalog, so a binding cannot separate dev from prod.
Catalog per environment	Bind <code>dev</code> , <code>test</code> and <code>prod</code> to their workspaces.	Domains are schemas: one owner, shared storage and schema names.
Catalog per environment and domain	<code>prod_sales</code> , <code>dev_sales</code> : each bound to its environment's workspaces, with its own managed location. Dev gets a read-only binding where it needs real data.	More catalogs to create, so script them. The naming rule must hold from the first catalog.

DEFAULT

FIG. 1.04 The layout decides whether a binding can draw the line between environments.

DEFAULT

A catalog per environment and domain, set to `ISOLATED`, bound read-write to its own environment's workspaces and read-only where others need to read.

SIGNS YOU CHOSE WRONG

Developers hold `MODIFY` on prod tables. Prod is written from more than one workspace.

BEFORE YOU CHANGE IT

`ISOLATED` cuts off every workspace at once, grants or not, until bindings are added. List the workspaces that used the catalog in 30 days (audit table, Public Preview), get the platform owner's approval for the list, include the workspace you run the CLI from, and run both commands below back to back. Rollback: `--isolation-mode OPEN`.

```
SELECT workspace_id, COUNT(*) AS events
FROM system.access.audit
WHERE service_name = 'unityCatalog' AND event_date >= current_date() - 30
      AND request_params['full_name_arg'] LIKE 'prod_sales.%'
GROUP BY workspace_id;
-- only events with full_name_arg: ask owners too
```

CHECK IT

Isolate the prod catalog, then bind it. From the read-only workspace a write now fails, even with `MODIFY`.

```
databricks catalogs update prod_sales --isolation-mode ISOLATED
databricks workspace-bindings update-bindings catalog prod_sales --json '{"add":[{"workspace_id":1111,"binding_type":"BINDING_TYPE_READ_WRITE"},
{"workspace_id":2222,"binding_type":"BINDING_TYPE_READ_ONLY"}]}'
```

RELATED **1.03** Finance wants its own metastore **1.05** The migration moved every table to Unity Catalog

1.05 The migration moved every table to Unity Catalog. A nightly job still reads last month's data.

ENGINES **JOBS** PIPELINES COMPUTE STORAGE DELTA **UNITY CATALOG** WAREHOUSE SECURITY COST

The job uses **two-part names**. On a workspace whose default catalog is still **hive_metastore**, `sales.orders` resolves to the old copy, and the job succeeds on stale data.

Cost of failure **Wrong data**
Status **GA** · Verified **Oct 2026**

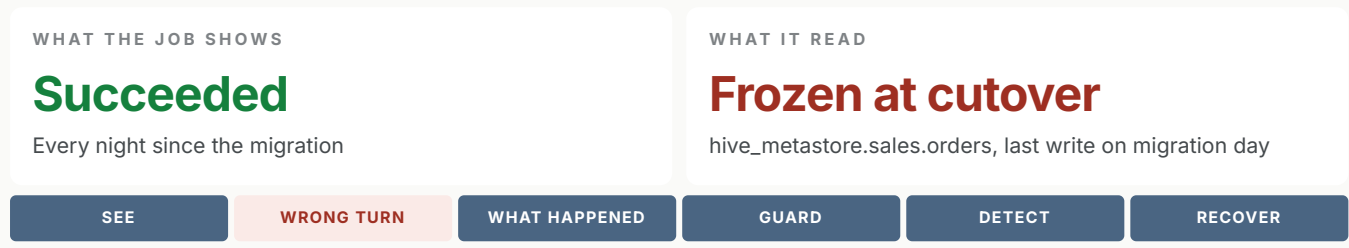


FIG. 1.05 The run status covers the job, not the catalog it read from.

WRONG TURN

"Once every table exists in Unity Catalog, every query uses it."

WHAT HAPPENED

A two-part name resolves through the session's `USE CATALOG`, then the cluster's `spark.databricks.sql.initial.catalog.namespace`, then the workspace default catalog. Workspaces not auto-enabled for Unity Catalog kept `hive_metastore` as that default, and the old tables were left in place.

1 GUARD

Set the workspace default catalog in Admin Settings, Advanced. Clusters and SQL warehouses pick it up only on restart. New code uses three-part names.

```
SELECT current_catalog();
-- run it inside the job, not in your notebook
```

2 DETECT

Compare the last write. `DESCRIBE HISTORY` needs Delta; for other Hive tables use the newest file time.

```
DESCRIBE HISTORY
  sales_prod.sales.orders LIMIT 1;
SELECT MAX(
  _metadata.file_modification_time)
FROM hive_metastore.sales.orders;
```

BEFORE YOU CHANGE IT

Disable legacy access is workspace-wide. List every job and warehouse query that still names `hive_metastore`, move all jobs to Runtime 13.3 LTS or above, and get the workspace owner's sign-off. You can switch it back for 30 days; after that the workspace is Unity Catalog only, permanently.

RELATED **1.04** A developer dropped a production table from the dev workspace

1.06 Your notebook reads `abfss://` paths on the old cluster

1.06 Your notebook reads abfss:// paths on the old cluster. On serverless the same cell fails.

ENGINES

JOBSPipelines

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The old cluster reached storage with an **account key** or a **mount** that Unity Catalog never saw. Serverless and standard access mode reach cloud storage only through **external locations**.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	OLD CLUSTER, MOUNT OR KEY IN SPARK CONFIG	SERVERLESS OR STANDARD ACCESS MODE
Path authorised by	The account key or the mount's credential	An external location and its storage credential
Who is checked	Nobody: every user of the cluster has the key's access	The user's <code>READ FILES</code> or <code>WRITE FILES</code> grant
Storage key in Spark config	Accepted	Not settable on serverless
Outcome	Reads succeed	Permission error on the first read

FIG. 1.06 The same path, two ways to authorise it. Only one of them is governed by Unity Catalog.

CAUSE

Keys and mounts give every user of the cluster the same access, so Unity Catalog cannot govern them. Serverless accepts only a short list of Spark properties, and storage keys are not on it. Standard access mode runs user code with low privileges, so file access goes through external locations.

FIX

Register the storage once. Create a storage credential backed by an Azure Databricks access connector: its managed identity has no secret to expire. Then add an external location and a grant, and give raw files a volume.

```
CREATE EXTERNAL LOCATION landing
URL 'abfss://landing@stlakeprod.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL ac_lake_prod);
GRANT READ FILES ON EXTERNAL LOCATION landing TO `data-engineers`;

CREATE EXTERNAL VOLUME raw.files.landing
LOCATION 'abfss://landing@stlakeprod.dfs.core.windows.net/files';
```

PREVENT

Remove storage keys from cluster configs and replace `/mnt` paths as you migrate. When no job uses a mount, turn on **Disable DBFS root and mounts**. A path that works on one cluster only is a path Unity Catalog cannot see.

- RELATED
- [1.05](#) The migration moved every table to Unity Catalog
- [1.07](#) Another team points its tool at the folder behind a managed table

1.07 Another team points its tool at the folder behind a managed table. Its numbers stop matching yours.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Path access to managed tables is **not supported**, and their folders are named by **UUID**. A reader that lists Parquet files without the Delta log counts superseded files too.

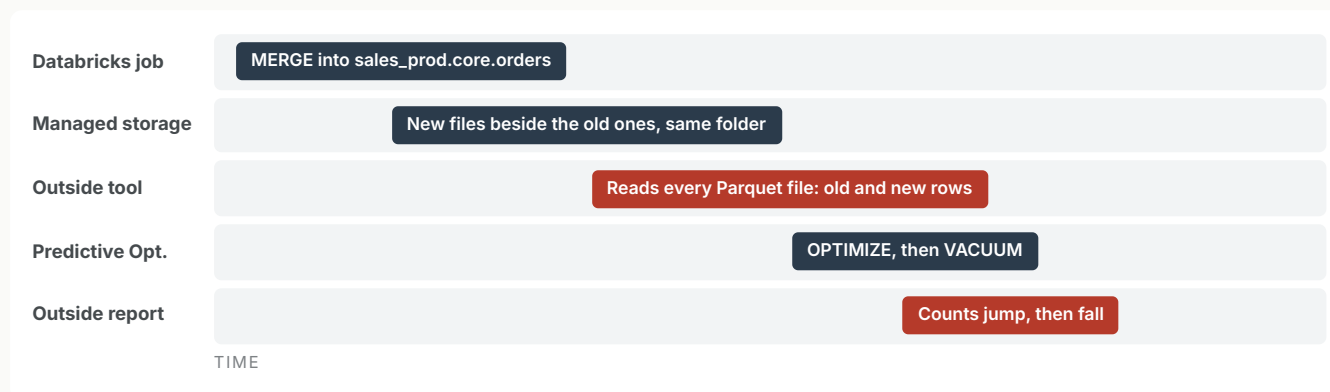
Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

FIG. 1.07 Databricks reads through the Delta log. The outside tool reads the folder, so every rewrite changes its answer.

SEEN IN

The outside tool: a report, a pipeline or another engine's query

CAUSE LIVES IN

The read path: a folder listing that ignores the Delta log

CAUSE

A Delta table is its log plus the files the log lists. After `MERGE`, `UPDATE` or `OPTIMIZE`, superseded files stay in the folder until `VACUUM`. With deletion vectors, a file still holds rows that were deleted. Unity Catalog does not support path access to managed tables, so nothing warns the reader.

FIX

Give each outside reader a governed route to the table, not the folder. Engines use the Unity REST API or the Iceberg REST catalog with credential vending. Another organisation or region uses OpenSharing. Fabric uses a mirrored Azure Databricks catalog. Outside writes to managed tables are a separate, optional route: the Unity REST API on tables with catalog commits (GA), in preview (Beta in the April 2026 release notes, so check the Previews page). It needs `MODIFY`, Spark 4.0 or 4.1, Delta Spark 4.3.0 or above and the Unity Catalog Spark client 0.5.0 or above. Until it is GA, keep writers inside Databricks.

CONFIRM IN BOTH ENGINES

```
-- Metastore: External data access on
GRANT EXTERNAL USE SCHEMA
ON SCHEMA sales_prod.core TO `bi-reader`;
```

```
-- Same version, same answer, both sides
SELECT COUNT(*) FROM sales_prod.core.orders
VERSION AS OF 412;
```

`EXTERNAL USE SCHEMA` is not part of `ALL PRIVILEGES`, and only the catalog owner can grant it.

RELATED **1.01** Every table is external so the data stays in your own storage

1.06 Your notebook reads `abfss://` paths on the old cluster

1.08 Jobs that read one storage account run far slower than the rest. The Azure bill shows data transfer.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The storage account is in **another region**. Every file request crosses regions, adding latency, and Azure bills the bytes as **inter-region transfer**.

Cost of failure **Slow queries**Status **GA** · Verified **Oct 2026**

WORKSPACE AND METASTORE

West Europe

Compute, catalogs and most storage

STORAGE ACCOUNT STLAKENEU

North Europe

Every read crosses regions

FIG. 1.08 The metastore is regional. An external location can still point at storage anywhere.

MEASURE FIRST

- 1 Find where the slow table's files live.

```
DESCRIBE DETAIL legacy_neu.core.orders;
-- location: abfss://...@stlakeneu.dfs.core.windows.net/...
```

- 2 Compare the storage region with the workspace region.

```
az storage account show -n stlakeneu --query primaryLocation
az databricks workspace show -n dbw-prod -g rg-data --query location
```

- 3 In Azure Cost Management, filter the storage account to the Bandwidth meter category.

CAUSE

Each file read is a request to a storage endpoint in the other region, so latency adds up per file. Small files make it worse: more files, more round trips. Azure bills the transferred bytes, so the cost grows with every scan.

FIX

Keep data and compute in one region. Give each catalog a managed location in the workspace region. Data that must stay in the other region is read through OpenSharing from that region's metastore, or copied on a schedule with **DEEP CLONE**. Clones do not work on tables with row filters or column masks, or on materialized views: rebuild those from their source.

```
CREATE OR REPLACE TABLE sales_prod.core.orders
DEEP CLONE legacy_neu.core.orders;
-- a rerun commits only what changed since the last clone
```

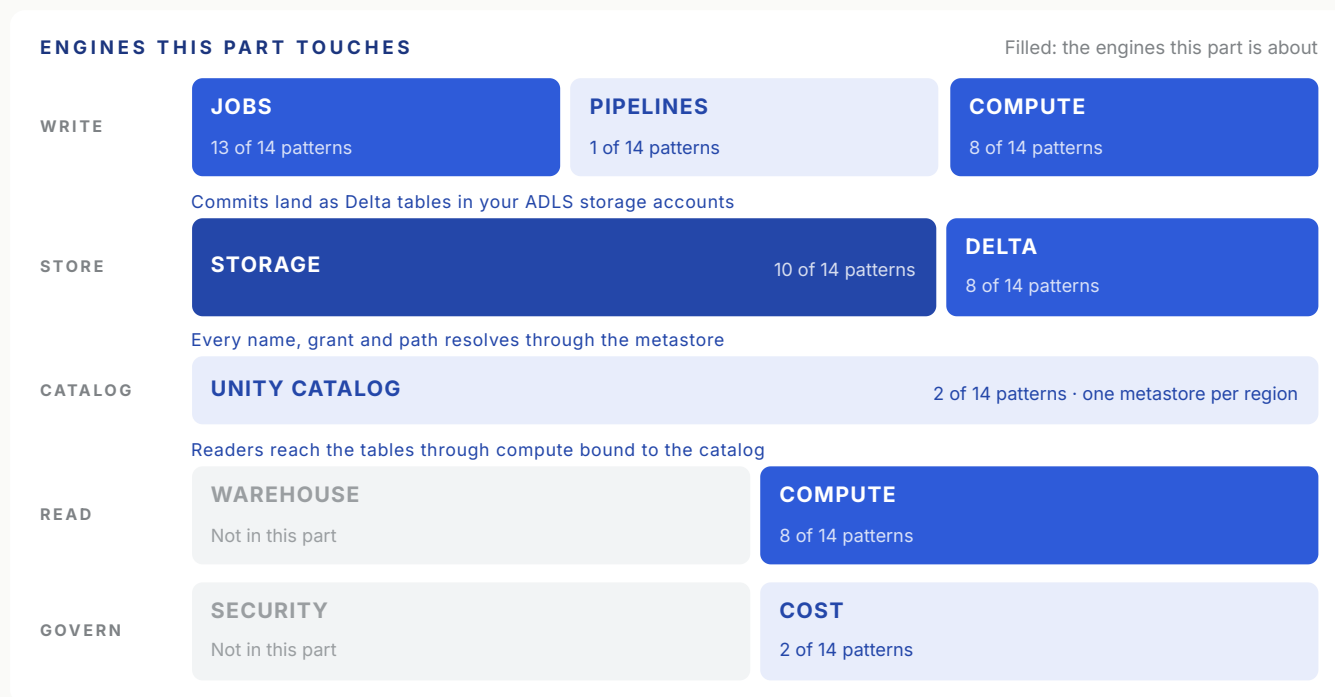
EXPECTED RESULT

Scan time on the cloned table matches other local tables, and the bandwidth line for the account goes flat.

RELATED [1.03 Finance wants its own metastore](#) [1.01 Every table is external so the data stays in your own storage](#)

02 Ingestion and Streaming.

How files and records arrive and become tables: Auto Loader discovery and schemas, checkpoints, triggers, streaming writes to Delta and Lakeflow Connect. Most of these failures report success, because the checkpoint decides what was read and the run status never looks at it.



NO.	PATTERN	JOBS	COMPUTE	STORAGE	DELTA	UC	COST	PAGE
2.01	Auto Loader processed the same file twice. Duplicate rows appeared downstream.	●	●	●	●	●	●	39
2.02	Auto Loader runs for hours on a folder with millions of files. It finds nothing new.	●	●	●	●	●	●	40
2.03	You switched Auto Loader from directory listing to file notifications. Some files were never loaded.	●	●	●	●	●	●	41
2.04	Every Auto Loader batch succeeded. Part of the data never reached a column.	●	●	●	●	●	●	42
2.05	The same Auto Loader code infers amount as a double in dev. In prod it comes out as a string.	●	●	●	●	●	●	43
2.06	You deleted the checkpoint to fix a stuck stream. It reloaded months of data.	●	●	●	●	●	●	44
2.07	Two streaming queries share one checkpoint folder. Both report healthy batches.	●	●	●	●	●	●	45
2.08	You changed the streaming query. The restart fails on the old checkpoint.	●	●	●	●	●	●	46
2.09	Trigger once loaded the whole backlog in one batch. Available now ran for hours before it stopped.	●	●	●	●	●	●	47
2.10	Your trigger fires every 60 seconds. Each batch takes 90.	●	●	●	●	●	●	48

NO.	PATTERN	JOBS	COMPUTE	STORAGE	DELTA	UC	COST	PAGE
2.11	A stream writes to Delta every 30 seconds. Small files pile up and reads slow down.							49
2.12	A foreachBatch MERGE failed and the batch was retried. The audit table now holds it twice.							50
2.13	Auto Loader's first run on two million files takes most of a day. Each batch holds 1,000 of them.							51
2.14	You need SQL Server and Salesforce data in the lakehouse every hour. Three teams propose three tools.							52

2.01 Auto Loader processed the same file twice. Duplicate rows appeared downstream.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Auto Loader loads each file **path** exactly once. A file re-sent under a new name, or rewritten while `cloudFiles.allowOverwrites` is on, is new input to it, so only an **idempotent MERGE** downstream keeps the table clean.

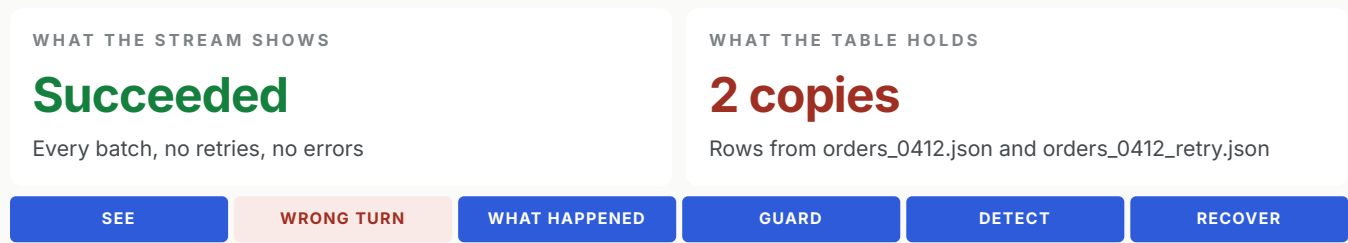
Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

FIG. 2.01 Exactly once holds for each file path. It says nothing about the rows inside.

WRONG TURN

"Auto Loader is exactly once, so the table cannot hold the same order twice."

WHAT HAPPENED

The sender timed out and uploaded the same orders again with a new file name. A new path is a new file, so Auto Loader loaded it. With `cloudFiles.allowOverwrites` set to `true`, a rewritten file is loaded again in full, and in file notification mode one unchanged file can even arrive twice, by event and by listing. With the default `false`, which version of a rewritten file gets loaded is not guaranteed.

1 GUARD

Treat landed files as immutable and leave `allowOverwrites` at `false`. Keep the source file on every bronze row, and load silver with **MERGE** on the business key, never with a plain append.

```
raw = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", schema_path)
    .load("/Volumes/sales_prod/raw/landing/orders/")
    .select("?", "_metadata.file_path",
        "_metadata.file_modification_time"))
```

2 DETECT

Find keys that arrived in more than one file.

```
SELECT order_id, COUNT(*) AS copies,
    COLLECT_SET(file_path) AS files
FROM sales_prod.raw.orders
GROUP BY order_id
HAVING COUNT(*) > 1;
```

3 RECOVER

Bronze may keep both copies: it records what arrived. Rebuild the affected silver rows with a **MERGE** that keeps one row per `order_id`, the one with the latest `file_modification_time`. From then on the merge absorbs any re-send.

RELATED [2.12](#) A foreachBatch MERGE failed and the batch was retried [2.06](#) You deleted the checkpoint to fix a stuck stream

2.02 Auto Loader runs for hours on a folder with millions of files. It finds nothing new.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Directory listing lists the whole load path on every trigger, and the checkpoint keeps every file ever seen in **RocksDB**. Turn on **file events** for the external location and move processed files out of the path.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

EVERY TRIGGER

Millions of files listed

to find a handful of new ones, or none

EVERY START

State for every file seen

tracked in the checkpoint for exactly once

FIG. 2.02 The cost grows with what is in the folder, not with what arrives.

MEASURE FIRST

- 1 Split each batch into finding files and writing them.

```
p = q.lastProgress
print(p["durationMs"]["latestOffset"], p["durationMs"]["addBatch"])
print(p["sources"][0]["metrics"]["numFilesOutstanding"])
```

- 2 Count tracked files.

```
SELECT COUNT(*) FROM cloud_files_state('/Volumes/sales_prod/ops/chk/orders');
```

CAUSE

Directory listing, the default, repeats the full listing every trigger, so `latestOffset` dominates.

FIX

Turn on **Enable file events** on the external location (default for locations created since 12 February 2026). The location owner approves it: the service adds resources in the storage account. Runtime 18 LTS streams use it by default; before that set `cloudFiles.useManagedFileEvents`. Move one stream first, then the rest; revert with `false`. Run at least every 7 days to stay incremental. From Runtime 16.4, `cleanSource` moves files after `retentionDuration` (30 days by default): archive outside the load path.

```
df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.useManagedFileEvents", "true")
    .option("cloudFiles.cleanSource", "MOVE")
    .option("cloudFiles.cleanSource.retentionDuration", "7 days")
    .option("cloudFiles.cleanSource.moveDestination",
        "/Volumes/sales_prod/raw/landing/_archive/orders/")
    .load("/Volumes/sales_prod/raw/landing/orders/"))
```

EXPECTED RESULT

`latestOffset` stops growing with the folder, list transactions fall away, and on 18 LTS `cloud_files_state` shows in `discovery_mode` how each file was found.

RELATED [2.03](#) You switched Auto Loader to file notifications

[2.13](#) Auto Loader's first run on two million files takes most of a day

2.03 You switched Auto Loader from directory listing to file notifications. Some files were never loaded.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Classic notifications see only events that reach the queue. Auto Loader acts on **FlushWithClose**, some Azure API usage does not emit it, and rarely events are lost. Use **file events** on the external location, or schedule a **backfill**.

Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

	DIRECTORY LISTING	FILE NOTIFICATIONS, CLASSIC
Discovery	Lists the load path on each trigger	Event Grid puts events on a queue the stream reads
Setup	Read access	Subscription and queue, 500 per account
No flush event	Found on the next listing	Not seen until a backfill
Event lost	Found on the next listing	Never seen without <code>backfillInterval</code>
Premium	Works	Not supported: no queue storage

FIG. 2.03 Switching modes keeps exactly once for what the stream sees, not for what it misses.

CAUSE

Listing compares the folder with the checkpoint, so every file is found eventually. A classic notification stream knows only what the queue tells it, delivery is not guaranteed, and an upload without `FlushWithClose` leaves nothing to act on. Nothing fails.

FIX

Turn on file events for the external location (default for new ones since 12 February 2026). Runtime 18 LTS streams then use them by default; before that set `cloudFiles.useManagedFileEvents`. Stop classic streams and delete their queues first, one stream at a time. Databricks then runs backfills, and the first run lists the path once. If you keep classic mode, set a backfill; it creates no duplicates.

```
.option("cloudFiles.useNotifications", "true")
.option("cloudFiles.backfillInterval", "1 day")    # classic mode only
```

PREVENT

After any change of mode, compare every file under the path with the checkpoint, in Spark, so listing runs in parallel.

```
from pyspark.sql import functions as F
key = lambda c: F.regexp_extract(c, "/landing/orders/(.*)$", 1)
done = spark.sql(f"SELECT path FROM cloud_files_state('{chk}') "
                "WHERE commit_time IS NOT NULL").select(key("path").alias("k"))
files = (spark.read.format("binaryFile")
        .option("recursiveFileLookup", "true").load(src)
        .select(key("path").alias("k")))    # listing runs distributed
missed = files.join(done, "k", "left_anti")
```

RELATED [2.02](#) Auto Loader runs for hours on a folder with millions of files

[1.06](#) Your notebook reads abfss:// paths on the old cluster

2.04 Every Auto Loader batch succeeded. Part of the data never reached a column.

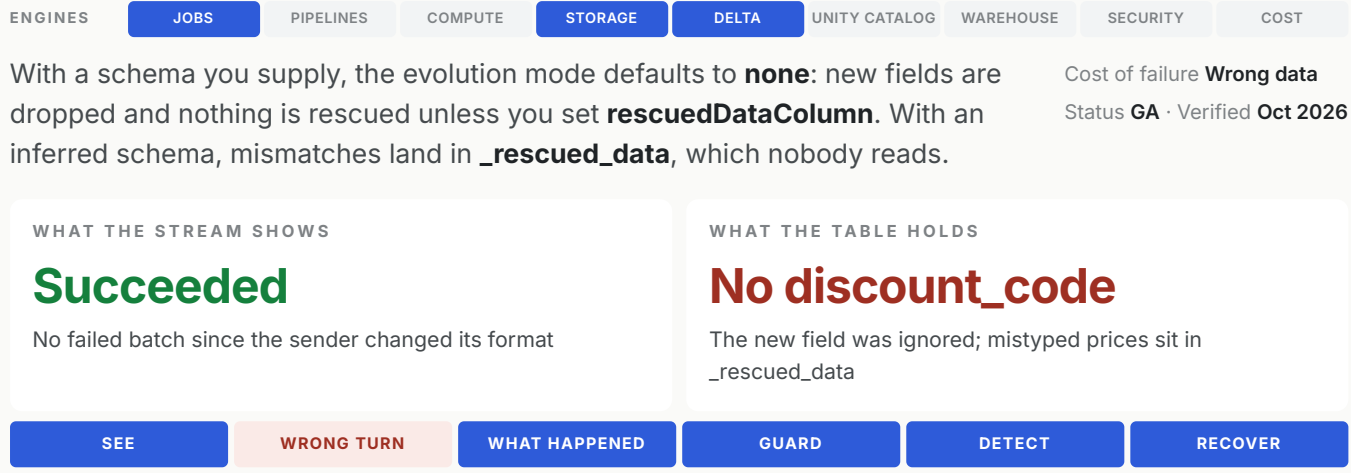


FIG. 2.04 The parser never failed, so neither did the batch.

WRONG TURN

"If a record does not fit the schema, Auto Loader fails the stream."

WHAT HAPPENED

Five modes exist. `addNewColumns`, the default for an inferred schema, and `addNewColumnsWithTypeWidening` stop the stream on a new field, and a restart picks it up. `failOnNewColumns` keeps failing until you change the schema. `rescue` never evolves and rescues new fields. This job passed a schema, so the mode was `none`: new fields ignored, no rescue. Type and case mismatches go to the rescued data column when there is one. A malformed JSON or CSV record in the default `PERMISSIVE` mode is not kept unless the schema has a corrupt-record column.

1 GUARD

Name the rescued column and choose the mode yourself. With a rescued column, `FAILFAST` fails only on records that cannot be parsed at all; type mismatches are rescued.

```
.schema(order_schema)
.option("rescuedDataColumn", "_rescued_data")
.option("cloudFiles.schemaEvolutionMode", "rescue")
.option("mode", "FAILFAST")
```

2 DETECT

Alert on rescued rows per file. On Databricks Runtime 16.4 and above, `cloud_files_state` also shows `SKIPPED_CORRUPTED` files.

```
SELECT file_path, COUNT(*) AS n
FROM sales_prod.raw.orders
WHERE _rescued_data IS NOT NULL
GROUP BY file_path;
```

3 RECOVER

Add the missing column, then fill it from the rescued JSON with `_rescued_data:discount_code` in an `UPDATE`. Rows lost before the rescued column existed come back only by reloading their files under a new checkpoint (2.06).

RELATED [2.05](#) The same Auto Loader code infers different types in prod [3.09](#) A source renamed a column

2.05 The same Auto Loader code infers amount as a double in dev. In prod it comes out as a string.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Auto Loader infers a schema once, from the first **50 GB or 1,000 files** it finds, and stores it in **schemaLocation**. JSON and CSV columns are strings unless **cloudFiles.inferColumnTypes** is on, and then each type depends on what the sample held.

Cost of failure **Rework**
Status **GA** · Verified **Oct 2026**

	DEV	PROD
Files in the sample	Forty clean test files	The first 1,000 found, one with "N/A" in amount
Schema stored in	The dev <code>schemaLocation</code>	The prod <code>schemaLocation</code> , kept until it evolves
Inferred type of amount	double	string
Downstream SUM(amount)	Works	Fails, or every query adds its own cast

FIG. 2.05 Same code, different files in the sample, a different schema stored for good.

CAUSE

Inference runs when a stream starts with an empty `schemaLocation`, and later runs reuse the stored schema. So the schema changes per environment, or whenever someone points a stream at a new or wiped location. Without `inferColumnTypes`, JSON, CSV and XML columns are all strings, nested fields too. With it, one odd value in the sample decides the type.

FIX

Pin the columns that matter with schema hints and let inference handle the rest. A value that does not fit a hinted type goes to `_rescued_data` instead of changing the type. For columns that grow later, `int` to `long` or a wider `decimal`, the Public Preview mode `addNewColumnsWithTypeWidening` (Runtime 16.4 and above) widens the type; the Delta target needs `delta.enableTypeWidening`.

```
.option("cloudFiles.inferColumnTypes", "true")
.option("cloudFiles.schemaHints",
      "amount DECIMAL(18,2), order_ts TIMESTAMP")
.option("cloudFiles.schemaLocation",
      "/Volumes/sales_prod/ops/chk/orders/schema")
```

PREVENT

Keep the hints in the repository with the job, one `schemaLocation` per stream. Before promotion, compare `DESCRIBE TABLE` on the bronze table in each environment. A type that differs is a missing hint.

RELATED [2.04](#) Every Auto Loader batch succeeded [3.09](#) A source renamed a column

2.06 You deleted the checkpoint to fix a stuck stream. It reloaded months of data.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Cost of failure **Wrong data**

Status **GA** · Verified **Oct 2026**

The checkpoint is the stream's only record of what it has read. An emptied or new checkpoint **begins fresh**: Auto Loader loads every file in the path again and a Delta source rereads the table. Restart from a **known position** instead.

YOU'D EXPECT (A CACHE YOU CAN CLEAR)

"The checkpoint holds temporary progress. Delete it and the stream starts clean from now."

IN STRUCTURED STREAMING

Deleting the checkpoint files, or moving the query to a new location, makes the next run begin fresh. Auto Loader's `cloudFiles.includeExistingFiles` is `true` by default, so every file in the path loads again. A restart on the old checkpoint replays the batch planned in the previous run, so a stuck batch usually needs more compute, not a new checkpoint. A new checkpoint restarts batch IDs at 0, so writes using `txnAppId` also need a new one.

Do this. Keep the checkpoint and fix the batch. If you must start a new one, start it before the old one stopped: `startingVersion` for a Delta source; for Auto Loader, `modifiedAfter` set to the oldest file `cloud_files_state` shows without a `commit_time`, minus a safety margin. A cutoff from the last committed file skips older files still outstanding, for good. Write with `MERGE` so the overlap is harmless, then reconcile the folder against the new checkpoint (2.03).

RELATED [2.07](#) Two streaming queries share one checkpoint folder [2.08](#) You changed the streaming query

2.07 Two streaming queries share one checkpoint folder. Both report healthy batches.

Each query needs **its own checkpoint** in durable storage. A second query on the same folder reads the first one's offsets and commits as its own, so it **skips or replays** data while both report healthy batches.

Cost of failure **Wrong data**
Status **GA** · Verified **Oct 2026**

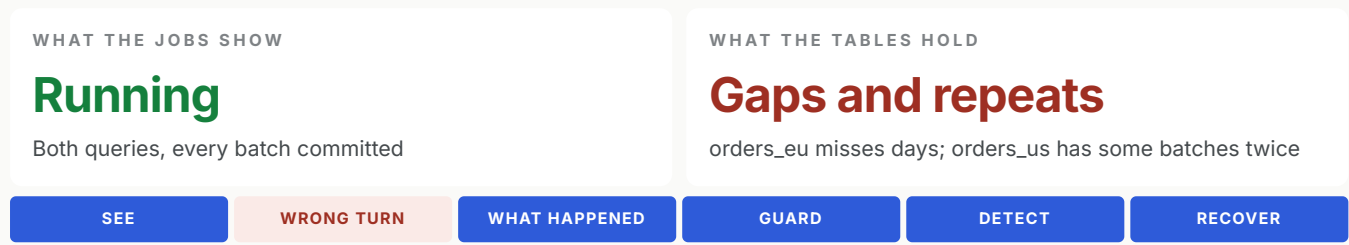


FIG. 2.07 The checkpoint records progress. Nothing records which query wrote it.

WRONG TURN

"The checkpoint is a progress log. A copied notebook can keep the same path."

WHAT HAPPENED

A checkpoint holds the query's id, the offsets planned for each batch, the commits and any state. The copied notebook kept `checkpointLocation`, so each query took the other's last offsets as its own. Other ways to lose a checkpoint: a local or `/tmp` path that goes with the cluster, and a storage lifecycle policy that deletes old files and corrupts the state.

1 GUARD

Derive the path from the target table, in a volume kept for checkpoints, with no lifecycle policy on its storage. Spark trims old checkpoint files itself, keeping `spark.sql.streaming.minBatchesToRetain` batches, 100 by default; do not delete them by hand.

```
def checkpoint_for(table):
    return f"/Volumes/sales_prod/ops/chk/{table}"

(df.writeStream
  .option("checkpointLocation", checkpoint_for("core.orders_eu"))
  .toTable("sales_prod.core.orders_eu"))
```

2 DETECT

A query's id is stored in its checkpoint. Two jobs that log the same id share one.

```
q = (df.writeStream ... .start())
print(q.id, q.name)
# log it from every job, compare
```

3 RECOVER

Stop both. Leave the folder with the query that created it. Give the other a new path and start it from a known position (2.06), then repair both tables with **MERGE** on the business key.

RELATED [2.06](#) You deleted the checkpoint to fix a stuck stream [2.08](#) You changed the streaming query

2.08 You changed the streaming query. The restart fails on the old checkpoint.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The checkpoint identifies sources by **position** and stores the **schema of any state**. Filters, projections, rate limits and triggers can change; a new source path, topic, grouping key or **stateful operator** needs a new checkpoint.

Cost of failure **Outage**Status **GA** · Verified **Oct 2026**

CHANGE	EXAMPLE	RESTART ON THE OLD CHECKPOINT
Add or remove a filter	<code>.where("status <> 'test'")</code>	Allowed
Projection, same output schema	<code>to_json(...)</code> in place of a string column	Allowed
Rate limit or trigger interval	<code>maxFilesPerTrigger</code> 500 to 2000	Allowed, from the next batch planned
Source path or Kafka topic	<code>landing/orders</code> to <code>landing/orders_v2</code>	Not allowed: results unpredictable
Grouping keys, aggregates, dedup keys	<code>groupBy("store")</code> to <code>groupBy("store", "day")</code>	Not allowed: state cannot be restored

FIG. 2.08 The change that looks smallest, one more grouping column, is the one the state cannot absorb.

CAUSE

State is saved with its schema and restored only into the same schema, so any addition, removal or schema change to a streaming aggregation, `dropDuplicates`, stream-stream join or `mapGroupsWithState` is not allowed. Sources are matched by their position in the plan, so adding or swapping one breaks the match too.

FIX

Give the changed query a new checkpoint and a starting point at the old one. For a Delta source, the old query's last progress shows the table version it reached (`endOffset.reservoirVersion`). `startingVersion` is inclusive and a batch can stop part way through a version, so start at that version, not the next. Start aggregations early enough to cover their window, and write with `MERGE` so the overlap does no harm.

```
(spark.readStream
  .option("startingVersion", 4812)  # the version the old stream reached
  .table("sales_prod.core.orders"))
```

PREVENT

Classify every change to a streaming query against this list in review. A stateful change ships with its new checkpoint path and starting version in the same pull request. Run a runtime upgrade on a staging copy of the stream before production.

RELATED [2.06](#) You deleted the checkpoint to fix a stuck stream [2.07](#) Two streaming queries share one checkpoint folder

2.09 Trigger once loaded the whole backlog in one batch. Available now ran for hours before it stopped.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Trigger.Once is deprecated from Databricks Runtime 11.3 LTS and ignores rate limits. **availableNow** reads what was there when the run started, in batches sized by the **rate limits**, then stops.

Cost of failure **Outage**

Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (A NIGHTLY BATCH JOB)

"A scheduled stream reads everything new in one pass and stops when it is done."

WITH STRUCTURED STREAMING TRIGGERS

Trigger.Once runs one batch with no rate limit, so a large backlog becomes one oversized batch that can run out of memory, and a restart replays the same batch. **availableNow** processes everything available at start in as many batches as the rate limits need (2.13), then stops, and a restart resumes from the last committed batch. Files that land during the run wait for the next one, so freshness is the schedule plus the run time.

Do this. Use `trigger(availableNow=True)` on every `writeStream` in the task; a stream without a trigger never stops. Give the task a timeout, and schedule it at the freshness you need.

RELATED [2.13](#) Auto Loader's first run on two million files takes most of a day [5.14](#) One test file started one run

2.10 Your trigger fires every 60 seconds. Each batch takes 90.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

When a batch outlasts the interval, the next one starts **as soon as it finishes**, so the trigger no longer sets the pace. While **processedRowsPerSecond** stays below **inputRowsPerSecond**, the backlog only grows.

Cost of failure **Slow queries**Status **GA** · Verified **Oct 2026**

CONFIGURED

Every 60 seconds

processingTime trigger

ACTUAL

90 seconds, back to back

each batch starts behind the last

FIG. 2.10 A shorter trigger cannot help. Only a faster batch, or a smaller one, can.

MEASURE FIRST

- 1 Compare the two rates and see where the 90 seconds go.

```
p = q.lastProgress
print(p["inputRowsPerSecond"], p["processedRowsPerSecond"])
print(p["durationMs"]) # addBatch, latestOffset, queryPlanning, ...
```

- 2 Read the phase: `addBatch` is your write, `latestOffset` and `getBatch` are the source, `queryPlanning` is planning.

- 3 Watch the backlog: `numFilesOutstanding` for Auto Loader, `maxOffsetsBehindLatest` for Kafka.

CAUSE

Usually `addBatch`. A `foreachBatch` that writes several tables runs them one after another. A `MERGE` into a target with no clustering on the key reads most of the table each batch. A Python UDF or an external call per row sets the pace for the whole batch.

FIX

Keep one `foreachBatch`: a stream per sink reads the source twice. On classic compute, persist the batch, write each sink from it, then unpersist. Serverless raises an exception on `persist`, `cache` and `unpersist`, so write the sinks from the batch without them, and run it with `availableNow`: serverless rejects `processingTime` triggers. Cluster the merge target on its key, and recluster existing files once with `OPTIMIZE ... FULL` (Runtime 16.4 LTS and above); plain `OPTIMIZE` leaves them. Add workers only if `addBatch` still dominates.

```
ALTER TABLE sales_prod.core.orders CLUSTER BY (order_id);
OPTIMIZE sales_prod.core.orders FULL; -- once; can take hours on a large table
```

EXPECTED RESULT

Processed rate stays above input rate until the backlog reaches zero, and batch duration then settles under the 60 second interval.

RELATED [2.11](#) A stream writes to Delta every 30 seconds [3.08](#) A `MERGE` that changes 100 rows rewrites much of the table

2.11 A stream writes to Delta every 30 seconds. Small files pile up and reads slow down.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Every micro-batch commits its own files, so a **short trigger** means many small files. **Auto compaction** after each write, Optimized writes during it and **Predictive Optimization** on managed tables keep the count down without a separate job.

Cost of failure **Slow queries**Status **GA** · Verified **Oct 2026**

WRITER, 30 SECOND TRIGGER

2,880 commits a day

each one adds new files, however few rows

READERS

More files per query

more opens and more log to read

FIG. 2.11 The writer pays per commit. The readers pay per file.

MEASURE FIRST

- 1 Divide size by file count.

```
DESCRIBE DETAIL sales_prod.core.events; -- numFiles, sizeInBytes
```

- 2 Check whether anything compacts the table. Auto compaction shows in history as **OPTIMIZE** with **auto** set to **true**.

```
DESCRIBE HISTORY sales_prod.core.events LIMIT 50;
```

CAUSE

Auto compaction and Optimized writes are always on for **MERGE**, **UPDATE** and **DELETE**, not for streaming appends. Auto compaction runs on the writing cluster after a write succeeds and compacts only files not compacted before. Predictive Optimization runs **OPTIMIZE** separately, on serverless compute, and only for Unity Catalog managed tables.

FIX

Turn both on for the table. **auto** lets Databricks tune the target file size to the table. Make the table managed so Predictive Optimization covers it too (1.01), and use the longest trigger the consumers accept.

```
ALTER TABLE sales_prod.core.events SET TBLPROPERTIES (
  'delta.autoOptimize.optimizeWrite' = 'true',
  'delta.autoOptimize.autoCompact' = 'auto');
```

EXPECTED RESULT

History shows automatic **OPTIMIZE** entries after writes, **numFiles** levels off, and average file size climbs towards the tuned target.

RELATED **3.03** OPTIMIZE runs every hour and writes keep getting slower

1.01 Every table is external so the data stays in your own storage

2.12 A foreachBatch MERGE failed and the batch was retried. The audit table now holds it twice.

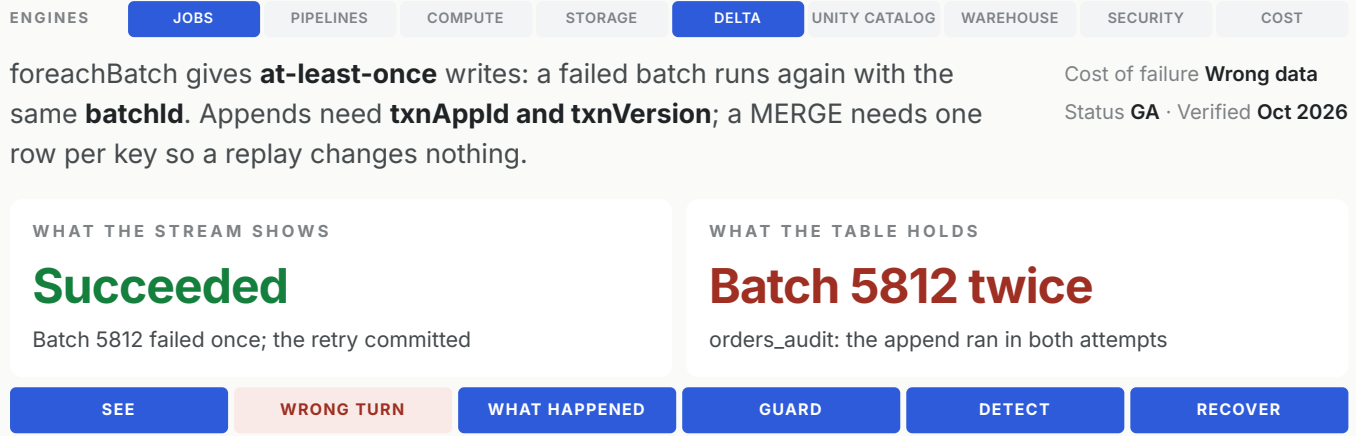


FIG. 2.12 Each write in the function is its own transaction. The retry repeats all of them.

WRONG TURN

"Delta is transactional, so a retried batch cannot write twice."

WHAT HAPPENED

The function appended the batch to `orders_audit`, then merged it into `orders`. The merge failed, the batch ran again, and the append committed twice.

1 GUARD

Append all of the batch tagged with its `batch_id`, and merge only the latest row per key. Persisting the batch is optional on classic compute and fails on serverless, so the code leaves it out.

```
from delta.tables import DeltaTable
from pyspark.sql import Window, functions as F
def upsert(batch_df, batch_id):
    (batch_df.withColumn("batch_id", F.lit(batch_id)).write.mode("append")
     .option("txnAppId", "orders_audit_v1").option("txnVersion", batch_id)
     .saveAsTable("sales_prod.core.orders_audit")) # full batch, no dedup
    w = Window.partitionBy("order_id").orderBy(F.desc("event_ts"), F.desc("seq"))
    latest = batch_df.withColumn("rn", F.row_number().over(w)).where("rn = 1").drop("rn")
    (DeltaTable.forName(spark, "sales_prod.core.orders").alias("t")
     .merge(latest.alias("s"), "t.order_id = s.order_id")
     .whenMatchedUpdateAll("s.event_ts >= t.event_ts")
     .whenNotMatchedInsertAll().execute())
```

2 DETECT

The same event twice within one batch is a replay.

```
SELECT batch_id, order_id, seq, COUNT(*)
FROM sales_prod.core.orders_audit
GROUP BY ALL HAVING COUNT(*) > 1;
```

3 RECOVER

Delete the second copy of each replayed batch. A new checkpoint restarts at batch 0, so change `txnAppId` with it or Delta skips those batches.

RELATED [2.01](#) Auto Loader processed the same file twice

[3.07](#) Your MERGE works alone and fails when two jobs run it at once

2.13 Auto Loader's first run on two million files takes most of a day. Each batch holds 1,000 of them.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

cloudFiles.maxFilesPerTrigger defaults to **1,000** files per batch, so a first run over 2 million existing files is 2,000 batches, each with its own commit. Size the backfill by **bytes** instead.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

1,000

Files per micro-batch by default. With `maxBytesPerTrigger` also set, whichever limit is reached first applies. From Databricks Runtime 18.0 the options page calls both dynamically configured.

Source: Microsoft Learn, Auto Loader options, Oct 2026

HOW YOU WILL NOTICE

Batch after batch reads about 1,000 files, the cluster sits half idle and the backlog falls slowly.

```
m = q.lastProgress["sources"][0]
print(m["numInputRows"],
      m["metrics"]["numFilesOutstanding"])
```

GUARD

For the backfill, raise the file limit and let bytes decide the batch size.

```
.option("cloudFiles.maxFilesPerTrigger",
        "20000")
.option("cloudFiles.maxBytesPerTrigger",
        "20g")
```

WORKAROUNDS

Run the first load with `availableNow` on larger compute, then go back to the usual size and schedule; rate limits can change between restarts on the same checkpoint. Or split by time: a batch read for files up to a cutoff, and the stream with `modifiedAfter` a safety margin before it, so late or slow uploads are not skipped; `MERGE` absorbs the overlap, and a reconcile (2.03) proves nothing fell between. On Runtime 18 LTS, read `batchSizeNumFiles` and `batchSizeNumBytes` in the progress metrics before tuning by hand: the dynamic setting may already be enough. The first run lists the whole path even with file events on.

NEIGHBOURING LIMITS

50 GB

Or 1,000 files, whichever comes first: the schema inference sample

7 days

Longest gap between runs before file events lists the whole path again

1,000

Default `maxFilesPerTrigger` for a Delta table source

RELATED [2.09](#) Trigger once loaded the whole backlog in one batch

[2.02](#) Auto Loader runs for hours on a folder with millions of files

2.14 You need SQL Server and Salesforce data in the lakehouse every hour. Three teams propose three tools.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The **Lakeflow Connect** connectors for SQL Server and Salesforce are generally available. They track changes for you and write **streaming tables** governed by Unity Catalog. Copy pipelines and **JDBC notebooks** stay for sources no connector covers.

Cost of failure **Rework**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Data Factory copy to storage, then Auto Loader	Sources with no managed connector. Teams that already run Data Factory and its integration runtimes.	Two schedulers and two sets of credentials. Watermarks, deletes and schema drift are your code, and the files in between need their own monitoring.
JDBC read in a notebook task	A one-off pull, or a query no connector supports.	Change tracking, deletes and history are hand-written. Each new table adds a full query against the source.
Lakeflow Connect managed connector DEFAULT	SQL Server through change tracking or CDC, with a gateway that captures changes continuously. Salesforce incrementally on SystemModstamp . Serverless ingestion pipelines, SCD type 2 built in, credentials in a Unity Catalog connection.	A column type changes at the source: not evolved automatically. Salesforce formula fields need full snapshots by default, and hard deletes need a full refresh. The SQL Server gateway runs all day on classic compute, so the VM and DBUs bill whatever the schedule. If it is stopped, changes can be lost and need a full refresh.

FIG. 2.14 The connector replaces the change tracking code. The other two options make it yours.

DEFAULT

A managed connector where one is generally available, with the ingestion pipeline scheduled hourly as a Lakeflow Job (the gateway is not scheduled). HubSpot and the query-based connectors (Oracle, PostgreSQL, MySQL and others, no gateway) became GA in May 2026; check each connector's status, as many are in preview. Events your own applications produce go to Zerobus Ingest (GA since February 2026), straight into a Unity Catalog table.

CHECK IT

The gateway needs change tracking or CDC on the primary instance; read replicas are not supported. Confirm it at the source, then that the destination moves every hour.

SIGNS YOU CHOSE WRONG

Deleted source rows still show in the lakehouse. A watermark column nobody owns. A notebook holding a password. Hourly full scans at the source.

```
-- on SQL Server: tables with change tracking
SELECT OBJECT_NAME(object_id) FROM sys.change_tracking_tables;

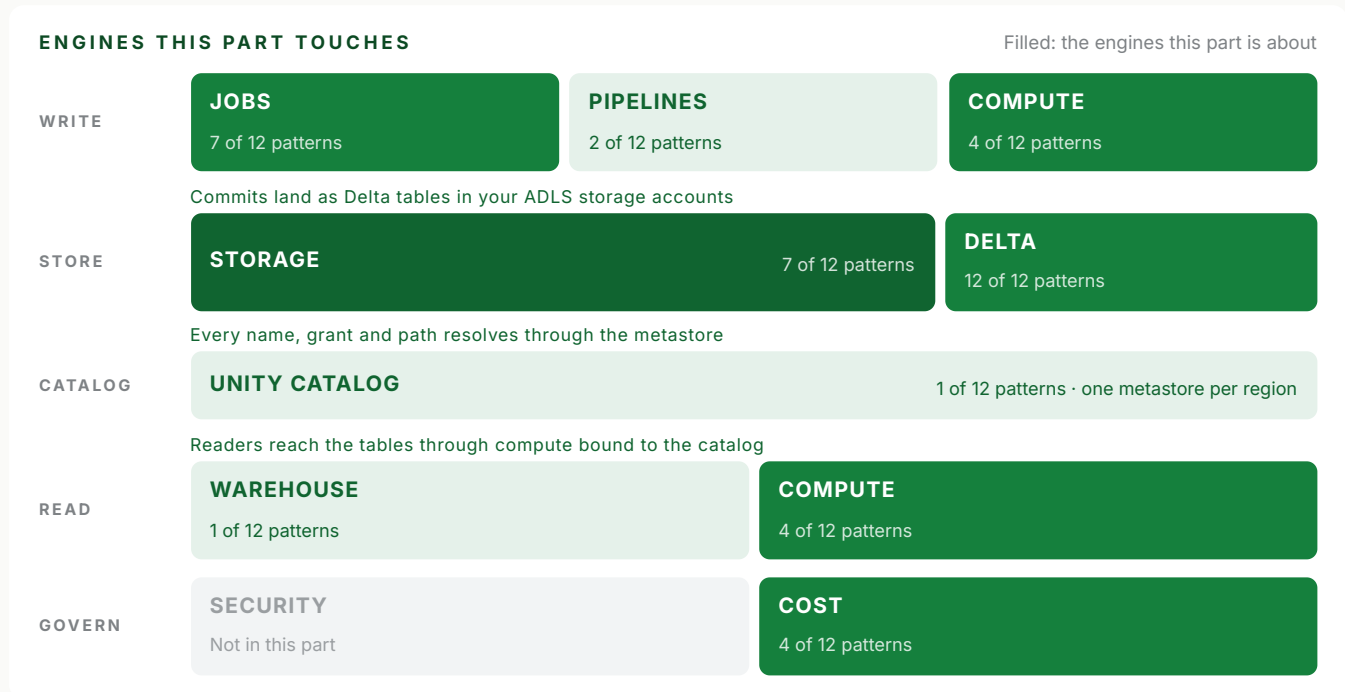
-- in Databricks: one write per scheduled run
DESCRIBE HISTORY sales_prod.erp.customers LIMIT 5;
```

RELATED [5.07](#) You switched a pipeline to continuous [2.01](#) Auto Loader processed the same file twice

03

Delta Lake and Table Layout.

How tables are laid out and kept: clustering, OPTIMIZE and VACUUM, retention, deletion vectors, MERGE, schema change, change data feed, restore and clone. A Delta table is its log plus the files the log lists, and most of these failures come from treating it as a folder of files.



NO.	PATTERN	JOBS	PIPE	COMPUTE	STORAGE	DELTA	COST	PAGE
3.01	Every query on a 500 million row table does a full scan. Someone suggests partitioning by customer.	●	●	●	●	●	●	55
3.02	Liquid clustering is defined on the table. No data has been clustered.	●	●	●	●	●	●	56
3.03	OPTIMIZE runs every hour. Writes to the table keep getting slower.	●	●	●	●	●	●	57
3.04	Time travel to yesterday works. Time travel to last month fails.	●	●	●	●	●	●	58
3.05	You deleted rows and ran VACUUM. Storage did not go down.	●	●	●	●	●	●	59
3.06	An older reader cannot open your table. Another one returns rows you deleted.	●	●	●	●	●	●	60
3.07	Your MERGE works when it runs alone. Two jobs running it at once fail.	●	●	●	●	●	●	61
3.08	A MERGE that changes 100 rows rewrites a large part of the table.	●	●	●	●	●	●	62
3.09	A source renamed a column. Schema evolution added a new one and left the old one empty.	●	●	●	●	●	●	63
3.10	A Change Data Feed reader was paused for ten days. After the restart, changes are missing.	●	●	●	●	●	●	64

NO.	PATTERN	JOB	PIPE	COMPUTE	STORAGE	DELTA	COST	PAGE
3.11	You restored a table to last week. Downstream streams broke or kept the bad rows.	●	●	●	●	●	●	65
3.12	You cloned a table for testing. The clone costs as much as the original.	●	●	●	●	●	●	66

3.01 Every query on a 500 million row table does a full scan. Someone suggests partitioning by customer.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Queries skip files only when per-file **statistics** can rule them out, and that needs rows with similar keys in the same files. Under 100 TB, do not partition: use **liquid clustering**, and **CLUSTER BY AUTO** on managed tables.

Cost of failure **Slow queries**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Partition by a filter column	Tables of 100 TB and up, filtered on a low-cardinality column such as a date, when a test shows it beats clustering. Whole partitions drop or overwrite cleanly.	The table is under 100 TB or a partition holds under 1 GB: thousands of small files. <code>customer_id</code> makes a folder per customer. Changing the column means rewriting the table.
Z-order with scheduled OPTIMIZE	Existing partitioned tables filtered on one or two high-cardinality columns.	Every run rewrites the files it covers, and the gain fades with each added column. Predictive Optimization never runs <code>ZORDER</code> , so the job stays yours.
Liquid clustering DEFAULT	Up to 4 keys, low or high cardinality. Keys change with <code>ALTER TABLE</code> without a rewrite. On managed tables with Predictive Optimization on, <code>CLUSTER BY AUTO</code> picks keys from the query history.	Keys need statistics, which external tables collect on the first 32 columns only. Data is clustered by <code>OPTIMIZE</code> or large enough writes, not by the definition (3.02).

FIG. 3.01 All three arrange rows so file statistics can rule files out. They differ in who keeps the arrangement up.

DEFAULT

Liquid clustering on every new table: `CLUSTER BY AUTO` on managed tables, one or two filter columns as manual keys on external tables. Partition only at 100 TB or more, by date, and only if it measures faster.

SIGNS YOU CHOSE WRONG

Thousands of partition folders holding files of a few MB. A nightly `ZORDER` job that runs longer each month. A filter column placed after the 32nd, so its files have no statistics.

CHECK IT

See how the table is laid out, then convert a partitioned table in place (Runtime 18.1 and above): same table, history and grants. Check first: it is not a streaming table or materialized view in a pipeline, and no OpenSharing share filters it by partition. The protocol rises to writer 7 and reader 3 and cannot be lowered, so readers need 13.3 LTS or above. Batch writes carry on; streams may need a restart. Reclustering existing files needs `FULL` once, a rewrite for off-peak hours.

```
DESCRIBE DETAIL sales_prod.core.orders;
-- partitionColumns, clusteringColumns, numFiles, sizeInBytes

ALTER TABLE sales_prod.core.orders
REPLACE PARTITIONED BY WITH CLUSTER BY (order_date, customer_id);
OPTIMIZE sales_prod.core.orders FULL;
```

Before 18.1, a CTAS rebuild loses history, grants, properties, constraints and tags. Compare files read with files pruned in the query profile.

RELATED 3.02 Liquid clustering is defined. No data has been clustered

3.03 OPTIMIZE runs every hour and writes keep getting slower 1.01 Every table is external so the data stays in your own storage

3.02 Liquid clustering is defined on the table. No data has been clustered.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

`CLUSTER BY` sets the keys; it moves no data. Rows are clustered by **OPTIMIZE**, run by you or by **Predictive Optimization**, and on write only for some operations above a **size threshold**.

Cost of failure **Slow queries**Status **GA** · Verified **Oct 2026**

WHAT THE TABLE SHOWS

Clustered by customer_id

clusteringColumns in DESCRIBE DETAIL

WHAT THE FILES HOLD

0 OPTIMIZE runs

Every file written by small MERGE batches

SEE

WRONG TURN

WHAT HAPPENED

GUARD

DETECT

RECOVER

FIG. 3.02 The keys are table metadata. The file layout only changes when something rewrites the files.

WRONG TURN

"Once the table has `CLUSTER BY`, every write keeps it clustered."

WHAT HAPPENED

Clustering on write covers `INSERT INTO`, CTAS and RTAS, `COPY INTO` from Parquet and append writes, and only when one transaction passes a threshold: on a managed table 64 MB for one key up to 1 GB for four, and four times that on other tables. This table is fed by `MERGE` every five minutes and is external, so Predictive Optimization never runs and no job runs `OPTIMIZE`. Keys added later leave old files as they were.

1 GUARD

Managed tables: check the state first, because Predictive Optimization is on by default for accounts created since 11 November 2024 and in supported regions. Enable it only where it is off (it bills serverless compute: agree it with the cost owner; `INHERIT` reverts). Use `CLUSTER BY AUTO`. External tables: schedule `OPTIMIZE`.

```
DESCRIBE CATALOG EXTENDED sales_prod; -- PO state
ALTER CATALOG sales_prod ENABLE PREDICTIVE OPTIMIZATION;
OPTIMIZE sales_ext.core.events; -- external: daily job
```

2 DETECT

No `OPTIMIZE` in the history means nothing was clustered. Managed tables: the PO log (Public Preview, 180 days).

```
DESCRIBE HISTORY sales_ext.core.events;
SELECT operation_type, start_time
FROM system.storage
    .predictive_optimization_operations_history
WHERE catalog_name = 'sales_prod'
    AND schema_name = 'core'
    AND table_name = 'events';
```

3 RECOVER

Run `OPTIMIZE` once: it clusters incrementally. After a key change, `OPTIMIZE ... FULL` (Databricks Runtime 16.4 LTS and above) reclusters all existing data, so run it outside load windows. With `CLUSTER BY AUTO`, a small or rarely queried table may get no keys at all, and that is expected.

RELATED **3.01** Every query on a 500 million row table does a full scan

3.03 `OPTIMIZE` runs every hour and writes keep getting slower

1.01 Every table is external so the data stays in your own storage

3.03 OPTIMIZE runs every hour. Writes to the table keep getting slower.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

An hourly **OPTIMIZE** pays for compute start-up 24 times a day to compact one hour of small files, and with **ZORDER** or without deletion vectors it also conflicts with running writers. On managed tables let **Predictive Optimization** decide when.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

HAND-SCHEDULED JOB

24 runs a day

ZORDER rewrites, cluster start-up, MERGE conflicts if Z-ordered

PREDICTIVE OPTIMIZATION

Runs when it pays

Serverless, per table, logged in a system table

FIG. 3.03 The schedule was set once. The table, its writers and the platform kept changing.

MEASURE FIRST

- 1 Read what each run did, and how long the writes take.

```
DESCRIBE HISTORY sales_prod.core.orders LIMIT 48;
-- OPTIMIZE rows: numRemovedFiles, numAddedFiles, p50FileSize
-- MERGE rows: executionTimeMs, week by week
```

- 2 Without deletion vectors, or with Z-order, search the writer's retries for `ConcurrentDeleteReadException`.

CAUSE

Bin-packing is idempotent, so a run only compacts files written since the last one, and each run still starts compute. **ZORDER BY** is not idempotent and rewrites the files it covers. With deletion vectors on Runtime 14.3 LTS and above, **OPTIMIZE** cannot conflict with **UPDATE**, **DELETE** or **MERGE** unless **ZORDER BY** is used; otherwise it removes files a running **MERGE** read, and the writer retries.

FIX

Stop the hourly job. On managed tables, check Predictive Optimization is on (`DESCRIBE SCHEMA EXTENDED`): it runs **OPTIMIZE**, **VACUUM** and **ANALYZE** when they pay off, and never **ZORDER**, so move Z-ordered tables to liquid clustering (3.01). **VACUUM** follows `delta.deletedFileRetentionDuration`; from Runtime 18.0 a **RETAIN** value is ignored except 0 hours. For append writers to external tables, turn on auto compaction and Optimized writes (always on for **MERGE**, **UPDATE**, **DELETE**) and run **OPTIMIZE** daily, outside the write window.

```
ALTER SCHEMA sales_prod.core ENABLE PREDICTIVE OPTIMIZATION; -- if off
ALTER TABLE sales_ext.core.events SET TBLPROPERTIES (
  'delta.autoOptimize.autoCompact' = 'auto',
  'delta.autoOptimize.optimizeWrite' = 'true');
```

EXPECTED RESULT

Compute spent on **OPTIMIZE** falls, write durations stop climbing and conflict retries disappear. The Public Preview `predictive_optimization_operations_history` system table shows each run.

RELATED **3.01** Every query on a 500 million row table does a full scan

3.07 Your MERGE works alone and fails when two jobs run it at once **2.11** A stream writes to Delta every 30 seconds

3.04 Time travel to yesterday works. Time travel to last month fails.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Time travel needs both the **log** and the **data files** of the old version. The log is kept 30 days by default, but the data files only **7 days**, and that shorter one decides.

Cost of failure **Data loss**Status **GA** · Verified **Oct 2026**

7 days

Default `delta.deletedFileRetentionDuration`. **VACUUM** may delete files older than this, and from Databricks Runtime 18.0, or 12.2 for managed tables, time travel to an older version is blocked even before **VACUUM** runs.

Source: Microsoft Learn, Work with table history, Oct 2026

HOW YOU WILL NOTICE

An audit or a rebuild asks for the table as of a month ago, and the query fails. On older runtimes it may fail later, on files **VACUUM** already deleted.

```
SELECT * FROM sales_prod.core.orders
TIMESTAMP AS OF '2026-09-01';
-- fails: older than the retention
```

GUARD

Set both properties on tables that need a longer window. The log setting must be at least the file setting.

```
ALTER TABLE sales_prod.core.orders
SET TBLPROPERTIES (
  'delta.deletedFileRetentionDuration'
    = 'interval 30 days',
  'delta.logRetentionDuration'
    = 'interval 30 days');
```

WORKAROUNDS

Every rewrite kept in the window costs storage, so do not stretch retention to months. For long history, write a scheduled **DEEP CLONE** snapshot or keep an append-only history table. From Runtime 18.0, **VACUUM** ignores **RETAIN** except 0 hours, so the table property is the only dial; **RETAIN 0 HOURS** removes every unreferenced file, which a running reader or writer may still need. Time travel, **RESTORE** and clones fail on tables with row filters or column masks, under ABAC and on materialized views unless the running identity is exempt (7.17): rebuild those from source.

NEIGHBOURING LIMITS

30 days

Default log retention: how far **DESCRIBE HISTORY** reaches

1,000

Files listed by **VACUUM ... DRY RUN**, at most

24 hours

Wait between the two runs of **DROP FEATURE ... TRUNCATE HISTORY**

RELATED **3.05** You deleted rows and ran **VACUUM** **3.11** You restored a table to last week

7.17 A row filter or mask blocks your own recovery jobs

3.05 You deleted rows and ran VACUUM. Storage did not go down.

ENGINES

JOB

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

With **deletion vectors**, `DELETE` only marks rows, and the file stays referenced. `VACUUM` removes files that are unreferenced and older than the **retention**, and Azure **soft delete** may keep even those billed.

Cost of failure **Cost**

Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (A DATABASE OR PLAIN PARQUET)

"DELETE removes the rows and VACUUM gives the space back, so storage drops the same day."

IN A DELTA TABLE ON AZURE STORAGE

`DELETE`, `UPDATE` and `MERGE` write a small deletion vector beside the Parquet file, which keeps the deleted rows until it is rewritten. Rewritten files then wait out `delta.deletedFileRetentionDuration`, 7 days by default, before `VACUUM` may remove them. If blob soft delete is on for the storage account, deleted files are kept and billed at the same rate as live data for its retention, 1 to 365 days.

Do this. Run `REORG TABLE ... APPLY (PURGE)` to rewrite files with soft-deleted rows, then `VACUUM` after the retention has passed; on managed tables Predictive Optimization runs that `VACUUM`. For an erasure request, record the time of the `REORG` and vacuum to it. From Runtime 18.0, `VACUUM ... RETAIN n HOURS` is ignored unless n is 0, so shorten the wait only through the table property, and not below 7 days. Check the soft delete retention on the account, and expect storage to fall in steps, days after the delete.

RELATED [3.04](#) Time travel to last month fails [3.06](#) An older reader cannot open your table

[1.02](#) You dropped a migrated table the way you always did

3.06 An older reader cannot open your table. Another one returns rows you deleted.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Deletion vectors raise the table's **reader protocol**. A client that checks it and does not support them fails; a tool that reads the **Parquet files** without the log returns deleted rows. Upgrade the reader, or **DROP FEATURE**.

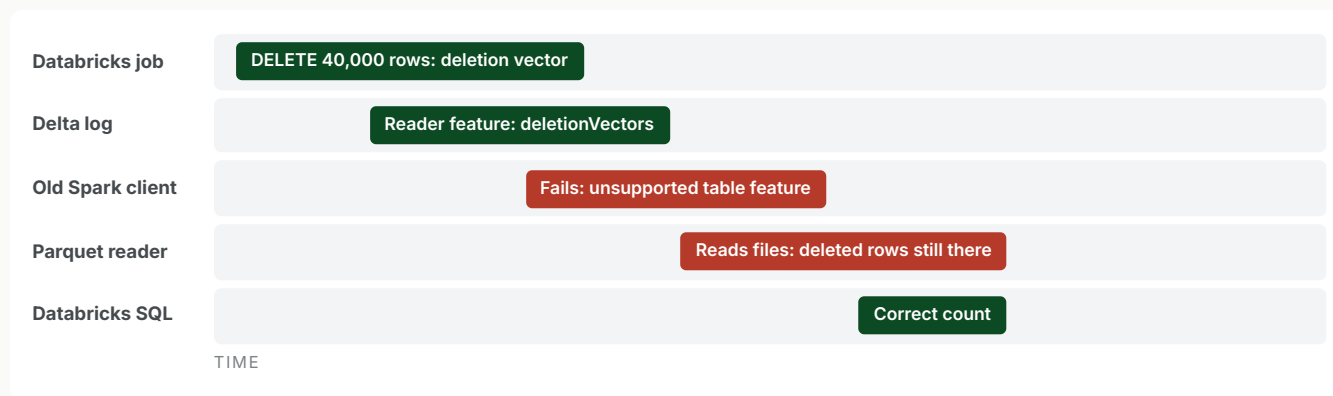
Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

FIG. 3.06 One commit, three readers. Only the one that applies the deletion vector gets the right rows.

SEEN IN

The outside reader: an old Spark job with open source Delta, an old runtime, a tool that reads Parquet

CAUSE LIVES IN

The table protocol, raised when deletion vectors were turned on, often by a workspace setting

CAUSE

Reading needs Databricks Runtime 12.2 LTS or above, or open source Delta 2.3.0 or above. OpenSharing recipients need Runtime 14.1 or delta-sharing-spark 3.1. The workspace setting **Auto-enable deletion vectors** turns them on for new tables, so nobody decided. A Parquet reader cannot apply a deletion vector at all.

FIX

Upgrade the reader, or give it a governed route that reads through the log (1.07). Reads through the Unity REST API are GA, but writes to managed tables from outside Delta clients need catalog commits and are in Public Preview, so keep writers inside Databricks. Where neither is possible, drop the feature on Runtime 16.3 or above: it rewrites the files, adds `checkpointProtection` and lowers the protocol so open source clients can read. If a client still refuses, add `TRUNCATE HISTORY`: two runs 24 hours apart, and history older than 24 hours is gone.

CONFIRM IN BOTH ENGINES

```
SHOW TBLPROPERTIES sales_prod.core.orders;
-- delta.minReaderVersion
-- delta.feature.deletionVectors
```

```
ALTER TABLE sales_prod.core.orders
DROP FEATURE deletionVectors;
-- then rerun the old reader
```

DROP FEATURE conflicts with concurrent writes, and streams reading the table stop and must restart.

RELATED [3.05](#) You deleted rows and ran `VACUUM` [1.07](#) Another team points its tool at the folder behind a managed table [3.07](#) Your `MERGE` works alone and fails when two jobs run it at once

3.07 Your MERGE works when it runs alone. Two jobs running it at once fail.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Delta detects conflicts at **file** level: two MERGEs that read the same files conflict even when their rows differ. **Row-level concurrency** needs **deletion vectors**, no partitions and Runtime 14.3 LTS or above.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	ONE JOB	TWO JOBS AT ONCE
Files read	Every file ON can match	The same files, in both jobs
Check at commit	Nothing else committed	Did another commit add or remove files I read?
Partitioned or no deletion vectors	Succeeds	Can fail with ConcurrentAppendException or ConcurrentDeleteReadException
Both change the same row	Succeeds	Can conflict, even with row-level concurrency

FIG. 3.07 Alone there is nothing to conflict with. Together, overlap in what each job read is enough.

CAUSE

An **ON** condition with only the business key lets each MERGE read the whole table, so every file the other job writes is a conflict. Row-level concurrency turns on by itself for **UPDATE**, **DELETE** and **MERGE** when its requirements are met. Partitioned tables never get it.

FIX

On an unpartitioned or liquid clustered table, turn on deletion vectors and run 14.3 LTS or above. On a partitioned table, put explicit partition values in **ON** so each job reads only its own partitions. Jobs that change the same rows must run one after the other.

```
ALTER TABLE sales_prod.core.orders
SET TBLPROPERTIES ('delta.enableDeletionVectors' = true);

MERGE INTO sales_prod.core.orders t
USING eu_updates s
ON t.order_id = s.order_id AND t.region = 'EU' -- partitioned: name it
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

PREVENT

A retry around the MERGE hides the conflict until volume grows. Make the jobs touch disjoint data, then prove it in test by starting both at the same moment.

RELATED

3.03 OPTIMIZE runs every hour and writes keep getting slower

3.08 A MERGE that changes 100 rows rewrites much of the table

3.06 An older reader cannot open your table

3.08 A MERGE that changes 100 rows rewrites a large part of the table.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Without deletion vectors, MERGE **copies every unchanged row** of each file it touches. With the keys spread over every file, 100 changes touch hundreds of files. **Deletion vectors** and **clustering on the merge key** fix the two halves.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

NUMTARGETROWSUPDATED

100

Rows the business changed

NUMTARGETROWSCOPIED

61,000,000

Unchanged rows written again, 412 files

FIG. 3.08 One MERGE in DESCRIBE HISTORY, in this example. The cost follows the files touched, not the rows changed.

MEASURE FIRST

- 1 Read the metrics of the last MERGE runs.

```
DESCRIBE HISTORY sales_prod.core.customers LIMIT 5;
-- numTargetRowsUpdated, numTargetFilesRemoved, numTargetRowsCopied
```

- 2 Check `delta.enableDeletionVectors` in `SHOW TBLPROPERTIES`.
- 3 In the query profile, compare files read by the target scan with `numFiles` from `DESCRIBE DETAIL`.

CAUSE

Without deletion vectors, one changed row means the whole file is rewritten. The table is laid out by load date, so 100 changed customers sit in 100 different files. The `ON` clause holds only the key, so the scan of the target cannot skip anything.

FIX

Turn on deletion vectors: MERGE marks the old rows and writes only the changed ones. Cluster the target on the merge key so matches fall in few files and the scan prunes. Add known limits to `ON`, such as a date range, when the source only touches recent rows. Random keys such as UUIDs spread each batch over the whole key range and prune little; add a column the changes share, such as a date. On a partitioned table, convert it first (3.01).

```
ALTER TABLE sales_prod.core.customers
SET TBLPROPERTIES ('delta.enableDeletionVectors' = true);
ALTER TABLE sales_prod.core.customers CLUSTER BY (customer_id);
OPTIMIZE sales_prod.core.customers FULL; -- once; plain OPTIMIZE skips old files
```

EXPECTED RESULT

`numTargetRowsCopied` falls close to zero and MERGE time follows the rows changed, not the table size. Deleted rows are removed from files later, by `OPTIMIZE` or Predictive Optimization.

RELATED [3.07](#) Your MERGE works alone and fails when two jobs run it at once

[3.01](#) Every query on a 500 million row table does a full scan [3.05](#) You deleted rows and ran VACUUM

3.09 A source renamed a column. Schema evolution added a new one and left the old one empty.

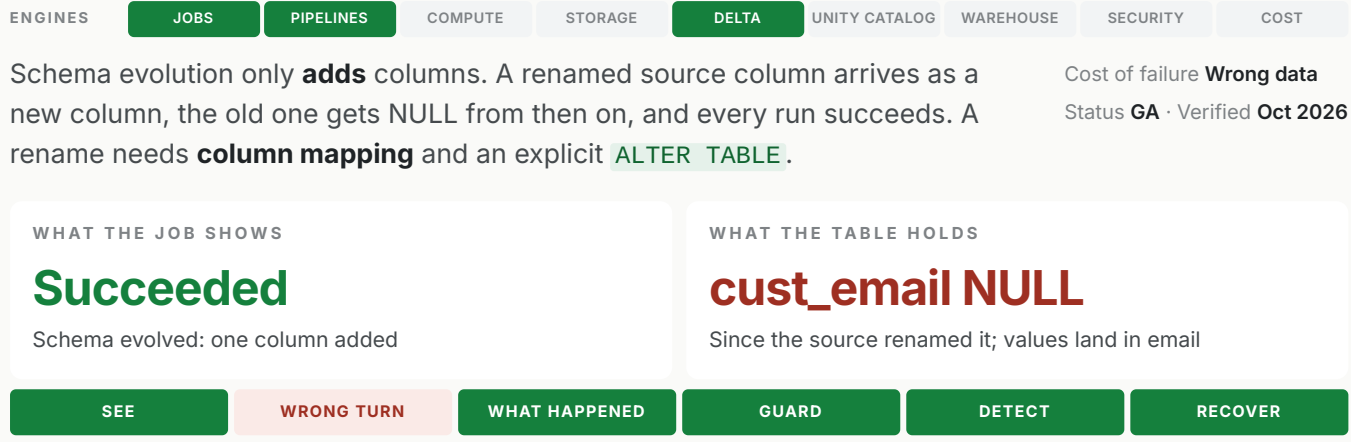


FIG. 3.09 Evolution matches columns by name. A new name is a new column.

WRONG TURN

"With schema evolution on, the table follows whatever the source does."

WHAT HAPPENED

`mergeSchema`, `MERGE WITH SCHEMA EVOLUTION` and, from Runtime 18.1, `INSERT WITH SCHEMA EVOLUTION` add columns and nested fields. They never rename or drop, and change types only by widening, on tables with `delta.enableTypeWidening`. The renamed field matched nothing, so it was added, and the old column gets NULL. Reports still read the old one.

1 GUARD

Turn evolution on per write, not for the session with `spark.databricks.delta.schema.autoMerge.enabled`. Before the write, fail when a target column is missing from the source, leaving out columns your pipeline fills itself.

```
AUDIT = {"load_ts", "ingested_by"} # set by the pipeline
missing = set(spark.table(target).columns) - set(src.columns) - AUDIT
if missing:
    raise ValueError(f"source no longer sends {missing}")
```

2 DETECT

Count by load day.

```
SELECT date(load_ts) AS d,
       count_if(cust_email IS NULL) AS old_null,
       count_if(email IS NOT NULL) AS new_set
FROM sales_prod.core.customers
GROUP BY 1 ORDER BY 1 DESC;
```

3 RECOVER

Pause the writers and streams first, or a write in between adds `email` again. Set `delta.columnMapping.mode` to `name`, copy `email` into `cust_email`, drop `email`, then `RENAME COLUMN cust_email TO email`. Resume the writers last. Renames and drops break change data feed reads across them, and streams need `schemaTrackingLocation`.

RELATED **3.10** A Change Data Feed reader missed changes

2.04 Every Auto Loader batch succeeded

2.05 The same Auto Loader code infers different types in prod

3.10 A Change Data Feed reader was paused for ten days. After the restart, changes are missing.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Change data files follow the table's **VACUUM retention**, 7 days by default. A reader that falls further behind cannot read the gap, and restarting it at the latest version **skips the gap** without an error.

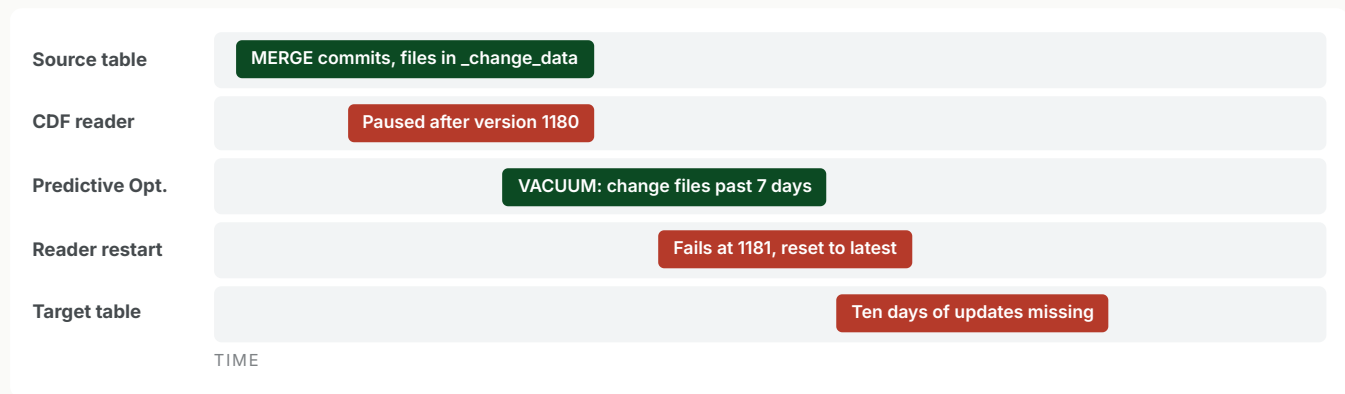
Cost of failure **Data loss**Status **GA** · Verified **Oct 2026**

FIG. 3.10 The source kept its promise for 7 days. The reader assumed it would wait forever.

SEEN IN

The target table, days later: counts drift and deleted customers are still there

CAUSE LIVES IN

The source table's retention and the version the reader resumed from

CAUSE

With legacy change data feed (`delta.enableChangeDataFeed`), change records live in `_change_data` files, which `VACUUM` deletes like data files, or come from the log, which keeps 30 days. A range older than that cannot be read.

FIX

Set `delta.deletedFileRetentionDuration` above the longest outage you accept, and alert when the stored version lags the latest. After a gap, never jump ahead: rebuild the target from a snapshot and resume from that snapshot's version. A streaming reader with a checkpoint does the bookkeeping, but still needs a lag alert. On Runtime 19 and above, Delta tables with row tracking (managed or external) and managed Iceberg v3 tables get automatic change data feed, GA since 1 September 2026, computed at query time. Migrate by unsetting `delta.enableChangeDataFeed`: both cannot be used together, and external Iceberg clients cannot query it. It reads only versions the table keeps, so the alert stays.

CONFIRM IN BOTH ENGINES

```
DESCRIBE HISTORY sales_prod.core.orders
LIMIT 1;
-- latest version vs reader's stored
```

```
SELECT _change_type, count(*)
FROM table_changes(
  'sales_prod.core.orders', 1181, 1260)
GROUP BY 1;
```

Apply `delete`, upsert from `update_postimage` and `insert`, and ignore `update_preimage` unless you keep history.

RELATED [3.04](#) Time travel to last month fails [3.11](#) You restored a table to last week

[2.07](#) Two streaming queries share one checkpoint folder

3.11 You restored a table to last week. Downstream streams broke or kept the bad rows.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

RESTORE does not rewind the log. It adds a new commit that removes and re-adds files with **dataChange** set to true, so streaming readers see it as an **upstream change**, not as an undo.

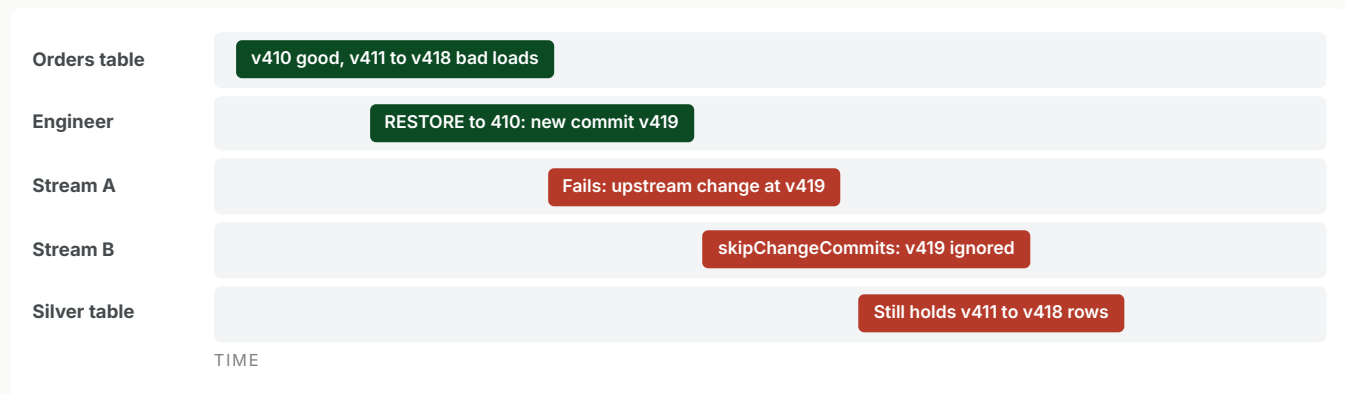
Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

FIG. 3.11 The source looks like last week again. Everything that already read the bad versions does not.

SEEN IN

Streaming jobs and the silver and gold tables they feed

CAUSE LIVES IN

The **RESTORE** commit on the source table

CAUSE

Commit v419 removes the files added since 410 and re-adds files removed since then, for example by **OPTIMIZE**. A Delta stream fails on commits that remove files. With **skipChangeCommits** it skips v419, so the bad rows stay downstream. A stream that processes v419, such as one using the older **ignoreChanges**, reads the re-added files again: duplicates. A new stream without **startingVersion** reads the snapshot once and is fine. **RESTORE** fails once **VACUUM** has removed the old files. It also fails on tables with row filters or column masks, under ABAC and on materialized views unless the running identity is exempt (7.17).

FIX

For a few bad rows, prefer **DELETE** or **MERGE**: readers of the change data feed then get proper **delete** records. For a full restore, plan downstream too: stop readers, restore the source, restore or rebuild each downstream table to the matching point, and start streams on new checkpoints with **startingVersion** 420, the restore commit plus one. Rebuild materialized views from source.

CONFIRM IN BOTH ENGINES

```
DESCRIBE HISTORY sales_prod.core.orders
LIMIT 3;
-- RESTORE: numRemovedFiles,
-- numRestoredFiles
```

```
SELECT count(*) FROM sales_prod.core.orders
VERSION AS OF 410;
SELECT count(*)
FROM sales_prod.silver.orders;
```

RESTORE needs **MODIFY** on the table. Downstream counts must match the source again before you close the incident.

RELATED [3.10](#) A Change Data Feed reader missed changes [3.04](#) Time travel to last month fails

[7.17](#) A row filter or mask blocks your own recovery jobs

3.12 You cloned a table for testing. The clone costs as much as the original.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

CLONE without a keyword is a **deep clone**: it copies every data file. A **shallow clone** copies only metadata, and in Unity Catalog **VACUUM** keeps the source files the clone still reads.

Cost of failure **Cost**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Deep clone	An independent copy that survives VACUUM and drops of the source. Rerunning CREATE OR REPLACE ... DEEP CLONE copies only what changed. Good for archives and moves.	Every test table costs a full copy in storage and compute. History, comments and Unity Catalog tags are not copied. Neither clone works on tables with row filters, column masks or ABAC, or on materialized views, unless the running identity is exempt (7.17).
Shallow clone DEFAULT	Seconds to create, no data copied. Writes to the clone go to its own files. VACUUM on the source, or on a managed clone, keeps files any clone still needs.	Managed to managed (13.3 LTS) and external to external (14.3 LTS) only. No shallow clone of a shallow clone, no OpenSharing. A long-lived clone keeps old source files, so source storage stops shrinking.
Sampled copy with CTAS	A small, filtered or masked subset with no link to production.	Tests need full volume or the real file layout. Row filters and masks are not copied (7.06).

FIG. 3.12 The clone type decides who pays for the files and how long they stay.

DEFAULT
Shallow clones for tests, created per run and dropped at the end. Deep clones only for copies that must outlive the source or leave its region.

SIGNS YOU CHOSE WRONG
Test schemas as large as production. Source storage that stays flat after **VACUUM** because old clones still reference files. Clones nobody owns. A source **DROP** that fails because clones exist: **FORCE** breaks every clone, so list them first.

CHECK IT
The clone result shows what was copied. A shallow clone cannot be replaced, so drop and recreate it to refresh.

```
CREATE TABLE dev_sales.core.orders_test
SHALLOW CLONE sales_prod.core.orders;
-- num_copied_files = 0, copied_files_size = 0

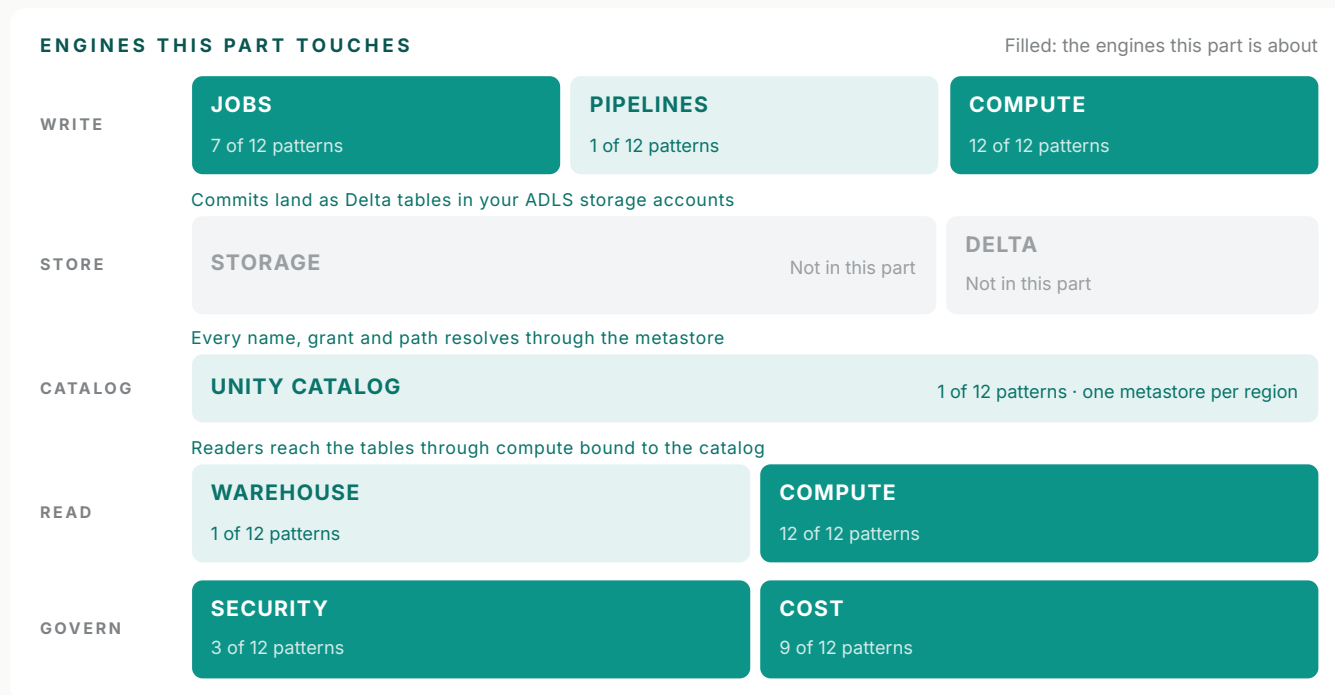
DROP TABLE dev_sales.core.orders_test; -- at the end of the test run
```

RELATED **1.08** Jobs that read one storage account run far slower than the rest **3.04** Time travel to last month fails

7.06 The table has a row filter. A copy made with CTAS has none

04 Compute and Clusters.

What runs the work: job, all-purpose and serverless compute, autoscaling, Spot VMs on Azure, policies, pools, runtimes and access modes. Most compute bills are set by values nobody chose: a missing auto termination, a policy default read as a limit, a pool that never empties.



NO.	PATTERN	JOBS	PIPE	COMPUTE	UC	SEC	COST	PAGE
4.01	A job runs every five minutes and starts a new cluster each time. Startup takes longer than the work.	●	●	●	●	●	●	69
4.02	You run jobs on an all-purpose cluster to skip cold starts. Costs tripled.	●	●	●	●	●	●	70
4.03	Autoscaling added workers and the job got slower. It also costs more than the fixed cluster did.	●	●	●	●	●	●	71
4.04	A streaming job autoscaled to the maximum and stayed there. Batch jobs with the same settings shrink.	●	●	●	●	●	●	72
4.05	Jobs on Spot VMs fail at random around 9am. At night the same jobs never fail.	●	●	●	●	●	●	73
4.06	A data scientist started a 100-node cluster. It ran for a week and nobody noticed.	●	●	●	●	●	●	74
4.07	The cluster policy is set. Users still create clusters that ignore it.	●	●	●	●	●	●	75
4.08	Auto termination is set to 60 minutes. The cluster never stops.	●	●	●	●	●	●	76
4.09	Your pool keeps instances idle. Jobs still wait, and Azure still charges.	●	●	●	●	●	●	77
4.10	You upgraded the runtime. The job still succeeds and returns different numbers.	●	●	●	●	●	●	78

NO.	PATTERN	JOB	PIPE	COMPUTE	UC	SEC	COST	PAGE
4.11	A Spark config set on the cluster is ignored at runtime. On serverless it cannot be set at all.	●	●	●	●	●	●	79
4.12	Code that runs on a dedicated cluster fails on standard access mode. The error names an API you have always used.	●	●	●	●	●	●	80

4.01 A job runs every five minutes and starts a new cluster each time. Startup takes longer than the work.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A classic **job cluster** is created for one run and terminated after it, so every run pays for VM allocation, the runtime image and libraries. For short, frequent jobs use **serverless jobs** with the performance target **PERFORMANCE_OPTIMIZED**.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
New job cluster per run	Full control of VM type, init scripts and JARs. Billed at the jobs compute rate, and nothing runs between runs.	Startup is minutes of VM allocation, image and library installs, every run. At a five-minute interval the next trigger arrives before the last run is done.
Job cluster from a warm pool	Idle pool instances skip VM allocation. A preloaded runtime skips the image download.	Azure bills idle pool VMs: minimum idle ones around the clock, the rest until idle termination. Init scripts and libraries still install on every run (4.09).
Serverless jobs, performance mode DEFAULT	Starts and scales fast, with no idle VMs to pay for. Databricks recommends serverless as the default for notebook, Python and wheel tasks.	Code needs RDDs, init scripts or unsupported Spark properties (4.11, 4.12). Standard mode is cheaper but can take 4 to 6 minutes to start.

FIG. 4.01 The work takes 90 seconds in every option. What changes is how much of each run is spent getting a machine.

DEFAULT

Serverless jobs for short, frequent work, in performance mode when the interval is tight. Classic job clusters for long runs and code that needs them.

SIGNS YOU CHOSE WRONG

Setup takes longer than execution. Runs pile up. Someone proposes an always-on all-purpose cluster (4.02).

CHECK IT

Find tasks that spend longer starting than working. A task with no cluster key runs on serverless.

```
SELECT job_id, task_key,
       avg(setup_duration_seconds) AS setup_s,
       avg(execution_duration_seconds) AS exec_s
FROM system.lakeflow.job_task_run_timeline
WHERE period_start_time >= current_date() - INTERVAL 7 DAYS
GROUP BY ALL HAVING setup_s > exec_s ORDER BY setup_s DESC;
```

```
resources:
  jobs:
    orders_5min:
      schedule: {quartz_cron_expression: "0 0/5 * * * ?", timezone_id: UTC}
      performance_target: PERFORMANCE_OPTIMIZED
      tasks:
        - task_key: load
          notebook_task: {notebook_path: ../src/load_orders.py}
```

RELATED **4.02** You run jobs on an all-purpose cluster to skip cold starts **4.09** Your pool keeps instances idle. Jobs still wait **8.06** Every serverless job starts fast

4.02 You run jobs on an all-purpose cluster to skip cold starts. Costs tripled.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A job task on all-purpose compute is billed at the **all-purpose DBU rate**, which is higher than **jobs compute**. The cluster kept warm for it also bills **VMs and DBUs** while it waits.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (A SHARED SPARK CLUSTER ON PREMISES)

"The cluster is already running and paid for. Jobs that use it are free, and they skip the startup."

IN AZURE DATABRICKS

Compute is billed by SKU, not by machine. A task pointed at an existing all-purpose cluster bills at the all-purpose rate. Between runs the cluster keeps billing Azure VMs and DBUs until auto termination stops it, and work that keeps it busy stops that too (4.08). Databricks does not recommend all-purpose compute for production jobs.

Do this. Find the jobs from the task timeline: `usage_metadata.job_id` is never set for work on all-purpose compute, so the billing table alone cannot name them. Price each cluster by `usage_metadata.cluster_id` and compare rates in `system.billing.list_prices`. Move short, frequent jobs to serverless jobs (4.01) and the rest to job clusters. Remove `existing_cluster_id` from production job definitions, and keep all-purpose clusters for interactive work.

```
WITH ap AS (
  SELECT workspace_id, cluster_id, cluster_name FROM system.compute.clusters
  WHERE cluster_source IN ('UI', 'API')
  QUALIFY row_number() OVER (PARTITION BY workspace_id, cluster_id
                             ORDER BY change_time DESC) = 1),
runs AS (
  SELECT DISTINCT workspace_id, job_id, cluster_id FROM (
    SELECT workspace_id, job_id, explode(compute_ids) AS cluster_id
    FROM system.lakeflow.job_task_run_timeline
    WHERE period_start_time >= current_date() - INTERVAL 30 DAYS))
SELECT r.job_id, ap.cluster_name, sum(u.usage_quantity) AS cluster_dbus
FROM runs r JOIN ap USING (workspace_id, cluster_id)
JOIN system.billing.usage u ON u.workspace_id = r.workspace_id
  AND u.usage_metadata.cluster_id = r.cluster_id
WHERE u.usage_date >= current_date() - INTERVAL 30 DAYS
GROUP BY ALL ORDER BY cluster_dbus DESC;
-- cluster_dbus covers everything on that cluster, interactive work included
```

RELATED **4.01** A job runs every five minutes and starts a new cluster each time **4.08** Auto termination is set to 60 minutes

C.01 All-purpose compute, jobs compute, serverless compute and SQL warehouses

4.03 Autoscaling added workers and the job got slower. It also costs more than the fixed cluster did.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Autoscaling adds workers when **tasks queue**. New VMs join minutes later, often after the stage that asked for them. A **skewed** stage keeps one task busy while the new workers sit idle and billed.

Cost of failure **Slow queries**Status **GA** · Verified **Oct 2026**

FIXED, 8 WORKERS

45 minutes

Every worker busy from the first stage

AUTOSCALING, 2 TO 20 WORKERS

65 minutes

Starts small, grows late, 18 idle on a skewed stage

FIG. 4.03 In this example, the same job on the same data. Autoscaling starts at the minimum and reacts after the queue forms.

MEASURE FIRST

- 1 Count nodes and CPU per minute during a run.

```
SELECT date_trunc('MINUTE', start_time) AS minute,
       count(DISTINCT instance_id) AS workers,
       round(avg(cpu_user_percent + cpu_system_percent)) AS cpu
FROM system.compute.node_timeline
WHERE cluster_id = '0930-061500-k2x7pq4a' AND NOT driver
GROUP BY 1 ORDER BY 1;
```

- 2 Line up **UPSIZE_COMPLETED** cluster events with the Spark UI stage timeline.
- 3 In the slow stage, compare the longest task with the median. A large gap is skew.

CAUSE

The cluster starts at the minimum, so early stages run on 2 workers. Scale-up waits for queued tasks, then for Azure VMs. A stage with a few huge partitions queues nothing, or scales and leaves the new workers idle.

Optimized autoscaling on Premium (the Standard tier retired on October 1, 2026) scales job compute down after 40 seconds of low use and checks shuffle files first.

FIX

Give short, predictable batch jobs a fixed size. Fix skew in the code, not with more workers. Keep autoscaling for long jobs whose stages differ a lot in width, or move the job to serverless.

```
new_cluster:
  node_type_id: Standard_E8ds_v5
  num_workers: 8    # fixed: short job, steady volume
  # spark_version: the 18 LTS key from databricks clusters spark-versions
```

EXPECTED RESULT

Run time back at the fixed-cluster figure, a flat worker count, and fewer DBUs per run.

Go deeper: PySpark Handbook 4.0, skew and adaptive query execution.

RELATED **4.04** A streaming job autoscaled to the maximum and stayed there

4.01 A job runs every five minutes and starts a new cluster each time **4.06** A data scientist started a 100-node cluster

4.04 A streaming job autoscaled to the maximum and stayed there. Batch jobs with the same settings shrink.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Classic compute autoscaling has **limits scaling down** Structured Streaming: it removes nodes that are idle, and a stream rarely leaves one idle. Databricks recommends **Lakeflow Spark Declarative Pipelines** with **enhanced autoscaling** for streaming.

Cost of failure **Cost**
Status **GA** · Verified **Oct 2026**

	BATCH JOB, AUTOSCALING 2 TO 16	STREAM, SAME CLUSTER SETTINGS
Shape of the work	Stages finish and the job ends	Micro-batches every few seconds, forever
Scale-up	When tasks queue in a wide stage	When a burst of input queues tasks
Scale-down signal	Workers fall idle between stages	Every worker runs a task in each batch
After the burst	Back to 2 workers, then terminated	16 workers, billed all day

FIG. 4.04 Batch work drains, so idle workers appear. A stream spreads small tasks over every worker and never drains.

CAUSE

Cluster autoscaling was built for batch stages. Each micro-batch spreads its partitions over all executors, so no node stays idle long enough to be removed. Enhanced autoscaling in a pipeline watches task slot use and the task queue, and shuts down under-used nodes without failing tasks. On serverless pipelines it is always on.

FIX

Move the stream into a pipeline with enhanced autoscaling, or run it serverless. A stream that stays a classic job gets a fixed size, set for its steady input rate. Serverless jobs accept only bounded triggers such as `AvailableNow`, so on serverless jobs the stream becomes a scheduled catch-up run. A continuous pipeline still restarts for runtime upgrades and infrastructure maintenance: set a maintenance window (day, start hour, time zone; September 2026), and note that a serverless cluster's maximum lifetime can restart it outside the window.

```
{
  "clusters": [
    {
      "label": "default",
      "autoscale": {
        "min_workers": 2,
        "max_workers": 8,
        "mode": "ENHANCED"
      }
    }
  ]
}
```

PREVENT

Give streaming job clusters `num_workers`, not `autoscale`. Review any cluster whose worker count in `system.compute.node_timeline` sits at its `max_autoscale_workers` for hours. A stream that needs more workers at peak belongs in a pipeline.

RELATED

4.03 Autoscaling added workers and the job got slower

5.07 You switched a pipeline to continuous

2.10 Your trigger fires every 60 seconds

4.05 Jobs on Spot VMs fail at random around 9am. At night the same jobs never fail.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Azure evicts Spot VMs when it needs the **capacity**, and demand peaks in business hours. Keep the driver and a base of workers **on demand** with **first_on_demand**, and run the rest as **Spot with fallback**.

Cost of failure **Outage**Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
All on demand ON_DEMAND_AZURE	No evictions. Run time is predictable, so SLAs hold. A capacity reservation group also guarantees the VMs exist.	Highest VM price. Azure bills a reservation used or not, and it cannot be used with Spot.
Spot workers, no fallback SPOT_AZURE	Lowest VM price for dev and batch that can rerun overnight.	An evicted worker comes back only if Spot capacity does. Its shuffle files go with it, so stages rerun. An All Spot pool puts the driver on Spot too.
On-demand base, Spot with fallback DEFAULT	Driver and a few workers on demand. A running Spot worker that is evicted is replaced by on demand when Spot is short.	Spot VMs that fail during setup are not replaced, so the cluster can start short. Savings shrink on days the fallback runs.

FIG. 4.05 The price setting only decides whether price can evict you. Capacity decides the rest.

DEFAULT

Production: **first_on_demand** covering the driver and enough workers to finish at reduced speed, the rest **SPOT_WITH_FALLBACK_AZURE**, and **spot_bid_max_price** at -1 so price never evicts.

Streams and tight SLAs: all on demand.

CHECK IT

Set it in the job cluster or fix it in a policy, then confirm evictions in the cluster events.

```
azure_attributes:
  availability: SPOT_WITH_FALLBACK_AZURE
  first_on_demand: 3          # driver plus two workers on demand
  spot_bid_max_price: -1      # evict on capacity only, never on price
```

```
databricks clusters events --json '{"cluster_id": "0930-061500-k2x7pq4a",
  "event_types": ["NODES_LOST", "ADD_NODES_FAILED"]}'
```

RELATED [4.03](#) Autoscaling added workers and the job got slower [4.09](#) Your pool keeps instances idle. Jobs still wait

[2.06](#) You deleted the checkpoint to fix a stuck stream

4.06 A data scientist started a 100-node cluster. It ran for a week and nobody noticed.

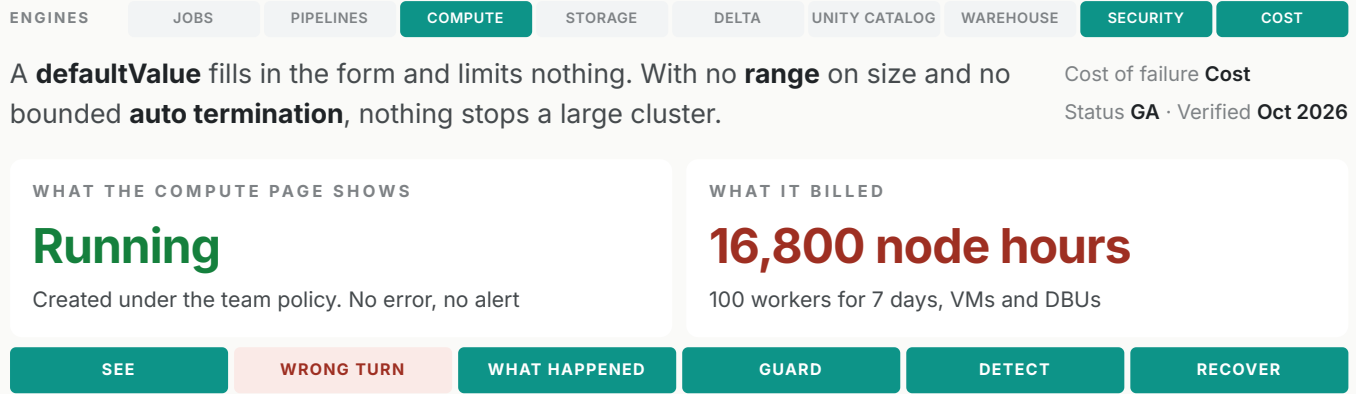


FIG. 4.06 A policy only limits what it marks as a limit. Everything else is a suggestion.

WRONG TURN

"Everyone creates clusters through our policy, so size and idle time are capped."

WHAT HAPPENED

The policy had `"autoscale.max_workers": {"type": "unlimited", "defaultValue": 100}`: a default pre-fills the form and any value is accepted. Auto termination was open, and 0 means never.

1 GUARD

Cap size and DBUs per hour with `range`. Give auto termination a `minValue`, since a range with only `maxValue` accepts 0. Then **Enforce all**.

```
{
  "autoscale.max_workers": {
    "type": "range",
    "maxValue": 10,
    "defaultValue": 4,
    "num_workers": {
      "type": "range",
      "maxValue": 10,
      "isOptional": true,
      "dbus_per_hour": {
        "type": "range",
        "maxValue": 40,
        "autotermination_minutes": {
          "type": "range",
          "minValue": 10,
          "maxValue": 60,
          "defaultValue": 30
        }
      }
    }
  }
}
```

2 DETECT

A job every 30 minutes pages on-call: over 20 nodes now, or over 8 running hours in the last day.

```
SELECT cluster_id,
       count(DISTINCT date_trunc('MINUTE', start_time)) / 60 AS hrs_up,
       count(DISTINCT CASE WHEN start_time > now() - INTERVAL 10 MINUTES
                           THEN instance_id END) AS nodes_now
FROM system.compute.node_timeline
WHERE start_time > now() - INTERVAL 24 HOURS
      AND cluster_id IN (SELECT cluster_id FROM system.compute.clusters
                        WHERE cluster_source IN ('UI', 'API'))
GROUP BY ALL HAVING nodes_now > 20 OR (nodes_now > 0 AND hrs_up > 8);
```

3 RECOVER

Terminate it, tell the owner, move it under the bounded policy (4.07). Budget alerts (8.04) lag up to 24 hours.

RELATED [4.07](#) The cluster policy is set. Users still create clusters that ignore it [4.08](#) Auto termination is set to 60 minutes [8.04](#) One team used 70 percent of the spend

4.07 The cluster policy is set. Users still create clusters that ignore it.

ENGINES

JOBSPipelinesCOMPUTEStorageDeltaUnity CatalogWarehouseSecurityCOST

A policy governs only the people who must use it. **Allow unrestricted cluster creation** skips policies, **Personal Compute** is open to all users by default, and a cluster on a **pool** takes the pool's VM type.

Cost of failure **Cost**
Status **GA** · Verified **Oct 2026**

	WHAT THE POLICY AUTHOR EXPECTED	WHAT USERS ACTUALLY HAVE
Entitlement	Users hold <code>CAN_USE</code> on one team policy	A group holds Allow unrestricted cluster creation, so Unrestricted is in the list
Built-in policies	Only the team policy	Personal Compute: all users by default, 72-hour auto termination, no limit on count
VM type	<code>node_type_id</code> allowlist	Cluster attached to a pool, which uses its own instance type; users hold Allow pool creation
Existing clusters	New rules apply at once	Old settings stay until the cluster is enforced, marked Out of compliance

FIG. 4.07 The policy is correct. Four other routes to compute were left open.

CAUSE

Policies work through permissions. Workspace admins always keep the unrestricted entitlement. Anyone else given it, often through a broad group, can pick the Unrestricted policy. Pools carry their own instance type, so a VM allowlist in the policy never sees them unless the policy also controls `instance_pool_id`.

FIX

Remove Allow unrestricted cluster creation and Allow pool creation from user groups. Restrict Personal Compute in the account console, or grant it to named groups only. Give each group one policy that pins pools as well as VM types, and give automation identities `CAN_USE` on a job policy instead of the entitlement.

```
{
  "instance_pool_id": {
    "type": "forbidden",
    "hidden": true
  },
  "node_type_id": {
    "type": "allowlist",
    "values": ["Standard_E8ds_v5", "Standard_E16ds_v5"]
  },
  "cluster_type": {
    "type": "fixed",
    "value": "all-purpose"
  }
}
```

PREVENT

After each change, sign in as a non-admin test user and open the create compute page: only the governed policies should appear. Review entitlements on every group each quarter, and run **Enforce all** whenever a policy changes. A policy that someone can step around controls nothing.

RELATED **4.06** A data scientist started a 100-node cluster **4.09** Your pool keeps instances idle. Jobs still wait **7.02** The grants look right. Users still get access denied

4.08 Auto termination is set to 60 minutes. The cluster never stops.

- ENGINES
- JOBS
- PIPELINES
- COMPUTE
- STORAGE
- DELTA
- UNITY CATALOG
- WAREHOUSE
- SECURITY
- COST

A cluster is idle only when **every command has finished**: Spark jobs, **Structured Streaming**, **JDBC calls** and web terminal sessions. One of them running somewhere keeps it alive.

Cost of failure

Cost

Status

GA

Verified

Oct 2026

YOU'D EXPECT (A VM SHUTDOWN SCHEDULE OR A LAPTOP'S SLEEP TIMER)

"Idle means nobody is typing. When I close the notebook and go home, the timer starts."

IN AZURE DATABRICKS

The timer measures time since the last command on the cluster, from anyone. A stream left running in a notebook, a BI tool that refreshes over JDBC, or a job scheduled on the cluster every 30 minutes all count as activity. The inactivity period runs from 10 to 10,000 minutes, and 0 turns it off.

Do this. Open the cluster's Spark UI and look for an active streaming query or a job that keeps arriving. Stop the stream or move it to a job (4.04), point BI tools at a SQL warehouse, and move scheduled work to job compute (4.02). Then bound `autotermination_minutes` in every policy with a `minValue` (4.06) and check `auto_termination_minutes` in `system.compute.clusters` for clusters with 0 or nothing set.

RELATED

4.06

A data scientist started a 100-node cluster

4.02

You run jobs on an all-purpose cluster to skip cold starts

6.02

Auto stop is 10 minutes

COMPARE
STORAGE
INGESTION
DELTA
COMPUTE
JOBS
SQL
SECURITY
PLATFORM
CI/CD
AI
SCENARIOS

4.09 Your pool keeps instances idle. Jobs still wait, and Azure still charges.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Idle pool instances cost **no DBUs**, but Azure bills the **VMs**. A pool saves only VM allocation; the runtime, init scripts and libraries still load unless the runtime is **preloaded**.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (FROM THE WORD POOL)

"Idle instances are free until a cluster takes them, and a cluster on the pool is ready at once."

IN AN AZURE DATABRICKS POOL

Minimum idle instances never terminate, and each is a running VM on the Azure bill. A cluster that picks a runtime other than the preloaded one downloads it. Init scripts and libraries install on every start. When idle plus used instances reach the maximum capacity, the request fails with `INSTANCE_POOL_MAX_CAPACITY_FAILURE`. A pool shared by teams has no quota per team. Databricks recommends serverless over pools where the workload supports it.

Do this. Preload the runtime your jobs use. Set minimum idle to 0 and use idle instance auto termination to keep VMs between runs, for example 20 minutes for jobs an hour apart. Give the driver its own on-demand pool, give each team its own pool, and put chargeback tags on the pool: pool tags reach the Azure VMs, cluster tags do not. For guaranteed capacity, a pool can launch against an Azure capacity reservation group (`capacity_reservation_group` in `azure_attributes`): on-demand only, VNet-injected workspaces only, billed by Azure used or not, and an edit that omits the field clears it.

RELATED **4.01** A job runs every five minutes and starts a new cluster each time

4.07 The cluster policy is set. Users still create clusters that ignore it **8.07** The Databricks usage report says one figure

4.10 You upgraded the runtime. The job still succeeds and returns different numbers.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

From Databricks Runtime 17.0, Apache Spark 4 runs with **ANSI SQL mode on by default**, and 18 LTS keeps it. Some expressions now fail, and a few return **different values** without any error. Test each **LTS** jump on real data.

Cost of failure **Wrong data**
Status **GA** · Verified **Oct 2026**

	15.4 LTS	18 LTS
Spark, Scala, end of support	3.5, 2.12, August 19, 2027	4.1, 2.13, June 10, 2029
size(NULL)	-1	NULL, so a filter on size < 1 drops rows
CAST('n/a' AS INT)	NULL	Error: the value is malformed
input_file_name()	Supported	Not supported (since 17.3 LTS): <code>_metadata.file_path</code> is the full path
NULL struct in MERGE, UPDATE	Expanded to a struct of NULL fields	Stays NULL, so <code>s IS NULL</code> matches more rows
Scala JAR built for 2.12	Loads	Fails to load

FIG. 4.10 Errors are the easy part. A changed NULL in a filter fails nothing and shrinks a total.

CAUSE

A runtime bundles Spark, Delta, Python, Scala and libraries. Spark 4 turned ANSI mode on, which makes bad casts and overflows fail and changes `size` on NULL. 18 LTS also runs on JDK 21, blocks time travel past the retention, ignores `VACUUM RETAIN` except 0 hours (3.04) and writes partition columns into new Parquet files.

FIX

Run the job on both runtimes into two schemas and compare both ways. Fix differences in code, for example `try_cast` where NULL was intended. Set `spark.sql.ansi.enabled` to false only as a short bridge.

```
SELECT count(*) FROM (
  SELECT * FROM sales_dev.rt_154.daily_sales
  EXCEPT ALL
  SELECT * FROM sales_dev.rt_18.daily_sales);
-- swap the two tables and run again: 0 and 0 means the same rows
```

PREVENT

Use LTS runtimes only, and the same one in dev, test and prod, set once in a Declarative Automation Bundles target, and pinned in the policy with a `spark_version` regex. Move to the next LTS well before end of support, with the comparison run as the gate.

RELATED

4.11 A Spark config set on the cluster is ignored at runtime

9.01 The bundle deployed to production

5.15 A notebook works when you run it

4.11 A Spark config set on the cluster is ignored at runtime. On serverless it cannot be set at all.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A property set in the **session** overrides the compute setting, and Spark accepts any key, typos included. **Serverless** has no compute-level Spark config and accepts only **six properties** in the session.

Cost of failure **Rework**
Status **GA** · Verified **Oct 2026**

	CLASSIC COMPUTE	SERVERLESS NOTEBOOKS AND JOBS
Where to set it	Compute Spark config, or <code>spark.conf.set</code> in the session	Only in the session, or <code>SET</code> in SQL
Which properties	Almost any; unknown keys are stored without a warning	Execution timeout, time parser policy, session time zone, shuffle partitions, ANSI mode, max partition bytes
Job moved from a job cluster	<code>spark_conf</code> in the cluster spec applies	No cluster spec, so the settings are gone
Any other property	Applied	The call fails

FIG. 4.11 The same setting has a different home, or no home, depending on the compute.

CAUSE

Compute-level properties apply to every notebook and job on the compute. A notebook, a shared library or an earlier cell that sets the same key wins for that session. Properties such as executor memory are read when the compute starts, so changing them later does nothing to the running executors. On standard access mode with Databricks Runtime 19 and above, JVM options, extra classpaths, `spark.jars` and isolation settings are refused when the cluster is created.

FIX

Read the value inside the job, not in your notebook. Set session properties in code at the top of the entry point, so they move with the job to any compute.

```
for k in ["spark.sql.shuffle.partitions", "spark.sql.ansi.enabled",
        "spark.sql.session.timeZone"]:
    print(k, spark.conf.get(k, "not set"))

spark.conf.set("spark.sql.session.timeZone", "Europe/London") # works on serverless
```

PREVENT

Keep a short list of Spark properties each job needs and why, in the repository with the job. Before a move to serverless, check every cluster `spark_conf` key against the six supported ones. Most tuning keys exist because classic compute needed them; serverless picks shuffle partitions itself.

RELATED

4.12 Code that runs on a dedicated cluster fails on standard access mode

4.10 You upgraded the runtime. The job returns different numbers

1.06 Your notebook reads abfss:// paths on the old cluster

4.12 Code that runs on a dedicated cluster fails on standard access mode. The error names an API you have always used.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Standard access mode runs many users' code on one cluster and isolates each of them. User code reaches Spark through **Spark Connect**, so **RDD APIs** are gone, and init scripts and JARs need the Unity Catalog **allowlist**.

Cost of failure **Rework**
Status **GA** · Verified **Oct 2026**

	DEDICATED ACCESS MODE	STANDARD ACCESS MODE
Who runs code	One user or one group	Many users, isolated from each other
RDD APIs, <code>sc.parallelize</code> , <code>df.rdd</code>	Work	Not supported
Init scripts, JARs, Maven	Any	Only allowlisted paths and coordinates
ML runtime, GPU, Hive UDFs	Supported	Not supported; Unity Catalog UDFs instead
Local files, DBFS, <code>spark-submit</code>	Full access	Low-privilege user; JAR task instead of <code>spark-submit</code>

FIG. 4.12 Sharing a cluster safely costs the APIs that reach past Spark into the JVM or the machine.

CAUSE

On standard access mode user code is separated from the Spark driver and runs as a low-privilege user. APIs that hand out the JVM context or the local machine would break that isolation, so they are not offered. Spark Connect also resolves names at execution time, so some errors appear later than they used to.

FIX

Rewrite RDD code as DataFrame code. Put init scripts and JARs in a volume and allowlist the folder; this needs `MANAGE ALLOWLIST` on the metastore. Keep dedicated access mode for ML runtime, GPU and code that truly needs the JVM.

```
df = spark.createDataFrame(rows, ["id", "amount"]) # was sc.parallelize(rows).toDF()

databricks artifact-allowlists update INIT_SCRIPT --json '{"artifact_matchers": [
  {"artifact": "/Volumes/ops_prod/compute/scripts/", "match_type": "PREFIX_MATCH"}]}'
```

PREVENT

Before moving clusters to standard access mode, search the code for `sparkContext`, `.rdd`, `sc.` and `/dbfs/`. The update call replaces the whole allowlist, so read it first with `get`. Serverless shares most of these limits, so code that passes here is close to ready for serverless.

- RELATED
- 4.11

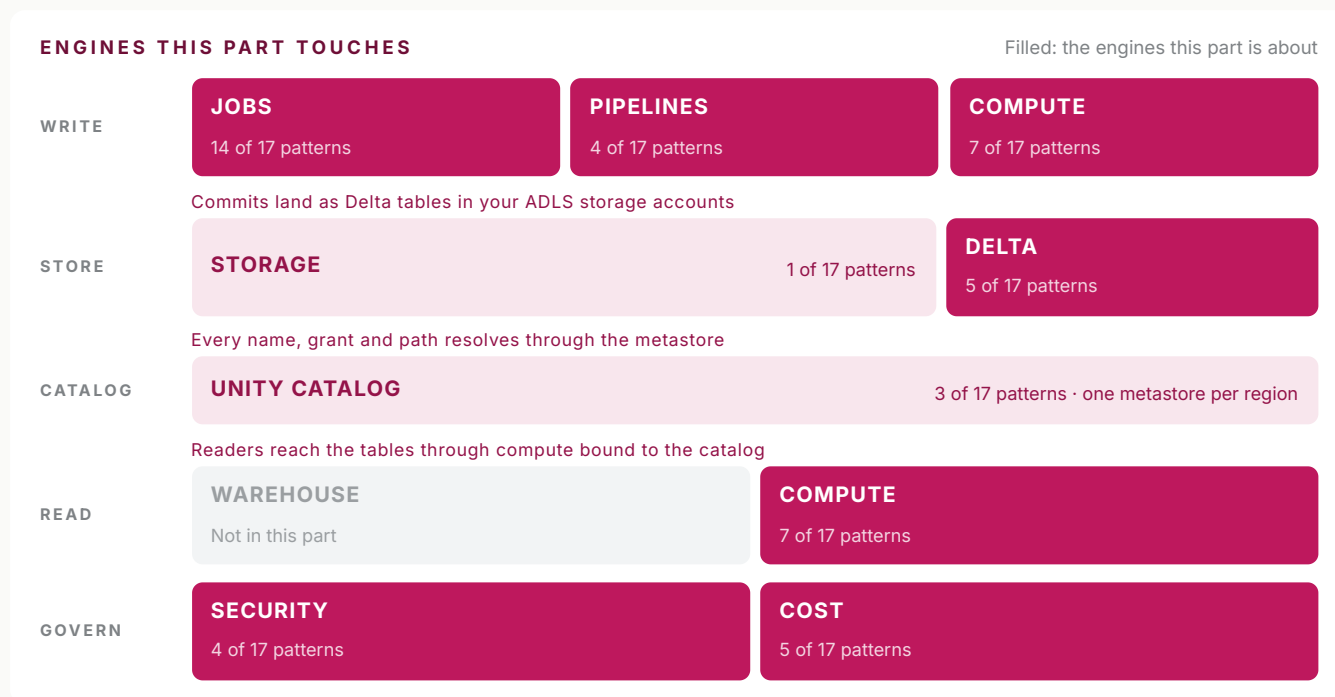
A Spark config set on the cluster is ignored at runtime
- 1.06

Your notebook reads `abfs://` paths on the old cluster
- C.02

Standard access mode, dedicated access mode and the legacy no-isolation mode

05 Jobs and Pipelines.

How work is ordered, started and repeated: Lakeflow Jobs with their tasks, triggers, retries and parameters, and Lakeflow Spark Declarative Pipelines with expectations and refresh modes. A green run reports that the code finished, and says nothing about the rows it wrote.



NO.	PATTERN	JOBS	PIPE	COMPUTE	DELTA	SEC	COST	PAGE
5.01	The job succeeded and every task is green. One task wrote nothing.	●	●	●	●	●	●	83
5.02	A failed task retried three times. Every attempt hit the same error.	●	●	●	●	●	●	84
5.03	The task timeout is 30 minutes. The run went on for two hours.	●	●	●	●	●	●	85
5.04	You cloned a production job to test it. It wrote to production tables.	●	●	●	●	●	●	86
5.05	Tasks share one job cluster. One slow or failing task holds up the rest.	●	●	●	●	●	●	87
5.06	Pipeline expectations fail on thousands of rows. The update still completes.	●	●	●	●	●	●	88
5.07	You switched a pipeline to continuous for fresher data. The bill went up ten times.	●	●	●	●	●	●	89
5.08	The pipeline reprocesses all of history on every update. On test data nobody noticed.	●	●	●	●	●	●	90
5.09	The pipeline runs clean when you start it. Run from the production job, it fails.	●	●	●	●	●	●	91
5.10	A downstream task ran on yesterday's data. Its upstream refresh never ran.	●	●	●	●	●	●	92

NO.	PATTERN	JOBS	PIPE	COMPUTE	DELTA	SEC	COST	PAGE
5.11	Task values work when you test the notebook. In the job, the next task gets the default.	●	●	●	●	●	●	93
5.12	A for each task loops over 1,000 tables. The list does not fit and the loop crawls.	●	●	●	●	●	●	94
5.13	The job is scheduled every 15 minutes. It starts every 20.	●	●	●	●	●	●	95
5.14	One test file started one run. In production, files landing over 20 minutes start run after run.	●	●	●	●	●	●	96
5.15	A notebook works when you run it. As a job task it fails.	●	●	●	●	●	●	97
5.16	An auditor asks for a job run from four months ago. The Runs tab stops at 60 days.	●	●	●	●	●	●	98
5.17	Every table has a hand-built freshness check. A schema nobody wrote checks for went stale for a week.	●	●	●	●	●	●	99

5.01 The job succeeded and every task is green. One task wrote nothing.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A task **succeeds** when its code finishes without an uncaught exception. Writing zero rows raises nothing, so the run stays green until you add an **output check** that fails the task.

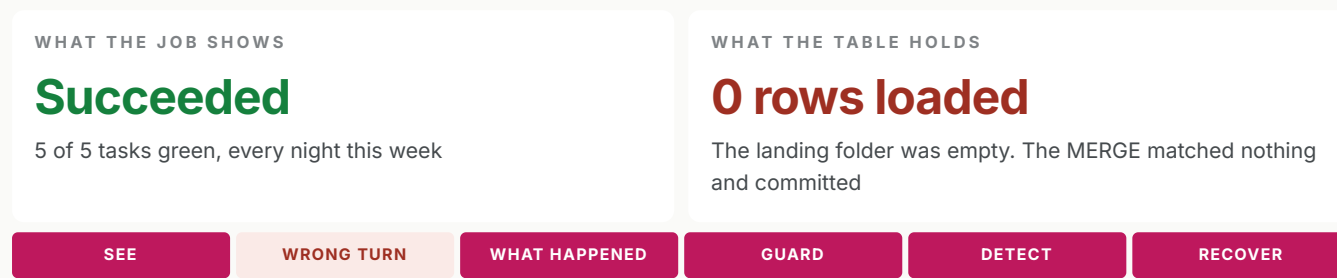
Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

FIG. 5.01 The task status describes the process. Only a check you write describes the output.

WRONG TURN

"A green run means the tables were loaded."

WHAT HAPPENED

An empty source, a filter that matches nothing, a join against an empty lookup, or an exception caught and printed in a **try** block all end the same way: the notebook finishes and the task succeeds. The error surfaces later in the task or report that reads the table, so it points at the wrong place.

1 GUARD

Count the stored result, not the DataFrame: `df.count()` followed by `df.write` runs the query twice, and the two runs can read different data. Land the result once, check it against a floor passed as a parameter, then load. Publish the count as a task value.

```
df.write.mode("overwrite").saveAsTable("sales_prod.stg.orders_load")
stg = spark.table("sales_prod.stg.orders_load")
rows = stg.count()
if rows < int(dbutils.widgets.get("min_rows")):
    raise ValueError("orders: fewer rows than min_rows, stopping the load")
dbutils.jobs.taskValues.set(key="rows_written", value=rows)
stg.write.mode("append").saveAsTable("sales_prod.core.orders")
```

2 DETECT

Each Delta commit records what it wrote. Look for loads that wrote nothing.

```
DESCRIBE HISTORY sales_prod.core.orders
LIMIT 10;
-- operationMetrics: numOutputRows,
-- numTargetRowsInserted for MERGE
```

3 RECOVER

Fix the source, then load the missing day with **Run now with different parameters**. Re-raise errors your code catches. Send failed checks to the same destination as failed runs, so a data failure pages the same person.

RELATED [5.06](#) Pipeline expectations fail on thousands of rows [5.10](#) A downstream task ran on yesterday's data

[9.07](#) A job failed. You heard about it from a business user

5.02 A failed task retried three times. Every attempt hit the same error.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A retry starts the task again **from the beginning**, with the same code, settings and network path. It helps with transient errors only. For anything else, fix the cause and **repair the run**.

Cost of failure **Outage**Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (RETRY POLICIES IN SCHEDULERS AND ADF)

"Three retries will get it through. Whatever went wrong will have cleared by the third attempt, and if not, the whole job reruns."

IN LAKEFLOW JOBS

A task retry waits for the retry interval, counted from the start of the failed attempt, then runs the task again. A firewall rule, a missing grant or a wrong path fails every attempt. If you set a timeout, it applies to each retry. Most configurations do not retry by default, but serverless jobs add retries through auto-optimization unless you clear it in the retry policy. A **Repair run** reruns only the failed or canceled tasks and their dependents, with the job's current settings, and accepts new parameter values. Successful tasks are not rerun.

Do this. Read the first attempt's error before anything else. Give retries, with an interval, only to tasks that call services that fail briefly; tasks that fail on configuration get none. After the fix, repair the run instead of starting it again. A repair starts each task from the beginning, so a task that wrote part of its output must be safe to rerun: **MERGE** on keys, not a blind append. Clear serverless auto-optimization on tasks that must run at most once.

RELATED **5.03** The task timeout is 30 minutes **5.01** The job succeeded and every task is green

9.07 A job failed. You heard about it from a business user

5.03 The task timeout is 30 minutes. The run went on for two hours.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A task timeout applies to **each attempt**, not to the task as a whole. With three retries and **retry on timeout**, a 30 minute limit allows four attempts. Only the **job timeout** caps the whole run.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

	TESTED ONCE, NO RETRIES	PRODUCTION: 3 RETRIES, RETRY ON TIMEOUT
Timeout clock	30 minutes for the one attempt	Starts again with every attempt
Attempts allowed	1	4
Longest task	30 minutes, then Timed Out	Four times 30 minutes, plus retry intervals
Job timeout	0, which means none	0, so nothing stops the run

FIG. 5.03 The same task timeout, with and without retries.

CAUSE

Under **Metric thresholds**, a task's **Timeout** is the maximum time for one run of the task, and the docs state that with retries it applies to each retry. `retry_on_timeout` is false by default; once it is on, a timed-out attempt starts again. The **Warning** field only raises an event. The job's own `timeout_seconds` defaults to 0, no limit.

FIX

Cap the run at the job level, keep task timeouts per attempt, and do not retry tasks that time out for a real reason.

```
resources:
  jobs:
    nightly_sales:
      timeout_seconds: 5400          # whole run, every task and retry
      health:
        rules:
          - metric: RUN_DURATION_SECONDS
            op: GREATER_THAN
            value: 2700              # duration warning at 45 minutes
      email_notifications:           # without a target the rule sends nothing
        on_duration_warning_threshold_exceeded: [oncall-data@contoso.com]
      tasks:
        - task_key: load_orders
          timeout_seconds: 1800      # per attempt
          max_retries: 1
          retry_on_timeout: false
```

PREVENT

Check what is deployed, not what you remember setting: `databricks jobs get` returns `timeout_seconds` for the job and for each task. A duration rule only sets the threshold: nothing is sent until a notification target is set for **Duration Warning**, an email list or a system destination (9.07).

RELATED **5.02** A failed task retried three times **9.08** The duration alert fires after the job finishes

5.04 You cloned a production job to test it. It wrote to production tables.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A clone copies the **job settings**, and the notebooks still name **production catalogs**, so the test writes there. Pass the catalog as a **job parameter** and let each deployment target set it.

Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

WHAT THE TEST JOB SHOWS

Succeeded

Test copy of nightly_sales, run by an engineer at 15:10

WHAT PRODUCTION HOLDS

Test rows

prod_sales.core.orders, written by two jobs that day

SEE

WRONG TURN

WHAT HAPPENED

GUARD

DETECT

RECOVER

FIG. 5.04 Nothing in the copied job says which environment it belongs to.

WRONG TURN

"A copy of the job is a test environment."

WHAT HAPPENED

The copy has the same tasks and points at the same notebooks, which name **prod_sales.core.orders** in full. It runs as its creator, the default **Run as**, who holds **MODIFY** on production for support work. The write succeeds.

1 GUARD

Read the catalog from a widget whose default is dev, and set it from a job parameter. With Declarative Automation Bundles, each target sets the variable.

```
dbutils.widgets.text("catalog", "dev_sales")          # the default is never prod
target = dbutils.widgets.get("catalog") + ".core.orders"
df.write.mode("append").saveAsTable(target)

# databricks.yml: job parameter catalog has default ${var.catalog}
targets:
  prod:
    variables:
      catalog: prod_sales
```

2 DETECT

Every commit records the user and the job that wrote it.

```
DESCRIBE HISTORY prod_sales.core.orders
LIMIT 20;
-- userName and job.jobId per commit
```

3 RECOVER

Delete the test rows by key first, so downstream readers get proper deletes. Use **RESTORE** only with the downstream steps in 3.11; it does not apply to tables with row filters or column masks, or to materialized views. Then bind **prod_sales** read-write to the production workspace only (1.04), and take **MODIFY** on production away from people.

RELATED **1.04** A developer dropped a production table from the dev workspace **3.11** You restored a table to last week

9.01 The bundle deployed to production

5.05 Tasks share one job cluster. One slow or failing task holds up the rest.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A shared job cluster saves start-up time, but parallel tasks share **one driver and one set of workers**. **Serverless compute for jobs** is the default for tasks that support it. On classic compute, share a cluster only between tasks that run one after another.

Cost of failure **Outage**Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
One shared job cluster for every task	One start-up per run. The cluster starts with the first task that uses it and stops after the last one.	Parallel tasks compete for the same executors, so one heavy task slows the light ones. A driver that runs out of memory fails every task on it. Sized for the largest task, it idles through the small ones.
A job cluster per task or per chain	Each task isolated and sized for its own work. A failure stays in its branch.	Every cluster pays its own start-up, so short tasks spend more time starting than working. Libraries still go on each task, never on a shared cluster spec.
Serverless compute for jobs DEFAULT	No cluster to size or start. The Performance optimized mode starts fast; standard mode uses fewer DBUs and starts in about 4 to 6 minutes.	Code needs Spark settings serverless does not accept, init scripts or a chosen VM type, or a source sits on a network that serverless network connectivity cannot reach.

FIG. 5.05 A shared cluster is scoped to one job run. Inside that run, everything on it competes.

DEFAULT

Serverless for new jobs. On classic compute, one job cluster per chain of dependent tasks, and a separate one for each parallel branch that matters on its own.

SIGNS YOU CHOSE WRONG

A task's duration depends on what else ran beside it. One out-of-memory error fails three unrelated tasks. Start-up time is a large share of a short job.

CHECK IT

See which tasks shared compute, and how long each spent starting against working.

```
SELECT task_key, compute_ids, setup_duration_seconds, execution_duration_seconds
FROM system.lakeflow.job_task_run_timeline
WHERE job_id = '123' AND period_start_time > current_date() - INTERVAL 7 DAYS
ORDER BY period_start_time;
```

RELATED **4.01** A job runs every five minutes and starts a new cluster each time

4.02 You run jobs on an all-purpose cluster to skip cold starts **8.06** Every serverless job starts fast

5.06 Pipeline expectations fail on thousands of rows. The update still completes.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

An expectation with no action is **warn**: bad rows are written and counted. Only **expect_or_drop** keeps them out, and only **expect_or_fail** stops the update. A quarantine table is something you build.

Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

WHAT THE PIPELINE SHOWS

Completed

Every dataset green in the graph

WHAT THE TABLE HOLDS

8,412 bad rows

No customer_id, written to orders_silver and counted in the event log

SEE

WRONG TURN

WHAT HAPPENED

GUARD

DETECT

RECOVER

FIG. 5.06 The expectation did its job: it counted. Nobody told it to stop anything.

WRONG TURN

"An expectation is a rule, so a broken rule fails the pipeline."

WHAT HAPPENED

In Lakeflow Spark Declarative Pipelines, `dp.expect` and SQL `EXPECT` without `ON VIOLATION` keep invalid rows and record them. `expect_or_drop` removes them before the write. `expect_or_fail` fails the update and rolls back a table update; in a triggered pipeline, other parallel flows carry on. On classic compute, expectations need the Advanced product edition.

1 GUARD

Choose the action per rule: fail on rows that must never pass, drop rows you can lose, warn only on what you are measuring.

```
from pyspark import pipelines as dp

@dp.table(name="orders_silver")
@dp.expect_or_fail("has_order_id", "order_id IS NOT NULL")
@dp.expect_or_drop("has_customer", "customer_id IS NOT NULL")
def orders_silver():
    return spark.readStream.table("orders_bronze")
```

2 DETECT

Warn and drop rules report passed and failed rows on the Data quality tab and in the event log.

```
SELECT timestamp, details
FROM event_log(TABLE(
    sales_prod.core.orders_silver))
WHERE event_type = 'flow_progress'
ORDER BY timestamp DESC LIMIT 5;
```

3 RECOVER

Build the quarantine yourself: a second table from the same source with the rule inverted, so dropped rows are kept and can be replayed. Rows written under warn stay until you delete them.

RELATED **5.01** The job succeeded and every task is green

5.08 The pipeline reprocesses all of history on every update

5.07 You switched a pipeline to continuous for fresher data. The bill went up ten times.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A **triggered** update processes what is there and stops. A **continuous** pipeline keeps its compute running until someone stops it, so you pay for every hour, busy or idle.

Cost of failure **Cost**
Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (A SCHEDULER)

"Continuous means it checks more often. If data arrives hourly, it does an hour's work and waits cheaply in between."

IN LAKEFLOW SPARK DECLARATIVE PIPELINES

Continuous mode processes data as it arrives, and the compute stays up between arrivals. By default, streaming flows check every five seconds, and complete queries every minute when all sources are Delta. The docs place continuous at updates every 10 seconds to a few minutes, and triggered at every 10 minutes, hourly or daily. Serverless pipelines add a standard performance mode that uses fewer DBUs; a continuous pipeline reaches it only when run from a continuous job.

Do this. If the freshness target is 10 minutes or more, keep the pipeline triggered and schedule it from a job. If it must be continuous, run it from a continuous job with the pipeline itself left on triggered, and clear **Performance optimized** where the start-up delay is acceptable. Set **pipelines.trigger.interval** on the slow-changing tables, so they are not recomputed every minute.

RELATED

5.08 The pipeline reprocesses all of history on every update

8.02 You moved to serverless to save money

8.06 Every serverless job starts fast

5.08 The pipeline reprocesses all of history on every update. On test data nobody noticed.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A dataset defined by a **batch read** is a materialized view. It refreshes incrementally only on **serverless** and only when its query allows it; otherwise it is recomputed in full. Append-only sources belong in **streaming tables**.

Cost of failure **Cost**
Status **GA** · Verified **Oct 2026**

	STREAMING TABLE	MATERIALIZED VIEW
Defined by	<code>spark.readStream</code> , or <code>STREAM</code> in SQL	<code>spark.read</code> , or a plain <code>SELECT</code>
Update reads	New rows only	Changed data, when incremental refresh applies
On classic compute	New rows only	Always a full recompute
SUM over DOUBLE	New rows only	Full recompute, even on serverless

FIG. 5.08 On a small test set a full recompute takes seconds. On five years of orders it takes the night.

CAUSE

In Lakeflow Spark Declarative Pipelines, incremental refresh needs serverless; classic compute always recomputes. On serverless, non-deterministic functions other than time functions in `WHERE`, and `SUM` or `AVG` over `FLOAT` or `DOUBLE`, force a full recompute.

FIX

Read append-only sources as streams (`spark.readStream`, Auto Loader). For views that must stay batch, use serverless, cast money to `DECIMAL`, and enable deletion vectors and row tracking on sources; with row tracking, Runtime 19 and above needs no change data feed property, older readers need `delta.enableChangeDataFeed`. Ask the engine before you deploy, then check each refresh.

```
EXPLAIN CREATE MATERIALIZED VIEW sales_prod.core.daily_sales AS
SELECT order_date, sum(CAST(amount AS DECIMAL(18,2))) AS amount
FROM sales_prod.core.orders GROUP BY order_date;
-- Incremental Update Eligibility: can it refresh incrementally?

SELECT timestamp, message FROM event_log(TABLE(sales_prod.core.daily_sales))
WHERE event_type = 'planning_information' ORDER BY timestamp DESC;
-- FULL_RECOMPUTE: not incremental. ROW_BASED, APPEND_ONLY: incremental
```

PREVENT

`EXPLAIN` shows eligibility, not what the default `AUTO` cost model will pick. Where a full recompute would be a cost incident, the owner can add `REFRESH POLICY INCREMENTAL STRICT` (Beta): a refresh that cannot run incrementally fails and the view stays stale, so route the failure to a person (9.07). `INCREMENTAL` falls back to full. A full refresh also clears streaming checkpoints: set `pipelines.reset.allowed` to false where the source keeps a short history.

RELATED **5.07** You switched a pipeline to continuous **6.05** A materialized view stopped refreshing

5.09 The pipeline runs clean when you start it. Run from the production job, it fails.

ENGINES **JOBS** PIPELINES COMPUTE STORAGE DELTA **UNITY CATALOG** WAREHOUSE SECURITY COST

Your copy runs as **you**, on compute kept warm between updates, with retries off. The production pipeline runs as its **Run as** identity, on fresh compute, with retries on. The first difference to bite is usually a **missing grant**.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	YOUR DEV PIPELINE, STARTED BY HAND	PROD PIPELINE, RUN FROM A JOB
Runs as	You, the creator, by default	Its Run as identity, a service principal
Compute	Reused between updates, kept for 2 hours by default	New for each update, shut down when it ends
Retries	Off, so errors show at once	On for recoverable errors
Grants used	Yours, including READ VOLUME on landing	The service principal's: none on landing
Outcome	Completed	Fails on the first read, after retries

FIG. 5.09 The code is the same. Who runs it, and how, is not.

CAUSE

In Lakeflow Spark Declarative Pipelines, the old development and production modes now depend on how an update starts. From the editor or **Run now**, it reuses compute and skips retries. From a job or the API, it restarts compute on recoverable errors, retries, and shuts down when done. The identity comes from the pipeline's **Run as** setting, and in Unity Catalog pipelines table ownership follows it. That identity needs **USE CATALOG**, **USE SCHEMA** and **CREATE TABLE** on the target, **MODIFY** on existing tables, read access to every source, and read on the source code.

FIX

Grant to an account group that holds the production service principal, never to the engineer who built the pipeline.

```
GRANT USE CATALOG ON CATALOG prod_sales TO `pipelines-prod`;
GRANT USE SCHEMA, CREATE TABLE ON SCHEMA prod_sales.core TO `pipelines-prod`;
GRANT USE SCHEMA ON SCHEMA prod_sales.raw TO `pipelines-prod`;
GRANT READ VOLUME ON VOLUME prod_sales.raw.landing TO `pipelines-prod`;
```

PREVENT

Before go-live, start one update of the production pipeline from its job, not from the editor, and read its event log. Give the dev pipeline a dev service principal too, so missing grants show up in dev first. A run that only passes with your own rights has not been tested.

RELATED **9.05** The production deployment runs as the person who deployed it
7.02 The grants look right. Users still get access denied **5.15** A notebook works when you run it

5.10 A downstream task ran on yesterday's data. Its upstream refresh never ran.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Tasks on the branch an If/else condition did not take are **Excluded**, and Run if treats excluded tasks as **successful**. A task with another parent that succeeded runs anyway, on whatever the table held before.

Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

WHAT THE JOB SHOWS

Succeeded

refresh_orders Excluded, build_report Succeeded

WHAT THE REPORT USED

Yesterday's orders

sales_prod.core.orders, last written the night before

SEE

WRONG TURN

WHAT HAPPENED

GUARD

DETECT

RECOVER

FIG. 5.10 An excluded parent satisfies All succeeded. It does not refresh anything.

WRONG TURN

"If the refresh does not run, the report that depends on it will not run either."

WHAT HAPPENED

`build_report` depends on `refresh_orders` and `load_customers` with Run if **All succeeded**, the default. `refresh_orders` sat on the false branch of an If/else, so it was excluded. Excluded counts as success, and `load_customers` succeeded, so the report ran.

1 GUARD

The reader checks freshness itself: skip maintenance commits and count any other commit since the run started, not a list of expected operations. `UPDATE`, `DELETE`, `COPY INTO` and `CREATE OR REPLACE TABLE AS SELECT` load data too.

```
from delta.tables import DeltaTable
from pyspark.sql import functions as F
start_ms = int(dbutils.widgets.get("start_ms")) # {{job.start_time.timestamp_ms}}
skip = ["OPTIMIZE", "VACUUM START", "VACUUM END", "SET TBLPROPERTIES"]
writes = (DeltaTable.forName(spark, "sales_prod.core.orders").history(100)
          .where(~F.col("operation").isin(skip))
          .where(F.unix_millis("timestamp") >= start_ms))
if writes.count() == 0:
    raise RuntimeError("orders was not written in this run")
```

2 DETECT

Compare both tasks per run.

```
SELECT job_run_id, task_key, result_state
FROM system.lakeflow
     .job_task_run_timeline
WHERE job_id = '123'
     AND task_key IN ('refresh_orders',
                     'build_report');
```

3 RECOVER

Rerun the refresh and the report for the affected days. To stop everything after a branch, make later tasks depend on that branch only, so exclusion cascades.

RELATED [5.11](#) Task values work when you test the notebook

[5.01](#) The job succeeded and every task is green

5.11 Task values work when you test the notebook. In the job, the next task gets the default.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Outside a job, `taskValues.get` returns its `debugValue` and set does nothing, so a test never checks the task key. In the job the key must match the upstream **task key** exactly, or you get an error or your default.

Cost of failure **Wrong data**
Status **GA** · Verified **Oct 2026**

	RUN INTERACTIVELY	RUN AS A JOB TASK
<code>set()</code>	Does nothing	Stores the value; its JSON form may be up to 48 KiB
<code>get()</code>	Returns <code>debugValue</code> , or raises <code>TypeError</code> without one	Returns the value set by the task named in <code>taskKey</code>
<code>taskKey</code>	Never checked	The upstream task's key, not the notebook name
Key spelled differently	Still returns <code>debugValue</code>	Returns default if given, else <code>ValueError</code>
Outcome	The test passes	The default flows into the next step

FIG. 5.11 The interactive test exercises the debug path only.

CAUSE

`taskKey` must be the key of the task that called `set`, and `key` must match what it set. An unknown task raises `ValueError`; an unknown key raises it too, unless you gave a `default`, which is then returned without a word. A job run holds up to 250 task values.

FIX

Pass values as task parameters with a dynamic value reference, so the job resolves them, and read them as widgets. Do not give a default to a value that must exist.

```
- task_key: publish
depends_on: [{task_key: load_orders}]
notebook_task:
  notebook_path: ../src/publish.py
  base_parameters:
    rows: "{{tasks.load_orders.values.rows_written}}" # read with
dbutils.widgets.get("rows")
```

PREVENT

Test the hand-off in a job run, never only in the notebook. Keep values small and JSON-serialisable: a list of keys, a count or a status. An If/else condition accepts only numbers, strings and booleans from a task value; booleans arrive as the strings `"true"` and `"false"`.

RELATED **5.10** A downstream task ran on yesterday's data **5.12** A for each task loops over 1,000 tables
5.15 A notebook works when you run it

5.12 A for each task loops over 1,000 tables. The list does not fit and the loop crawls.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The **Inputs** box of a For each task holds 5,000 characters, and iterations run **one at a time** unless you set Concurrency, which goes up to **100**.

Cost of failure **Rework**Status **GA** · Verified **Oct 2026**

100

Iterations of one For each task that can run at the same time. The default concurrency is 1, and a For each task cannot contain another For each task.

Source: Databricks REST API reference, Jobs create; Microsoft Learn, Use a For each task, Oct 2026

HOW YOU WILL NOTICE

A pasted array of 1,000 table names is rejected at 5,000 characters. With the default concurrency of 1, the run takes the sum of every iteration.

```
for_each_task:
  inputs: "{{tasks.list.values.t}}"
  concurrency: 20
  task:
    task_key: load_one
```

GUARD

Build the list in an upstream task named **list** and pass it as a task value: a reference may resolve to up to 48 KB.

```
rows = spark.sql(
  "SHOW TABLES IN sales_prod.raw")
names = [r.tableName
  for r in rows.collect()]
dbutils.jobs.taskValues.set(
  key="t", value=names)
```

WORKAROUNDS

Pass groups instead of single items: 50 inputs of 20 tables each, with the nested task looping inside. For many small, similar loads, one task that parallelises in Spark beats a thousand task runs, each with its own start-up and history row. Raise concurrency only as far as the sources and compute can take.

NEIGHBOURING LIMITS

48 KB

Value behind a task value reference in Inputs

2,000

Tasks running at once per workspace. Fixed

750

Run job and For each parent tasks running at once per workspace. Fixed

RELATED **5.11** Task values work when you test the notebook

4.01 A job runs every five minutes and starts a new cluster each time

5.13 The job is scheduled every 15 minutes. It starts every 20.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A job allows **one active run** by default. When a run takes 20 minutes, the next tick waits in the **queue** or is skipped, so the run length sets the interval.

Cost of failure **Outage**Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (CRON ON A SERVER)

"Every tick starts a run. If the last one is still going, the new one runs beside it."

IN LAKEFLOW JOBS

Maximum concurrent runs is 1 by default. A tick that finds a run active is queued when queueing is on, the default for jobs created in the UI after 15 April 2024, and starts when the active run ends; queued runs wait up to 48 hours. With queueing off, the tick is skipped. The scheduler is not built for low latency: runs can start several minutes late. Hourly schedules in a time zone with daylight saving can be skipped or delayed by an hour or two when the clocks change. Quartz cron starts with a seconds field: `0 0/15 * * * ?`.

Do this. Measure runs from their time slices in `system.lakeflow.job_run_timeline`: `run_duration_seconds` and `queue_duration_seconds` are 0 for multi-task jobs and jobs not created through the legacy API. For time queued, `databricks jobs get-run` returns `queue_duration` in milliseconds. If runs outlast the interval, make them shorter or the interval longer. Raise maximum concurrent runs only when two runs can safely overlap, which two **MERGE** runs into one table usually cannot (3.07). Use UTC for anything hourly, and check a job that never runs for **PAUSED** in its trigger.

```
SELECT run_id, min(period_start_time) AS started,
       sum(timestampdiff(SECOND, period_start_time, period_end_time)) AS run_s
FROM system.lakeflow.job_run_timeline
WHERE job_id = '123' AND period_start_time >= current_date() - INTERVAL 2 DAYS
GROUP BY run_id ORDER BY started;
-- the gap between consecutive started values is the real interval
```

RELATED **5.14** One test file started one run **3.07** Your MERGE works alone and fails when two jobs run it at once

9.08 The duration alert fires after the job finishes

5.14 One test file started one run. In production, files landing over 20 minutes start run after run.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A file arrival trigger checks for new files about **every minute** and starts a run when it finds any. A drop that lands over 20 minutes starts many runs. **Wait after last change** holds the run until the drop is complete.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

	TEST: ONE FILE DROPPED	PRODUCTION: 120 FILES OVER 20 MINUTES
Checks	Best effort, every minute	Best effort, every minute
New files found	On one check	On most checks for 20 minutes
Runs started	One	One after another while files keep landing
Each run reads	The whole drop	Whatever has landed so far, part of the drop

FIG. 5.14 The trigger sees arrivals, not batches. Nothing tells it the drop is complete.

CAUSE

Only new files trigger runs, and each check that finds some can start one. Without rate settings nothing waits for the last file. Without file events on the external location, a workspace allows 50 jobs with file arrival triggers, each watching at most 10,000 files.

FIX

Debounce the trigger and make each run take everything new since the last one, with Auto Loader and an available-now trigger. Both settings have a minimum of 60 seconds.

```
trigger:
  pause_status: UNPAUSED
  file_arrival:
    url: /Volumes/sales_prod/raw/landing/stores/
    wait_after_last_change_seconds: 300    # each new file restarts the wait
    min_time_between_triggers_seconds: 900 # at most one run per 15 minutes
```

PREVENT

Turn on file events for the external location. Never write a job that processes "the file that triggered it": the trigger does not hand over a file list. When the input is a table, use a table update trigger instead, generally available for Unity Catalog tables, with up to 10 tables and an any or all updated condition.

RELATED **2.01** Auto Loader processed the same file twice **5.13** The job is scheduled every 15 minutes

2.09 Trigger once loaded the whole backlog in one batch

5.15 A notebook works when you run it. As a job task it fails.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The task runs as the job's **Run as** identity, from the top, with only the **parameters** the job passes. With a **Git source** it runs the code from the branch, tag or commit as it stood when the run started.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	YOU, IN THE NOTEBOOK	JOB TASK
Identity	You	Run as: the job's creator by default, or a service principal
Code	The copy you are editing	A workspace path, or a Git snapshot taken at run start
State	Variables and installs from earlier cells	Fresh, top to bottom
Widgets	Values you typed	Job and task parameters, matched by exact key
Parameter not passed	Your typed value	The widget default, without an error
Outcome	Works	Wrong date, missing grant or old code

FIG. 5.15 Same notebook, different identity, code version and inputs.

CAUSE

The creator is the default Run as identity, so grants differ the moment it moves to a service principal. A task pinned to a Git commit keeps running that commit; a branch runs its latest commit, recorded in the task run details. If a job parameter and a task parameter share a key, the job parameter wins. `dbutils.notebook.run` starts a separate ephemeral job and passes only strings. Output over 30 MB cancels the run.

FIX

Make the notebook refuse to guess. Fail when a required parameter is missing, and print what arrived.

```
dbutils.widgets.text("run_date", "")
run_date = dbutils.widgets.get("run_date")
if not run_date:
    raise ValueError("run_date was not passed by the job")
print(dbutils.widgets.getAll())
```

PREVENT

Set Run as to a service principal before the first production run, not after the creator leaves. Point production at a Git branch or tag through a deployment, not at a hand-edited commit. Test with **Run now with different parameters** on a fresh job run, not on the cluster you developed on.

RELATED **5.04** You cloned a production job to test it **9.05** The production deployment runs as the person who deployed it **4.11** A Spark config set on the cluster is ignored at runtime

5.16 An auditor asks for a job run from four months ago. The Runs tab stops at 60 days.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Lakeflow Jobs keeps run history for **60 days**. The **system.lakeflow** tables keep run records for 365 days. Anything older, and every notebook output, survives only if you **copy it out**.

Cost of failure **Rework**Status **GA** · Verified **Oct 2026**

60

Days of job run history kept in the workspace, in the UI and the Jobs API. Databricks recommends exporting results you need before they expire.

Source: Microsoft Learn, Monitoring and observability for Lakeflow Jobs, Oct 2026

HOW YOU WILL NOTICE

The run is gone from the Runs tab and the API. System tables still have its record, though not its output.

```
SELECT run_id, result_state,
       period_start_time
FROM system.lakeflow.job_run_timeline
WHERE job_id = '123'
      AND period_start_time
          < current_date() - 60;
```

GUARD

Copy run records into your own table every day, so they outlive both windows.

```
MERGE INTO ops_prod.audit.job_runs t
USING (SELECT * FROM
       system.lakeflow.job_run_timeline
       WHERE period_end_time >
            current_date() - 3) s
ON t.workspace_id = s.workspace_id
AND t.run_id = s.run_id
AND t.period_start_time
    = s.period_start_time
WHEN NOT MATCHED THEN INSERT *;
```

WORKAROUNDS

The three-day window covers records that arrive late; most land within an hour. Runs over an hour are stored as hourly slices, and only the last slice carries **result_state**. Export notebook output to HTML from the run page, and deliver cluster logs to a volume (9.10).

NEIGHBOURING LIMITS

365 days

Run records in system.lakeflow timeline tables

1 hour

Typical delay before a run appears there

1 region

Each system table covers workspaces in its own region

RELATED

9.09 You run 200 jobs**7.10** An audit needs two years of access history**9.10** The job cluster terminated

5.17 Every table has a hand-built freshness check. A schema nobody wrote checks for went stale for a week.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A hand-built check covers only its own table. **Anomaly detection**, in Public Preview, watches every table in a schema for **freshness** and **completeness**, learned from each table's history.

Cost of failure **Wrong data**Status **Public Preview** · Verified **Oct 2026**

WHAT THE CHECKS SHOW

42 of 42 pass

One hand-built freshness check per table in core

WHAT FINANCE HELD

7 days stale

fx_rates: no commit since its feed stopped

SEE

WRONG TURN

WHAT HAPPENED

GUARD

DETECT

RECOVER

FIG. 5.17 A missing check looks exactly like a passing one: nothing alerts.

WRONG TURN

"Every table that matters has a freshness check, so a stale table would alert."

WHAT HAPPENED

Checks were written table by table in **core**. The **finance** schema came later from another team, with none. When its feed stopped, the load job still succeeded.

1 GUARD

Enable it on the schema, or from the catalog for up to 50 schemas at once. A table turns Unhealthy when a commit is unusually late or a day's rows fall below range. It needs **MANAGE** on the schema or catalog and bills serverless DBUs. Results land in a system table that only account admins can read until they grant the alert owner **USE CATALOG** on **system**, **USE SCHEMA** on **data_quality_monitoring** and **SELECT** on **table_results**.

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.dataquality import AnomalyDetectionConfig, Monitor
w = WorkspaceClient()
sid = w.schemas.get("sales_prod.finance").schema_id
w.data_quality.create_monitor(Monitor(object_type="schema", object_id=sid,
    anomaly_detection_config=AnomalyDetectionConfig()))
```

2 DETECT

Alert on the results table (also Public Preview) to on-call (9.07).

```
SELECT catalog_name, schema_name,
    table_name, max(event_time) AS seen
FROM system.data_quality_monitoring
    .table_results
WHERE status = 'Unhealthy'
    AND event_time > now() - INTERVAL 6 HOURS
GROUP BY ALL;
```

3 RECOVER

Fix the feed, rerun what read the stale days. Keep hand checks for hard deadlines, rules that must fail the job (5.01), views and foreign tables, which it skips, and new tables still in Training.

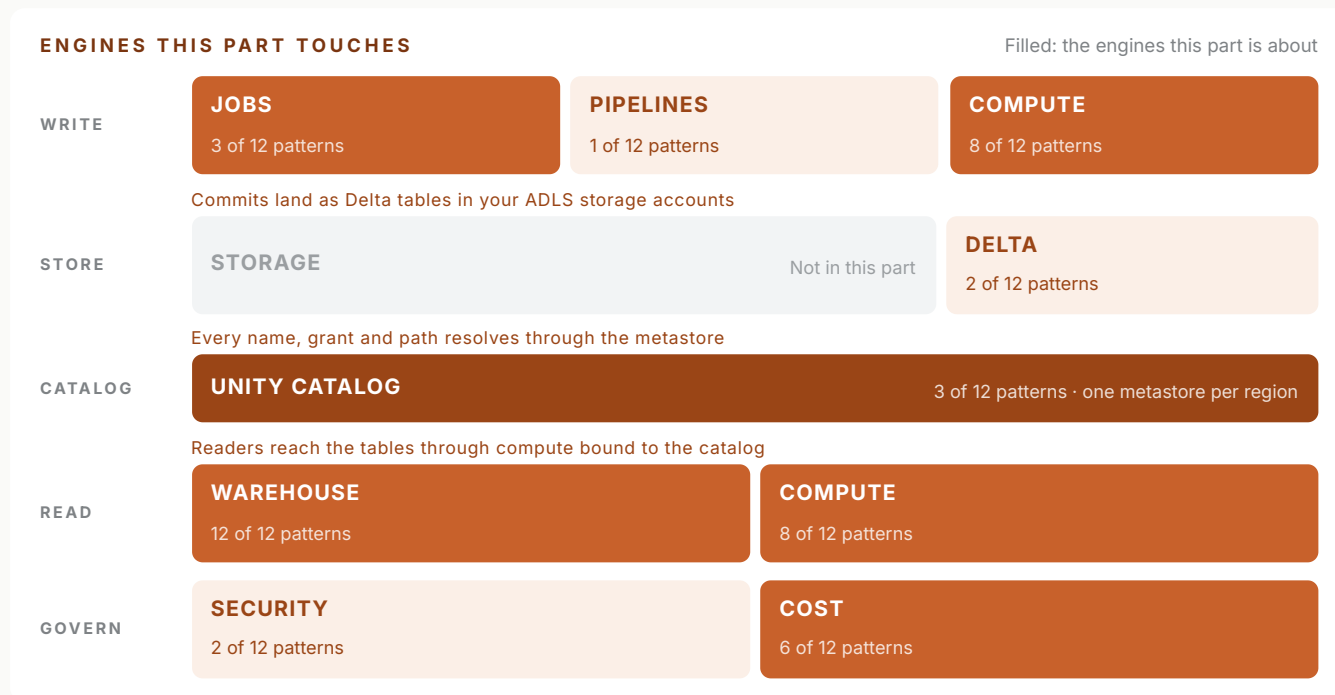
RELATED **5.01** The job succeeded and every task is green

6.08 The SQL alert fired six hours late

9.07 A job failed. You heard about it from a business user

06 Databricks SQL and BI.

How data is served: SQL warehouses, federation, materialized views, dashboards, alerts and Power BI. Through most of these failures the warehouse shows Running, which says nothing about whether an answer is fast, current, or meant for the person reading it.



NO.	PATTERN	JOBS	COMPUTE	DELTA	UC	WH	COST	PAGE
6.01	You moved the warehouse to X-Large. Queries are no faster and still queue.							102
6.02	Auto stop is 10 minutes. The warehouse is never warm when users arrive.							103
6.03	One warehouse serves everyone. One heavy query slows every dashboard.							104
6.04	A federated query is a hundred times slower than querying the source directly.							105
6.05	A materialized view stopped refreshing. Users see three-day-old data.							106
6.06	Someone ran SELECT * on a 10 TB table. It ran to completion.							107
6.07	A shared dashboard shows viewers data they cannot query themselves.							108
6.08	The SQL alert fired six hours after the threshold was crossed.							109
6.09	A Power BI DirectQuery report sends hundreds of queries to the warehouse.							110
6.10	A view or row filter works on a SQL warehouse. The same query fails from an older cluster.							111

NO.	PATTERN	JOB	COMPUTE	DELTA	UC	WH	COST	PAGE
6.11	Photon is on. The query plan shows most operators running outside Photon.							112
6.12	A dashboard is fast for the first user and slow for the next. Or it shows a result from before the last load.							113

6.01 You moved the warehouse to X-Large. Queries are no faster and still queue.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Cluster size sets the compute each query gets. The **number of clusters** sets how many queries run at once. Queueing is a cluster count problem, and a slow scan is usually a table problem.

Cost of failure **Cost**

Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
One large cluster	Heavy single queries: big joins and aggregations that spill to disk on a smaller size.	Many users at once. Pro and classic plan for about 10 concurrent queries per cluster, so the eleventh queues however large the cluster is. You pay for X-Large all day.
Small size, high max clusters	Many short dashboard queries at the same time.	Heavy queries spill and run long. On pro and classic, clusters are added by the estimated time to clear running and queued queries (2 to 6 minutes adds one, 6 to 12 two, 12 to 22 three) or after a 5 minute queue wait, and each takes minutes to start.
Serverless, sized by evidence DEFAULT	Start one size larger and size down. Raise the size when the query profile shows spill. Raise max clusters when Peak Queued Queries stays above 0. Intelligent Workload Management adds clusters quickly.	Queries are slow because they scan everything. No warehouse setting fixes a full scan; table layout does.

FIG. 6.01 Size and cluster count answer different questions. Measure which one you have before paying for either.

DEFAULT

Serverless for every user-facing warehouse. Size from spill, max clusters from queue time, and fix slow scans in the tables, not the warehouse.

SIGNS YOU CHOSE WRONG

A bigger size leaves query times unchanged. The warehouse sits at max clusters and every query is still slow. Queue time stays high on a 4X-Large with one cluster.

CHECK IT

Split a week of queries into time spent queued, time spent spilling, and the rest. Queue time calls for clusters, spill calls for size, and the rest calls for query and layout work.

```
SELECT date_trunc('HOUR', start_time) AS hr,
       percentile(waiting_at_capacity_duration_ms, 0.9) / 1000 AS p90_queue_s,
       sum(spilled_local_bytes) / 1e9 AS spill_gb,
       percentile(execution_duration_ms, 0.9) / 1000 AS p90_exec_s
FROM system.query.history
WHERE compute.warehouse_id = '4f2a9c1d7e3b8a60'
      AND start_time > current_timestamp() - INTERVAL 7 DAYS
GROUP BY 1 ORDER BY 1;
```

Go deeper: SQL Handbook 4.0, reading the query profile.

RELATED **6.03** One warehouse serves everyone **3.01** Every query on a 500 million row table does a full scan

8.02 You moved to serverless to save money

6.02 Auto stop is 10 minutes. The warehouse is never warm when users arrive.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A pro or classic warehouse takes **several minutes** to start, so any idle gap longer than auto stop ends in a wait. Serverless starts in **seconds**, which makes a short **auto stop** cheap rather than painful.

Cost of failure **Slow queries**Status **GA** · Verified **Oct 2026**

	SERVERLESS WAREHOUSE	PRO OR CLASSIC WAREHOUSE
Start time	2 to 6 seconds	About 4 minutes
Default auto stop	10 minutes	45 minutes
Minimum auto stop	5 minutes in the UI, 1 minute through the API	10 minutes
First query after idle	Waits seconds	Waits for the whole start
Longer auto stop	Not needed	Pays for idle hours, still cold every morning

FIG. 6.02 The same auto stop value is a cost saving on one type and a daily wait on the other.

CAUSE

Pro and classic warehouses provision their clusters in your Azure subscription on every start. A short auto stop means many cold starts; a long one means paying for idle clusters and still starting cold at 8am.

FIX

Move user-facing warehouses to serverless and set auto stop just under the usual gap between queries: 5 minutes is the UI minimum, 1 minute is allowed through the API. A serverless warehouse bills until it stops, so a refresh every few minutes keeps it running all day. Then measure the start wait in query history.

```
databricks warehouses edit 4f2a9c1d7e3b8a60 --json '{"warehouse_type": "PRO",
  "enable_serverless_compute": true, "auto_stop_mins": 5}'
```

```
SELECT date(start_time) AS day,
  max(waiting_for_compute_duration_ms) / 1000 AS worst_start_wait_s
FROM system.query.history WHERE compute.warehouse_id = '4f2a9c1d7e3b8a60'
GROUP BY 1 ORDER BY 1 DESC;
```

PREVENT

A warehouse that must stay pro or classic gets a SQL task in a Lakeflow Job that warms it before the working day. Check `system.compute.warehouse_events` for **STARTING** events in working hours: each one is a user who waited.

RELATED **6.01** You moved the warehouse to X-Large **8.02** You moved to serverless to save money

4.01 A job runs every five minutes and starts a new cluster each time

6.03 One warehouse serves everyone. One heavy query slows every dashboard.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Queries on a warehouse share its **concurrency**, and a running query is never interrupted for a shorter one. Split warehouses by **workload**, not by team, so dashboards never wait behind ad hoc scans or refresh jobs.

Cost of failure **Slow queries**Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
One shared warehouse	One configuration, one bill line, one cache that every query can hit.	An analyst's full scan or a dashboard refreshing every minute fills the slots. Short queries queue behind them, and nobody can tell which workload costs what.
Warehouse per team	Cost per team through warehouse tags. Teams manage their own size.	Each team mixes heavy and light work again. Many warehouses sit idle, and the same dashboard query runs on five of them.
Warehouse per workload DEFAULT	BI serving, ad hoc analysis, and scheduled refreshes and alerts each get their own serverless warehouse, size, timeout and tags.	Needs CAN USE grants per group and a rule for what runs where. Without the rule, people use whichever warehouse is fastest.

FIG. 6.03 Isolation comes from separating workloads with different shapes, not from separating people.

DEFAULT

Three warehouses: BI with more max clusters, ad hoc with a larger size and a short timeout (6.06), and one for scheduled dashboard updates, refreshes and alerts. Schedule dashboards no more often than the data changes. Warehouse tags give cost per workload; query tags (GA) split one warehouse's history by team: `SET QUERY_TAGS = 'team:finance'`.

CHECK IT

Group a week on the shared warehouse by source. If dashboards queue while ad hoc work holds most task time, split them.

```
SELECT CASE WHEN query_source.dashboard_id IS NOT NULL THEN 'dashboard'
            WHEN query_source.job_info IS NOT NULL THEN 'job'
            ELSE 'ad hoc' END AS workload,
       count(*) AS queries,
       avg(waiting_at_capacity_duration_ms) / 1000 AS avg_queue_s,
       sum(total_task_duration_ms) / 3.6e6 AS task_hours
FROM system.query.history
WHERE compute.warehouse_id = '4f2a9c1d7e3b8a60'
      AND start_time > current_timestamp() - INTERVAL 7 DAYS
GROUP BY 1;
```

RELATED **6.01** You moved the warehouse to X-Large **6.09** A Power BI DirectQuery report sends hundreds of queries

6.06 Someone ran `SELECT *` on a 10 TB table

6.04 A federated query is a hundred times slower than querying the source directly.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Only the parts that **push down** run in the source database. The rest is copied over **JDBC** and processed in Databricks, and federated queries get **no result or disk cache**.

Cost of failure **Slow queries**Status **GA** · Verified **Oct 2026**

RUN IN POSTGRES SQL

6 seconds

Index lookup, a few thousand rows

SAME QUERY THROUGH PG_CRM

10 minutes

40 million rows read over one connection

FIG. 6.04 The filter that the source would use did not reach the source.

MEASURE FIRST

- 1 See the SQL that Databricks sends to the source, and which filters it carries.

```
EXPLAIN FORMATTED
SELECT customer_id, sum(amount) FROM pg_crm.public.orders
WHERE lower(channel) = 'web' AND order_date >= '2026-09-01'
GROUP BY customer_id;
```

- 2 In the query profile, compare rows read from the foreign table with rows returned.
- 3 Compare **DESCRIBE TABLE** types with the source, so a type cast is not mistaken for a speed problem.

CAUSE

Pushdown covers filters, projections, limits and aggregates, but only some functions inside filters. For PostgreSQL, window functions never push down. A join with a Delta table runs in Databricks; join pushdown between foreign tables is in Public Preview. With no result or disk cache, every refresh goes back to the source.

FIX

Add a pushable predicate with **AND** next to one that is not. Data read often belongs in a materialized view, and high volumes in a Lakeflow Connect pipeline. The view refreshes on **SCHEDULE: TRIGGER ON UPDATE** watches Delta sources and cannot see a change in the source database.

```
CREATE MATERIALIZED VIEW sales_prod.core.crm_orders_recent
  SCHEDULE EVERY 1 HOUR
AS SELECT order_id, customer_id, channel, amount, order_date
FROM pg_crm.public.orders
WHERE order_date >= '2026-01-01';
```

EXPECTED RESULT

The plan shows the date filter in the remote query, dashboards read the materialized view in seconds, and the source sees one read an hour instead of one per viewer.

RELATED **6.05** A materialized view stopped refreshing **2.14** You need SQL Server and Salesforce data every hour

C.13 OpenSharing, Lakehouse Federation, external data access and Fabric mirroring

6.05 A materialized view stopped refreshing. Users see three-day-old data.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A failed refresh leaves the **last successful result** in place, so every query still succeeds. The refresh runs as the **owner** on a serverless pipeline, and by default only the owner hears that it failed.

Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

WHAT THE DASHBOARD SHOWS

Loads in 0.4 s

Every visual renders, no error

WHAT THE VIEW HOLDS

Monday 03:30

Refresh failing since an upstream column was renamed

SEE

WRONG TURN

WHAT HAPPENED

GUARD

DETECT

RECOVER

FIG. 6.05 A materialized view answers from its stored result, whether or not the last refresh worked.

WRONG TURN

"The view has a refresh schedule, so the data is current."

WHAT HAPPENED

A **SCHEDULE** clause creates a system-managed job, and each refresh runs as the view's owner on a serverless pipeline. A renamed column, a lost grant, or an owner who left makes the refresh fail. The stored result stays queryable. Failure emails for a **SCHEDULE** clause are in Beta and go to the owner only by default. **TRIGGER ON UPDATE** (GA) refreshes when Delta sources change, so it cannot fire before the load lands, but it fails the same way.

1 GUARD

Refresh business views from a SQL task in a Lakeflow Job with failure notifications to the team, owned by a service principal, not a person. **DROP SCHEDULE** removes a trigger too. Name the columns: a **SELECT *** definition breaks on the next upstream change.

```
ALTER MATERIALIZED VIEW sales_prod.bi.daily_revenue DROP SCHEDULE;
-- the job's SQL task now runs this, with notifications on failure
REFRESH MATERIALIZED VIEW sales_prod.bi.daily_revenue;
```

2 DETECT

Read the last refreshes from the event log. Only the owner can query it directly.

```
SELECT timestamp,
       details:update_progress.state AS st
FROM event_log(TABLE(
       sales_prod.bi.daily_revenue))
WHERE event_type = 'update_progress'
ORDER BY timestamp DESC LIMIT 5;
```

3 RECOVER

Fix the definition with explicit columns, run **REFRESH**, and confirm the refresh type in the event log. Then give the dashboard a "data as of" tile from **max()** of the source timestamp in the view, so the next failure shows on screen.

RELATED **6.08** The SQL alert fired six hours late **9.07** A job failed. You heard about it from a business user

5.06 Pipeline expectations fail on thousands of rows

6.06 Someone ran `SELECT *` on a 10 TB table. It ran to completion.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The system default for `STATEMENT_TIMEOUT` is **2 days**, and the practical ceiling is **24 hours**. Set a lower workspace value from what your statements really need, and remember that a **session** value overrides it.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

172,800

Seconds, the default `STATEMENT_TIMEOUT` on Databricks SQL and its maximum: 0 to 2 days is allowed. Clusters running over 24 hours are recycled and a statement still running then can be ended, so treat 86,400 as the ceiling.

Source: Microsoft Learn, `STATEMENT_TIMEOUT` and Create a SQL warehouse, Oct 2026

HOW YOU WILL NOTICE

Hours-long statements in query history. Look at every statement type: job `MERGE`, `INSERT` and CTAS on the same warehouses hit the same limit.

```
SELECT statement_type,
       percentile(total_duration_ms, 0.99)
         / 60000 AS p99_min,
       max(total_duration_ms) / 60000
         AS max_min
FROM system.query.history
WHERE compute.type = 'WAREHOUSE'
      AND start_time > current_timestamp()
        - INTERVAL 30 DAYS
GROUP BY 1 ORDER BY max_min DESC;
```

GUARD

Before you change it: the query on the left is the inventory. The rule: the highest p99 across statement types, leaving out jobs that set their own session value, plus a margin (here 2 hours), never above 24 hours. The platform owner approves; removing the parameter rolls it back. Then add it in workspace admin settings, Compute, SQL warehouses, SQL configuration parameters.

```
STATEMENT_TIMEOUT 7200

-- check what applies in a session
SET STATEMENT_TIMEOUT;
```

WORKAROUNDS

Order of precedence is session, then warehouse, then workspace. A job that needs longer runs `SET STATEMENT_TIMEOUT = 14400` (up to 86400) in its own session, so ETL keeps working under a tight workspace value. Any user can do the same, so the timeout stops accidents, not intent. Per-warehouse timeouts, short for BI and long for ETL, are in Beta: an admin enables the Warehouse Statement Timeout preview, then sets them through the warehouses API.

NEIGHBOURING LIMITS

10

Concurrent queries per cluster, the sizing rule for pro and classic

1,000

Queued queries per warehouse, all types

24 h

Clusters running longer are recycled; design statements to finish within it

RELATED **6.03** One warehouse serves everyone **6.01** You moved the warehouse to X-Large

8.04 One team used 70 percent of the spend

6.07 A shared dashboard shows viewers data they cannot query themselves.

ENGINES JOBS PIPELINES COMPUTE STORAGE DELTA **UNITY CATALOG** WAREHOUSE SECURITY COST

The default publish option, **Share data permissions**, runs every viewer's queries with the **publisher's** permissions. Row filters and masks that test the viewer's identity see the publisher instead.

Cost of failure **Security exposure**
Status **GA** · Verified **Oct 2026**

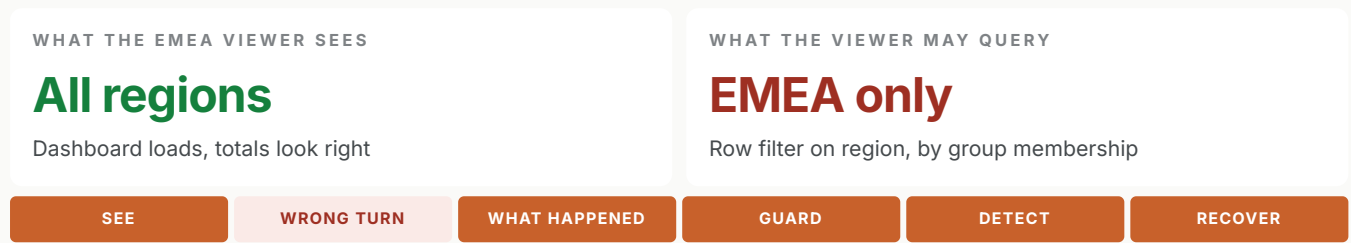


FIG. 6.07 The row filter worked. It filtered for the publisher, who may see every region.

WRONG TURN

"Unity Catalog row filters follow whoever opens the dashboard."

WHAT HAPPENED

Publishing offers **Share data permissions**, the default, and **Individual data permissions**. With the default, the publisher's credential is embedded and viewers see what the publisher sees, even with no grant on the tables. Viewer-based row-level security does not apply. Warehouse access always comes from the publisher. If the publisher loses a grant or leaves, every dashboard they published breaks at once.

1 GUARD

Publish dashboards over filtered or masked tables with Individual data permissions. Where viewers must not need grants, publish with a service principal as the publisher credential, granted **SELECT** only on the aggregate tables the dashboard reads.

```
databricks lakeview get-published 01ef7c2a9b3d4e5f
-- "embed_credentials": true means viewers run as the publisher
```

2 DETECT

List whose privileges dashboard queries used.

```
SELECT query_source.dashboard_id,
       executed_as, count(*) AS n
FROM system.query.history
WHERE query_source.dashboard_id
      IS NOT NULL
GROUP BY ALL ORDER BY n DESC;
```

3 RECOVER

Republish with the right option, then review who the dashboard is shared with, including account users outside the workspace. Treat what viewers already saw as disclosed, and say so.

RELATED **7.04** Users can read a view whose source table they cannot see

7.07 A row filter using `current_user` works in a notebook **C.12** Row filters, column masks, dynamic views and ABAC policies

6.08 The SQL alert fired six hours after the threshold was crossed.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

An alert does not watch the data. It runs its query on a **schedule** and checks the result, so the delay is the schedule interval plus the **freshness** of the table it reads.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	TEST CONDITION IN THE EDITOR	SCHEDULED ALERT
When it checks	When you click	On each tick of its cron schedule
Data it reads	The table as it is now	The table as of its last load
Notifications	Shown on screen	On first trigger. Repeats only with a notification frequency set
Threshold crossed at 09:00	Seen at 09:00	Emailed at 15:00 on a 6-hourly schedule
Query returns no rows	You see an empty grid	The empty result state you set, which may be OK

FIG. 6.08 The condition you tested is right. When it is checked, and against what, is set elsewhere.

CAUSE

The alert ran on the schedule copied from a daily report, every 6 hours. It also reads a table loaded hourly, so a 5-minute schedule alone would still see data up to an hour old. After the first trigger, an alert sends nothing more until it returns to OK, unless a notification frequency is set. Alerts do not support queries with parameters.

FIX

Set the schedule from the latency you need, and alert on freshness in the same query so a stalled load cannot look like a quiet day. Compute staleness over the whole table: with a date filter, a day with no load returns NULL and the alert never fires. Give the alert a warehouse that is already running or starts in seconds.

```
SELECT
  (SELECT coalesce(sum(amount), 0) FROM sales_prod.core.payments
   WHERE status = 'FAILED' AND updated_at >= current_date()) AS failed_amount,
  (SELECT coalesce(timestampdiff(MINUTE, max(updated_at), current_timestamp()), 99999)
   FROM sales_prod.core.payments) AS mins_stale; -- no date filter
-- schedule: 0 0/5 * * * ? alert 1: failed_amount > 10000
-- alert 2: mins_stale > 90 (an empty table counts as stale)
```

PREVENT

An alert cannot be fresher than its table. If the data lands every 6 hours, a 5-minute schedule adds cost and no speed. Put checks that must fire within minutes where the data lands: a **SQL alert task** (GA) as the last task of the loading job, which evaluates the alert when the load ends, whatever schedule the alert has. It needs a current alert, not a legacy one, takes no parameters and runs on a serverless or pro warehouse. An expectation in Lakeflow Spark Declarative Pipelines also fires at load time.

RELATED

9.07 A job failed. You heard about it from a business user

6.05 A materialized view stopped refreshing

9.08 The duration alert fires after the job finishes

6.09 A Power BI DirectQuery report sends hundreds of queries to the warehouse.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

In **DirectQuery**, each visual sends at least one query every time a page loads or a slicer moves. Keep dimensions in **Import or Dual** mode and point DirectQuery at small, **pre-aggregated** tables.

Cost of failure **Slow queries**Status **GA** · Verified **Oct 2026**

ONE VIEWER, ONE SLICER CLICK

14 queries

12 visuals plus cross-highlighting

60 VIEWERS AT 9 AM

800+ queries in minutes

Queued on a warehouse sized for 20 at once

FIG. 6.09 The report has one model. The warehouse sees every visual of every viewer as its own query.

MEASURE FIRST

- 1 Count Power BI queries per minute and their queue time.

```
SELECT date_trunc('MINUTE', start_time) AS m, count(*) AS queries,
       max(waiting_at_capacity_duration_ms) / 1000 AS worst_queue_s
FROM system.query.history
WHERE client_application ILIKE '%power%bi%'
      AND start_time > current_timestamp() - INTERVAL 1 DAY
GROUP BY 1 ORDER BY queries DESC LIMIT 20;
```

- 2 In Power BI Desktop, Performance Analyzer shows each visual's queries and source time.
- 3 Note each table's storage mode.

CAUSE

Every visual, slicer and cross-highlight becomes SQL. Dimensions in DirectQuery are joined again in every query, and a result over 1,000,000 rows fails. At about 10 concurrent queries per cluster, a busy page queues.

FIX

Set dimensions to Dual, keep DirectQuery for the fact table only, and point it at an aggregate. Turn on the Apply button for slicers, and drop interactions nobody uses. Serve the report from its own serverless warehouse with max clusters sized to the morning peak. The view refreshes when **orders** changes, at most every 15 minutes; **SCHEDULE** would refresh on the clock whether or not the data moved.

```
CREATE MATERIALIZED VIEW sales_prod.bi.sales_daily
  TRIGGER ON UPDATE AT MOST EVERY INTERVAL 15 MINUTES
AS SELECT order_date, region, product_id,
       sum(amount) AS amount, count(*) AS orders
FROM sales_prod.core.orders GROUP BY ALL;
```

EXPECTED RESULT

A slicer click sends one query per fact visual, each reading thousands of rows, and peak queue time drops to zero.

RELATED **6.03** One warehouse serves everyone **6.01** You moved the warehouse to X-Large

3.01 Every query on a 500 million row table does a full scan

6.10 A view or row filter works on a SQL warehouse. The same query fails from an older cluster.

ENGINES

JOBSPipelines

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

On **dedicated access mode**, views over tables you cannot read, row filters and column masks need **Databricks Runtime 15.4 LTS** or above and serverless compute in the workspace, which does the **filtering**.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	SQL WAREHOUSE OR STANDARD ACCESS MODE	DEDICATED ACCESS MODE, RUNTIME 14.3 LTS
Who enforces	The compute itself	Cannot: user code shares the JVM
Writes	Supported	Needs Runtime 16.3 or above
View, no grant on its table	Returns rows	Fails
Row filter or mask	Filtered rows, masked values	Cannot be accessed

FIG. 6.10 The policy is the same on both sides. Only one side can run it safely.

CAUSE

A dedicated cluster runs user code in the same process that reads the data, so it cannot hide rows from that code. From 15.4 LTS it passes such queries to serverless compute, which applies the view, filter or mask. Below 12.2 LTS, filtered tables return no data on any access mode.

FIX

Find dedicated clusters on older runtimes and upgrade them to 15.4 LTS or above, or move the workload to standard access mode. Serverless compute must be enabled in the workspace. The filtering is billed as serverless usage under its own product.

```
SELECT cluster_id, cluster_name, dbr_version
FROM system.compute.clusters
QUALIFY row_number() OVER (PARTITION BY cluster_id ORDER BY change_time DESC) = 1
      AND data_security_mode = 'SINGLE_USER' AND delete_time IS NULL;

SELECT usage_date, sum(usage_quantity) AS dbus FROM system.billing.usage
WHERE billing_origin_product = 'FINE_GRAINED_ACCESS_CONTROL'
GROUP BY 1 ORDER BY 1 DESC;
```

PREVENT

Pin a minimum runtime in the cluster policy for dedicated compute, and test views and filters as a non-privileged user on each compute type you support.

- RELATED
- 4.12

Code that runs on a dedicated cluster fails on standard access mode
- 7.07
- A row filter using current_user works in a notebook

C.02

6.11 Photon is on. The query plan shows most operators running outside Photon.

Photon runs only the operators and expressions it supports. A Python, pandas or Scala **UDF**, or another unsupported step, sends that part of the work **back to Spark**. The **query profile** shows how much task time Photon really ran.

Cost of failure **Slow queries**
Status **GA** · Verified **Oct 2026**

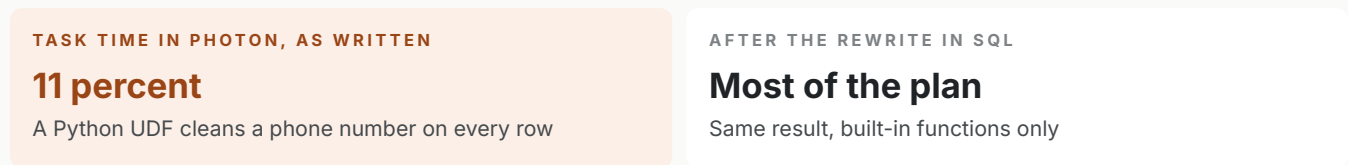


FIG. 6.11 Photon was on in both runs. Only the second gave it work it can do.

MEASURE FIRST

- 1 In the query profile, Execution details show the share of task time in Photon. Photon operators are purple, Spark ones grey.
- 2 Find the first grey operator above the scan. `BatchEvalPython` or `ArrowEvalPython` means a Python or pandas UDF.
- 3 If Photon already runs most of it, check reads and shuffle.

```
SELECT statement_id, total_duration_ms / 1000 AS secs,
       read_bytes / 1e9 AS gb_read, pruned_files, read_files,
       shuffle_read_bytes / 1e9 AS gb_shuffled
FROM system.query.history
WHERE statement_id = '01f0a7c3-5e2b-1d4f-9a8e-3c6b7d2e1f00';
```

CAUSE

The docs say Photon does not support UDFs, RDD or Dataset code. At an unsupported operation it falls back to Spark for the rest of that operation, and rows change format at each boundary. Results stay correct, so nothing fails. Queries under two seconds gain nothing, and an all-purple plan still waits on storage and shuffle.

FIX

Replace the UDF with built-in functions in the query. The docs rank built-in functions and SQL UDFs as the most efficient; pandas UDFs still leave Photon. Check the profile again after the change.

```
-- was: @udf("string") def norm_phone(s): ... re.sub(r"\D", "", s)[-10:]
SELECT customer_id,
       right(regexp_replace(phone, '^[^0-9]', ''), 10) AS phone_norm
FROM sales_prod.core.customers;
```

EXPECTED RESULT

The profile shows most task time in Photon and no Python operator, and the query runs faster. If the share is high and the time has not moved, the work is in the scan or the shuffle: fix the layout (3.01) or the size (6.01).

RELATED **6.01** You moved the warehouse to X-Large **7.08** Row filters slow every query on the table

3.01 Every query on a 500 million row table does a full scan

6.12 A dashboard is fast for the first user and slow for the next. Or it shows a result from before the last load.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The warehouse **result cache** is dropped when a table changes, so it never serves stale rows. The **dashboard cache** can serve results up to **24 hours** old even after the data changed. A data load does not refresh it; a dashboard schedule or a dashboard task does.

Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

	WAREHOUSE RESULT AND DISK CACHE	AI/BI DASHBOARD CACHE
Where it lives	Each warehouse cluster; the remote result cache is serverless only, for JDBC, ODBC and the Statement API	Shared with shared data permissions, per user with individual data permissions
How long	24 hours, lost when the cluster stops (except remote)	24 hours, best effort
Table loaded at 07:00	Entry invalidated, next query sees new rows	06:00 result can show until 06:00 next day
current_timestamp() in the query	Not served from cache	Still cached

FIG. 6.12 Two caches answer the same dashboard. Only one of them watches the tables.

CAUSE

The first viewer warmed a cache the next cannot use: another cluster of a multi-cluster warehouse, a restarted warehouse, or individual data permissions. Stale numbers come from the dashboard cache, which a load does not refresh. A restart only empties the caches; slowness under load is queueing (6.03).

FIX

Refresh the dashboard from the job that loads its tables. A dashboard task warms the shared cache only with shared data permissions; with individual permissions each viewer has their own cache (6.07). Cache hits never reach the warehouse, so old numbers with no recent rows in query history point here.

```
tasks: # after the load task in the same job
  - task_key: refresh_sales_dashboard
    depends_on: [{task_key: load_gold_orders}]
    dashboard_task: {dashboard_id: 01ef7c2a9b3d4e5f}

SELECT max(start_time), count_if(from_result_cache) AS cached, count(*) AS n
FROM system.query.history WHERE query_source.dashboard_id = '01ef7c2a9b3d4e5f'
AND start_time > current_timestamp() - INTERVAL 1 DAY;
```

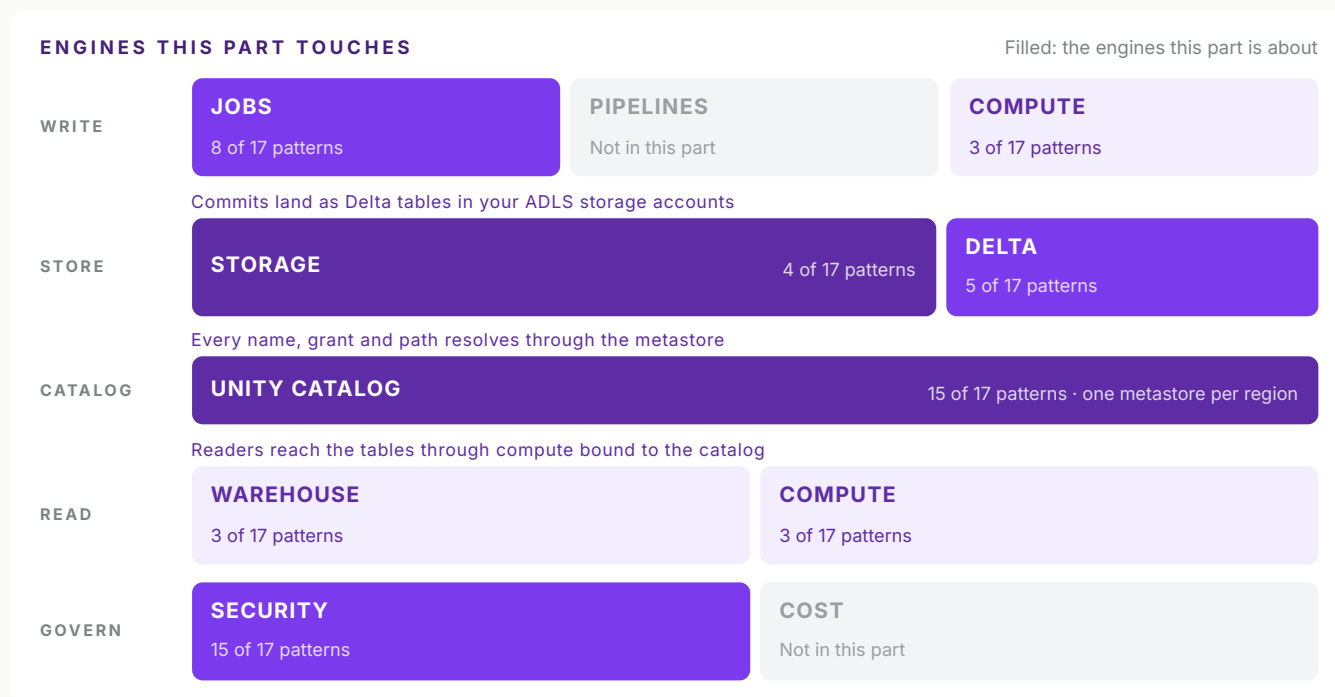
PREVENT

Give each dashboard a "data as of" tile from **max()** of the load timestamp. Keep **SET use_cached_result = false** for benchmarks only.

RELATED **6.05** A materialized view stopped refreshing **6.07** A shared dashboard shows viewers data they cannot query
6.03 One warehouse serves everyone

07 Security and Governance.

Who can see what, and through which route: privileges and groups, external locations, row filters and column masks, lineage, audit, secrets, identities and the move off the Hive metastore. Most exposures here come from a second route to the data that the grant never covered.



NO.	PATTERN	JOBS	COMPUTE	STORAGE	DELTA	UC	SEC	PAGE
7.01	You granted SELECT on a schema. Nobody can query its tables.							116
7.02	The grants look right. Users still get access denied.							117
7.03	You revoked a user's access. They still read the table from an old cluster.							118
7.04	Users can read a view whose source table they cannot see.							119
7.05	You created an external location. People still read the storage directly.							120
7.06	The table has a row filter. A copy made with CTAS has none.							121
7.07	A row filter using current_user works in a notebook. In a job it returns no rows.							122
7.08	Row filters slow every query on the table. Even users who see everything wait.							123
7.09	Lineage shows some tables and misses others. The gaps follow how the code was written.							124
7.10	An audit needs two years of access history. The audit table starts a year ago.							125

NO.	PATTERN	JOB	COMPUTE	STORAGE	DELTA	UC	SEC	PAGE
7.11	A secret showed up in notebook output. Redaction was on.	●	●	●	●	●	●	126
7.12	The Key Vault secret was rotated. New runs work and the long-running jobs fail.	●	●	●	●	●	●	127
7.13	An engineer left. Their personal token still runs 12 production jobs.	●	●	●	●	●	●	128
7.14	You removed someone from an Entra group. They kept access for hours.	●	●	●	●	●	●	129
7.15	The Unity Catalog migration finished. Some tables were silently skipped.	●	●	●	●	●	●	130
7.16	A customer asked to be erased. The rows are gone from Gold and still exist in five other places.	●	●	●	●	●	●	131
7.17	A row filter or mask blocks your own recovery, clone and sharing jobs. The data owner cannot test the policy either.	●	●	●	●	●	●	132

7.01 You granted SELECT on a schema. Nobody can query its tables.

ENGINES

JOBSS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Schema-level **SELECT** does reach every current and future table in it. A query also needs **USE CATALOG** on the catalog and **USE SCHEMA** on the schema, and only their **owners** or holders of **MANAGE** can grant those.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (SQL SERVER)

"GRANT SELECT ON SCHEMA lets the group read every table in it. Nothing else is needed."

IN UNITY CATALOG

Reading a table takes three grants: **SELECT** on the table or an ancestor, **USE SCHEMA** on its schema and **USE CATALOG** on its catalog. Usage privileges are the gate that lets catalog and schema owners keep control, whatever a table owner grants. **ALL PRIVILEGES** on a schema still needs **USE CATALOG** from above, and it leaves out **MANAGE**, **EXTERNAL USE SCHEMA**, **EXTERNAL USE LOCATION** and **READ METADATA**. Users with **BROWSE** see the tables without usage grants, so seeing a table proves nothing about reading it.

Do this. Grant the whole chain to an account group in one script: **USE CATALOG** on `sales_prod`, **USE SCHEMA** and **SELECT** on `sales_prod.core`. Use **CREATE TABLE** on the schema for writers, never **ALL PRIVILEGES**. Check with **SHOW GRANTS `analysts` ON CATALOG sales_prod**, then run a query as a member before you close the request.

RELATED **7.02** The grants look right. Users still get access denied **C.03** Metastore, catalog, schema and workspace

7.02 The grants look right. Users still get access denied.

ENGINESJOBSPIPELINESCOMPUTESTORAGEDELTAUNITY CATALOGWAREHOUSESECURITYCOST

The grant went to an **account group**. The users were added to a **workspace-local group** with the same name, which Unity Catalog cannot see. Service principals are named by **application ID**, not display name.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	ACCOUNT GROUP	WORKSPACE-LOCAL GROUP
Source in settings	Account	Workspace
Created by	Account admin, SCIM or automatic identity management	Workspace admin in one workspace
Built-in example	<code>account users</code>	<code>users, admins</code>
Unity Catalog grants	Allowed	Not allowed
Member runs the query	Rows returned	PERMISSION_DENIED, while Catalog Explorer shows a grant to that name

FIG. 7.02 Two groups can carry one name. Unity Catalog only knows the account one.

CAUSE

Workspace-local groups are legacy. They cannot be granted Unity Catalog privileges, so a grant to `data-analysts` reaches the account group of that name, and members added in the workspace UI get nothing. The same applies to `users`: grant to `account users`. A service principal in a `GRANT` is its application ID in backticks, and a job fails when its run-as principal is not the one in the grant.

FIX

See who the grant reaches, test membership as the failing user, and name service principals by ID.

```
SHOW GRANTS ON SCHEMA sales_prod.core;
SELECT is_account_group_member('data-analysts'); -- run as the failing user

GRANT USE CATALOG ON CATALOG sales_prod TO `6a1f0c2e-8b4d-4f7a-9c3e-2d5b7e1a9f40`;
GRANT USE SCHEMA, SELECT ON SCHEMA sales_prod.core TO `6a1f0c2e-8b4d-4f7a-9c3e-2d5b7e1a9f40`;
```

PREVENT

Convert workspace-local groups: rename each with a `(workspace)` suffix, create the group in Entra ID and sync it, move permissions with the UCX group migration workflow or the Permissions API, then delete the local group. Grant to groups, never to people, and make groups the owners of production catalogs and schemas.

RELATED [7.01](#) You granted SELECT on a schema [7.14](#) You removed someone from an Entra group

[C.03](#) Metastore, catalog, schema and workspace

7.03 You revoked a user's access. They still read the table from an old cluster.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The table they read is the **Hive metastore** copy. Legacy table access control applies only where it is switched on, and a **no-isolation shared** cluster gives every user **full access** to Hive data.

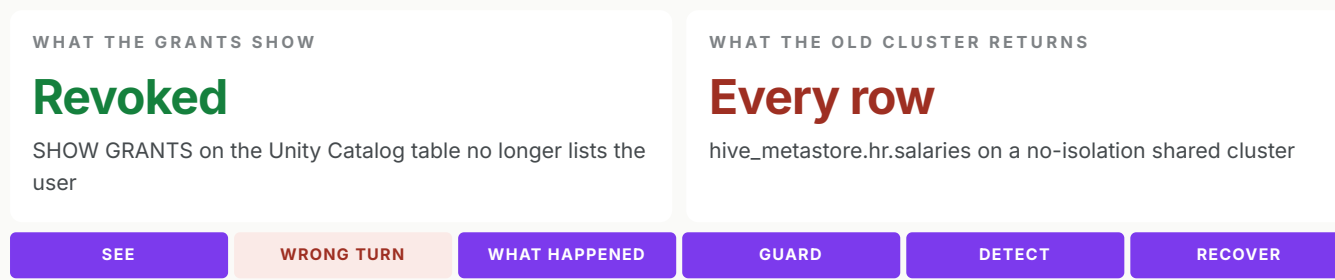
Cost of failure **Security exposure**Status **GA** · Verified **Oct 2026**

FIG. 7.03 The revoke worked on the governed table. The user went to the ungoverned copy.

WRONG TURN

"A revoke takes effect everywhere the data can be read."

WHAT HAPPENED

The migration created the Unity Catalog table and left the Hive table in place. Hive metastore table access control is enforced only on clusters with it enabled and on SQL warehouses. Without it, all users have full access to Hive data, and a no-isolation shared cluster enforces nothing. The user's notebook still names the Hive table, so the revoke never applied to it.

1 GUARD

In Workspace admin, Security, turn on **Enforce User Isolation** and **Disable legacy access**. Both take effect when a cluster restarts. Disabling legacy access needs Runtime 13.3 LTS or above and becomes permanent after 30 days. List what still runs in legacy modes first, and have the platform owner sign off.

```
SELECT cluster_id, cluster_name, owned_by, data_security_mode FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY cluster_id ORDER BY change_time DESC) rn
  FROM system.compute.clusters)
WHERE rn = 1 AND delete_time IS NULL
AND data_security_mode IN ('NONE', 'LEGACY_TABLE_ACL', 'LEGACY_PASSTHROUGH');
```

2 DETECT

Find Hive tables that were synced but still exist and hold data.

```
SHOW TBLPROPERTIES
  hive_metastore.hr.salaries
  ('upgraded_to');
DESCRIBE TABLE EXTENDED
  hive_metastore.hr.salaries;
```

3 RECOVER

Retire the legacy clusters. Before any **DROP**, read **Type** and **Location** in the describe output. Dropping a **MANAGED** Hive table deletes its files, and a table synced **AS EXTERNAL** reads those same files. Drop only **EXTERNAL** ones, or a copy the Unity Catalog table does not use. Otherwise leave it and disable legacy access. Review who used the clusters.

RELATED [1.05](#) The migration moved every table to Unity Catalog [7.15](#) The Unity Catalog migration finished

[C.02](#) Standard access mode, dedicated access mode and the legacy no-isolation mode

7.04 Users can read a view whose source table they cannot see.

ENGINES

JOB

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Unity Catalog checks the **view owner's** rights on the source tables. A reader needs **SELECT on the view** and the usage grants above it, and nothing on the tables. A grant on a view is a **grant on its data**.

Cost of failure **Security exposure**
Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (DEDICATED CLUSTERS ON RUNTIME 15.3 AND BELOW)

"A view checks my rights on its tables too. Revoke **SELECT** on the table and the view stops working for that person."

ON SQL WAREHOUSES, STANDARD ACCESS MODE AND DEDICATED COMPUTE ON 15.4 LTS AND ABOVE

The reader needs **SELECT** on the view plus **USE CATALOG** and **USE SCHEMA** on its parents. The owner's privileges cover the source tables. If the owner loses **SELECT** on a source, the view fails for everyone. Dedicated compute enforces this through serverless filtering, which is billed as serverless.

Do this. Review a view grant as you would a table grant on the same rows. Make a group the owner of shared views, so the view does not break when a person leaves. Build per-user rules with **is_account_group_member** inside the view, or put row filters and column masks on the table itself. List what each view exposes with **SHOW GRANTS ON TABLE** on the view. ABAC support for views is limited, and **CREATE OR REPLACE VIEW** drops governed tags and directly attached policies, so change views with **ALTER VIEW**. A table under a row filter, mask or ABAC policy also cannot be cloned or shared by an identity that is not exempt, and restore and time travel are restricted: see 7.17.

RELATED **7.06** The table has a row filter. A copy made with CTAS has none

7.17 A row filter or mask blocks your own recovery jobs **C.07** Views, materialized views, streaming tables and metric views

7.05 You created an external location. People still read the storage directly.

ENGINES JOBS PIPELINES COMPUTE **STORAGE** DELTA **UNITY CATALOG** WAREHOUSE **SECURITY** COST

Unity Catalog governs requests that come through Databricks. **Azure RBAC roles, account keys** and SAS tokens on the storage account still work, and those reads skip grants, **row filters**, masks and audit.

Cost of failure **Security exposure**
Status **GA** · Verified **Oct 2026**

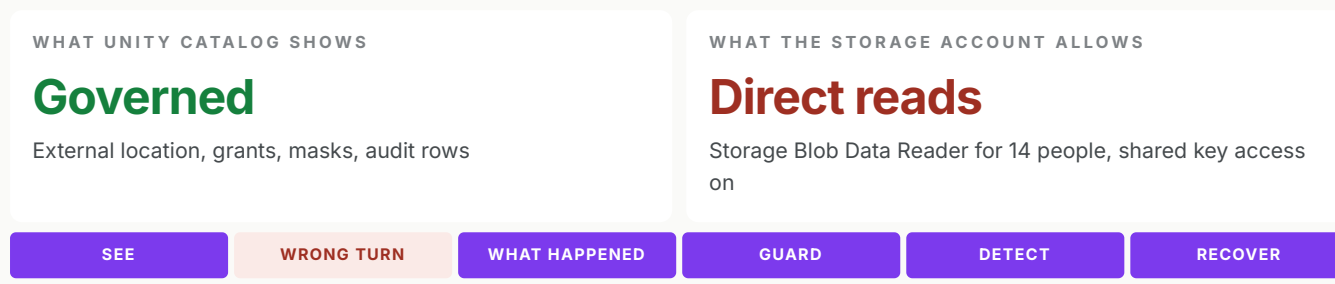


FIG. 7.05 The external location covers one route into the container. Azure still honours the others.

WRONG TURN

"Once the container is an external location, every read goes through Unity Catalog."

WHAT HAPPENED

The container had data roles from before the migration. Storage Explorer, a Python script or another Spark platform reads the files with those roles or the account key. Unity Catalog never sees the request, so masked columns and filtered rows arrive in clear, and `system.access.audit` has no row for it.

1 GUARD

Only the access connector's managed identity holds a data role on these containers. Roles inherited from the resource group or subscription count, and so do holders of `listKeys` (Owner, Contributor, Storage Account Contributor): they can fetch the account key, which reads everything. Set the firewall to selected networks with a resource instance rule for the connector. Leave shared keys on until step 2 has run.

```
az role assignment list --scope $SA_ID -o table --include-inherited \
--query "[].{who:principalName, role:roleDefinitionName}"
```

2 DETECT

Log 30 days to Log Analytics, then list key and SAS callers.

```
StorageBlobLogs
| where AccountName == "stlakeprod"
| where AuthenticationType
  in ("AccountKey", "SAS")
| summarize n = count()
  by CallerIpAddress,
  UserAgentHeader
```

3 RECOVER

Give each direct reader a governed route: a table, a volume, or `READ FILES`. Remove the role, then the mount. When step 2 shows no callers, the storage owner approves turning off shared keys. It can be turned back on, but every key and SAS client fails at once.

```
az storage account update \
-n stlakeprod -g rg-data \
--allow-shared-key-access false
```

RELATED [1.06](#) Your notebook reads abfss:// paths on the old cluster

[1.07](#) Another team points its tool at the folder behind a managed table

[C.05](#) Storage credentials, external locations, service credentials and the access connector

7.06 The table has a row filter. A copy made with CTAS has none.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Row filters and column masks belong to **one table**. A `CREATE TABLE AS SELECT` reads through the filter as the person running it and writes **plain rows**. Run by someone exempt, the copy holds **every row**.

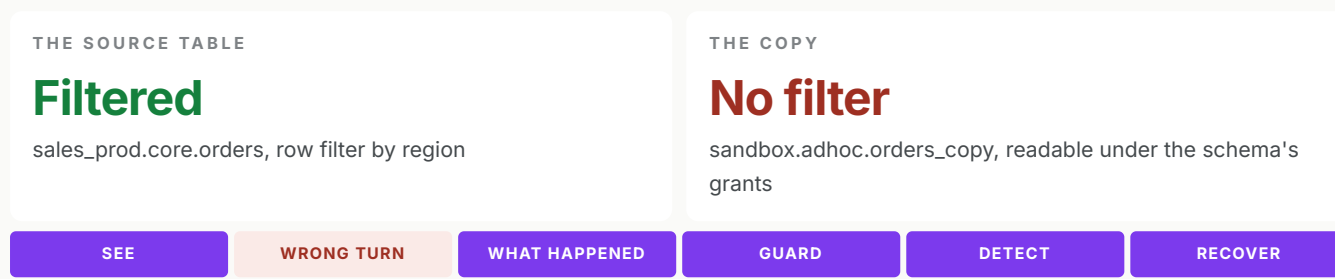
Cost of failure **Security exposure**Status **GA** · Verified **Oct 2026**

FIG. 7.06 The rule protects the table it is attached to. Rows written elsewhere carry no rule.

WRONG TURN

"The row filter protects the data wherever it goes."

WHAT HAPPENED

A member of the exempt group ran a CTAS into a sandbox schema. The new table took its grants from that schema, where a wider group has `SELECT`. Clones are blocked on filtered tables; CTAS, `INSERT ... SELECT` and DataFrame writes are not.

1 GUARD

Keep the exempt group small and deny it `CREATE TABLE` wherever wider groups read. An ABAC policy at catalog level covers tables in that catalog whose columns carry the governed tag. A copy in another catalog is outside it, and column tags do not reach a copy unless something applies them. Have an exempt person write copies only into governed schemas, or tag the copy.

```
CREATE POLICY region_rows ON CATALOG sales_prod
ROW FILTER sales_prod.sec.region_filter
TO `account users` EXCEPT `sales-admins`
FOR TABLES
MATCH COLUMNS has_tag('region') AS r
USING COLUMNS (r);
```

2 DETECT

Lineage shows tables written from the source.

```
SELECT DISTINCT
  target_table_full_name, created_by
FROM system.access.table_lineage
WHERE source_table_full_name =
  'sales_prod.core.orders'
AND target_table_full_name
  IS NOT NULL;
```

3 RECOVER

Drop each copy that has no reason to exist. Protect the ones that stay: governed column tags inside the policy's catalog, else `SET ROW FILTER`. Then check the audit log for who read the copies while they were open.

RELATED [7.17](#) A row filter or mask blocks your own recovery jobs [7.08](#) Row filters slow every query on the table

[C.12](#) Row filters, column masks, dynamic views and ABAC policies

7.07 A row filter using current_user works in a notebook. In a job it returns no rows.

ENGINESJOBSPIPELINESCOMPUTEDELTAUNITY CATALOGWAREHOUSESECURITYCOST

In a job, the session user is the **run-as identity**. For a **service principal**, `current_user()` returns its application ID, so an email lookup matches nothing. Test **account groups** with `is_account_group_member` instead.

Cost of failure **Wrong data**
Status **GA** · Verified **Oct 2026**

	NOTEBOOK, RUN BY YOU	JOB, RUN AS A SERVICE PRINCIPAL
<code>current_user()</code> returns	Your email address	The application ID, a UUID
Lookup on email in a mapping table	One match	No match
<code>is_account_group_member()</code> checks	Your account groups	The service principal's groups
Rows returned	Your region	None
Run status	Succeeded	Succeeded, with an empty output

FIG. 7.07 Same function, same table. The identity running the query changed.

CAUSE

Filter functions run with the owner's rights, except `session_user()` and `is_account_group_member()`, which see the person or principal running the query. The docs list `current_user()` as deprecated, an alias for `session_user()`, so new filters should not use it. A job runs as its run-as identity, and for a service principal that identity is a UUID. A filter keyed on email addresses never matches it.

FIX

Decide by account group, and add the job's service principal to the group that should see the rows.

```
CREATE OR REPLACE FUNCTION sales_prod.sec.region_filter(region STRING)
RETURN is_account_group_member('sales-admins')
      OR is_account_group_member(concat('sales-', lower(region)));

ALTER TABLE sales_prod.core.orders SET ROW FILTER sales_prod.sec.region_filter ON (region);
```

PREVENT

Test every filtered table as the job's run-as identity before release, not as yourself. A task that reads a filtered table should fail when it reads zero rows: an empty result from a filter looks the same as no data.

RELATED **7.08** Row filters slow every query on the table **5.15** A notebook works when you run it
C.12 Row filters, column masks, dynamic views and ABAC policies

7.08 Row filters slow every query on the table. Even users who see everything wait.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The filter runs on every query, for every user. A **mapping table lookup**, a **Python UDF** or an expression that can throw stops the optimiser from **pruning**. The docs advise simple SQL **CASE** logic on group membership.

Cost of failure **Slow queries**Status **GA** · Verified **Oct 2026**

FILTER AS WRITTEN

EXISTS over a mapping table

A subquery on `sec.user_regions` in every query, plus four unused arguments

FILTER AS THE DOCS ADVISE

Group checks in CASE

One column argument, SQL only, no table lookup

FIG. 7.08 The same rule, written two ways. Only one leaves the optimiser room to skip files.

MEASURE FIRST

- 1 Read the function the table uses.

```
DESCRIBE TABLE EXTENDED sales_prod.core.orders; -- row filter name and columns
DESCRIBE FUNCTION EXTENDED sales_prod.sec.region_filter;
```

- 2 Open the query profile of a slow query. Look for the extra join or subquery, a Python UDF node, and files read against files pruned.
- 3 Count the arguments and **AND** conjuncts. Each argument column is read even when the query does not use it.

CAUSE

Mapping tables and subqueries add work to each query. Python UDFs offer fewer optimisations than SQL. Expressions that can raise errors, such as ANSI division, block pushdown, because an error could leak values before filtering. A filter cannot read a table that has its own filter or mask, and each distinct column mask is evaluated per query.

FIX

Rewrite in SQL with group checks, one argument and `try_` functions. When the same rule covers many tables, use an ABAC policy, which the docs say is evaluated more efficiently.

```
CREATE OR REPLACE FUNCTION sales_prod.sec.region_filter(region STRING)
RETURN CASE WHEN is_account_group_member('sales-admins') THEN TRUE
             ELSE is_account_group_member(concat('sales-', lower(region))) END;
-- keep lower(): 'EMEA' must still match the group sales-emea
```

EXPECTED RESULT

The profile shows no join to the mapping table, and file pruning matches the same query on an unfiltered copy. Query time moves close to the unfiltered one.

RELATED [7.07](#) A row filter using `current_user` works in a notebook

[7.06](#) The table has a row filter. A copy made with CTAS has none

[C.12](#) Row filters, column masks, dynamic views and ABAC policies

7.09 Lineage shows some tables and misses others. The gaps follow how the code was written.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Lineage is captured for **Unity Catalog** tables read and written through DataFrames or SQL. Path references lose **column lineage**, RDDs and renames lose it entirely, and the system tables keep **365 days**.

Cost of failure **Rework**

Status **GA** · Verified **Oct 2026**

365

Days of history in system.access.table_lineage and column_lineage. Catalog Explorer shows lineage captured since 1 September 2024.

Source: Microsoft Learn, View data lineage using Unity Catalog, Oct 2026

HOW YOU WILL NOTICE

A table has upstream rows only as a path, or none at all. Check what was captured.

```
SELECT source_table_full_name,
       source_path, entity_type,
       event_time
FROM system.access.table_lineage
WHERE target_table_full_name =
      'sales_prod.core.orders'
ORDER BY event_time DESC
LIMIT 20;
```

GUARD

Read and write by table name. Column lineage needs names on both sides.

```
(df.write.mode("append")
 .saveAsTable(
   "sales_prod.core.orders"))
# not .save("abfss://..."):
# table lineage keeps a path,
# column lineage is lost
```

WORKAROUNDS

Move Hive tables to Unity Catalog; nothing in **hive_metastore** is captured. Replace RDD code with DataFrames. Rename rarely, since lineage does not follow a rename. Jobs started with **runs submit** are captured but do not appear as jobs in the graph. Python UDFs keep table lineage but hide the column mapping. Copy the lineage tables to your own table if you need more than a year.

NEIGHBOURING LIMITS

13.3 LTS

Runtime for column lineage in pipeline workloads

11.3 LTS

Runtime for streaming lineage between Delta tables

BROWSE

Minimum privilege on the catalog to see lineage

- RELATED **7.10** An audit needs two years of access history
- 1.07** Another team points its tool at the folder behind a managed table
- 7.06** The table has a row filter. A copy made with CTAS has none

7.10 An audit needs two years of access history. The audit table starts a year ago.

ENGINES JOBS PIPELINES COMPUTE STORAGE DELTA UNITY CATALOG WAREHOUSE SECURITY COST

`system.access.audit` keeps **365 days** at no charge, and it is in **Public Preview**. Anything older exists only if you **archived** it yourself, so start the copy on day one.

Cost of failure **Rework**
Status **Public Preview** · Verified **Oct 2026**

365

Days of audit events kept in `system.access.audit`. Workspace events stay in the workspace region; account events are global.
Source: Microsoft Learn, Monitor account activity with system tables, Oct 2026

HOW YOU WILL NOTICE

The earliest event is about a year old, whatever the auditor asks for.

```
SELECT MIN(event_date)
FROM system.access.audit;

SELECT event_time,
       user_identity.email,
       request_params['full_name_arg']
FROM system.access.audit
WHERE event_date >=
       current_date() - INTERVAL 30 DAYS
       AND action_name = 'getTable';
```

GUARD

Stream the table into your own, with tight grants.

```
(spark.readStream
  .option("skipChangeCommits",
    "true")
  .table("system.access.audit")
  .writeStream
  .option("checkpointLocation",
    ckpt)
  .trigger(availableNow=True)
  .toTable("ops_prod.audit.events"))
```

WORKAROUNDS

Run the copy as a daily job and cluster the archive by `event_date`. Azure diagnostic settings can also send audit logs to a storage account with a lifecycle policy (Premium plan). Databricks advises the system table over exports, since exported logs are yours to secure. Configurable system table retention is in Beta.

NEIGHBOURING LIMITS

365

Days in `system.query.history`

395

Days of free retention once configurable retention (Beta) is on

3,650

Days, the most configurable retention (Beta) allows

RELATED [7.09](#) Lineage shows some tables and misses others [S.19](#) Who read the sensitive table last quarter [8.05](#) Your monitoring queries cost more than the jobs they watch

7.11 A secret showed up in notebook output. Redaction was on.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Redaction replaces only the **literal value**; any transformation prints it in clear.

Secret scope ACLs work **per scope**, while **Unity Catalog secrets** are granted

per secret. The real control is **who can read**.

Cost of failure **Security exposure**Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (A PASSWORD VAULT)

"[REDACTED]" means the value cannot leave.

Access to one secret means access to that secret only."

IN AZURE DATABRICKS

Values read with `dbutils.secrets.get` or a Spark config reference are redacted in notebook output and Spark logs, but not in driver stdout and stderr. Splitting or reversing the string defeats redaction, and the docs say it cannot stop a user with access from seeing the value. Secret scopes grant **READ**, **WRITE** and **MANAGE** on a whole scope, and a Key Vault-backed scope exposes every secret in its vault. Unity Catalog secrets, GA since August 2026, are granted one by one with **READ SECRET**, **WRITE SECRET** and **REFERENCE SECRET**, and every read is audited. Creating one needs **CREATE SECRET** and **USE SCHEMA** on the schema, and a schema holds 100 secrets while a metastore holds 1,000. Notebooks and jobs need Runtime 17.3 LTS or above or serverless environment 4 and above. SQL warehouses cannot read a secret directly, only through a Python UDF declared with a **SECRETS** clause on Pro and serverless warehouses, and dedicated compute cannot call such a UDF.

Do this. Move shared secrets to Unity Catalog secrets and grant **READ SECRET** to job service principals, not to people. Key Vault-backed (external) Unity Catalog secrets are Beta, enabled from the Previews page, so keep Key Vault-backed scopes with one vault each until they are GA. For Azure services, replace the secret with a service credential on an access connector (GA, Runtime 16.2 and above), so there is nothing to print. Review who reads each scope:

```
for s in $(databricks secrets list-scopes -o json | jq -r '(.scopes // .)[].name'); do
  echo "== $s"; databricks secrets list-acls "$s"
done
```

RELATED [7.12](#) The Key Vault secret was rotated [7.13](#) An engineer left. Their personal token still runs 12 production jobs

[C.15](#) Databricks-backed scopes, Key Vault-backed scopes, Unity Catalog secrets and service credentials

7.12 The Key Vault secret was rotated. New runs work and the long-running jobs fail.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A Key Vault-backed scope reads the vault, but a **Spark config or environment variable** reference is resolved at **cluster start**. Clusters and streams started before the rotation keep the **old value** until they restart.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	JOB ON A NEW JOB CLUSTER	CLUSTER OR STREAM STARTED BEFORE ROTATION
<code>dbutils.secrets.get</code> in the run	New value	New value only when called again; a value held in a variable or connection pool stays old
<code>{{secrets/kv-sales/sql-pwd}}</code> in Spark config or env	Resolved at start: new value	Resolved at start: old value
Old password disabled at the database	No effect	Login fails on the next connection
Outcome	Runs succeed	Fails until restarted

FIG. 7.12 The vault holds one new value. Each running process holds whatever it read last.

CAUSE

The docs say it plainly: if a secret referenced in Spark config or an environment variable changes, restart the cluster to fetch it again. Long-running clusters, continuous jobs and streams also hold values their code read at start. The old password was disabled at once, so every holder failed together.

FIX

Rotate with overlap: create the new credential, update the vault, restart the long-running consumers, and only then disable the old one. A job cluster cannot be restarted by hand: cancel the run and start a new one, or pause and resume a continuous job. Read secrets where they are used.

```
def sql_password():
    return dbutils.secrets.get("kv-sales", "sql-pwd") # read per connection

databricks clusters restart 0915-081200-abcd1234 # all-purpose cluster
databricks jobs cancel-run 8841203 # job cluster: then run again
```

PREVENT

Prefer Entra ID authentication through a service credential, so no password needs rotating, or a Unity Catalog secret with per-secret grants. A Key Vault-backed scope needs the vault access policy model, which clashes with RBAC-only landing zone policies. A firewalled vault must allow trusted Microsoft services; a network security perimeter in Secure by perimeter mode blocks even those. Before deleting a scope, search job and cluster configs for its name.

RELATED

[7.11](#) A secret showed up in notebook output

[7.13](#) An engineer left. Their personal token still runs 12 production jobs

[C.15](#) Databricks-backed scopes, Key Vault-backed scopes, Unity Catalog secrets and service credentials

7.13 An engineer left. Their personal token still runs 12 production jobs.

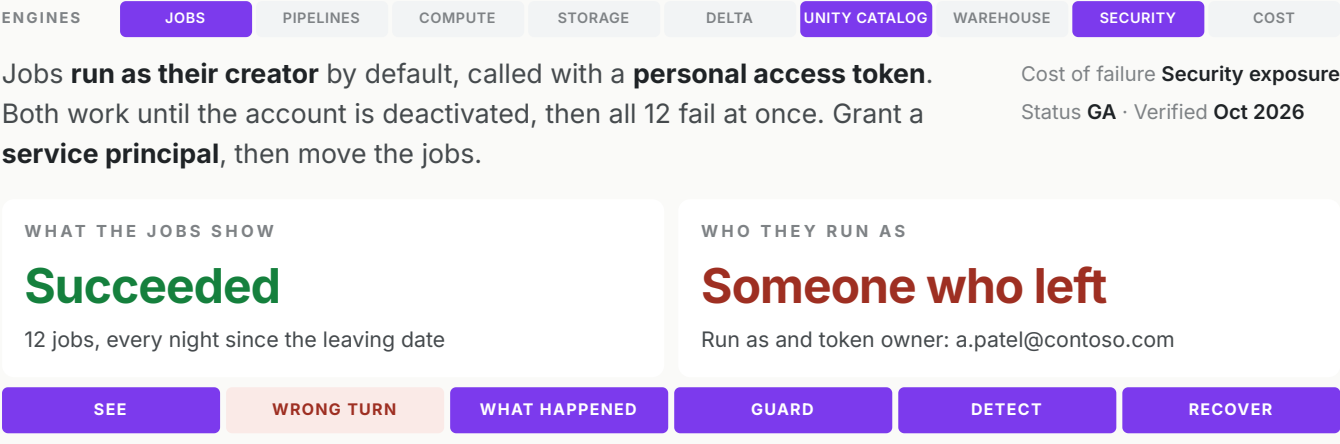


FIG. 7.13 Green runs prove the identity still works, which is the problem.

WRONG TURN

"Offboarding removes their access everywhere, so anything still running is fine."

WHAT HAPPENED

The account stayed enabled for a handover. The token had the 730-day default lifetime and ran nightly, so the 90-day unused rule never fired.

1 GUARD

Grant first, through a group, per the lineage query. Switch one job, check it, then the rest. You need **CAN MANAGE** or ownership of the job and the Service Principal User role on the principal, which needs read on the code.

```
GRANT USE CATALOG ON CATALOG sales_prod TO `jobs-sales-prod`;
GRANT USE SCHEMA, SELECT, MODIFY ON SCHEMA sales_prod.core TO `jobs-sales-prod`;
SELECT DISTINCT source_table_full_name, target_table_full_name
FROM system.access.table_lineage WHERE entity_metadata.job_info.job_id = '4711'
AND event_date >= current_date() - INTERVAL 90 DAYS;
databricks jobs update --json '{"job_id": 4711, "new_settings": {"run_as": {"service_principal_name": "6a1f0c2e-8b4d-4f7a-9c3e-2d5b7e1a9f40"}}}'
```

2 DETECT

Jobs not run as a service principal. Rows from before December 2025 have only **run_as**.

```
SELECT job_id, name,
       coalesce(run_as_user_name, run_as) AS who
FROM system.lakeflow.jobs
QUALIFY ROW_NUMBER() OVER (PARTITION BY workspace_id, job_id ORDER BY change_time DESC) = 1
AND delete_time IS NULL
AND NOT coalesce(run_as_user_name, run_as)
RLIKE '^[0-9a-f-]{36}$';
```

3 RECOVER

Move the scheduler to OAuth M2M. Revoke the person's tokens with **token-management delete** and cap new ones with **maxTokenLifetimeDays**. Never make the principal an admin.

RELATED **9.05** The production deployment runs as the person who deployed it **S.20** Identities for 200 jobs

7.14 You removed someone from an Entra group. They kept access for hours.

ENGINES **JOBS** PIPELINES COMPUTE STORAGE DELTA **UNITY CATALOG** WAREHOUSE **SECURITY** COST

Group membership reaches Databricks on a **sync schedule**, not at once. Cost of failure **Security exposure**
Automatic identity management refreshes on use; **SCIM provisioning** syncs Status **GA** · Verified **Oct 2026**
every 20 to 40 minutes. Grants outside the group stay regardless.

40

Minutes since the last sync before a job or token request waits for fresh group membership under automatic identity management. Between 10 and 40 minutes the refresh runs in the background.
Source: Microsoft Learn, Sync users and groups automatically from Microsoft Entra ID, Oct 2026

HOW YOU WILL NOTICE

Audit rows for the person after the removal time.

```
SELECT event_time, service_name,
       action_name
FROM system.access.audit
WHERE user_identity.email =
      'a.patel@contoso.com'
AND event_time >
      '2026-09-30 14:00:00'
ORDER BY event_time;
```

GUARD

For an urgent removal, act in Databricks too, and revoke tokens.

```
databricks token-management list \
  --created-by-username \
  a.patel@contoso.com
databricks token-management \
  delete <token-id>
```

WORKAROUNDS

Remove the user in the account console when access must end now. With SCIM, Entra schedules each sync after a delay that grows with users and groups; **Clear current state and restart synchronization** forces one. Hours of access usually mean another route: a direct grant, a workspace-local group, or a second group. Automatic identity management syncs nested groups and service principals; SCIM does neither.

NEIGHBOURING LIMITS

5 min

Browser logins refresh membership after this gap

20 to 40 min

SCIM provisioning sync interval

90 days

Unused personal access tokens are revoked

RELATED **7.13** An engineer left. Their personal token still runs 12 production jobs
7.02 The grants look right. Users still get access denied **S.20** Identities for 200 jobs

7.15 The Unity Catalog migration finished. Some tables were silently skipped.

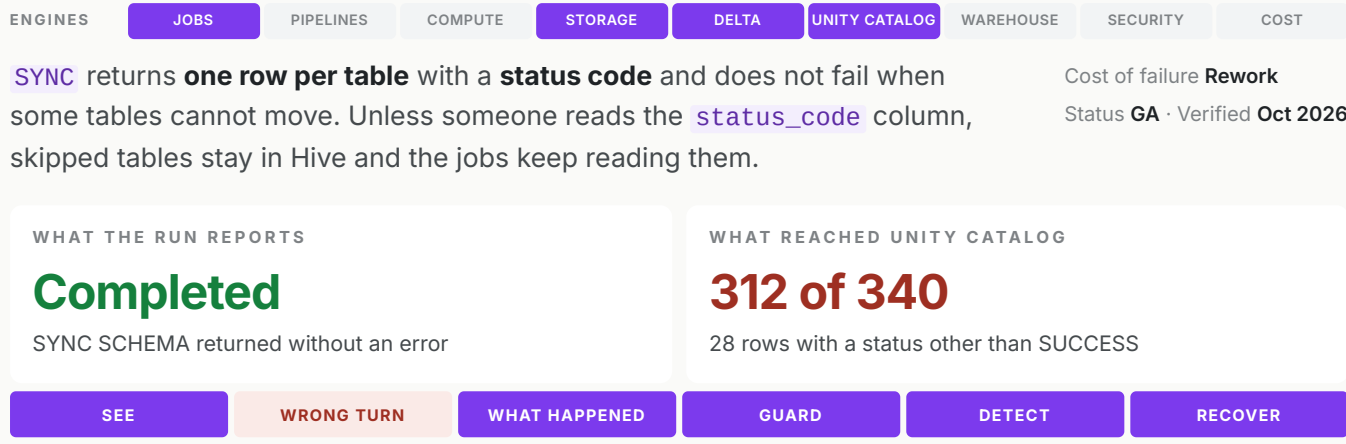


FIG. 7.15 The command succeeded. The result set held the failures.

WRONG TURN

"SYNC SCHEMA either migrates the schema or fails."

WHAT HAPPENED

Managed tables in DBFS root return `DBFS_ROOT_LOCATION`. Managed tables elsewhere return `NOT_EXTERNAL` until you add `AS EXTERNAL`. Hive SerDe tables, views, overlapping paths and names already taken return their own codes, and a table over 600 seconds returns `TIMEOUT`. UCX, a Databricks Labs project without an SLA, migrates DBFS root tables with `DEEP CLONE` but only Delta or Parquet ones.

1 GUARD

Dry run every schema and stop on anything that will not move.

```
res = spark.sql("SYNC SCHEMA sales_prod.core FROM hive_metastore.sales DRY RUN")
bad = res.filter("status_code <> 'DRY_RUN_SUCCESS'")
display(bad.select("source_name", "status_code", "description"))
assert bad.count() == 0, "tables that SYNC will not migrate"
```

2 DETECT

Compare counts on both sides.

```
SHOW TABLES IN hive_metastore.sales;
SELECT COUNT(*)
FROM sales_prod.information_schema
.tables
WHERE table_schema = 'core';
```

3 RECOVER

Per code: `DEEP CLONE` for DBFS root tables, convert SerDe tables to Delta, recreate views from `SHOW CREATE TABLE`, raise `spark.databricks.sync.command.task.timeout`; `AS EXTERNAL` for the `NOT_EXTERNAL` ones. A table synced that way shares its files with the Hive original, so never drop that original (7.03). Repoint jobs, then disable legacy access. Where SYNC cannot move a table, Hive metastore federation (GA) mirrors it in a foreign catalog with Unity Catalog grants and audit while it stays in Hive.

RELATED [1.05](#) The migration moved every table to Unity Catalog [7.03](#) You revoked a user's access

[S.05](#) Hive metastore to Unity Catalog

7.16 A customer asked to be erased. The rows are gone from Gold and still exist in five other places.

ENGINES JOBS PIPELINES COMPUTE **STORAGE** DELTA UNITY CATALOG WAREHOUSE **SECURITY** COST

A **DELETE** on one table hides rows from queries. The data still sits in **raw files**, upstream tables, old **Parquet files** until **REORG** and **VACUUM**, storage **soft delete**, and logs that captured it.

Cost of failure **Security exposure**
Status **GA** · Verified **Oct 2026**

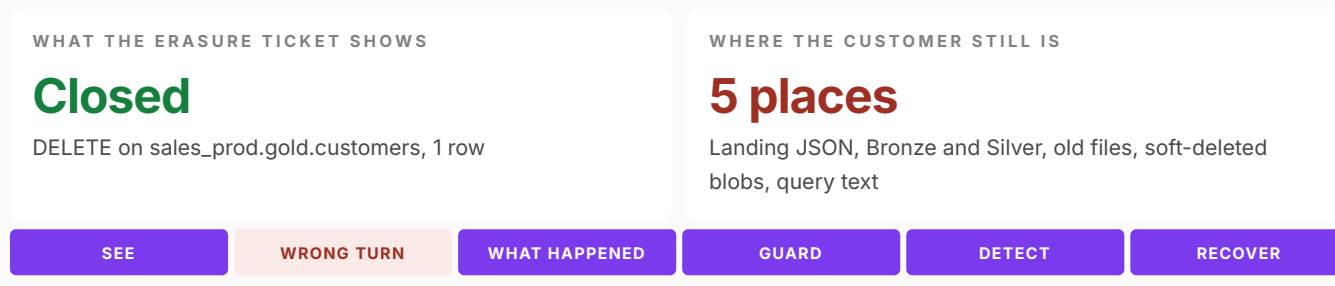


FIG. 7.16 The table people query is clean. The lake behind it is not.

WRONG TURN

"The customer is gone from the Gold table, so the customer is erased."

WHAT HAPPENED

Gold comes from Silver, Silver from Bronze, Bronze from JSON in a landing volume; none was touched. With deletion vectors, **DELETE** only marks rows. Old files stay readable by time travel until **VACUUM**, change data feed files and a test deep clone hold the rows too, and blob soft delete keeps removed files. Query history and the audit archive hold an email typed into a **WHERE** clause.

1 GUARD

One job, driven by a request table, from the source down: raw files, Bronze, Silver, lookup tables, Gold. Streams reading them need **skipChangeCommits**; materialized views handle deletes on refresh.

```

MERGE INTO sales_prod.bronze.customers t
USING (SELECT customer_id FROM ops_prod.privacy.erasure_requests WHERE status = 'OPEN') r
ON t.customer_id = r.customer_id WHEN MATCHED THEN DELETE;
REORG TABLE sales_prod.bronze.customers APPLY (PURGE); -- rewrites files holding them
VACUUM sales_prod.bronze.customers; -- 7 days later: old files leave storage

```

2 DETECT

Find every table that can hold the key, then count the key in each.

```

SELECT table_catalog,
       table_schema, table_name
FROM system.information_schema
     .columns
WHERE column_name IN
      ('customer_id', 'email');

```

3 RECOVER

Set blob soft delete shorter than the erasure deadline; it cannot be cut short once a file is deleted. Delete the key from deep clones and exports. Statement text stays stored in **system.query.history**; it shows as redacted except to account admins and **databricks_pii_access**, so restrict both and **system.query**. Purge your own audit archive. The data protection owner signs the checklist.

RELATED **3.05** You deleted rows and ran **VACUUM** **3.10** A Change Data Feed reader missed changes
S.17 Patient records from four systems

7.17 A row filter or mask blocks your own recovery, clone and sharing jobs. The data owner cannot test the policy either.

ENGINESJOBSPIPELINESCOMPUTEDELTAUNITY CATALOGWAREHOUSESECURITYCOST

Clones, **OpenSharing** and **time travel** fail on filtered tables unless the identity is exempt, and a table-level filter has no exemption. Use **ABAC policies** with **EXCEPT** one group of **service principals**, and keep people out of it.

Cost of failure **Data loss**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Table-level row filter or mask	One table, one function, set in one statement.	Clones, time travel and sharing do not work and nobody can be exempted. A restore or DR copy means removing the filter first.
ABAC policy, people in EXCEPT	An admin can fix a table fast.	The group grows, and its members see every row and write unfiltered CTAS copies (7.06). A policy takes 20 principals.
ABAC policy, service principals in EXCEPT DEFAULT	Clone, share, restore and DR jobs run as named principals. People ask for a time-boxed exemption.	The group, policies and tags need owners and a review.

FIG. 7.17 Recovery tools read the whole table. The policy has to know which identities may.

DEFAULT

Policies, not table-level filters, on tables you may clone, share or restore. One account group, **policy-exempt-sp**, in **EXCEPT**: the clone and DR principal, the share owner, and the owner of each pipeline over the table. Time travel under ABAC is Beta for qualifying tables and compute: test restore first.

WHO OWNS WHAT

A security group holds **MANAGE** and owns the policies and the exempt group, so a data owner cannot exempt themselves. A governed tag needs **ASSIGN** on the tag and **APPLY TAG** on the object, and whoever changes a tag changes which policy applies. Give **ASSIGN** to the governance group only. Test in a test catalog first, as the exempt principal and as a reader.

CHECK IT

Review the **EXCEPT** lists quarterly and these changes weekly.

SIGNS YOU CHOSE WRONG

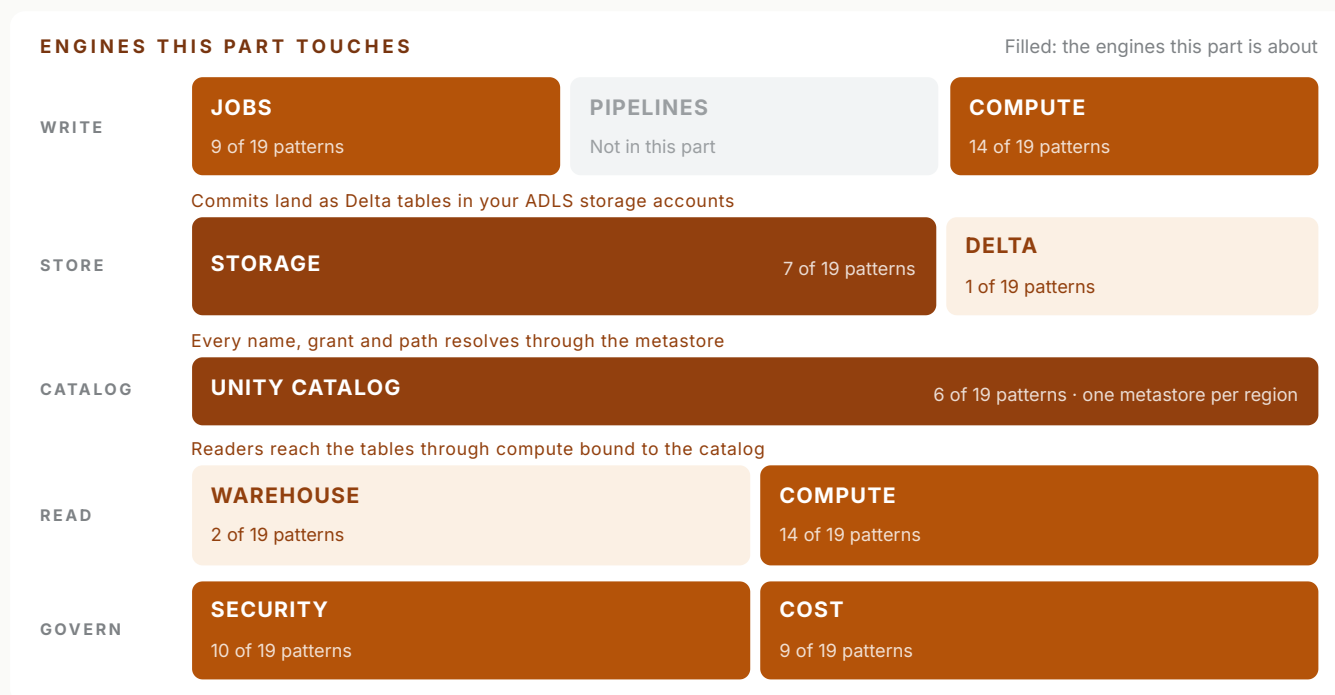
A DR clone fails when you need it. Someone drops a policy to run it and forgets. A share fails because its owner is a person. The exempt group outgrows the readers.

```
SELECT event_time, user_identity.email, action_name, request_params
FROM system.access.audit
WHERE service_name = 'unityCatalog' AND event_date >= current_date() - INTERVAL 7 DAYS
  AND action_name IN ('createPolicy', 'updatePolicy', 'deletePolicy',
'updateEntityTagAssignment');
```

RELATED7.06 The table has a row filter. A copy made with CTAS has none8.09 Leadership asks what happens if West Europe goes down7.08 Row filters slow every query on the table

08 Platform, Networking and Cost.

How the platform holds up and what it costs: regions and disaster recovery, outbound routes, storage firewalls and private DNS, vCPU quota, admin roles, billing system tables, serverless against classic and budgets. Most of these failures are set up on the day a workspace is created and found on the day something else changes.



NO.	PATTERN	JOBS	COMPUTE	STORAGE	UC	SEC	COST	PAGE
8.01	The Databricks bill doubled. Nobody can say which jobs caused it.	●	●	●	●	●	●	135
8.02	You moved to serverless to save money. Small, frequent queries now cost more.	●	●	●	●	●	●	136
8.03	Photon made the job faster. The job now costs more.	●	●	●	●	●	●	137
8.04	One team used 70 percent of the spend. You found out at month end.	●	●	●	●	●	●	138
8.05	Your monitoring queries over system tables cost more than the jobs they watch.	●	●	●	●	●	●	139
8.06	Every serverless job starts fast. The nightly batch has hours to spare.	●	●	●	●	●	●	140
8.07	The Databricks usage report says one figure. The Azure bill says another.	●	●	●	●	●	●	141
8.08	The storage account is geo-redundant. You assume the platform survives a region outage.	●	●	●	●	●	●	142
8.09	Leadership asks what happens if West Europe goes down. Nobody has an RPO.	●	●	●	●	●	●	143
8.10	Clusters in a new workspace cannot reach PyPI or Maven. The old workspace never had this problem.	●	●	●	●	●	●	144

NO.	PATTERN	JOB	COMPUTE	STORAGE	UC	SEC	COST	PAGE
8.11	The storage firewall admits the workspace subnets. Serverless jobs on the same data get 403.	●	●	●	●	●	●	145
8.12	You added a private endpoint rule for serverless. It has said PENDING for two days.	●	●	●	●	●	●	146
8.13	The private endpoint exists. Queries still go over the public internet, or fail.	●	●	●	●	●	●	147
8.14	At 9am clusters fail to start with a quota error. Yesterday the same jobs ran.	●	●	●	●	●	●	148
8.15	The only account admin left. Nobody can add users, create a metastore or change account settings.	●	●	●	●	●	●	149
8.16	Anyone on the internet can reach the workspace login page. Security wants it closed without locking out the admins.	●	●	●	●	●	●	150
8.17	Auditors ask who holds the encryption keys for your data and your notebooks. The answer is Microsoft.	●	●	●	●	●	●	151
8.18	A serverless notebook reaches the internet. The same code fails as a production job.	●	●	●	●	●	●	152
8.19	One workspace for everything, or one per team? Nobody agrees, and the metastore is shared anyway.	●	●	●	●	●	●	153

8.01 The Databricks bill doubled. Nobody can say which jobs caused it.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Spend splits only along what **system.billing.usage** records: **custom tags** and workload IDs. The tags were set on some clusters and never reached serverless, and the untagged share is the part that grew.

Cost of failure **Cost**Status **Public Preview** · Verified **Oct 2026**

WHAT THE TAG AUDIT SHOWS

Every cluster tagged

team and cost_center set on all classic compute

WHAT THE USAGE HOLDS

38 percent untagged

Serverless jobs and notebooks, clusters retagged while running

SEE

WRONG TURN

WHAT HAPPENED

GUARD

DETECT

RECOVER

FIG. 8.01 In this example the audit covers clusters. The bill also covers compute that has no cluster to tag.

WRONG TURN

"Tag every cluster and the bill splits itself by team."

WHAT HAPPENED

A cluster tag applies only after a restart, so long-running clusters kept billing without it. Serverless notebooks, jobs and pipelines have no cluster: their usage carries custom tags only through a serverless usage policy, in Public Preview. Clusters created from a pool pass only workspace and pool tags to their Azure VMs.

1 GUARD

Set tags where nobody can skip them: a compute policy on classic compute (the finance team's below), job tags (they propagate to job clusters), a usage policy per team for serverless.

```
{ "custom_tags.team": { "type": "fixed", "value": "finance" } }
```

2 DETECT

Price 30 days by tag. Null is untagged.

```
SELECT u.custom_tags['team'] AS team,
       u.billing_origin_product AS product,
       SUM(u.usage_quantity *
           p.pricing.effective_list.default) AS usd
FROM system.billing.usage u
JOIN system.billing.list_prices p
  ON u.sku_name = p.sku_name
 AND u.usage_start_time BETWEEN
     p.price_start_time AND
     coalesce(p.price_end_time, now())
WHERE u.usage_date >= current_date() - 30
GROUP BY ALL ORDER BY usd DESC;
```

3 RECOVER

Attribute past spend through `usage_metadata.job_id` and `identity_metadata.run_as`, joined to `system.lakeflow.jobs` for names and owners. Then tag at source and restart long-running clusters.

`list_prices` holds list prices, not your discount. Records arrive within about 12 hours.

RELATED **8.04** One team used 70 percent of the spend **8.07** The Databricks usage report says one figure

4.06 A data scientist started a 100-node cluster

8.02 You moved to serverless to save money. Small, frequent queries now cost more.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Serverless charges a higher **DBU rate** that already includes the VMs, and bills until **auto stop**. Work that arrives every few minutes keeps it running all day; it saves where work comes in **bursts with idle gaps**.

Cost of failure **Cost**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Classic everywhere	Long, steady work on compute sized for it. VMs and DBUs sit on separate meters, so Azure VM discounts apply to the VM half.	Idle time. A classic cluster or warehouse bills VMs and DBUs until it stops. Pro and classic SQL warehouses default to 45 minutes of auto stop, minimum 10.
Serverless everywhere	No VMs to size or forget, quick starts, Photon always on. Serverless SQL warehouses default to 10 minutes of auto stop.	A dashboard refreshes every 5 minutes, so the warehouse never reaches auto stop and bills all day at the serverless rate. Long steady jobs pay that rate every hour.
By workload shape DEFAULT	Serverless for interactive, bursty and short scheduled work, with auto stop set below the usual gap: 5 minutes in the UI, as low as 1 through the API. Long steady batch on classic job compute or serverless standard mode.	Needs a monthly look at cost per workload rather than per product, and someone who owns the refresh schedules.

FIG. 8.02 Classic bills a lower DBU rate plus the VMs. Serverless bills one higher rate, plus networking items, and nothing once stopped.

DEFAULT

Choose by how the work arrives. Serverless where it is bursty, with auto stop tight; classic or standard mode where it runs for hours at a steady load.

SIGNS YOU CHOSE WRONG

A serverless warehouse bills 24 hours a day for a few dozen small queries. Serverless jobs run for hours at full load every night. Nobody knows the refresh schedule of the dashboards on a warehouse.

CHECK IT

Count the hours each serverless warehouse billed per day. A warehouse used for small queries should not bill close to 24.

```
SELECT usage_metadata.warehouse_id, usage_date,
       SUM(usage_quantity) AS dbu,
       COUNT(DISTINCT date_trunc('HOUR', usage_start_time)) AS hours_billed
FROM system.billing.usage
WHERE billing_origin_product = 'SQL' AND product_features.is_serverless
      AND usage_date >= current_date() - INTERVAL 14 DAYS
GROUP BY ALL ORDER BY hours_billed DESC;
```

RELATED

[8.06](#) Every serverless job starts fast

[6.02](#) Auto stop is 10 minutes

[4.02](#) You run jobs on an all-purpose cluster to skip cold starts

8.03 Photon made the job faster. The job now costs more.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

On classic compute, Photon **raises the DBU count** for the same VM. A job saves money only when the **speed-up** outweighs that uplift, and that is settled per job, by measuring.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (SQL WAREHOUSES AND SERVERLESS)

"Photon is always on there and runs queries faster. Faster runs mean a smaller bill, so it pays for itself everywhere."

ON CLASSIC JOBS AND ALL-PURPOSE COMPUTE

Photon is on by default for new classic compute and can be turned off. Turning it on increases the DBU count per VM hour; the VM price stays the same. Queries that already finish in under two seconds gain little, and code paths Photon does not support, such as RDD code, fall back to the standard engine while the whole cluster bills at the Photon count. Prepaid DBCU also drain faster, because they are deducted per DBU.

Do this. Decide per job. Replay one fixed input: pin the source tables to a Delta version or a dated snapshot, run the job on it twice with the same cluster size, once with Photon and once without, and repeat three times. Compare DBUs per run in `system.billing.usage`, split by `product_features.is_photon`, and add the VM hours from Azure. Two different weeks of live data differ in volume and skew, so they prove nothing. Keep Photon where cost per run falls. Long scans, joins and aggregations usually win; short tasks and Python-heavy code often do not. Set `runtime_engine` to `PHOTON` or `STANDARD` explicitly in the job definition so the choice survives the next template change.

RELATED **8.02** You moved to serverless to save money **4.03** Autoscaling added workers and the job got slower

8.07 The Databricks usage report says one figure

8.04 One team used 70 percent of the spend. You found out at month end.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

One account-wide figure hides a single team's growth. **Budgets** scoped by **tag** email at up to four thresholds, and **compute policies** cap what a team can start. Neither stops spend once it is running, and each needs a named **owner**.

Cost of failure **Cost**Status **Public Preview** · Verified **Oct 2026**

WHAT THE ACCOUNT BUDGET SHOWS

No alert

One budget for the account, threshold not reached until day 26

WHAT THE USAGE HOLDS

70 percent, one team

Large clusters at maximum size since day 3

SEE

WRONG TURN

WHAT HAPPENED

GUARD

DETECT

RECOVER

FIG. 8.04 In this example the total stayed plausible while one team's share grew all month.

WRONG TURN

"A monthly cost review is soon enough, and a budget alert will stop the overspend."

WHAT HAPPENED

One account budget, one threshold. Budgets use list price in USD, can lag by up to 24 hours and do not stop compute; **Block usage** covers Unity Gateway only. The clusters had no size limit.

1 GUARD

Give each team a budget on its **team** tag (budgets are GA), alerting at 50, 80 and 100 percent. Name an owner per budget and per cap, who answers the alert. Cap compute by policy, with auto termination above 0. Exception route: a time-boxed cap raise, approved by the team's cost owner and the platform owner, reviewed at expiry.

```
{ "custom_tags.team": { "type": "fixed", "value": "marketing" },
  "dbus_per_hour": { "type": "range", "maxValue": 40 },
  "autotermination_minutes": { "type": "range",
    "minValue": 10, "maxValue": 60, "defaultValue": 30 } }
```

2 DETECT

Week-to-date DBUs per team, as a scheduled SQL alert.

```
SELECT custom_tags['team'] AS team,
       SUM(usage_quantity) AS dbu
FROM system.billing.usage
WHERE usage_unit = 'DBU'
      AND usage_date >=
      date_trunc('WEEK', current_date())
GROUP BY ALL ORDER BY dbu DESC;
```

3 RECOVER

Shrink the clusters behind the spike and move the team onto its policy. Tag its serverless work with a serverless usage policy (Public Preview). Send the weekly figure to the owner too.

RELATED [8.01 The Databricks bill doubled](#) [4.06 A data scientist started a 100-node cluster](#)

[4.07 The cluster policy is set. Users still create clusters that ignore it](#)

8.05 Your monitoring queries over system tables cost more than the jobs they watch.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

System tables are tables, read by **your compute** at its rate. Billing records land within about **12 hours** and job timelines within an hour, so a five-minute refresh pays to read the **same rows** again.

Cost of failure **Cost**
Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (AZURE MONITOR AND BUILT-IN METRICS)

"Monitoring data is free to query and fresh to the minute, so refresh the dashboard as often as you like."

WITH DATABRICKS SYSTEM TABLES

Queries on `system.billing` and `system.lakeflow` run on whatever warehouse or cluster you point at them, at its usual rate. `system.billing.usage` records usually arrive within 12 hours; job timeline records within an hour of the period end. A dashboard that refreshes every five minutes, scans a year of usage and keeps a large warehouse from reaching auto stop can cost more than many of the jobs it reports on.

Do this. Run monitoring on one small serverless SQL warehouse with a short auto stop. Filter on `usage_date` and `period_start_time` so each query reads days, not the full year. Refresh job health hourly and cost daily. Write a daily summary table and point dashboards at it, not at the raw tables. Then watch the watcher: filter `usage_metadata.warehouse_id` to the monitoring warehouse and check what it costs each month.

RELATED

9.09 You run 200 jobs

8.02 You moved to serverless to save money

6.03 One warehouse serves everyone

8.06 Every serverless job starts fast. The nightly batch has hours to spare.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Serverless jobs run in **performance optimised** mode unless you say otherwise. Cost of failure **Cost**

Standard mode uses the same SKU and fewer DBUs, in exchange for **4 to 6 minutes** more startup latency. Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Performance optimised for every job	Fast start and fast run. Right for jobs with a tight deadline, jobs that users trigger and wait for, and short frequent runs.	Applied to everything, including batch that finishes hours before anyone reads the result. Those runs pay for speed nobody uses.
Classic job compute for batch	Long, steady jobs where you choose the VM size, Spot and the runtime.	Cluster sizing, policies and startup time come back as your work. Short jobs pay a cluster start every run.
Standard mode for batch with slack	Nightly and hourly batch whose deadline is well after its usual finish. Same SKU, fewer DBUs, no clusters to manage.	The downstream deadline is tight, or a frequent schedule stacks the extra 4 to 6 minutes of start latency into a backlog.
DEFAULT		

FIG. 8.06 The setting is per job and applies only to its serverless tasks. Decide it from the deadline, not the default.

DEFAULT

Standard mode for any serverless job with slack before its deadline. Performance optimised only where someone waits for the result. Set it explicitly in the bundle.

SIGNS YOU CHOSE WRONG

Every serverless DBU in the bill is **PERFORMANCE_OPTIMIZED**. Nightly jobs finish at 02:10 for a report read at 09:00. Nobody can say which jobs have a deadline.

CHECK IT

Set the mode on the job, then see where serverless DBUs go by mode over the last 30 days.

```
resources:
  jobs:
    orders_nightly:
      name: orders_nightly
      performance_target: STANDARD
      # job level, not task level
```

```
SELECT product_features.performance_target,
       billing_origin_product,
       SUM(usage_quantity) AS dbu
FROM system.billing.usage
WHERE product_features.is_serverless
      AND billing_origin_product
      IN ('JOBS', 'DLT')
      AND usage_date >=
      current_date() - INTERVAL 30 DAYS
GROUP BY ALL ORDER BY dbu DESC;
```

RELATED 8.02 You moved to serverless to save money 5.13 The job is scheduled every 15 minutes

4.01 A job runs every five minutes and starts a new cluster each time

8.07 The Databricks usage report says one figure. The Azure bill says another.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The usage system table prices **DBUs at list**. The Azure invoice adds the **VMs, disks and networking** behind classic compute and applies your discounts and prepurchase, so the two never match, and should not.

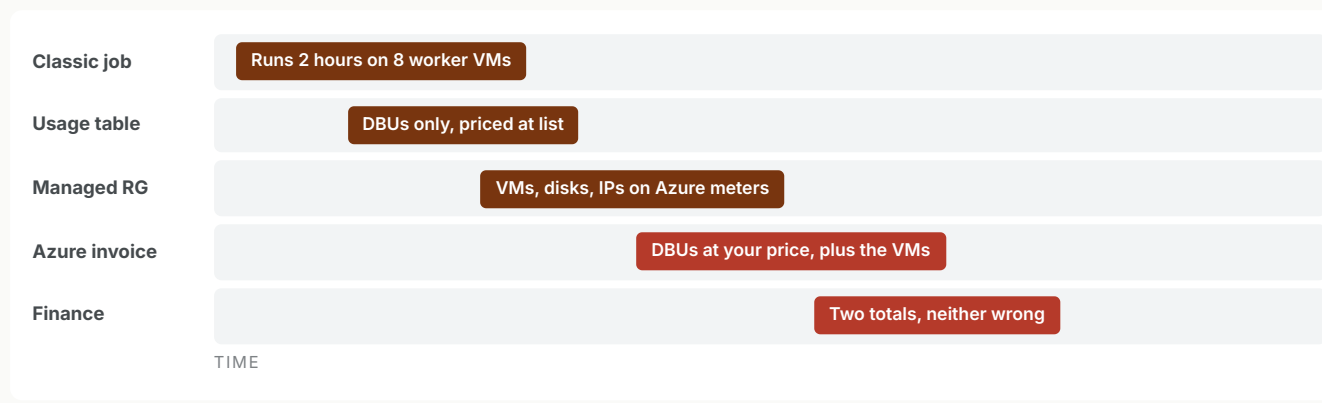
Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

FIG. 8.07 One run produces two kinds of charge, recorded in two places at two kinds of price.

SEEN IN

Finance's month-end report: the usage dashboard and Azure Cost Management disagree

CAUSE LIVES IN

What each side counts: DBUs at list price, or every Azure meter at your price

CAUSE

Classic compute bills twice: DBUs for Databricks, plus VMs, managed disks and networking for the nodes in the workspace's managed resource group. `system.billing.list_prices` holds list prices, not discounts. Pre-purchased DBCU cover DBU usage only. Serverless DBUs include the compute, but data transfer, NAT gateway and private endpoint usage are billed on top.

FIX

Reconcile in two halves. Compare DBU quantities, not money, between the usage table and the Azure Databricks meters in Cost Management. Then take infrastructure from Cost Management, filtered to the managed resource group and grouped by the `ClusterId` or `JobId` tag Databricks puts on each VM. Report cost per job as both halves added together.

CONFIRM IN BOTH ENGINES

```
-- DBUs per day, one workspace
SELECT usage_date,
       SUM(usage_quantity) AS dbu
FROM system.billing.usage
WHERE workspace_id = '1234567890'
      AND usage_unit = 'DBU'
GROUP BY ALL ORDER BY usage_date;
```

```
# Azure Cost Management, cost analysis
# scope: the workspace subscription
Filter  Resource group = databricks-rg-*
Group by Tag: JobId
Metric  Amortised cost, daily
```

Clusters created from a pool tag their VMs with workspace and pool tags only, so group those by pool.

RELATED [8.01](#) The Databricks bill doubled [8.03](#) Photon made the job faster

[4.09](#) Your pool keeps instances idle. Jobs still wait

8.08 The storage account is geo-redundant. You assume the platform survives a region outage.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Geo-redundant storage copies files to another region, asynchronously. It does not copy the **workspace** or the **metastore**: notebooks, jobs, clusters, tables and grants live in a regional control plane and metastore.

Cost of failure

Outage

Status

GA

Verified

Oct 2026

YOU'D EXPECT (VMS, DATABASES AND STORAGE ON AZURE)

"The data is on GRS, so it already sits in the paired region. If West Europe fails, we fail over the storage accounts, point the platform at them and carry on."

IN AZURE DATABRICKS

The control plane survives a zone failure and recovers within about 15 minutes, but a workspace and its metastore belong to one region. Jobs, notebooks, cluster and warehouse definitions, secrets, Unity Catalog metadata and grants are not in your storage account, so GRS never copies them. Microsoft's DR guidance says not to rely on geo-redundant storage to duplicate the workspace's root storage across regions. A failed-over storage account is files with no workspace, catalog or jobs to read them. DR also copies corruption: a bad write in the primary is replicated to the secondary.

Do this. Treat GRS as a copy of files, nothing more. Write a DR plan with a secondary workspace and metastore in another region, deploy jobs, clusters, grants and secrets there as code, and copy tables by tier with deep clone or managed disaster recovery (8.09). Keep time travel and backups for corruption, which DR does not fix; time travel, **RESTORE** and clones do not apply to tables with row filters or column masks, or to materialized views. Then run a failover test on a schedule: an untested plan often fails when it is needed.

RELATED

8.09 Leadership asks what happens if West Europe goes down

3.11 You restored a table to last week

1.08 Jobs that read one storage account run far slower than the rest

8.09 Leadership asks what happens if West Europe goes down. Nobody has an RPO.

ENGINES

JOBSS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

DR starts with an **RPO and RTO per tier** of data, then a secondary workspace and metastore filled by **infrastructure as code** and **deep clone**, or by managed disaster recovery, which is gated and does not cover everything.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Rebuild after the outage	No second region to pay for. Code is in Git; data is reloaded from sources.	RTO is days, RPO depends on source retention. Nobody has practised the rebuild.
Managed disaster recovery	Continuous replication of Unity Catalog managed table data, external table and volume metadata, views, functions and grants, plus notebooks, jobs, warehouses, clusters and draft dashboards. Failover in minutes.	Gated Preview, by application. Needs Premium, the Mission Critical add-on and serverless on both workspaces; billed at that rate. Not replicated: managed volume data, materialized views, streaming tables, pipelines, secrets, models, serving endpoints, vector indexes, published dashboards, shares. Tables with row filters, masks or ABAC tags fail to replicate (7.17).
By tier, as code plus deep clone DEFAULT	Jobs, clusters, grants and secrets deployed to both regions as code, secondary schedules paused. Tier 1 tables deep cloned to secondary-region storage on a schedule that meets their RPO, registered in the secondary metastore; the rest rebuilt from source.	Clone jobs need an owner. Clones fail on tables with row filters, column masks or ABAC policies unless the job's identity is exempt (7.17). Materialized views and streaming tables are rebuilt, not copied.

FIG. 8.09 The RPO of a cloned table is the age of its last clone. Agree it per tier before choosing the tool.

DEFAULT

RPO and RTO per tier, signed off. Metadata as code in both regions, tier 1 clones on schedule and registered as external tables in the secondary metastore, a failover test on the calendar.

SIGNS YOU CHOSE WRONG

The plan names GRS as the DR copy. A second region reads tier 1 through OpenSharing, which reads the primary's storage. Nobody has tested the secondary.

CHECK IT

A clone registered only in the primary metastore is unreadable in an outage. Register the clone path in the secondary metastore and read the last clone's age there: the real RPO.

```
-- primary metastore, scheduled clone job
CREATE OR REPLACE TABLE sales_prod.dr.orders
DEEP CLONE sales_prod.core.orders
LOCATION 'abfss://dr@stlakeneu.dfs.core.windows.net/orders';
-- secondary metastore, once; a rerun copies only changes
CREATE TABLE sales_dr.core.orders
LOCATION 'abfss://dr@stlakeneu.dfs.core.windows.net/orders';
DESCRIBE HISTORY sales_dr.core.orders LIMIT 1;
-- timestamp of the last CLONE = data you would lose today
```

RELATED

8.08 The storage account is geo-redundant

1.08 Jobs that read one storage account run far slower than the rest

9.11 At 02:00 on-call must pause a production job

8.10 Clusters in a new workspace cannot reach PyPI or Maven. The old workspace never had this problem.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Since **31 March 2026**, new Azure virtual networks have no **default outbound access**. A VNet-injected workspace with secure cluster connectivity needs an explicit way out: a **NAT gateway** or firewall routing on both subnets.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	OLD WORKSPACE, VNET FROM 2024	NEW WORKSPACE, VNET FROM MAY 2026
Cluster nodes	No public IPs (secure cluster connectivity)	No public IPs (secure cluster connectivity)
Route to the internet	Azure default outbound access, on an IP you do not own	None until you add one
pip and Maven installs	Work	Time out
Outcome	Jobs run	Library installs fail; cluster functionality is not assured

FIG. 8.10 The workspace template is the same. The VNet's creation date decides whether there is a way out.

CAUSE

Microsoft retired default outbound access for new VNets after 31 March 2026. Existing workspaces are not affected. A workspace with the default managed VNet gets a NAT gateway that Databricks creates in the managed resource group, at extra cost. With VNet injection the egress is yours to build, and an egress load balancer is the wrong choice: Databricks says not to use one with secure cluster connectivity because of port exhaustion risk.

FIX

Attach a NAT gateway to both workspace subnets. It also gives a stable egress IP for partner allow lists. If policy sends all traffic through a firewall, add a route table on both subnets instead and allow the Databricks service tags and package hosts.

```
az network public-ip create -g rg-net -n pip-dbw-egress --sku Standard
az network nat gateway create -g rg-net -n ng-dbw --public-ip-addresses pip-dbw-egress
az network vnet subnet update -g rg-net --vnet-name vnet-dbw -n snet-dbw-public \
  --nat-gateway ng-dbw
az network vnet subnet update -g rg-net --vnet-name vnet-dbw -n snet-dbw-private \
  --nat-gateway ng-dbw
```

PREVENT

Put the NAT gateway or route table in the same template as the VNet, and test a library install before handing a new workspace over. Record the egress IP wherever partners allow-list it. A new region or a rebuilt VNet is a new VNet, with no way out by default.

RELATED

8.11 The storage firewall admits the workspace subnets

8.13 The private endpoint exists, but queries still go public

S.18 Every source behind a firewall

8.11 The storage firewall admits the workspace subnets. Serverless jobs on the same data get 403.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Serverless compute runs in your **Databricks account**, not in your VNet, so subnet rules never match it. Admit it with a **network security perimeter** rule for the **AzureDatabricksServerless** service tag, in **Transition** mode, or an NCC private endpoint.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	CLASSIC JOB CLUSTER, VNET INJECTED	SERVERLESS JOB
Runs in	Your subnets, in your subscription	The serverless compute plane, in your Databricks account
What admits it	Virtual network rules for both workspace subnets	NSP inbound rule for the service tag, or an NCC private endpoint
Serverless subnet IDs in the firewall	Not needed	No longer accepted
Outcome	Reads succeed	403 Forbidden

FIG. 8.11 The firewall checks where a request comes from. Serverless comes from somewhere your subnet rules do not describe.

CAUSE

The firewall lists the classic subnets only. Accounts that allow-listed serverless subnet IDs can no longer connect either: Databricks moved serverless storage access to network security perimeters. Two settings break it after the rule exists: public network access set to **Secured by Perimeter** gives **PERMISSION_DENIED**, and **Enforced** access mode bypasses the storage firewall and blocks every service without a perimeter rule.

FIX

For same-region ADLS, create a perimeter in the workspace region, associate the storage account in **Transition** mode, and add an inbound rule for **AzureDatabricksServerless.WestEurope**. Keep public access on **Enabled from selected networks**. Traffic stays on service endpoints, uncharged. Use NCC private endpoints for accounts with public access disabled, and for databases; each bills per hour. Then confirm from serverless:

```
-- serverless notebook or SQL warehouse, after the rule is in place
LIST 'abfss://raw@stlakeprod.dfs.core.windows.net/orders/' LIMIT 5;
```

PREVENT

Before moving jobs to serverless, list every storage account they read and its firewall settings, and add the perimeter rule first. The storage owner and the network security lead approve each association. Stay in **Transition**, which Databricks recommends; it checks perimeter rules first, then the old firewall. Move to **Enforced** only after perimeter diagnostic logs show every other service is covered; roll back with Change access mode. The perimeter is GA; firewall support for workspace storage, which onboards it for you, is in preview.

- RELATED
- 8.12 Your serverless private endpoint has said PENDING for two days

1.06 Your notebook reads abfss:// paths on the old cluster

S.18 Every source behind a firewall

8.12 You added a private endpoint rule for serverless. It has said PENDING for two days.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Databricks creates the private endpoint, but the **owner of the target resource** must approve it. A rule left **PENDING** expires after **14 days**, and Azure bills it every hour it exists.

Cost of failure **Outage**Status **GA** · Verified **Oct 2026**

14

Days a private endpoint rule can stay PENDING, REJECTED or DISCONNECTED before it becomes EXPIRED. Only an approval on the target resource moves it to ESTABLISHED.

Source: Microsoft Learn, Manage private endpoint rules, Oct 2026

FIG. 8.12 The NCC asks for the connection. The resource owner grants it.

HOW YOU WILL NOTICE

Serverless still gets 403 from a storage account with public access disabled. The rule's state in the NCC never changes.

```
databricks account network-connectivity \
  list-private-endpoint-rules ncc-abc123
# connection_state: PENDING
```

GUARD

Whoever owns the resource approves it in the portal under Networking, Private endpoint connections, or by CLI.

```
az network private-endpoint-connection \
  list --id "$STORAGE_ID" -o table
az network private-endpoint-connection \
  approve --id "$CONNECTION_ID" \
  --description "Databricks NCC"
```

WORKAROUNDS

Raise the approval with the resource owner when you add the rule, not after. An EXPIRED rule is deleted and created again. After ESTABLISHED, wait for propagation, usually within 10 minutes but up to 24 hours, and restart running serverless resources. Delete rules nobody uses: Azure bills each one per hour in every state.

NEIGHBOURING LIMITS

10

NCCs per region in one Databricks account

100

Private endpoints per region, across those NCCs

50

Workspaces attached to one NCC

RELATED **8.11** The storage firewall admits the workspace subnets **8.13** The private endpoint exists, but queries still go public

S.18 Every source behind a firewall

8.13 The private endpoint exists. Queries still go over the public internet, or fail.

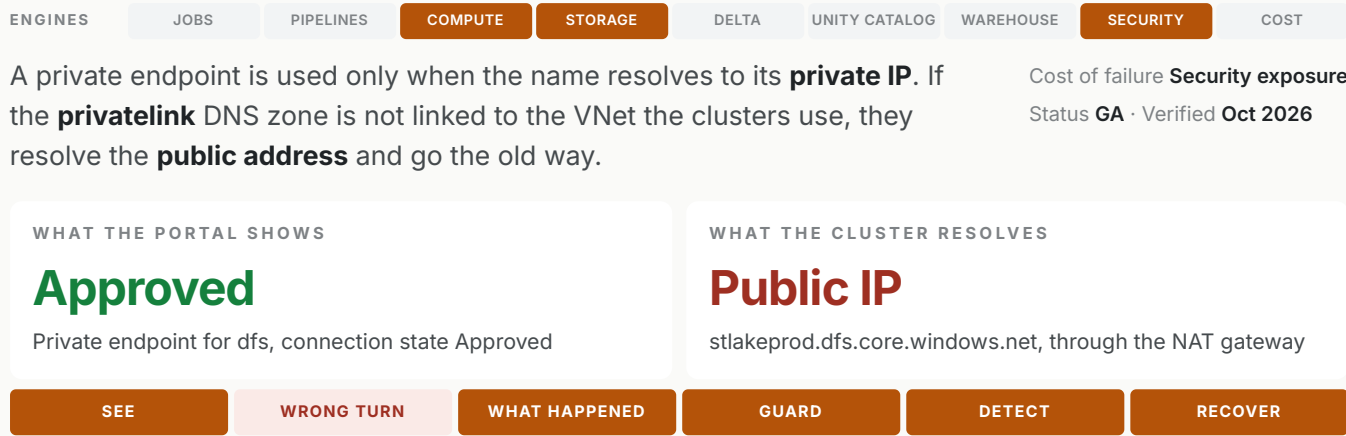


FIG. 8.13 The endpoint is healthy. Nothing sends traffic to it.

WRONG TURN

"The private endpoint is approved, so storage traffic now stays private."

WHAT HAPPENED

Public DNS answers with a CNAME to `stlakeprod.privatelink.dfs.core.windows.net`. Only a VNet linked to the `privatelink.dfs.core.windows.net` zone, or a DNS server that forwards to Azure DNS at `168.63.129.16` from a linked VNet, gets the private IP. The zone was linked to the hub, and the workspace VNet uses its own DNS. With public access open to selected networks, reads go out through the NAT gateway; with it disabled, they fail.

1 GUARD

Link both zones, `dfs` and `blob`, to the workspace VNet, or point its DNS at the hub resolver that forwards to Azure DNS. A VNet can be linked to only one zone of each name.

```
az network private-dns link vnet create -g rg-dns -n link-vnet-dbw \
  --zone-name privatelink.dfs.core.windows.net \
  --virtual-network "$VNET_ID" --registration-enabled false
```

2 DETECT

Resolve from a cluster, not your laptop.

```
import socket
h = "stlakeprod.dfs.core.windows.net"
print(socket.gethostbyname_ex(h))
# want: privatelink alias, 10.x IP
```

3 RECOVER

Link the zone and rerun the check on a new cluster. Before disabling public network access on a storage account, which also cuts every client not on a private endpoint, inventory its readers (jobs, serverless, partners, tools) and get the data owner's approval. Roll out one account at a time, dev first; revert by setting access back to selected networks. Name the owner of the hub DNS zones and forwarders, which every spoke depends on. Repeat for both subresources and add the check to the workspace build.

RELATED **8.11** The storage firewall admits the workspace subnets

8.10 Clusters in a new workspace cannot reach PyPI or Maven

S.18 Every source behind a firewall

8.14 At 9am clusters fail to start with a quota error. Yesterday the same jobs ran.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Classic clusters are VMs in **your subscription**, counted against Azure **vCPU quota** per region: a total and one per **VM family**. Every workspace in the subscription and region draws on the same numbers.

Cost of failure **Outage**Status **GA** · Verified **Oct 2026**

2

vCPU quotas every new VM must fit, per subscription and region: total regional vCPUs and the vCPUs of its VM family. Exceed either and the VM is not deployed.

Source: Microsoft Learn, Check vCPU quotas, Oct 2026

FIG. 8.14 Yesterday's run fitted. Today a new workspace in the same subscription started its jobs at the same minute.

HOW YOU WILL NOTICE

Clusters terminate with

AZURE_QUOTA_EXCEEDED_EXCEPTION, a client error naming the family, limit and usage.

```
az vm list-usage --location westeurope \
-o table | grep -i -e "Total Reg" \
-e "DSv2" -e "Ddsv5"
# CurrentValue near Limit = next launch
# that needs vCPUs fails
```

GUARD

Cap what each team can start, and keep node types to families with headroom.

```
{ "autoscale.max_workers": {
  "type": "range", "maxValue": 20 },
  "node_type_id": { "type": "allowlist",
    "values": ["Standard_D8ds_v5",
      "Standard_E8ds_v5"] } }
```

WORKAROUNDS

Request an increase on the Azure Quotas page under Compute; it needs Contributor, is usually answered within minutes, and a family increase raises the regional total with it. Quota costs nothing until used, so name a cost owner for each increase and review the standing quota every quarter against peak use. Stagger schedules that all start at 9am. Spread pools and policies across two VM families. Flexible node types fall back to a compatible family; Databricks treats that as a safety net, not the fix. Serverless runs in your Databricks account and does not use your subscription's quota.

NEIGHBOURING LIMITS

Spot

Spot vCPUs have their own quota, requested separately

2 IPs

Per node: one in each workspace subnet. A full subnet also stops launches

Capacity

Quota is permission, not stock. A stockout is a different error

RELATED **4.05** Jobs on Spot VMs fail at random around 9am **4.06** A data scientist started a 100-node cluster

8.10 Clusters in a new workspace cannot reach PyPI or Maven

8.15 The only account admin left. Nobody can add users, create a metastore or change account settings.

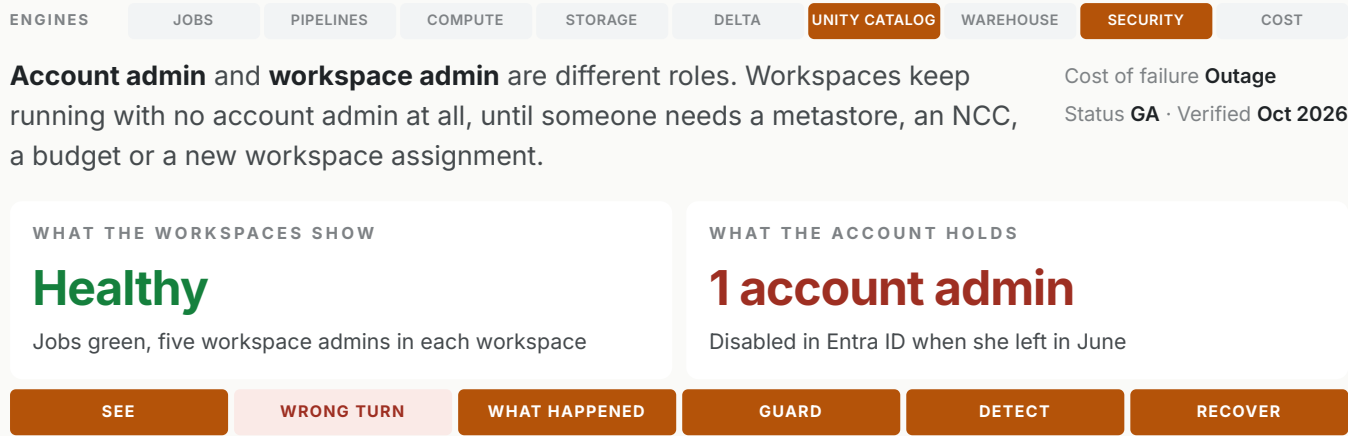


FIG. 8.15 In this example nothing fails for months. The first account-level change finds nobody who can make it.

WRONG TURN

"Several of us are workspace admins, and Azure Owners get admin anyway, so we are covered."

WHAT HAPPENED

Azure Contributors and Owners become workspace admins when they launch the workspace, never account admins. Account admins enable Unity Catalog, manage identities across workspaces, account settings and previews, and grant the admin roles. One person held the role, so nobody can now delegate it.

1 GUARD

Databricks recommends two or three trusted account admins, and groups for admin roles where possible. Add a break-glass identity under your emergency access procedure, and make a group, not a person, the metastore admin.

2 DETECT

List account admins weekly; alert below two.

```
databricks account users list \
  --attributes userName,roles -o json |
jq -r '.[ ] | select(.roles[]?.value
  == "account_admin") | .userName'
```

3 RECOVER

The documented bootstrap is a Microsoft Entra ID Global Administrator signing in at accounts.azuredatabricks.net, which makes them account admin. They assign the role in User management, Roles, then step back. If that fails, open a support case. Afterwards review the account admin list with every leaver.

RELATED **9.11** At 02:00 on-call must pause a production job

7.13 An engineer left. Their personal token still runs 12 production jobs **1.03** Finance wants its own metastore

8.16 Anyone on the internet can reach the workspace login page. Security wants it closed without locking out the admins.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Three controls close the front door: **IP access lists**, **front-end Private Link** and **context-based ingress**. Databricks now recommends ingress policies first, because they can be tested in **dry run** before they block anyone.

Cost of failure **Security exposure**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Workspace IP access lists	One API call per workspace and no network build. Block lists win over allow lists. Every region.	IPv4 only. The public IPs classic compute uses to reach the control plane must be on the allow list, or clusters cannot launch. Staff on home networks and SaaS tools lose access.
Front-end Private Link, public access off	No public path to the workspace. IP access lists never block Private Link traffic.	Needs VNet injection, private DNS every client resolves, and a separate browser_authentication workspace for SSO. Everyone arrives by VPN or ExpressRoute. Converting a workspace needs all compute stopped.
Context-based ingress, dry run first DEFAULT	Allow and deny rules on identity, request type and network source. Public access stays open only for named ranges or identities. Denials are logged in <code>system.access.inbound_network</code> .	Not in West India, Government or China regions. Private endpoint rules are Beta. IP access lists still apply: a request must pass both.

FIG. 8.16 Private Link removes the public path. Ingress policies and IP lists narrow it. Choose by who must still arrive from outside.

DEFAULT

Attach an ingress policy in dry run, read a week of denials, then enforce. Add Private Link where the network team owns the DNS, with one **browser_authentication** workspace per region under a delete lock. Entra Conditional Access targets the **AzureDatabricks** app (`2ff814a6-3304-4ab8-85cb-cd0e6f879c1d`): start report-only, exclude a break-glass account. Its Learn page is marked retired, so test it.

BEFORE YOU CHANGE IT

The security lead owns the policy and a second account admin approves it. Try a non-production workspace first. Before enforcing, allow your own egress IP, the NAT IP of every classic compute plane and a break-glass identity. Changes take about 10 minutes. Roll back from an allowed network: policy to dry run, or disable the IP list. With Private Link, keep public access on until **nslookup** gives the private IP from every client network.

```
databricks ip-access-lists update <list-id> --json '{"enabled": false}'
SELECT * FROM system.access.inbound_network ORDER BY event_time DESC LIMIT 50;
-- DRY_RUN_DENIAL rows would be blocked once enforced
```

RELATED

8.11 The storage firewall admits the workspace subnets

8.18 A serverless notebook reaches the internet

C.23 IP access lists, Private Link, VNet injection, NCC and serverless network policies

8.17 Auditors ask who holds the encryption keys for your data and your notebooks. The answer is Microsoft.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Customer-managed keys are three separate switches: **managed services** (the control plane), **workspace storage** and **managed disks**. The **compliance security profile** is a fourth, different decision, and it cannot be undone.

Cost of failure **Security exposure**Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Microsoft-managed keys	Nothing to run and no key to lose.	The auditor wants proof that you control and can revoke the keys behind notebooks, secrets and job results.
Customer keys on managed services and workspace storage DEFAULT	Managed services: notebook source, secrets, SQL queries and history, tokens, Git credentials. Workspace storage: DBFS root, job and SQL results, system data. Keys sit in Key Vault or Managed HSM.	A deleted or revoked key stops reads and writes on managed services once the key cache expires, and lost keys cannot be recovered. Existing data is not re-encrypted. Mounts and your own storage accounts are not covered.
Add managed disk keys and the compliance security profile	Disk keys cover classic data disks, not OS disks or serverless. The profile adds a hardened image, TLS 1.2 or higher in clusters and monitoring. HIPAA and PCI-DSS require it.	The profile forces automatic cluster updates, specific VM types and an add-on charge. Virtual network encryption is enforced from 1 February 2027. Disk keys cannot be disabled, and the profile cannot be removed once regulated data was processed.

FIG. 8.17 Each switch protects a different place. Say which places the auditor means before you choose.

DEFAULT

Keys for managed services and workspace storage, in a dedicated vault with soft delete and purge protection, pinned to a key version, with two named owners in security. Tables in your own storage accounts take their encryption from that account, so settle it with the storage owners.

BEFORE YOU CHANGE IT

Security approves the key owner, the platform lead the window. Try a non-production workspace first. Irreversible: managed services and disk keys cannot be switched off, and the profile only leaves with a new workspace once regulated data ran. Reversible: workspace storage, with `--key-source Default`. Rotation keeps the old key available for 24 hours. Disk keys need all compute stopped. Under the profile you restart clusters for host patches.

```
az databricks workspace update -n dbw-prod -g rg-data \
  --managed-services-key-name cmk-ms --managed-services-key-version <version> \
  --managed-services-key-vault https://kv-dbx-prod.vault.azure.net
az databricks workspace update -n dbw-prod -g rg-data --key-source Default
```

RELATED **8.09** Leadership asks what happens if West Europe goes down

8.16 Anyone on the internet can reach the workspace login page **9.11** At 02:00 on-call must pause a production job

8.18 A serverless notebook reaches the internet. The same code fails as a production job.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Serverless outbound traffic follows the workspace's **network policy**. In **restricted access** mode only allowed domains, storage accounts and Unity Catalog external locations are reachable, and everything else is dropped.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	DEV WORKSPACE, FULL ACCESS POLICY	PRODUCTION WORKSPACE, RESTRICTED POLICY
Outbound internet	Open	Only listed domains and storage accounts, Unity Catalog external locations and the workspace's own APIs
Direct storage paths	Allowed	Prohibited in notebooks and UDFs: use Unity Catalog or list the account
Call to a partner API or <code>pip install</code>	Works	Dropped unless the host is allowed
Outcome	Notebook succeeds	Job fails on a connection error

FIG. 8.18 The notebook was tested under a different policy from the one production runs under.

CAUSE

Each workspace is attached to one policy, set by account admins. Dev was left on full access, so nothing there tested the allow list. Restricted mode blocks package installs, external APIs and unlisted storage accounts, and model serving builds fail when PyPI or conda-forge are not allowed.

FIX

Put the production policy on dev in **dry run**, run the real jobs, then read the denials. Add each host as an allowed domain (100 per policy) and each storage account as an allowed storage destination (100 per policy, 2,500 destinations in all). Take the PyPI host names from the denials.

```
SELECT * FROM system.access.outbound_network
WHERE access_type = 'DRY_RUN_DENIAL'
      AND event_time >= current_timestamp() - INTERVAL 7 DAYS
ORDER BY event_time DESC;
```

PREVENT

The security lead approves the allow list and an account admin attaches it. Run the same policy on every workspace in the chain: dry run, restricted on one workspace, then the rest. Changes reach most workloads in about 10 minutes, but a mode switch needs serverless compute restarted, and a notebook with Spark can take up to 24 hours. Rollback: set allow all and restart. Databases and storage with public access off need NCC private endpoints, billed hourly (8.12). Dry run for external models and MCP needs the **All products** option.

- RELATED
- 8.11

The storage firewall admits the workspace subnets
- 8.12
- Your serverless private endpoint has said PENDING for two days

C.23

8.19 One workspace for everything, or one per team? Nobody agrees, and the metastore is shared anyway.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A region has **one metastore**, so workspaces do not hold the data boundary.

Catalogs and bindings do. Split workspaces by environment and region, and split teams with catalogs and groups.

Cost of failure **Rework**

Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
One workspace for everything	Nothing to build. One place for tokens, jobs and cluster policies.	Dev and prod share admins and compute policies. A binding cannot separate catalogs that all belong to the same workspace.
A workspace per team or domain	A hard edge when organisational rules say no assets are shared between business units.	Each one needs its own network, assignments, secrets and jobs. The guide advises against it for single teams or small projects. Notebooks are not shared across workspaces.
A workspace per environment and region	Prod workspaces are bound to prod catalogs. Teams get catalogs and groups. Each region has one metastore and an admin workspace for its Unity Catalog resources.	It must be scripted from the first workspace. Data in a second region needs OpenSharing: never register one table as external in two metastores.
DEFAULT		

FIG. 8.19 The metastore is regional and shared. Workspaces separate people and compute; catalogs separate data.

DEFAULT

Dev, test and prod workspaces per region, named by environment, region and purpose. One metastore per region. A catalog per environment and domain (1.04), bound to its environment's workspaces. Catalog owners are groups.

SIGNS YOU CHOSE WRONG

A new team means a new workspace ticket. Prod and dev catalogs show in one workspace's catalog list. Nobody can say which workspace administers the metastore. The platform has over 50 workspaces and no template.

LIMITS TO PLAN AROUND

The guide says not to exceed 50 to 100 workspaces without strong justification and full automation, and typically 3 to 10 catalogs per metastore. Hard limits: 5,000 groups per account and 1,000,000 tables per metastore. Raisable on request: 10,000 users and service principals per account, 1,000 catalogs per metastore.

BEFORE YOU CHANGE IT

Moving a team changes workspace URLs, tokens, job IDs and cluster policies. The platform owner approves the target layout and each catalog owner the bindings. Move one team first and keep its old workspace and binding open until its jobs have run. Isolating a live catalog cuts off every unbound workspace at once (1.04).

```
SELECT current_metastore();
SELECT catalog_name, catalog_owner FROM system.information_schema.catalogs;
databricks workspace-bindings get-bindings catalog prod_sales
```

RELATED

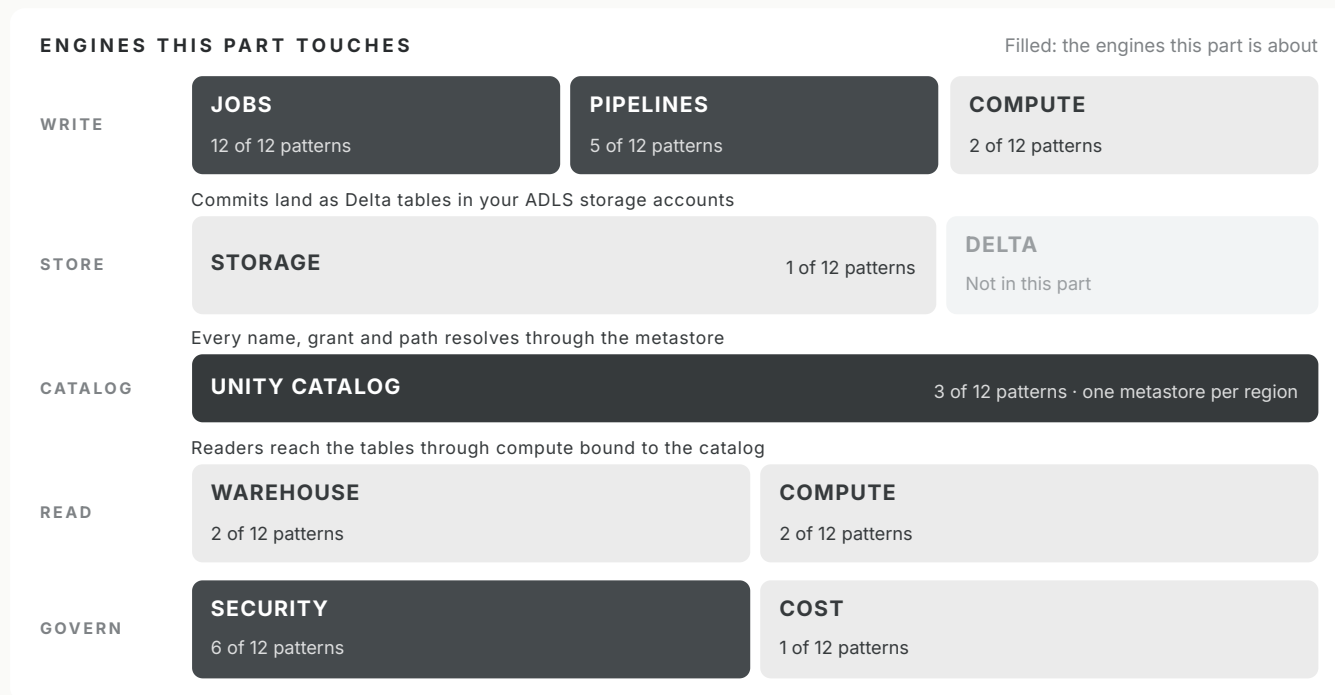
1.03 Finance wants its own metastore

1.04 A developer dropped a production table from the dev workspace

8.15 The only account admin left

09 CI/CD and Operations.

How changes ship and how failures surface: Declarative Automation Bundles, environments, run identity, alerts, health views and logs. A green deploy and a green run both say less than they seem, so check what was deployed, as whom, and who hears when it breaks.



NO.	PATTERN	JOBS	PIPE	COMPUTE	UC	WH	SEC	PAGE
9.01	The bundle deployed to production. The job uses the dev cluster settings.	●	●	●	●	●	●	156
9.02	The bundle deploy succeeded. The job you open in the workspace did not change.	●	●	●	●	●	●	157
9.03	Bundle validation passes on your laptop. The deploy fails in CI.	●	●	●	●	●	●	158
9.04	You ran bundle destroy. It deleted a job the bundle did not create.	●	●	●	●	●	●	159
9.05	The production deployment runs as the person who deployed it.	●	●	●	●	●	●	160
9.06	Every promotion is manual. Staging has drifted from production.	●	●	●	●	●	●	161
9.07	A job failed. You heard about it from a business user four hours later.	●	●	●	●	●	●	162
9.08	The duration alert fires after the job finishes.	●	●	●	●	●	●	163
9.09	You run 200 jobs. Nobody can say which ones are healthy.	●	●	●	●	●	●	164
9.10	The job cluster terminated. Its logs went with it.	●	●	●	●	●	●	165
9.11	At 02:00 the on-call engineer must pause a production job. They only have CAN MANAGE RUN.	●	●	●	●	●	●	166

NO.	PATTERN	JOB	PIPE	COMPUTE	UC	WH	SEC	PAGE
9.12	Every grant request, new external location and policy change goes to whoever answers first.	●	●	●	●	●	●	167

COMPARE

STORAGE

INGESTION

DELTA

COMPUTE

JOB

SQL

SECURITY

PLATFORM

CI/CD

AI

SCENARIOS

9.01 The bundle deployed to production. The job uses the dev cluster settings.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A variable with a **dev default** and no prod value, or a **BUNDLE_VAR_** environment variable left on the runner, wins without a word. Values set in a target come fourth in the **resolution order**.

Cost of failure **Wrong data**
Status **GA** · Verified **Oct 2026**

	DEPLOY TO DEV FROM A LAPTOP	DEPLOY TO PROD FROM CI
Command	<code>bundle deploy, dev is default: true</code>	<code>bundle deploy -t prod</code>
node_type	Set in the dev target	Never set in prod, so the default applies: the dev size
BUNDLE_VAR_catalog	Not set	Left on the runner by an old pipeline: <code>dev_sales</code>
Resolved values	Dev values, as intended	Dev size, dev catalog
Outcome	Correct job	Deploy succeeds; prod job reads and writes dev data

FIG. 9.01 Both deploys succeed. Only the resolved values show which environment the job really points at.

CAUSE

Declarative Automation Bundles resolve a variable from the first source that has it: `--var`, then `BUNDLE_VAR_` environment variables, then `.databricks/bundle/<target>/variable-overrides.json`, then the target's `variables`, then the top-level `default`. Job cluster overrides merge by `job_cluster_key`: a new key adds a cluster.

FIX

Give environment-specific variables no default, so a missing value fails the deploy. Set them in every target.

```
variables:
  catalog: { description: "Catalog for this target, no default on purpose" }
  node_type: { description: "Worker VM size, no default either" }
targets:
  prod:
    mode: production
    variables:
      catalog: sales_prod
      node_type: Standard_E8ds_v5
```

PREVENT

Fail the prod pipeline on any `BUNDLE_VAR_` it did not set itself. Print what resolved with `databricks bundle validate -t prod --output json | jq '.variables'`, diff it against a checked-in file, and read the job back with `databricks jobs get` after deploy.

RELATED

9.03 Bundle validation passes on your laptop

9.05 The production deployment runs as the person who deployed it

5.04 You cloned a production job to test it

9.02 The bundle deploy succeeded. The job you open in the workspace did not change.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A bundle updates only the jobs in its **deployment state**, so a hand-made job with the same name is left alone. The jobs it does own are **UI_LOCKED**, and the real drift route is **Disconnect from source**.

Cost of failure **Rework**Status **GA** · Verified **Oct 2026**

WHAT THE PIPELINE SHOWS

Deploy succeeded

Exit code 0 on every merge

WHAT THE JOB HOLDS

Last month's tasks

Made by hand; the bundle owns a twin with the same name

SEE

WRONG TURN

WHAT HAPPENED

GUARD

DETECT

RECOVER

FIG. 9.02 The deploy did exactly what the bundle said. It said nothing about the job everyone bookmarked.

WRONG TURN

"A successful deploy means the job I know by name now matches the repo."

WHAT HAPPENED

Declarative Automation Bundles keep each bundle's state under the target's root path, by default `/Workspace/Users/<deployer>/ .bundle/<bundle>/<target>`. Deploy updates what is in that state and creates the rest; it does not adopt an existing job by name, so the first deploy made a twin. Jobs a bundle deploys get `edit_mode` `UI_LOCKED` and show read-only in the UI. Anyone who can edit the job can press **Disconnect from source** and change it. From then on the job runs settings no commit holds, until the next deploy overwrites them and locks it again.

1 GUARD

Bind an existing job before the first deploy, so the deploy updates it instead of creating a twin. Give people `CAN_MANAGE_RUN` on prod jobs, not `CAN_MANAGE`, so nobody can disconnect them, and keep an emergency route (9.11).

```
databricks bundle deployment bind orders_daily 123456789 -t prod
databricks bundle deploy -t prod
```

2 DETECT

Look for twins, then for bundle jobs someone disconnected: they show `EDITABLE`.

```
databricks bundle summary -t prod
databricks jobs list --name orders_daily
databricks jobs get 123456789 \
  | jq '.settings.edit_mode'
```

3 RECOVER

For a twin, bind the real job and redeploy, or delete the hand-made one and move its schedule and permissions into the bundle. For a disconnected job, copy the UI change into the repo first, then redeploy, or the deploy throws the fix away. `update` and `reset` events in `system.access.audit` (Public Preview, 365 days) show who changed it. Azure diagnostic logs (Premium plan) are the route that avoids a preview table.

RELATED **9.04** You ran bundle destroy **9.06** Every promotion is manual **9.11** At 02:00 on-call must pause a production job

9.03 Bundle validation passes on your laptop. The deploy fails in CI.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

bundle validate checks configuration, not what the deploying identity may do. Cost of failure **Rework**

CI runs a different **CLI version** and signs in as a **service principal** that has none of your grants. Status **GA** · Verified **Oct 2026**

	YOUR LAPTOP	CI RUNNER
CLI version	Installed months ago	setup-cli@main: whatever main installs that day
Sign-in	Your OAuth login in ~/.databrickscfg	Service principal through workload identity federation
Permissions	Yours, often broad	Only what the service principal was granted
Outcome	Validate and deploy pass	Validate passes, deploy fails

FIG. 9.03 Validation passes in both places. The two runs differ in what they are allowed to do.

CAUSE

In Declarative Automation Bundles, deploy is the first step that acts as the CI identity. It may lack `CAN_USE` on a cluster policy or a catalog grant, or miss a value your laptop kept in `.databricks/bundle/<target>/variable-overrides.json`. A newer CLI can read fields differently, and `@main` changes under you.

FIX

Pin the action to a release tag, which also fixes the CLI version, or to a commit SHA, which nobody can move. Match `databricks_cli_version` in the bundle, sign in with workload identity federation, and deploy to staging on every merge.

```
# bundle: databricks_cli_version: '1.19.*'    GitHub Actions below
permissions: { id-token: write, contents: read }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: staging
    env:
      DATABRICKS_AUTH_TYPE: github-oidc
      DATABRICKS_HOST: ${vars.DATABRICKS_HOST}
      DATABRICKS_CLIENT_ID: ${vars.SP_CLIENT_ID}
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@v1.19.0    # never @main in a deploy pipeline
      - run: databricks bundle deploy -t staging
```

PREVENT

Keep the service principal's grants in code with the bundle, nothing a deploy needs in git-ignored files, and no personal tokens in CI. Upgrade the CLI in one reviewed commit.

RELATED **9.01** The bundle deployed to production **9.05** The production deployment runs as the person who deployed it

7.13 An engineer left. Their personal token still runs 12 production jobs

9.04 You ran bundle destroy. It deleted a job the bundle did not create.

ENGINESJOBSPIPELINESCOMPUTESTORAGEDELTAUNITY CATALOGWAREHOUSESECURITYCOST

A job that was **bound** to a bundle belongs to that bundle. **bundle destroy** deletes it with everything else, and the deletion **cannot be undone**.

Cost of failure **Outage**Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (A CLEANUP SCRIPT YOU WROTE)
"Destroy removes what this bundle created. Anything that existed before the bundle stays where it was."

WITH DECLARATIVE AUTOMATION BUNDLES
bundle deployment bind links an existing job or pipeline to a bundle resource, and the next deploy manages it. From then on it is in the bundle's state, whoever created it. bundle destroy permanently deletes the jobs, pipelines and artifacts the bundle deployed, bound ones included. It prompts first, unless --auto-approve is set, as it usually is in a script.

Do this. Before any destroy, run `databricks bundle summary -t <target>` and read every resource it lists. For each one that must survive, run `databricks bundle deployment unbind <key> -t <target>`, then destroy. Keep `destroy --auto-approve` out of any pipeline that can reach production, and write down which resources each bundle owns.

RELATED9.02 The bundle deploy succeeded9.06 Every promotion is manual1.02 You dropped a migrated table the way you always did

9.05 The production deployment runs as the person who deployed it.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Without **run_as**, bundle jobs run as the **deploying identity**. Deploy prod from a laptop and production runs on your **personal grants** until your account is disabled.

Cost of failure **Security exposure**
Status **GA** · Verified **Oct 2026**

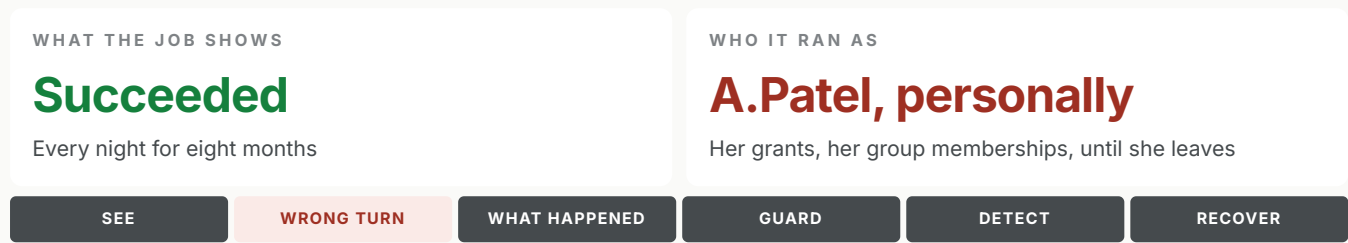


FIG. 9.05 The run status says the job works. It does not say whose access made it work.

WRONG TURN

"The target says `mode: production`, so the deployment is set up for production."

WHAT HAPPENED

In Declarative Automation Bundles, production mode validates; it does not choose an identity. When a user deploys, it checks that `run_as` and `permissions` are set, but `run_as` can still be that user.

1 GUARD

Deploy prod only from CI, set a service principal as `run_as`, and declare permissions so every deploy reapplies them.

```
targets:
  prod:
    mode: production
    run_as: { service_principal_name: "${var.prod_sp_app_id}" }
    permissions:
      - { level: CAN_MANAGE_RUN, group_name: data-ops }
```

2 DETECT

Live jobs not run as a service principal. Rows from before early December 2025 have only `run_as`, so `LIKE '%@%'` on `run_as_user_name` misses them.

```
SELECT name, coalesce(run_as_user_name, run_as) AS runs_as FROM system.lakeflow.jobs
QUALIFY ROW_NUMBER() OVER (PARTITION BY workspace_id, job_id
                           ORDER BY change_time DESC) = 1 AND delete_time IS NULL
AND NOT coalesce(run_as_user_name, run_as) RLIKE '^[0-9a-f-]{36}$';
```

3 RECOVER

Grant the service principal the Unity Catalog privileges the job needs, then redeploy from CI with `run_as` set. The deploying identity must be allowed to use that service principal. Run once in staging first: missing grants surface there, not at 02:00 in prod.

RELATED **9.03** Bundle validation passes on your laptop

7.13 An engineer left. Their personal token still runs 12 production jobs

7.07 A row filter using `current_user` works in a notebook

9.06 Every promotion is manual. Staging has drifted from production.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Promotion should be one pipeline deploying **the same commit** to every target. Declarative Automation Bundles are the recommended route; **Git folders** carry code but not job definitions, and **UI edits** are where drift starts.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Git folders, pulled by hand per workspace	Authoring and review. Code is versioned.	Job settings live only in the workspace, set by hand. Staging runs last month's code after a missed pull.
Terraform for everything	Workspaces, networking, catalogs, storage credentials and policies, with plan and review.	Every job change waits on an infrastructure review.
Bundles from one pipeline, Terraform for the platform DEFAULT	Jobs, pipelines and code in one bundle per project. A merge deploys staging, tests run, and the same commit goes to prod as a service principal.	Needs a staging workspace that mirrors prod's catalogs and grants. People who still edit jobs in the UI lose their changes on the next deploy.

FIG. 9.06 Drift comes from the steps people do by hand. The default removes them, not the people.

DEFAULT	SIGNS YOU CHOSE WRONG
One pipeline per bundle: validate, staging, tests, then the same commit to prod. Humans get <code>CAN_VIEW</code> or <code>CAN_MANAGE_RUN</code> on prod jobs, never <code>CAN_MANAGE</code> .	A promotion checklist in a wiki. Prod jobs changed in the UI with no commit behind them.

MIGRATE

Move prod jobs one at a time, the job owner and platform lead approving. Bind, deploy to staging first; the next deploy rewrites a bound job from the bundle. Roll back with `unbind` (the job stays) and `databricks jobs reset` from the export. Staging bills compute on every merge: test on a slice, pause its schedules, set a budget.

```
databricks jobs get 123456789 > before.json
databricks bundle generate job --existing-job-id 123456789 --bind
```

CHECK IT

List live jobs that no external deployment manages. In a bundle-only workspace, the list should be empty.

```
SELECT workspace_id, job_id, name, creator_user_name
FROM system.lakeflow.jobs
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY workspace_id, job_id ORDER BY change_time DESC) = 1
AND delete_time IS NULL AND deployment IS NULL;
-- deployment is set for jobs managed from outside, such as bundles,
-- in rows written since early December 2025
```

RELATED **9.02** The bundle deploy succeeded **9.05** The production deployment runs as the person who deployed it

5.04 You cloned a production job to test it

9.07 A job failed. You heard about it from a business user four hours later.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The only alert was an **email to the author**, sent after the last retry. Send failures to a **system destination** the on-call team watches, on every job, and give every job a **timeout**.

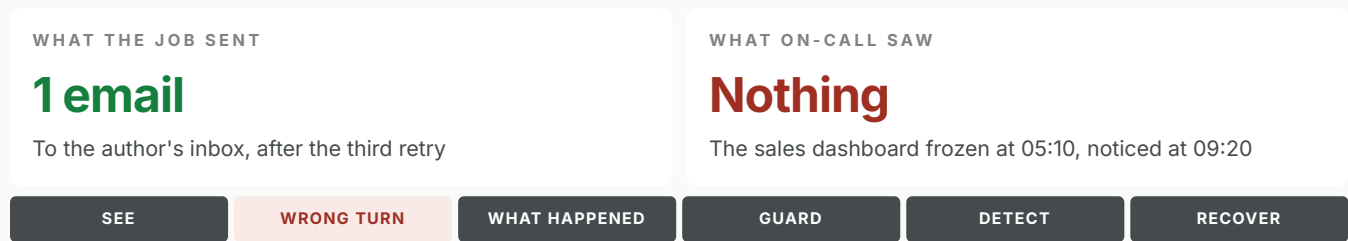
Cost of failure **Outage**Status **GA** · Verified **Oct 2026**

FIG. 9.07 In this example the notification worked as configured. It was configured for one person.

WRONG TURN

"The job emails on failure, so someone will know."

WHAT HAPPENED

Job-level notifications are not sent while failed tasks are retried, so three spaced retries delayed the only email by hours. It went to one person's inbox, through mail filters. A run that hangs never fails at all unless the job has a timeout.

1 GUARD

An admin creates a system destination for the on-call channel: Teams, Slack, PagerDuty or a webhook. Every job in the bundle sends failures there and has a timeout.

```
variables:
  oncall:
    lookup: { notification_destination: "Data on-call (Teams)" }
resources:
  jobs:
    orders_daily:
      timeout_seconds: 10800
      webhook_notifications:
        on_failure: [{ id: ${var.oncall} }]
```

2 DETECT

A scheduled SQL alert as a safety net: runs in the last day not ending `SUCCEEDED` or `SKIPPED`.

```
SELECT job_id, run_id, result_state
FROM system.lakeflow.job_run_timeline
WHERE result_state NOT IN ('SUCCEEDED',
  'SKIPPED') AND period_end_time
  > current_timestamp() - INTERVAL 1 DAY;
```

3 RECOVER

For critical tasks, add task notifications: they fire on each failed attempt unless **Mute notifications until the last retry** is selected. Each event takes up to three system destinations.

RELATED 9.08 The duration alert fires after the job finishes

5.02 A failed task retried three times

5.03 The task timeout is 30 minutes

9.08 The duration alert fires after the job finishes.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The alert reads **finished runs**, so it can only speak once the run ends. A **health rule** on run duration fires while the run is still going, and a **timeout** ends it.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

	IN TESTING: A 10-MINUTE RUN	IN PRODUCTION: A RUN HANGS AT 02:00
Alert source	SQL alert on run duration summed from <code>job_run_timeline</code> periods	The same alert
When rows land	Within an hour of the run's end	Hourly slices; duration known only at the end
Alert fires	Shortly after the run, which looks right	Only after the run ends or is killed
Outcome	Test passes	The 06:00 deadline is missed before anyone hears

FIG. 9.08 A query over run history measures runs that have ended. A late run has not ended yet.

CAUSE

System tables record runs after the fact: a run is split into one row per clock hour, only the last row carries `result_state`, and each row lands within about an hour of its period end. A duration check built on them can only report a run that is already over.

FIX

Add a health rule with a duration warning sent to the on-call destination, and a timeout as the hard stop. The warning fires during the run; the timeout marks it `Timed Out`.

```
resources:
  jobs:
    orders_daily:
      timeout_seconds: 10800          # hard stop at 3 hours
      health:                        # warn at 90 minutes
        rules: [{ metric: RUN_DURATION_SECONDS, op: GREATER_THAN, value: 5400 }]
      webhook_notifications:
        on_duration_warning_threshold_exceeded: [{ id: ${var.oncall} }]
```

PREVENT

Set the warning a little above the 95th percentile of successful runs over 30 days, the timeout well above it. Sum each run's periods, as in 5.13: `run_duration_seconds` is 0 for multi-task and non-legacy jobs. Jobs with no rule show an empty `health_rules` in `system.lakeflow.jobs` (rows since early December 2025).

```
WITH r AS (SELECT workspace_id, job_id, run_id, SUM(timestampdiff(SECOND,
  period_start_time, period_end_time)) AS secs FROM system.lakeflow.job_run_timeline
  WHERE period_start_time > current_timestamp() - INTERVAL 30 DAYS
  GROUP BY ALL HAVING MAX(result_state) = 'SUCCEEDED')
SELECT workspace_id, job_id, percentile(secs, 0.95) AS p95_secs FROM r GROUP BY ALL;
```

- RELATED
- 9.07 A job failed. You heard about it from a business user
- 5.03 The task timeout is 30 minutes
- 6.08 The SQL alert fired six hours late

9.09 You run 200 jobs. Nobody can say which ones are healthy.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The **Runs** page shows 60 days for the jobs you can see. The **system.lakeflow** tables keep 365 days for every workspace in the region and join to **billing**, so they hold the single view.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Jobs & Pipelines, Runs tab	Triage now: runs across all jobs and pipelines you can access, filtered by status, error code, run-as user or the last 48 hours. The matrix view shows each task's history.	You need trends, cost, or more than 60 days. One workspace at a time, and only what your permissions show.
Diagnostic logs to Log Analytics	Job events next to other Azure alerts, queried in KQL by the operations team.	Event records only: no task durations next to cost, and one more tool for data engineers to learn.
System tables dashboard DEFAULT	jobs, job_run_timeline and job_task_run_timeline are GA, cover every workspace in the region for 365 days, and join to system.billing.usage by job_id.	Rows land within about an hour, so it is a health view, not an alert. Pipeline tables are in Public Preview.

FIG. 9.09 Triage in the Runs tab, trends in system tables, alerts in the jobs themselves.

DEFAULT

One AI/BI dashboard on system tables: failure rate, duration trend and cost per job over 30 days, with owner and run-as. Alerts stay on the jobs (9.07, 9.08).

SIGNS YOU CHOSE WRONG

The weekly review starts by opening jobs one by one. Nobody knows which job got slower this month. A job fails one run in five and nobody has noticed.

CHECK IT

The core query: runs, unhealthy runs and the slowest run per job over seven days. Unhealthy is every end state other than **SUCCEEDED** and **SKIPPED**, so timeouts, errors, cancellations and blocked runs count too.

```
WITH j AS (SELECT * FROM system.lakeflow.jobs
  QUALIFY ROW_NUMBER() OVER (PARTITION BY workspace_id, job_id
    ORDER BY change_time DESC) = 1),
r AS (SELECT workspace_id, job_id, run_id, MAX(result_state) AS state,
  SUM(timestampdiff(SECOND, period_start_time, period_end_time)) AS secs
  FROM system.lakeflow.job_run_timeline
  WHERE period_start_time > current_timestamp() - INTERVAL 7 DAYS
  GROUP BY ALL)
SELECT j.name, COUNT(*) AS runs,
  COUNT_IF(r.state NOT IN ('SUCCEEDED', 'SKIPPED')) AS unhealthy,
  MAX(r.secs) AS slowest_s
FROM r JOIN j USING (workspace_id, job_id)
GROUP BY ALL ORDER BY unhealthy DESC, slowest_s DESC;
-- same state list as the 9.07 alert; NULL means still running
```

RELATED

8.05 Your monitoring queries cost more than the jobs they watch

9.07 A job failed. You heard about it from a business user

5.16 An auditor asks for a job run from four months ago

9.10 The job cluster terminated. Its logs went with it.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Job compute ends with the run. **Compute log delivery** to a **Unity Catalog volume** keeps driver and worker logs, in a folder named after the **cluster ID**.

Cost of failure **Rework**Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (A SERVER OR A LONG-LIVED CLUSTER)

"The logs are on the machine. After a failure I log in and read them."

ON AZURE DATABRICKS JOB COMPUTE

The cluster and its VMs go away when the run ends. Set a log path on the compute and Databricks delivers logs every five minutes, archived hourly, into a subfolder named after the `cluster_id`. A volume is the recommended destination; it needs Unity Catalog compute in standard access mode or dedicated to a user, and the compute owner needs `READ VOLUME` and `WRITE VOLUME`. Run history in the UI is kept for 60 days. The docs state no expiry for delivered logs, so the volume grows until you clean it.

Do this. Set `cluster_log_conf` with a `volumes` destination, such as `/Volumes/ops_prod/logs/compute`, on every job cluster in the bundle. To find a failed task's logs, take its `compute_ids` from `system.lakeflow.job_task_run_timeline` and open that folder: no searching by timestamp. Give `READ VOLUME` on that volume to the operations group only, because driver logs can hold stack traces, paths and values your code printed. Set a retention, for example 30 days, and delete older folders with a scheduled job; keep longer only what an audit needs. Export run results you must keep beyond 60 days, and send workspace diagnostic logs to Log Analytics when the operations team searches there.

RELATED **9.09** You run 200 jobs **5.16** An auditor asks for a job run from four months ago

4.08 Auto termination is set to 60 minutes

9.11 At 02:00 the on-call engineer must pause a production job. They only have CAN MANAGE RUN.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

CAN_MANAGE_RUN cancels runs but cannot pause a schedule. Use a **runbook job** run as a service principal, and put the pause in the **next deploy**.

Cost of failure **Outage**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Permanent CAN_MANAGE	Nothing to set up.	On-call can change or delete any prod job, all year (9.02).
Break-glass group for a few hours	CAN_MANAGE when the cause is unknown. Approved, logged, expires.	Approval is slower than the incident.
Runbook job DEFAULT	One job, run as a service principal, pauses or cancels listed prod jobs. Each use is a run with who and why.	The action is not in the runbook. Keep the group as fallback.

FIG. 9.11 The narrowest right that stops the damage, granted in seconds and visible afterwards.

DEFAULT

A runbook job in its own bundle, owned by the platform team. On-call has **CAN_MANAGE_RUN** on it. It runs as a service principal with **CAN_MANAGE** on the listed prod jobs through their bundles' permissions. After a pause, on-call opens a pull request setting `pause_status: PAUSED`: a deploy rewrites it.

SIGNS YOU CHOSE WRONG

On-call waits for an admin. Prod jobs show **EDITABLE** after an incident. A paused job runs again after the next merge.

NEW PROD JOB

Add its id to **ALLOWED** by pull request, grant the service principal in its bundle, test the runbook on it in staging.

CHECK IT

Core of the runbook; `job_id` and `reason` are job parameters.

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.jobs import JobSettings, PauseStatus
ALLOWED = {123456789, 223456789} # reviewed pull request only
job_id, reason = int(dbutils.widgets.get("job_id")), dbutils.widgets.get("reason")
assert job_id in ALLOWED and reason, "job not allowed or no incident number"
w = WorkspaceClient()
s = w.jobs.get(job_id).settings
new = {k: getattr(s, k) for k in ("schedule", "trigger", "continuous") if getattr(s, k)}
assert new, "job has no schedule, trigger or continuous block"
for block in new.values():
    block.pause_status = PauseStatus.PAUSED
w.jobs.update(job_id, new_settings=JobSettings(**new))
w.jobs.cancel_all_runs(job_id=job_id) # after the pause
print("paused", job_id, reason)
```

Review each use: run history and job events in `system.access.audit` (Public Preview) or Azure diagnostic logs.

RELATED

9.02 The bundle deploy succeeded

9.05 The production deployment runs as the person who deployed it

9.12 Every change goes to whoever answers first

9.12 Every grant request, new external location and policy change goes to whoever answers first.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Name an **owner group** for every catalog and an **approver per change type**. Cost of failure **Security exposure**
Agree **severity levels** and freshness targets before the first incident, not Status **GA** · Verified **Oct 2026**
during it.

OPTION	GOOD AT	BREAKS WHEN
Whoever answers first	Speed.	An unapproved external location appears. Nobody knows who decided.
One platform team approves all	One queue.	The queue stalls, and the team rules on data it does not know.
Owner per catalog, approver per change type DEFAULT	Owners decide data grants; platform and security decide platform changes. All in code or a ticket.	An owner group empties out. Needs a written table and a quarterly check.

FIG. 9.12 Anyone may be able to make the change. One named group answers for it.

APPROVER PER CHANGE TYPE

Grants: the catalog's owner group, in a reviewed pull request. **External locations, credentials, connections:** platform and security. **Filters, masks, policies:** the data protection owner. **Workspace settings, bindings, network:** the platform lead, with inventory, change window and rollback.

DEFAULT

Every catalog owned by a group. Every Gold table has an owner, a freshness target and a severity, kept in one table that the alerts read.

CHECK IT

Gold tables past their target, from the table's own commits, not lineage. Run it as a job. `OPTIMIZE` and `VACUUM` commits are skipped. History keeps 30 days by default, so a longer gap shows as late. A commit shows a write ran, not that rows arrived.

SEVERITY AND RESPONSE

1: a Gold table wrong or exposed; respond in 15 minutes, by a named on-call rota. With no rota, say "working hours" in the table. **2:** a Gold table past its freshness target; within an hour. **3:** anything else; next working day. Severity 1 and 2 get a blameless review within a week.

SIGNS YOU CHOSE WRONG

Catalogs owned by people who left. Severity argued during the incident.

```
OPS = "operation IN ('WRITE','MERGE','UPDATE','DELETE','COPY INTO','STREAMING UPDATE')"  
for r in spark.table("ops_prod.meta.freshness_slo").collect():  
    age = (spark.sql(f"DESCRIBE HISTORY {r.table_name}").where(OPS).selectExpr(  
        "timestampdiff(MINUTE, max(timestamp), current_timestamp())").first()[0])  
    if age is None or age > r.max_age_minutes:  
        print(r.table_name, r.owner_group, r.severity, age)
```

RELATED

9.11 At 02:00 on-call must pause a production job

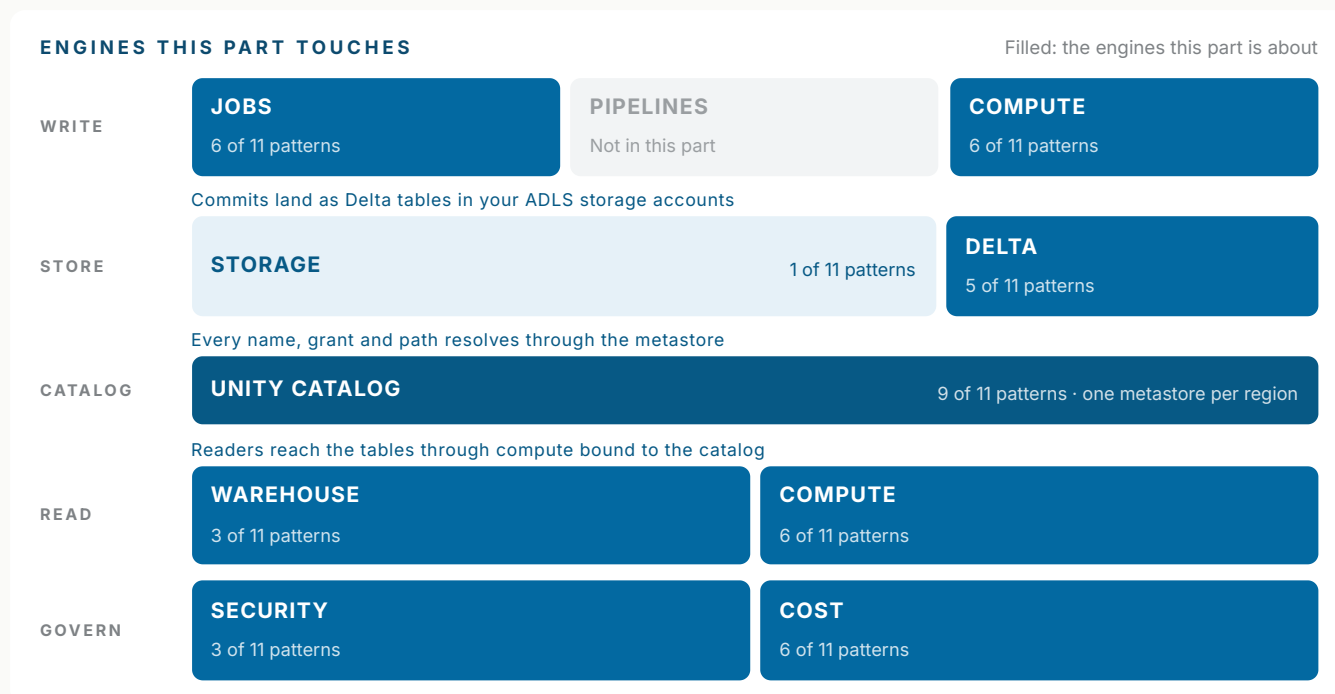
6.08 The SQL alert fired six hours late

9.09 You run 200 jobs

10

ML and AI Workloads.

Models, serving endpoints, batch scoring, AI Search, Genie and low-latency serving with Lakebase. Most failures here come from a copy: a model file, an index, a synced table or a second definition of revenue that drifted from its source.



NO.	PATTERN	JOBS	COMPUTE	DELTA	UC	WH	COST	PAGE
10.01	ETL jobs run on the GPU cluster built for training. The GPUs sit idle and you pay for them.	●	●	●	●	●	●	170
10.02	Models were saved to a volume as files. Nobody knows which version is in production.	●	●	●	●	●	●	171
10.03	A model serving endpoint answers ten requests an hour. It costs as much as a cluster.	●	●	●	●	●	●	172
10.04	You need predictions for 50 million rows every night. The job calls the serving endpoint row by row.	●	●	●	●	●	●	173
10.05	An app needs a customer's features in 10 milliseconds. A SQL warehouse takes a second.	●	●	●	●	●	●	174
10.06	Vector search returns documents you deleted last week. The table no longer has them.	●	●	●	●	●	●	175
10.07	Genie gives a different revenue figure from the dashboard for the same month.	●	●	●	●	●	●	176
10.08	A team shares one foundation model endpoint and one project burns the whole token budget.	●	●	●	●	●	●	177
10.09	The endpoint returns 200 for every request. Answers got worse a month ago.	●	●	●	●	●	●	178
10.10	Pay per token worked in the pilot. Production traffic makes the bill unpredictable.	●	●	●	●	●	●	179

NO.	PATTERN	JOB	COMPUTE	DELTA	UC	WH	COST	PAGE
10.11	Features in the online store differ from the ones the model trained on. Scores drift and no job fails.	●	●	●	●	●	●	180

COMPARE

STORAGE

INGESTION

DELTA

COMPUTE

JOB

SQL

SECURITY

PLATFORM

CI/CD

AI

SCENARIOS

10.01 ETL jobs run on the GPU cluster built for training. The GPUs sit idle and you pay for them.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Spark ETL runs on the **CPUs**. A GPU instance is billed by the hour whether its GPU works or not, and **Photon** is not supported on GPU instance types. Give ETL its own **CPU or serverless** compute.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

YOU'D EXPECT (A DATA SCIENCE WORKSTATION)

"One machine with a GPU runs everything. ETL simply does not use the GPU, so sharing the box costs nothing extra."

ON AZURE DATABRICKS GPU COMPUTE

Every node is a GPU VM: the Azure VM price and the DBUs per hour are set by the instance type, not by what your code uses. On the Azure pricing page (pay-as-you-go list, Oct 2026) an NV36ads A10 v5 is rated at 4 DBU per hour, a D4ds v5 at 1, before the VM charge. ETL on it also loses Photon, and Databricks Runtime for Machine Learning needs dedicated access mode, so the cluster cannot be shared in standard access mode either.

Do this. Split the job by task. ETL tasks run on serverless jobs compute or a CPU job cluster with Photon on Databricks Runtime 18 LTS; only the training task gets a GPU job cluster on the ML runtime, and it terminates when the task ends. Check [system.billing.usage](#) for GPU node types on all-purpose clusters. For short GPU work, AI Runtime (serverless GPU, A10 and H100) is in Public Preview.

RELATED **4.01** A job runs every five minutes and starts a new cluster each time **8.02** You moved to serverless to save money

10.04 You need predictions for 50 million rows every night

10.02 Models were saved to a volume as files. Nobody knows which version is in production.

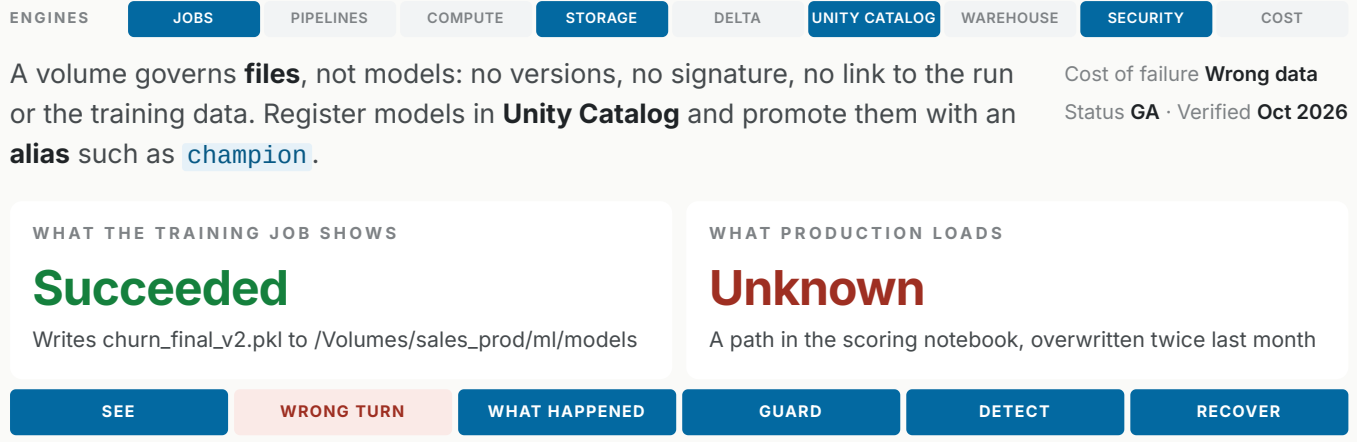


FIG. 10.02 The job reports that a file was written. Nothing records which file production uses.

WRONG TURN

"The volume is in Unity Catalog, so the model is governed and versioned."

WHAT HAPPENED

The volume controls who reads the folder. A pickle there has no version history, signature, run or training table behind it. Overwriting the file replaces the model in place, and the scoring notebook picks it up on its next run. Registry stages are not supported in Unity Catalog; use aliases.

1 GUARD

Register with a three-level name. MLflow 3 uses `databricks-uc` as the registry by default; the input example gives the signature Unity Catalog requires.

```
import mlflow
train = mlflow.data.load_delta(table_name="sales_prod.ml.churn_train")
with mlflow.start_run():
    mlflow.log_input(train, "training") # lineage to the training table
    mlflow.sklearn.log_model(sk_model=clf, name="model", input_example=X.head(5),
                           registered_model_name="sales_prod.ml.churn")
# promotion needs ownership of the model plus USE CATALOG and USE SCHEMA
mlflow.MlflowClient().set_registered_model_alias("sales_prod.ml.churn", "champion", 7)
```

2 DETECT

Find model files outside the registry, and what the alias points at.

```
LIST '/Volumes/sales_prod/ml/models/';

c = mlflow.MlflowClient()
v = c.get_model_version_by_alias(
    "sales_prod.ml.churn", "champion")
print(v.version, v.run_id)
```

3 RECOVER

Log each file still in use with `registered_model_name`, point the alias at the version production runs today, and load `@champion` in scoring. Run promotion as a service principal that owns the model: changing an alias needs ownership. Grant scorers `GRANT EXECUTE ON FUNCTION sales_prod.ml.churn TO `scoring-sp`` (models are functions in SQL) and make the old folder read-only.

RELATED [10.04](#) You need predictions for 50 million rows every night

[10.03](#) A model serving endpoint answers ten requests an hour [7.09](#) Lineage shows some tables and misses others

10.03 A model serving endpoint answers ten requests an hour. It costs as much as a cluster.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

An endpoint bills for its **provisioned concurrency** while it is up, not per request.

Scale to zero waits for 30 minutes without a request, so ten calls an hour keep it running all day. When inputs are known in advance, **score in batch** instead.

Cost of failure **Cost**

Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Always-on endpoint, scale to zero off	Capacity is there for every request. The right choice for production calls that need a steady response time.	Traffic is a few calls an hour: you pay for the minimum concurrency around the clock to serve them.
Scale to zero on	Nights and weekends cost nothing once the endpoint has been idle for 30 minutes.	Daytime calls every few minutes never reach 30 idle minutes. The first call after idle waits 10 to 20 seconds, sometimes minutes, with no SLA. Not for production.
Score in batch, serve the result DEFAULT	When the inputs exist before the request, a job scores every customer nightly and the app reads a table or a synced table. No endpoint at all.	The input only exists at request time, such as a basket or a typed message. Then keep an endpoint, sized to the traffic.

FIG. 10.03 The cost floor of an endpoint is its minimum concurrency multiplied by the hours it is up.

DEFAULT

Precompute scores in a job where the inputs allow it. Keep endpoints for request-time inputs: scale to zero in development, the smallest provisioned concurrency in production.

SIGNS YOU CHOSE WRONG

An endpoint's DBUs grow every hour of the day while its request count stays flat. The calling app retries on timeouts after quiet periods. Dev endpoints run all weekend.

CHECK IT

Rank endpoints by spend, then turn on scale to zero where a cold start is acceptable.

```
SELECT usage_metadata.endpoint_name, SUM(usage_quantity) AS dbus
FROM system.billing.usage
WHERE billing_origin_product = 'MODEL_SERVING'
      AND usage_date >= current_date() - INTERVAL 30 DAYS
GROUP BY ALL ORDER BY dbus DESC;

# update-config replaces the whole config: get, edit, put
databricks serving-endpoints get churn-dev -o json > cfg.json
# in cfg.json, set scale_to_zero_enabled true, keep the rest
databricks serving-endpoints update-config churn-dev --json @cfg.json
```

Go deeper: the old config stays live until the new one is ready, so a bad edit costs a retry, not an outage. Endpoint scale to zero is not Lakebase scale to zero (10.05). See C.22.

RELATED **10.04** You need predictions for 50 million rows every night **10.05** An app needs a customer's features in 10 milliseconds **8.01** The Databricks bill doubled

10.04 You need predictions for 50 million rows every night. The job calls the serving endpoint row by row.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A real-time endpoint is sized for interactive traffic. For a classical model, load it from the registry as a **Spark UDF** and score inside the job. Use **ai_query** when the model must stay behind an endpoint, such as a foundation model.

Cost of failure **Cost**Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Loop over a real-time endpoint	Reuses the endpoint the app already calls. The same model answers online and offline.	Every row crosses the network. Throughput stops at the endpoint's provisioned concurrency, and your loop owns batching, retries and partial failures.
ai_query against an endpoint	One SQL statement; Databricks parallelises the calls. <code>failOnError => false</code> keeps good rows and returns an error per bad one. Batch route for foundation models.	Cost follows tokens: run 1,000 rows, read <code>system.billing.usage</code> , multiply. An AI Gateway rate limit on the endpoint returns 429 to the batch (10.08). Not on classic SQL warehouses.
spark_udf in a job DEFAULT	The model is loaded on every executor and scales with the cluster. No endpoint. The job resolves <code>@champion</code> once and stamps the version on every row.	Library versions differ from training: use <code>env_manager="virtualenv"</code> . Deep learning models want GPU compute for the scoring task.

FIG. 10.04 Move the model to the data for classical models. Send the data to the model only when it cannot move.

DEFAULT

Classical models: `spark_udf` on job compute, writing a scores table. LLM tasks: `ai_query` in one statement over the whole table, with `failOnError => false`.

SIGNS YOU CHOSE WRONG

The nightly run takes hours while cluster CPU stays low. The endpoint is resized up every evening. A single failed call fails the whole run.

CHECK IT

An alias can move between runs, so pin the resolved version for audit and store it with the scores.

```
import mlflow
from pyspark.sql import functions as F
m = "sales_prod.ml.churn"
ver = mlflow.MlflowClient().get_model_version_by_alias(m, "champion").version
predict = mlflow.pyfunc.spark_udf(
    spark, f"models:{m}/{ver}", env_manager="virtualenv"
)(spark.table("sales_prod.core.customer_features")
    .withColumn("churn_score", predict())
    .withColumn("model_version", F.lit(ver))
    .write.mode("overwrite").saveAsTable("sales_prod.ml.churn_scores"))
```

RELATED **10.03** A model serving endpoint answers ten requests an hour **10.02** Models were saved to a volume as files
8.06 Every serverless job starts fast

10.05 An app needs a customer's features in 10 milliseconds. A SQL warehouse takes a second.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A SQL warehouse is built to scan, not to answer thousands of single-key lookups. Serve the row from **Lakebase**, through a **synced table** that follows the Delta table, or through **Feature Serving** when a model is the caller.

Cost of failure **Slow queries**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Query the SQL warehouse	No copy. Unity Catalog grants apply directly. Fine for dashboards and batch.	Each lookup is a full query. The warehouse runs all day for the app and queues when calls pile up.
Feature Serving endpoint	A REST lookup by key, with on-demand features computed by Unity Catalog functions. Models on Model Serving look features up automatically.	Needs the table published to an online store, plus one more endpoint to size and pay for.
Lakebase synced table DEFAULT	The app reads managed Postgres by primary key, with indexes. A pipeline follows the Delta table. GA on Azure since March 2026.	The copy lags by the sync mode and is read-only. Null keys are not synced. Lakebase runs in a subset of Azure regions.

FIG. 10.05 The lakehouse keeps the truth. A row store keeps a copy shaped for one-key reads.

DEFAULT

Triggerred for scheduled features, Continuous only where seconds matter, Snapshot when over about 10 percent of rows change per cycle. Sync work bills as Lakebase capacity units, about 150 rows a second per unit.

SIGNS YOU CHOSE WRONG

A warehouse never stops because an app polls it. The app caches features itself. A continuous sync follows a table rewritten nightly.

CHECK IT

Triggerred and Continuous read the change data feed, so turn it on first. Then create the synced table on a Lakebase branch.

```
ALTER TABLE sales_prod.core.customer_features
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

databricks postgres create-synced-table sales_prod.serving.customer_features --json '{
  "spec": {"source_table_full_name": "sales_prod.core.customer_features",
    "branch": "projects/app-db/branches/production",
    "primary_key_columns": ["customer_id"], "scheduling_policy": "TRIGGERED",
    "postgres_database": "appdb", "create_database_objects_if_missing": true}}'
```

GOVERNANCE

Unity Catalog row filters, column masks and grants do not follow the data into Postgres. Access is by Postgres role: `databricks_superuser` grants `SELECT`. The sync role owns the table, so put a view with row security on top, or sync only what every reader may see. Production branches scale to zero after 24 hours idle by default and resume in milliseconds. See C.22.

RELATED

10.03 A model serving endpoint answers ten requests an hour

6.01 You moved the warehouse to X-Large

10.06 Vector search returns documents you deleted last week

10.06 Vector search returns documents you deleted last week. The table no longer has them.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

A Delta Sync index in **AI Search** is its own copy of the rows. In **Triggered** mode it changes only when something calls a sync, and it only deletes what the **change data feed** of its source table shows as deleted.

Cost of failure **Wrong data**
Status **GA** · Verified **Oct 2026**

	SQL ON THE CHUNKS TABLE	DELTA SYNC INDEX, TRIGGERED
Reads	The latest table version	Its own copy of rows and embeddings
Changes arrive	On every commit	Through the change data feed, at the next sync
Document deleted, chunks kept	Chunk rows still there	Still returned, sync or not
Chunks deleted, no sync	Gone	Returned until the next sync

FIG. 10.06 The index follows the table it was built on, and only when it is told to.

CAUSE

Two copies sit between the document and the answer. A chunking job that only appends keeps the chunks of a deleted document. The index syncs from the chunks table: Continuous within seconds on always-on compute; Triggered only through the SDK, REST API or **Sync now**. Both apply the deletes in the change data feed.

FIX

Make chunks follow documents, deletes included, then sync in the next task.

```
MERGE INTO kb_prod.rag.chunks t USING kb_prod.rag.chunks_staged s
ON t.chunk_id = s.chunk_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE THEN DELETE; -- staged holds the full current set

# next task, after %pip install databricks-ai-search
from databricks.ai_search.client import AISearchClient
AISearchClient().get_index(index_name="kb_prod.rag.chunks_index").sync()
```

PREVENT

Pick the sync mode from how stale an answer may be: Triggered on a schedule for most corpora, Continuous only when seconds matter. An erasure request needs more: delete the chunks, sync, then **REORG TABLE kb_prod.rag.chunks APPLY (PURGE)** and **VACUUM**. The old rows and the change feed files survive time travel for the 7 day default retention, so sync inside that window. Verify: the chunks table returns no rows for the document ID, and an index query filtered on it returns none.

RELATED

3.10 A Change Data Feed reader missed changes

7.16 A customer asked to be erased

10.05 An app needs a customer's features in 10 milliseconds

10.07 Genie gives a different revenue figure from the dashboard for the same month.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

The definition of revenue lives in the dashboard's SQL, so **Genie** writes its own. Define it once as a **metric view** in Unity Catalog and point both at it, then pin it with **trusted assets** and benchmarks.

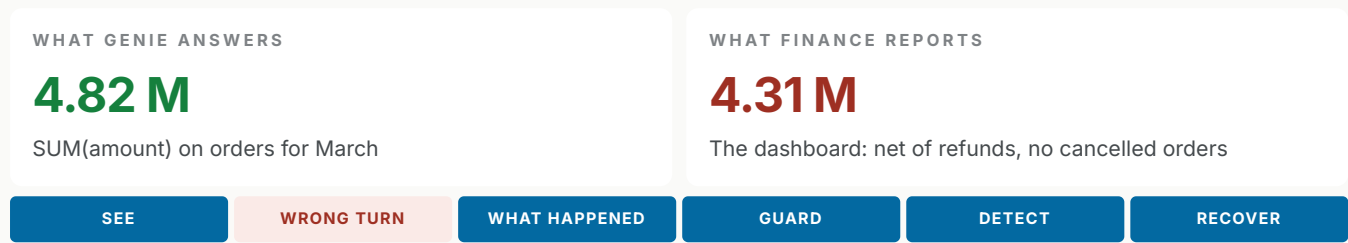
Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

FIG. 10.07 Both numbers come from the same table. Only one of them follows the business rule.

WRONG TURN

"Genie reads the same tables as the dashboard, so it gives the same numbers."

WHAT HAPPENED

A Genie Agent writes SQL from table metadata, instructions and examples. The refund and cancellation rules lived only in the dashboard's dataset. Permissions differ too: a dashboard shares the publisher's data permissions by default, Genie always queries as the user, so a row filter shrinks the user's total.

1 GUARD

Define it once. The dashboard dataset and the agent both read the metric view.

```
CREATE OR REPLACE VIEW sales_prod.core.revenue_mv WITH METRICS LANGUAGE YAML AS $$
version: 1.1
source: sales_prod.core.orders
filter: source.status <> 'CANCELLED'
fields:
  - name: Order Month
    expr: DATE_TRUNC('MONTH', source.order_date)
measures:
  - name: Net Revenue
    expr: SUM(source.amount) - SUM(source.refund_amount)
$$;
```

2 DETECT

Make this SQL the benchmark answer for the month's question. Rerun after every change.

```
SELECT `Order Month`,
       MEASURE(`Net Revenue`)
FROM sales_prod.core.revenue_mv
GROUP BY ALL ORDER BY 1;
```

3 RECOVER

Tell users of the Genie figure which number stands. Replace the raw orders table in the agent with the metric view. Save the monthly question as a trusted asset, so Genie marks the answer verified. Review the dashboard's permission mode.

RELATED [6.07](#) A shared dashboard shows viewers data they cannot query

[C.07](#) Views, materialized views, streaming tables and metric views

10.08 A team shares one foundation model endpoint and one project burns the whole token budget.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

An endpoint-level limit is **global**: when one project reaches it, every project is throttled. Add **per-user and per-group limits**, tag usage by project, and give a project that needs its own ceiling its own **model service** in Unity Gateway.

Cost of failure **Cost**
Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Endpoint-level limit	One number protects the provider quota.	One project reaches it and everyone gets 429.
Per-user and group limits	AI Gateway on the endpoint: a default per-user limit, custom ones for users, service principals and groups, in queries or tokens per minute.	At most 20 limits, 5 for groups. A rate per minute, not a budget. Projects on one service principal share a limit.
Model service per project DEFAULT	Unity Gateway. Own rate limits, request tags, inference table and EXECUTE grants. Fallbacks. GA since 4 August 2026.	Clients move to the <code>/ai-gateway/</code> route. Limits and guardrails skip <code>ai_query</code> batch jobs. Not on Azure Government.

FIG. 10.08 A limit shares capacity fairly only when each project has its own identity or its own service.

DEFAULT

A model service per project, tagged `project`, with a limit below the shared ceiling. Old endpoints keep per-group limits until clients move.

SIGNS YOU CHOSE WRONG

A batch job throttles an interactive app. One service principal serves three projects. Nobody can say which spent the tokens.

STATUS, FEATURE BY FEATURE

On the endpoint: usage tracking, rate limits and inference tables GA; fallbacks GA for external models only (`429` or `5XX`, two at most); guardrails Public Preview. In Unity Gateway: service policies (guardrails), the unified trace table and external model spend in budgets Beta. Its rate limits and fallbacks carry no label in the docs.

BEFORE YOU CHANGE IT

Enforce Unity Gateway is workspace-wide: legacy pay-per-token endpoints then fail. List callers, grant **EXECUTE** on the new services (`MODEL` grants do not carry over), test in dev, get the platform owner's sign-off. No revert steps are documented, so test one. Gateway tracking and inference tables are billed.

CHECK IT

Rank spend by tag and caller. The older route logs to `system.serving.endpoint_usage`.

```
SELECT request_tags['project'] AS project, requester, SUM(total_tokens) AS tokens
FROM system.ai_gateway.usage
GROUP BY ALL ORDER BY tokens DESC; -- tag: Databricks-Ai-Gateway-Request-Tags header

databricks serving-endpoints put-ai-gateway shared-llm --json '{"usage_tracking_config":
{"enabled":true,"rate_limits":[{"calls":600,"key":"user","renewal_period":"minute"}]}'
```

- RELATED
- [10.10](#) Pay per token worked in the pilot
- [10.09](#) The endpoint returns 200 for every request
- [7.17](#) A row filter or mask blocks your own recovery jobs

10.09 The endpoint returns 200 for every request. Answers got worse a month ago.

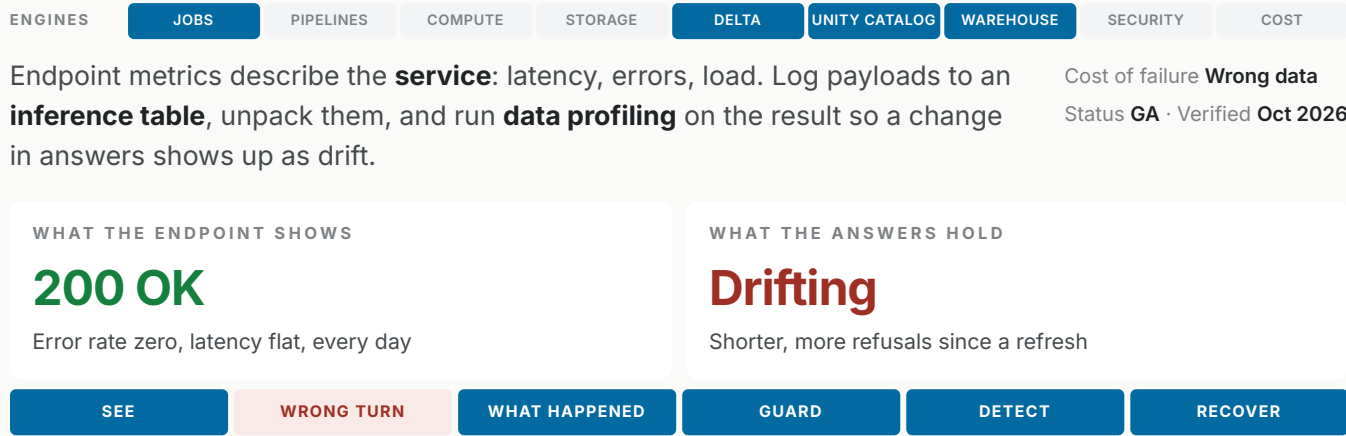


FIG. 10.09 The Metrics tab keeps 14 days and has no answer quality field.

WRONG TURN

"The endpoint is healthy, so the model is working."

WHAT HAPPENED

The index, prompt or upstream data changed and the endpoint answered without error. No payloads were kept to compare. Logging also stops quietly if the table's schema or name changes or it is deleted.

1 GUARD

Turn on inference tables in AI Gateway: rows arrive in seconds for custom models, within an hour for Foundation Model APIs. Route-optimised endpoints are Public Preview. Flatten the JSON.

```
databricks serving-endpoints put-ai-gateway support-bot --json '{"inference_table_config":
{"enabled":true,"catalog_name":"sales_prod","schema_name":"ml","table_name_prefix":"bot"}}'
```

```
CREATE OR REPLACE TABLE sales_prod.ml.bot_log AS -- refresh from a job
SELECT request_time AS ts, status_code, length(
  response:choices[0].message.content) AS answer_chars
FROM sales_prod.ml.bot_payload;
```

2 DETECT

Profile the flat table daily. Alert on `MAX(ts)` too: a stopped log looks like a quiet day.

```
day = AggregationGranularity.AGGREGATION_GRANULARITY_1_DAY
cfg = DataProfilingConfig(output_schema_id=schema.schema_id, assets_dir="/Workspace/Shared/dq",
  time_series=TimeSeriesConfig(timestamp_column="ts", granularities=[day]))
w.data_quality.create_monitor(monitor=Monitor(object_type="table",
  object_id=tbl.table_id, data_profiling_config=cfg))
```

3 RECOVER

Alert on `_drift_metrics`, find the change, and point the `champion` alias at the last good version. MLflow production monitoring (LLM judges on traces) is Beta.

RELATED [10.08](#) A team shares one foundation model endpoint [10.02](#) Models were saved to a volume as files

[6.07](#) A shared dashboard shows viewers data they cannot query

10.10 Pay per token worked in the pilot. Production traffic makes the bill unpredictable.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Pay-per-token bills what you use from shared capacity, with input and output token limits per minute. **Provisioned throughput** bills the capacity you hold, has no token limit and gives steady latency. Steady production traffic fits the second.

Cost of failure **Cost**

Status **GA** · Verified **Oct 2026**

OPTION	GOOD AT	BREAKS WHEN
Pay-per-token	No setup, no idle cost. Right for development and spiky traffic.	Shared capacity, variable latency. Limits on input and output tokens per minute and queries per hour, then 429. Retries add tokens.
Provisioned, on demand DEFAULT	Dedicated capacity, no token limit, up to 200 queries per second per workspace. Billed for the capacity you allocate; change or remove it any time.	Traffic is low or short: capacity bills while up. Only some models support it. Deployment can fail for lack of GPU capacity.
Provisioned, reserved	Lower unit rate for 1 or 3 months. Overflow goes to priority pay-per-token. GA.	Usage falls: the full term still bills. No cancelling. One model listed.

FIG. 10.10 Pay-per-token bills the traffic. Provisioned throughput bills the hours the capacity is up.

DEFAULT	SIGNS YOU CHOSE WRONG
Pay-per-token for development and bursts. When hourly volume is steady across the working day, move that model to provisioned throughput. Reserve after weeks of steady use.	429s at the same hours daily. Callers retry in loops. Token spend rises faster than users, and next month cannot be forecast.

STATUS AND LIMITS TO CHECK

Limits are per model: 200,000 input and 20,000 output tokens per minute for Claude models in the rate limits table. Priority pay-per-token (`service_tier="priority"`) costs more per token and is best-effort; it is Preview for the model listed. Provisioned Llama 4 Maverick is Public Preview.

CHECK IT

Chart tokens per hour for 30 days. A flat daytime plateau favours provisioned capacity, spikes favour pay-per-token. Price the plateau at the DBU rate on the Azure pricing page and compare. The model optimization info API gives the throughput increment.

```
SELECT date_trunc('HOUR', u.request_time) AS hr,
       SUM(u.input_token_count) AS in_tok, SUM(u.output_token_count) AS out_tok
FROM system.serving.endpoint_usage u
JOIN system.serving.served_entities e USING (served_entity_id)
WHERE e.endpoint_name = 'databricks-gpt-oss-120b'
      AND u.request_time >= current_timestamp() - INTERVAL 30 DAYS
GROUP BY ALL ORDER BY hr;
```

RELATED

10.08 A team shares one foundation model endpoint

10.04 You need predictions for 50 million rows every night

10.03 A model serving endpoint answers ten requests an hour

10.11 Features in the online store differ from the ones the model trained on. Scores drift and no job fails.

ENGINES

JOBS

PIPELINES

COMPUTE

STORAGE

DELTA

UNITY CATALOG

WAREHOUSE

SECURITY

COST

Training read feature values **as of each label**. The online store holds the **latest synced value**, so it can be older, newer or missing. Look features up the same way in both places and watch the **sync lag**.

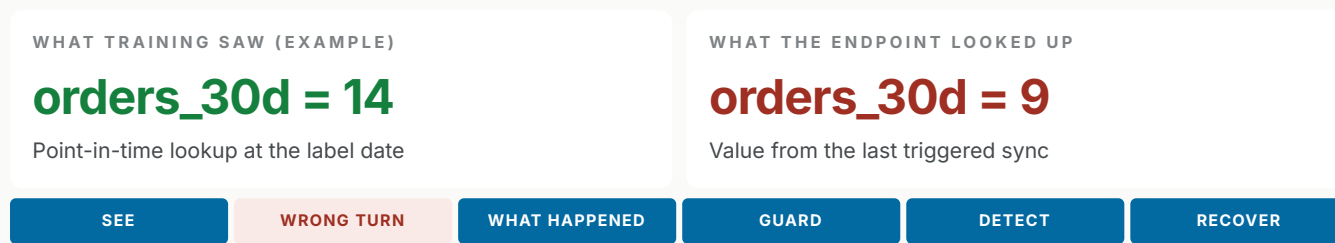
Cost of failure **Wrong data**Status **GA** · Verified **Oct 2026**

FIG. 10.11 Same table name, two different values. The endpoint answers 200 with a worse score.

WRONG TURN

"The online store is a copy of the feature table, so the model sees the same features."

WHAT HAPPENED

Three gaps. The copy lags: continuous sync has a 15 second minimum interval, triggered 5 minutes or more. A time series table is published as the **latest value per key only**, so online has no point-in-time lookup or [lookback_window](#). Null-key rows are left out, and a caller can override a looked-up feature.

1 GUARD

The logged model looks features up itself, so callers send keys only. Train with a point-in-time lookup, enable change data feed, and pick the mode by tolerable staleness. Online stores are GA, never scale to zero, and bill until deleted.

```
lk = [FeatureLookup(table_name="sales_prod.ml.customer_features",
    feature_names=["orders_30d"], lookup_key="customer_id",
    timestamp_lookup_key="label_ts")] # value as of the label
ts = fe.create_training_set(df=labels, feature_lookups=lk, label="churned")
fe.log_model(model=clf, artifact_path="model", flavor=mlflow.sklearn,
    training_set=ts, registered_model_name="sales_prod.ml.churn")
fe.publish_table(online_store=fe.get_online_store(name="churn-store"),
    source_table_name="sales_prod.ml.customer_features", publish_mode="CONTINUOUS",
    online_table_name="sales_prod.ml.customer_features_online") # lowest lag, highest cost
```

2 DETECT

Read the sync state. Compare one key in Delta and Postgres.

```
t = w.postgres.get_synced_table(
    "synced_tables/sales_prod.ml.cust_online")
print(t.status.detailed_state,
    t.status.last_sync_time)
```

3 RECOVER

Move to a faster mode or fix the sync. Retrain if the model learned values online cannot give. Drop request overrides of stored features. Alert on a stale [last_sync_time](#). Never patch Postgres: the next sync overwrites it.

RELATED [10.05](#) An app needs a customer's features in 10 milliseconds [10.02](#) Models were saved to a volume as files

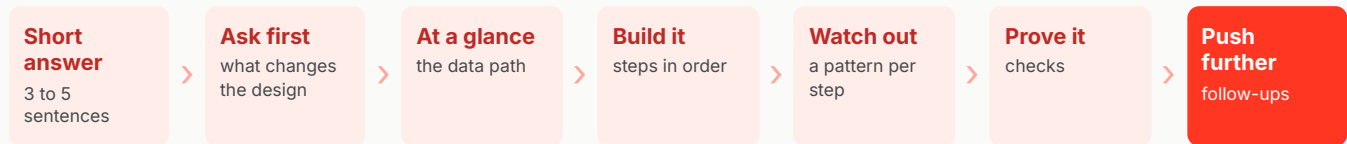
[10.03](#) A model serving endpoint answers ten requests an hour



System Design Scenarios.

Twenty-six design briefs, from a nightly file load to a platform handover. Each answer runs the way I'd explain it to a colleague: the short answer, the questions I'd ask first, the build in order, and the trap at each step. The traps link to the patterns in Parts 1 to 10.

HOW EACH ANSWER RUNS



BUILDING ON DATABRICKS

S.01	Nightly files from 120 stores	182
S.02	Twelve sources, one framework	185
S.03	A 3 billion row fact table	188
S.04	Three business units, one catalog design	191

FRESHNESS AND REAL TIME

S.09	Ten minutes behind Event Hubs	207
S.10	CDC from SQL Server	210
S.11	Records up to 30 days late	213
S.12	Operations dashboard in trading hours	215

SECURITY AND REGULATED DATA

S.17	Patient records from four systems	229
S.18	Every source behind a firewall	232
S.19	Who read the sensitive table last quarter	235
S.20	Identities for 200 jobs	238

MIGRATING INTO DATABRICKS

S.05	Hive metastore to Unity Catalog	194
S.06	Synapse Spark and ADF to Databricks	197
S.07	Classic clusters to serverless	200
S.08	On-premises Hadoop to Databricks	204

SERVING AND SHARING

S.13	Power BI for 2,000 users	218
S.14	External partners read part of Gold	221
S.15	Fabric reads Databricks Gold	224
S.16	Self-service SQL without a runaway bill	226
S.25	A model and its features behind an app	253

RUNNING THE PLATFORM

S.21	Dev, test and prod for three teams	241
S.22	The bill rose 60 percent	244
S.23	Recovering from a bad Monday release	247
S.24	Handover when the only engineer leaves	250
S.26	Recover the platform in another region	256

S.01 Design the nightly sales load for a retailer. Its 120 stores each drop a CSV file into an ADLS landing container by 02:00, and dashboards must be ready by 07:00.

Short answer. I'd land the files in a Unity Catalog volume and load them with one Lakeflow job that starts at 02:15 on serverless compute. The job runs a pipeline in Lakeflow Spark Declarative Pipelines: Auto Loader appends new files to Bronze, Silver keeps the latest file for each store and day, and Gold is a set of materialized views. A check task then compares the stores that arrived with a control table and names the missing ones. A second run at 05:15 picks up late stores, and the dashboard refreshes at 06:00.

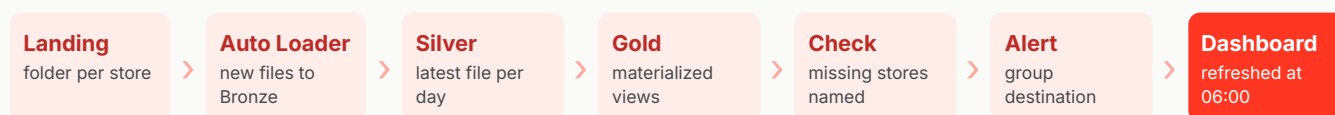
QUESTIONS TO ASK FIRST

- **How do stores reach the landing container, and which accounts are private?** Stores upload over the internet, so the landing account may need public access, scoped to a folder per store. The accounts behind Bronze to Gold stay private, and serverless needs its own path to them (S.18).
- **Can a store send its file again, or late?** Then Silver must replace that store's day, not add to it, and every upload needs a new file name.
- **If a store is missing at 02:15, do we wait?** Usually not. Publish on time, name the missing store, and load it later.
- **Is the CSV layout the same for every store?** One declared schema for all stores keeps Bronze stable. A store on old till software may need its own mapping.

WHICH TOOL, AND WHY

- **Lakeflow job with a pipeline.** My choice. Auto Loader remembers which files were loaded, the pipeline orders Bronze, Silver and Gold, and the job adds the schedule, the checks and the alerts.
- **Auto Loader in a notebook task.** Works well and gives full control of the write. I would then write the replace logic, the retries and the table maintenance myself.
- **COPY INTO from a SQL task.** Simple and idempotent for files. It has no expectations and no dependency graph between Bronze, Silver and Gold.
- **Azure Data Factory.** Keep it if the retailer already runs ADF for other loads. It can call the same Lakeflow job with the Azure Databricks Job activity, and everything after landing stays the same.

AT A GLANCE



One Lakeflow job on serverless compute, with runs at 02:15 and 05:15. The pipeline loads, the job checks, and the dashboard schedule publishes.

HOW I'D BUILD IT

1 Govern the landing container

I register the landing container as an external location and create an external volume on it, with one folder per store. Each store uploads with its own credential, scoped to its folder and allowed to write only. Engineers read the files through the volume with `READ VOLUME`, so every read goes through Unity Catalog. File names carry the store ID and an upload timestamp. Only the landing account takes uploads from outside. The same-region accounts behind Bronze to Gold deny public access and admit serverless through a network security perimeter rule for the `AzureDatabricksServerless` service tag.

Watch out. An Azure data role such as Storage Blob Data Reader on the account lets people read the files around Unity Catalog, and nothing in Databricks shows it. **7.05**

2 Load Bronze with Auto Loader

The pipeline's first table is a streaming table that reads the volume with Auto Loader. I declare the schema instead of inferring it: CSV columns arrive as strings unless `cloudFiles.inferColumnTypes` is on, and inference can change between runs. A rescued data column catches values that do not fit, and each row keeps `_metadata.file_path` and the file's modification time.

Watch out. A file that does not match the schema lands as rescued data or nulls while the run reports success. Count rescued rows per file. **2.04**

3 Make every resend a new file

Auto Loader loads each file name once. With `cloudFiles.allowOverwrites` at its default of false, a corrected file uploaded under the same name is ignored. So the rule for stores is a new name for every upload, and Silver decides which file wins. I leave overwrites off, because with them on a file rewritten in place can load twice.

Watch out. A corrected file with the old name never loads, and the run still succeeds. **2.01**

4 Replace each store's day in Silver

Silver is a materialized view over Bronze. For each store and business date it keeps only the rows from the latest file, chosen by a window over the file timestamp, then applies types and removes duplicate receipt lines. A resent file therefore replaces the whole day for that store, and receipts the store removed disappear too. On serverless, the pipeline refreshes the view incrementally where it can.

Watch out. A full refresh truncates Bronze and reloads only the files still in landing. I set `pipelines.reset.allowed` to false on Bronze. **5.08**

5 Build Gold as materialized views

Gold holds daily sales by store and product, and by region. Each is a materialized view in the same pipeline, so it refreshes after Silver in the same update. The pipeline maintains its tables, and I cluster the largest Gold view by business date.

Watch out. A materialized view that fails to refresh keeps serving its last result, so a dashboard can look normal with yesterday's numbers. **6.05**

6 **Orchestrate with one job**

A Lakeflow job runs at 02:15 and 05:15 on serverless compute in standard mode: a start-up of 4 to 6 minutes costs nothing against a five-hour window. Its tasks are the pipeline update and a SQL check task. The job runs as a service principal and is deployed with Declarative Automation Bundles.

Watch out. On a night with no files the pipeline update succeeds and writes nothing. The check task, not the pipeline, has to turn that into a failure. **5.01**

7 **Name missing stores and alert**

A control table lists every active store with its expected arrival time. The check task compares it with the files Bronze saw for the business date, writes one row per store to a run log table, and fails when stores are missing. The dashboard shows a missing store as missing, never as zero sales. Failure notifications go to a notification destination for the support group, not to one person's address. The explicit checks stay the primary guard. For tables nobody wrote a check for, anomaly detection in data quality monitoring, in Public Preview, adds a net over the Silver and Gold schemas for staleness and short row counts, and I would not depend on it alone.

Watch out. The check task covers only what the control table lists. A Gold table that quietly stops updating needs the schema-wide net. **5.17**

8 **Publish the dashboard**

The AI/BI dashboard reads Gold through a serverless SQL warehouse and is published with shared data permissions, so store managers see results without access to Silver. The dashboard keeps its own result cache, best effort and up to 24 hours old, and new data in Gold does not refresh it. A dashboard schedule at 06:00, after the late run, reruns its datasets so the first viewer opens tonight's numbers.

Watch out. With shared data permissions, viewers see whatever the publisher can read. The publisher should hold SELECT on Gold and nothing more. **6.07**

SECURITY IN ONE BREATH

Stores can only write to their own landing folder, and nobody reads landing outside Unity Catalog. The job and the pipeline run as a service principal that owns Bronze, Silver and Gold. Analysts get SELECT on the Gold schema only, and store managers see Gold through the dashboard.

HOW I'D KNOW IT WORKS

- The run log has one row per active store per business date, and missing stores are named by 02:45.
- Rows in each Bronze file match the line count of the file, and the rescued data column is empty.
- Resending a store's file replaces its day in Silver, and the totals do not double.
- At 06:00 the dashboard shows the latest business date, with missing stores flagged.

A WORKED EXAMPLE

Situation. For example, a retailer with 120 stores ran a notebook at 03:00 that read every file in the landing folder with inferred types and appended it to one table. Resent files doubled a store's sales about twice a month, and nobody knew which stores were missing until managers called.

Task. Make the load repeatable, make resends safe, and have the dashboard right by 07:00.

Action. In week 1 we created the volume, the control table and the Bronze table with a declared schema. In week 2 we built Silver and Gold as materialized views, the check task and the alert. In week 3 we ran the new job beside the old notebook and compared daily totals per store.

Result. In this example the parallel run found 3 days where the old table had doubled a store's sales after a resend. After cutover the job finished by 03:00 on a normal night, and missing stores were named in the support channel by 02:45.

IF THEY PUSH FURTHER

What if there were 1,200 stores, each sending a file every hour?

The design holds. I'd switch the job to a file arrival trigger on the volume, with file events on the external location, and set a wait after the last change so a burst of files starts one run. **5.14**

What if the dashboard had to be ready by 03:00?

Start at 02:05 and set the job's performance target to **PERFORMANCE_OPTIMIZED**. Start-up gets faster and the DBU rate goes up, so I'd do it for this job only. **8.06**

What if a store resends a file from last week?

Silver keeps the latest file per store and day, so that day is replaced and Gold follows on the next refresh. The control table should flag resends older than a few days, because they change numbers people have already reported.

S.02 Design a metadata-driven ingestion framework that loads twelve source systems of mixed types into Unity Catalog. About one new source arrives each month, and onboarding it should take configuration, not code.

Short answer. I'd keep the framework as configuration in Git: one YAML file per source, deployed with Declarative Automation Bundles. Sources that Lakeflow Connect covers use its managed connectors, created from the same files. Everything else lands as files in a volume and goes through one generic pipeline per source group in Lakeflow Spark Declarative Pipelines which defines its Bronze and Silver tables in a loop over the configuration. A new source of a known type is a new file and a pull request.

QUESTIONS TO ASK FIRST

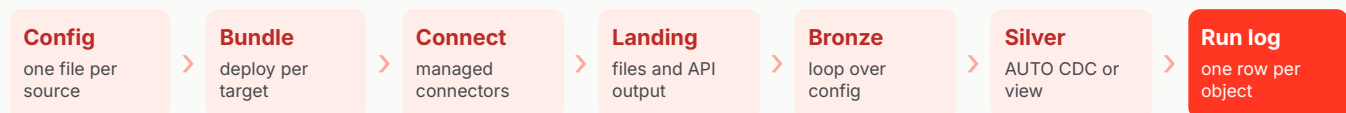
- **Which types are the twelve sources, and where do they sit?** Databases, SaaS applications, files and APIs each take a different path. Sources on premises or behind a firewall decide the networking first.
- **Does every table have a primary key or a reliable change column?** That decides between change capture, a cursor column and a full load. Without either, deletes are invisible.
- **Who adds a new source, and what review do they need?** If several teams add sources, the configuration needs a schema, validation in CI and a reviewer.
- **How fresh must each source be?** Most sources can share a daily run. A source that needs hourly data gets its own schedule in its file.

WHICH TOOL, AND WHY

- **Lakeflow Connect plus a config-driven pipeline.** My choice. Managed connectors take the sources they support, change capture included, and one generic pipeline handles files and APIs from the same configuration.

- **A notebook framework with a control table.** The familiar pattern from ADF days. It works, but I would write the watermarks, retries, schema handling and quality checks that pipelines already give me.
- **DLT-META from Databricks Labs.** An open-source framework that generates pipelines from onboarding files. Worth a look, but as a Labs project it has no Databricks support, so I would own its fixes.
- **Azure Data Factory with a control table.** Strong for on-premises sources through a self-hosted integration runtime. It splits the framework across two products and two places to monitor.

AT A GLANCE



One bundle deploys every source. Managed connectors and the generic pipeline both write Bronze, and Silver is built the same way for all of them.

HOW I'D BUILD IT

1 Define the source contract

Each source gets a YAML file in Git: source type, connection name, objects, load mode, key or cursor columns, schedule, owner and target schema. A JSON schema checks every file in CI, so a missing key column fails the pull request instead of the 02:00 run. Credentials never go in these files. They sit in Unity Catalog connections and Unity Catalog secrets, referenced by name. Secrets need Runtime 17.3 LTS or serverless environment 4 and above, and I check their release status first. Where Key Vault must stay the store, a Key Vault-backed scope needs the vault on the access policy model, not Azure RBAC. Catalog names come from the bundle target, never from the file.

Watch out. A config file that names the prod catalog directly sends a test deploy into production tables. **5.04**

2 Use managed connectors where they exist

For SQL Server, Salesforce, Workday and the other sources Lakeflow Connect covers, the framework creates a managed ingestion pipeline from the file. Databases with a reliable cursor column can use the query-based connectors, generally available since May 2026, which need no gateway. The log-based database connectors run a gateway on classic compute all day for each pipeline, so I count that as a standing cost. I check each connector's release status first, because many are still in Beta.

Watch out. A Beta connector can change between releases. Keep it away from loads with a hard deadline until it is generally available. **2.14**

3 Land everything else as files

APIs and file drops land raw in a volume under source, object and load date. A small Python task per API source writes each response as a JSON file and nothing else. Landing as files gives every source the same next step, and the raw data stays available for a replay.

Watch out. A task retry after a timeout repeats the whole call. Retries suit transient errors, so cap them and log each attempt. **5.02**

4 Generate Bronze in a loop

One generic pipeline per source group reads the group's config files when it starts and defines a streaming table per object, using Auto Loader with the declared schema. I wrap the table definition in a function and call it once per object, so each table binds its own path and options. Every Bronze row keeps the source, the file path and the load time.

Watch out. Remove an object from the config and its table becomes inactive on the next update: it still answers queries, but nothing updates it again. Retire objects with a flag, and drop their tables on purpose.

5 Build Silver with AUTO CDC

For file and API Bronze, Silver is an AUTO CDC flow keyed on the configured key columns and sequenced by the change timestamp or the load time. SCD type 1 is the default, and objects that need history set type 2 in their file. Deletes come through only where the source writes a delete flag. Connector tables already receive updates and deletes, so I never run AUTO CDC over them: Silver reads them directly or as a materialized view, or through change data feed for a stream (S.10).

Watch out. A renamed source column arrives as a new column while the old one fills with nulls. Each source needs a rule for renames. **3.09**

6 Check quality in the pipeline

Each object can list expectations in its file: key not null, dates in range, amounts numeric. The generic code applies them as drop expectations on the view that feeds Silver, and a quarantine table keeps the dropped rows per source. Rules that must stop the load use fail expectations instead.

Watch out. A plain expectation only counts violations. The pipeline succeeds with every bad row in Silver. **5.06**

7 Run the groups from one job per schedule

A Lakeflow job per schedule runs the connector pipelines and the generic pipelines as tasks, grouped by source system. A last task reads the pipeline event logs and writes one run log row per object: rows read, rows written, rows dropped and status. Failures notify the owner named in the config through a notification destination.

Watch out. A downstream task with the wrong Run if condition still runs when its upstream task was skipped, and builds Gold on stale Silver. **5.10**

8 Onboard a source

A new source of a known type is a config file, a pull request and a deploy. The bundle goes to the test target first, I run the new source once and compare row counts with the source, then the same commit goes to production. The first source of a new type, a new API style for example, needs code once.

Watch out. A bundle deployed to prod from a laptop runs every job as the person who deployed it, unless it sets run_as. **9.05**

SECURITY IN ONE BREATH

Source credentials live in Unity Catalog connections and secrets, never in config files or Git. Jobs and pipelines run as one service principal per source group, which can write only its own Bronze and Silver schemas. Engineers change sources only through pull requests, and analysts read Silver and Gold, never landing.

HOW I'D KNOW IT WORKS

- Every run has one run log row per active object, and the count matches the config.
- A test source added as a config file loads on its first run with no code change.
- Rerunning the same window adds no rows to Silver.
- A row deleted in a connector source, or flagged deleted in a file source, is gone from its Silver table after the next run.

IF THEY PUSH FURTHER**What if there were 300 sources instead of twelve?**

The config stays the same. I'd split pipelines and jobs by source group so one failure stops one group, and keep each managed connector within its recommended size, 250 tables per pipeline for SQL Server. **5.05**

What if a source cannot report deletes?

A cursor column sees inserts and updates only. I'd add a weekly full key comparison against the source and mark missing keys as deleted, or ask the source team for a delete log. **C.08**

What if one API source is slow and holds up its group?

Give it its own task with a timeout, so a slow API fails alone and the rest of the group carries on. **5.03**

S.03 Design a 3 billion row sales fact table that analysts filter by date and region, and that a nightly job updates with late corrections.

Short answer. I'd keep the fact as one managed Delta table with liquid clustering on sale date and region, and no partitions. Predictive Optimization keeps it clustered and vacuumed. The nightly job lands corrections in a staging table and runs one MERGE whose condition names the affected dates, so only files for those dates are touched. Analysts query the fact on a serverless SQL warehouse, and Power BI reads a daily aggregate instead of three billion rows.

QUESTIONS TO ASK FIRST

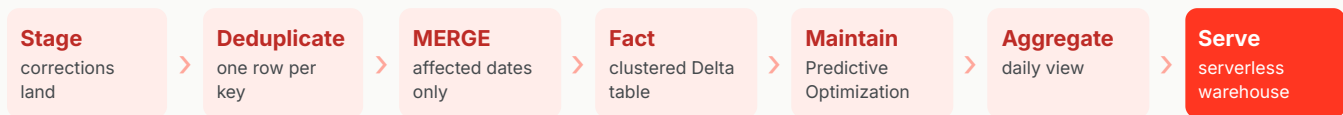
- **How many rows change each night, and how far back?** A few thousand corrections over the last 30 days is a different MERGE from a restatement of a whole quarter.
- **Which filters do analysts actually use?** The query history shows the columns in WHERE clauses. Clustering keys should match them.
- **Does anything else write to the table?** Two writers on one table can conflict. One nightly writer avoids the question.
- **Does any tool outside Databricks read the files?** Deletion vectors and liquid clustering raise the table protocol, and an older reader may not open it.

WHICH TOOL, AND WHY

- **Liquid clustering on sale date and region.** My choice. It skips files on both columns, reclusters incrementally, and lets me change keys later without rewriting the table.
- **Partition by date.** Years of daily partitions give thousands of folders and small files on correction days, and a region filter still reads every file in each day.

- **Partition by month with Z-order on region.** The older pattern. Z-order is not incremental, so each run rewrites the data again. On Runtime 18.1 and above, a partitioned Delta table can move to liquid clustering in place.
- **Automatic liquid clustering.** `CLUSTER BY AUTO` lets Predictive Optimization choose keys from the query history. Good when filters are unknown. Here they are known, so I set them.

AT A GLANCE



One writer, one table, one aggregate for reports. The layout follows the filters, and the MERGE follows the layout.

HOW I'D BUILD IT

1 Create the table managed and clustered

I create the fact as a managed table with `CLUSTER BY (sale_date, region)`, in dev first, and the platform owner signs off before prod. Liquid clustering takes up to four keys, and I keep two, because on tables under 10 TB extra keys weaken skipping on a single column. Deletion vectors and row tracking are on, so a MERGE marks changed rows instead of rewriting whole files.

Watch out. Clustering keys need statistics, and Delta collects them on the first 32 columns only. Keep `sale_date` and `region` among them. **3.01**

2 Load history once, then cluster it

The initial three billion rows go in with one CTAS from the old table, which clusters on write when the data is large enough. If the old table is already a partitioned Delta table, Databricks Runtime 18.1 and above convert it in place instead, with `ALTER TABLE ... REPLACE PARTITIONED BY WITH CLUSTER BY (sale_date, region)`, and readers and writers barely pause. Either way I then run `OPTIMIZE ... FULL` once, so every existing file is clustered on the new keys.

Watch out. Defining the keys clusters nothing by itself. Writes too small for clustering on write wait for `OPTIMIZE` or Predictive Optimization. **3.02**

3 Stage and deduplicate the corrections

The nightly job appends the night's corrections to a staging table with the business key, the sale date and a sequence number. Before anything touches the fact, it keeps the latest row per key, so the MERGE never sees two source rows for one target row.

Watch out. Two source rows that match one target row make the MERGE fail, and the whole night fails with it.

4 MERGE with the dates in the condition

The MERGE joins on the business key and on `sale_date`, with the list of dates present in staging written into the condition. Delta then skips every file outside those dates. With deletion vectors, changed rows are marked and only the new row versions are written.

Watch out. A MERGE that joins on the key alone reads the whole table to find matches and rewrites far more than it changes. **3.08**

5 Keep one writer

Only the nightly job writes the fact, with max concurrent runs set to 1. A backfill uses the same job with a date parameter, never a second notebook beside it. Liquid clustering with deletion vectors and row tracking gives row-level concurrency, which reduces conflicts; one writer removes them.

Watch out. Two MERGE jobs on overlapping dates still conflict, and one of the runs fails. **3.07**

6 Let Predictive Optimization maintain it

The table is managed, so Predictive Optimization runs OPTIMIZE, VACUUM and ANALYZE when it decides they pay. I remove the hand-scheduled OPTIMIZE job the old table had. Retention stays at the defaults unless the business needs older versions.

Watch out. An hourly OPTIMIZE on top of Predictive Optimization adds cost and competes with the nightly MERGE. **3.03**

7 Serve an aggregate to reports

Power BI reads a daily aggregate by date, region and product, a materialized view that refreshes incrementally after the nightly job. Sums are DECIMAL, because floating-point SUM and AVG are not refreshed incrementally. Analysts who need detail query the fact on a serverless SQL warehouse.

Watch out. A DirectQuery report on the detail fact sends queries for every visual and every user to the warehouse. **6.09**

8 Decide what time travel covers

Corrections rewrite history, so people will ask what the table said last month. Time travel reaches back as far as the deleted file retention, 7 days by default. On a Unity Catalog managed table like this one, older versions are blocked from Runtime 12.2 even before VACUUM runs. Time travel, RESTORE and clones also fail on a table with a row filter, column mask or ABAC policy unless the running identity is exempt (7.17), so once regional users get a filter here, an ordinary engineer has no recovery route. Month-end figures that must be reproducible go into a snapshot table.

Watch out. Raising log retention alone does not help. Time travel stops at the deleted file retention. **3.04**

SECURITY IN ONE BREATH

The nightly job runs as a service principal, the only identity with MODIFY on the fact. Analysts get SELECT on the schema through a group, and a row filter limits regional users to their region where needed. Nobody reads the table's folder directly.

HOW I'D KNOW IT WORKS

- A query filtered on one day and one region reads a small share of the files, shown as files pruned in the query profile.
- **DESCRIBE HISTORY** shows the nightly MERGE touching files for the corrected dates only.
- Predictive Optimization history lists OPTIMIZE and VACUUM on the table every few days.
- The daily aggregate matches the fact for a sample of dates after each night.

A WORKED EXAMPLE

Situation. For example, a 3 billion row fact was partitioned by day into about 2,600 partitions, and the nightly MERGE joined on the order key only. It ran for 3 hours and wrote about 400 GB to apply 50,000 corrected rows.

Task. Bring the nightly job under 30 minutes and make region filters fast.

Action. On Runtime 18.1 we converted the table in place with REPLACE PARTITIONED BY WITH CLUSTER BY on sale date and region, ran OPTIMIZE FULL, and added the staged dates to the MERGE condition. Readers kept the same table name throughout.

Result. In this example the nightly MERGE took 12 minutes and wrote under 5 GB. Analyst queries filtered on one month and one region read about 2 percent of the files.

IF THEY PUSH FURTHER

What if analysts start filtering by product more than by region?

Change the keys with `ALTER TABLE ... CLUSTER BY`. New writes and OPTIMIZE use them at once, and OPTIMIZE FULL reclusters the existing data when there is time. **C.09**

What if a tool outside Databricks must read the fact?

It needs a reader that supports deletion vectors and liquid clustering. Give it a governed route such as OpenSharing instead of the folder. **3.06**

What if a team wants a test copy of the fact?

A shallow clone in the dev catalog costs almost nothing until either side changes, and shallow is enough to test the nightly MERGE. Clones fail on a table with a row filter, column mask or ABAC policy unless the identity is exempt (7.17), so if the regional filter is on, I build the test table from a sample of Silver instead. **3.12**

S.04 Design the Unity Catalog layout for three business units that share customers, with dev, test and prod workspaces for each.

Short answer. I'd use the region's one metastore, with a catalog per environment and business unit, such as `prod_retail` and `dev_retail`. Each catalog is bound to its own environment's workspaces and stored in its unit's own storage account. Shared customers live once, in a `prod_customer` catalog owned by a data steward group and bound read-only to every workspace that needs it, with personal data masked by policy for dev. Groups come from Entra ID at account level, and every grant and every ownership goes to a group.

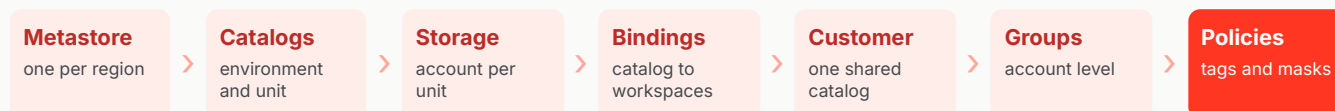
QUESTIONS TO ASK FIRST

- **Are all nine workspaces in one Azure region?** There is one metastore per region. Workspaces in another region need that region's metastore and OpenSharing between the two.
- **Who owns the customer record?** Shared data needs one owner and one place. Three copies drift within a month.
- **Must units be unable to see each other's data, or only unable to change it?** That decides between read grants across units and strict isolation.
- **Do developers need real data, and may dev compute reach prod storage?** A read-only binding still needs network reach from dev compute to the prod storage account. If policy forbids that, dev and test get a masked copy refreshed on the prod side, or read it through OpenSharing.

WHICH TOOL, AND WHY

- **Catalog per environment and unit.** My choice. Bindings separate environments, catalogs separate units, and storage follows the catalog.
- **Catalog per unit, schema per environment.** Fewer catalogs, but a binding cannot separate dev schemas from prod schemas inside one catalog.
- **A metastore per unit.** Not possible in one region: an account allows one metastore per region.
- **Catalog per environment, schema per unit.** Simple for one small team. One owner for three units and shared storage make each unit's boundary hard to draw.

AT A GLANCE



Nine workspaces, one metastore, about ten catalogs. Every line between environments is a binding, and every line between units is a grant.

HOW I'D BUILD IT

1 Attach every workspace to the region's metastore

All nine workspaces attach to the one metastore in the region. Each catalog gets its own managed location, so no data depends on metastore-level storage. Metastore admin rights go to a small platform group, never to unit leads.

Watch out. A design that needs a second metastore in the region stops at the second one. **1.03**

2 Create catalogs per environment and unit

Each unit gets `dev_`, `test_` and `prod_` catalogs, for example `prod_retail`, `prod_wholesale` and `prod_online`, each with a managed location in the unit's own storage account. Storage cost and network rules then follow the unit. Schemas inside follow the layers, `bronze`, `silver` and `gold`. A script creates the catalogs, so names never drift.

Watch out. Tables created by path end up external and miss Predictive Optimization. Create tables by name. **1.01**

3 Bind catalogs to environments

Every catalog is set to isolated and bound to its own environment's workspaces: `prod_retail` read-write to the retail prod workspace only. Where another environment needs to read, the binding is read-only. I change bindings in dev first, and removing a binding reverts it. A developer in a dev workspace then cannot write prod, even with MODIFY. On a catalog already in use, I first list from `system.access.audit`, in Public Preview, which workspaces read it, because isolation cuts off every workspace not bound, and the catalog owner approves the binding list.

Watch out. An unbound catalog is reachable from every workspace on the metastore. With grants alone, a developer can drop a prod table from dev. **1.04**

4 Keep shared customers in one catalog

The customer master lives in `prod_customer`, owned by a data steward group and written by one pipeline. It is bound read-only to the three prod workspaces and, for masked reads, to the dev and test workspaces, and each unit's groups get SELECT on its gold schema. The pipeline's service principal is the only identity exempt from the mask policies (7.17). Units join to it and never copy it.

Watch out. A unit that copies customers into its own catalog with CTAS loses every row filter and mask on the way. **7.06**

5 Give dev masked data

Dev and test workspaces get the same read-only binding on `prod_customer`. An ABAC column mask policy on the catalog masks every column with the governed tag `pii` for the dev engineer groups. Only stewards can set governed tags, so a new personal column is masked as soon as it is tagged, with no new grant. The binding also needs a network path: dev compute must reach the prod storage account. Where security policy keeps non-prod compute away from prod storage, a prod job writes a masked dev copy instead, or prod shares it through OpenSharing.

Watch out. A view over the customer table runs with its owner's rights. If its owner is a steward or an exempt identity, every reader sees unmasked columns (7.17). **7.04**

6 Use account groups for every grant

Groups come from Entra ID at account level, named by unit and role: `retail-engineers`, `retail-analysts`. Catalogs and schemas are owned by groups. Engineers get MODIFY in dev and SELECT in prod, and only service principals write prod.

Watch out. Workspace-local groups cannot hold Unity Catalog privileges. A grant to the wrong group of the same name looks right, and access still fails. **7.02**

7 Give each unit a production identity

Each unit's production jobs run as that unit's service principal, which holds MODIFY on its own prod catalog only. Jobs are deployed with Declarative Automation Bundles with `run_as` set to that principal.

Watch out. A bundle deployed to prod from a laptop runs every job as the person who deployed it, unless it sets `run_as`. **9.05**

8 Close direct storage access

Each unit's storage account gives data roles to the access connector and nobody else. People get no Azure data roles on these accounts, so every read goes through Unity Catalog and its audit log.

Watch out. A Storage Blob Data Reader role left on the account lets people read prod files with no Unity Catalog check. **7.05**

SECURITY IN ONE BREATH

Grants go to groups, ownership goes to groups, and prod writes go to service principals. Bindings stop cross-environment writes even when a grant is wrong. Personal data is tagged once by stewards and masked by policy wherever it is read.

HOW I'D KNOW IT WORKS

- From a dev workspace, an INSERT into a prod catalog fails on the binding, even for a user with MODIFY.
- The information schema lists no grants to individual users on prod catalogs.
- A dev or test engineer querying prod_customer sees masked email and phone columns.
- Every prod catalog, schema and table is owned by a group or a service principal.

IF THEY PUSH FURTHER**What if one unit moves to another Azure region?**

Its workspaces attach to that region's metastore. Shared customers reach it through OpenSharing, and lineage stops at the share. **1.08**

What if analysts in one unit need another unit's gold data?

Grant USE CATALOG, USE SCHEMA and SELECT to that analyst group, and bind the catalog read-only to their workspace. No copy. **7.01**

What if auditors ask who can see personal data?

List the grants from the information schema, the ABAC policies on the customer catalog, and actual reads from system.access.audit, in Public Preview. **S.19**

S.05 Plan the move of 4,000 Hive metastore tables and 300 jobs in four workspaces to Unity Catalog without a long freeze.

Short answer. I'd run the UCX assessment first to find every table, mount, cluster and job that touches the Hive metastore. Then I federate each workspace's Hive metastore into Unity Catalog, so jobs can switch to Unity Catalog names on their own dates. Tables move in waves: external Delta tables become managed with SET MANAGED MOVE once their readers use Unity Catalog names, DBFS root tables are deep cloned, and tables that must stay external go through SYNC. Each wave switches its jobs in the same weeks, and when the audit log shows nothing reading hive_metastore I turn on Disable legacy access, which becomes permanent after 30 days.

QUESTIONS TO ASK FIRST

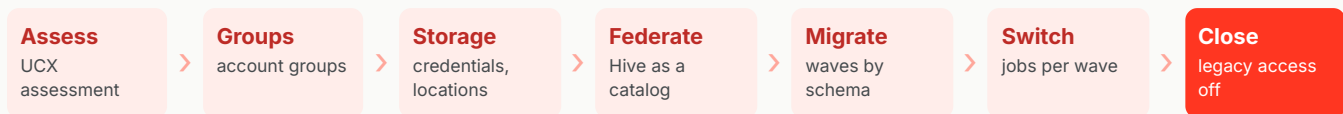
- **Which table types are in the 4,000?** External Delta, managed tables in the DBFS root, Parquet, CSV and Hive SerDe tables each take a different route.
- **Where does the data sit: DBFS root, mounts or ADLS paths?** Tables in the DBFS root are copied. Tables on ADLS can be registered or converted where they are.
- **Are the four workspaces in one region?** Then they share one Unity Catalog metastore, and table names repeated across workspaces need a naming rule.
- **When can each team change its jobs?** 300 jobs will not switch on one day. Each team needs a window that fits its own release.

WHICH TOOL, AND WHY

- **UCX with Hive metastore federation.** My choice. UCX assesses and migrates in bulk, and federation lets jobs move to Unity Catalog names before every table is converted.
- **SYNC and DEEP CLONE by hand.** Fine for a few hundred tables. For 4,000, the mapping, the order and the status tracking become the project.
- **The upgrade wizard in Catalog Explorer.** Good for one schema of external tables at a time. Not for four workspaces.

- **A freeze and a big copy.** Simple to explain and rarely possible: 300 jobs cannot all be retested in one weekend.

AT A GLANCE



Federation first, conversion in waves, the legacy path closed last. No job waits longer than its own table's switch.

HOW I'D BUILD IT

1 Assess with UCX

I install UCX, a Databricks Labs project without Databricks support, and run its assessment in each workspace. It lists tables by type and location, mounts, clusters without Unity Catalog access modes, and code that needs changes. The output goes into a tracker with one row per table and per job: owner, wave and status.

Watch out. Path-based writes never reach the metastore or lineage, so the assessment misses them. Search the code for `abfss://` and `/mnt` paths too. **7.09**

2 Move groups and run-as identities

UCX's group migration replaces workspace-local groups with account groups and copies their permissions. In the same step, production jobs move to service principals, because Unity Catalog checks the run-as identity.

Watch out. Grants to workspace-local groups do not exist in Unity Catalog. Access fails after the switch while the old grant still looks right. **7.02**

3 Register the storage

Every ADLS path behind a table or mount gets a storage credential on an access connector and an external location. UCX can create them from the mounts and paths it found. Mounts stay readable until their wave, so nothing breaks early. Fallback mode on the external locations lets unmigrated workloads keep their cluster credentials during the move. I turn it off per location once `system.access.audit` shows no fallback reads.

Watch out. Code that reads `abfss://` with a key in Spark config fails on standard access mode and serverless. Fix it in the same wave as its tables. **1.06**

4 Federate each Hive metastore

Hive metastore federation, generally available since March 2025, creates a foreign catalog that mirrors a workspace's Hive metastore, for example `hms_ws1`. Jobs can switch to three-part names now, with Unity Catalog grants, lineage and audit applied before any data moves. Writes through it go to the Hive metastore, so old and new jobs see the same tables. The foreign catalog's authorized paths must cover every table location, so I give common parent folders, not one path per table.

Watch out. `DROP SCHEMA ... CASCADE` through the federated catalog drops the Hive tables too, with no `UNDROP`. **1.02**

5 Migrate tables in waves

Each wave is a schema or a team. Federated external Delta tables become managed with `ALTER TABLE ... SET MANAGED MOVE`. The command runs on Databricks Runtime 17.3 LTS or serverless, and readers and writers need 15.4 LTS or above; on 15.3 or below, time travel works by version only. `MOVE` disables Hive access to the source, so every reader and writer must already use the Unity Catalog name. `COPY` leaves the Hive table live, and the two copies drift unless I stop its writers. Streams on the table fail and restart with the same settings. `UNSET MANAGED` rolls back within 14 days, and I rehearse it on a test table of each type before the first wave. Managed tables in the DBFS root are copied with `DEEP CLONE`: I stop their writers on the wave date, re-clone to pick up changes since the first copy, and switch readers to the new name. Tables that must stay external go through `SYNC`, Parquet tables become Delta first, and Hive SerDe tables are rewritten with CTAS.

Watch out. `SYNC` reports each table's outcome in `status_code`. A wave that only checks that the command finished misses the tables it skipped. [7.15](#)

6 Switch the jobs in the same wave

Each job in a wave moves to three-part names, runs on standard or dedicated access mode, and is compared with its last Hive output for a week. UCX's code linter flags what will not run on Unity Catalog compute.

Watch out. Code that uses RDDs or `sparkContext` fails on standard access mode. Those jobs need dedicated access mode or a rewrite. [4.12](#)

7 Set the default catalog

Once a workspace's jobs have switched, its default catalog becomes the production Unity Catalog catalog, so two-part names stop resolving to `hive_metastore`. Clusters and warehouses pick up the change when they restart.

Watch out. A job with two-part names on a workspace that still defaults to `hive_metastore` keeps reading the frozen copy, and succeeds. [1.05](#)

8 Retire the old path

Old Hive tables are renamed first, so a forgotten reader fails loudly, then dropped a month later. Clusters without Unity Catalog access modes are deleted and policies stop new ones. Before Disable legacy access, I query `system.access.audit`, in Public Preview, for any `hive_metastore` read in the last month, and the platform owner signs off. The setting can be turned back on for 30 days. After that the workspace is Unity Catalog only for good.

Watch out. An old no-isolation cluster still reads the Hive copy, so a user revoked in Unity Catalog keeps access until the cluster is gone. [7.03](#)

SECURITY IN ONE BREATH

Every grant is rebuilt on account groups, and every production job runs as a service principal. Unity Catalog audit covers federated tables from the day they are federated, so the trail starts before the data moves. Disable legacy access closes the last path Unity Catalog cannot see.

HOW I'D KNOW IT WORKS

- The tracker shows every table with a target and a status, and every SYNC and SET MANAGED outcome is recorded.
- Row counts and checksums match between each Hive table and its Unity Catalog table at the switch, after the final re-clone.
- No job, query or dashboard references hive_metastore for a month before legacy access is disabled.
- Every job runs as a service principal, and none reads through a mount.

A WORKED EXAMPLE

Situation. For example, four workspaces held 4,100 Hive tables: 2,300 external Delta tables on ADLS, 1,200 managed tables in the DBFS root, 450 Parquet tables and 150 CSV and SerDe tables. 300 jobs used two-part names, and 40 read through mounts.

Task. Move everything to Unity Catalog within two quarters without a platform freeze.

Action. In month 1 we ran the assessment, migrated groups and federated all four metastores. In months 2 to 5 we ran 12 waves of about 340 tables and 25 jobs each. In month 6 we set default catalogs, renamed the Hive tables and disabled legacy access.

Result. In this example the dry runs flagged 31 tables, mostly SerDe tables, which were rewritten with CTAS. No job stopped for longer than the short switch of its largest table.

IF THEY PUSH FURTHER**What if some tables must stay external because another tool writes them?**

Keep them external and run SYNC on a schedule, so changes in the Hive metastore reach Unity Catalog. Plan the other tool's move to a governed route later. **C.20**

What if a job cannot change before the deadline?

It keeps running against the federated catalog until its owner switches it. Disable legacy access waits for it, so the tracker names the job and the date. **1.05**

What if someone drops a migrated managed table the old way?

Unity Catalog keeps the files for the recovery period, 7 days by default, and UNDROP brings the table back. Tell teams before the first wave. **1.02**

S.06 Move 80 Synapse Spark notebooks and the ADF pipelines that call them onto Azure Databricks.

Short answer. I'd move the notebooks in groups that share tables, not one at a time. The data stays in the same ADLS accounts: I register it in Unity Catalog, so both platforms read the same files during the move. Each notebook changes to read tables by name and to use dbutils instead of mssparkutils, then runs in a Lakeflow job on serverless compute. ADF keeps orchestrating at first and calls each job with the Azure Databricks Job activity. Pipelines that only chain notebooks move into Lakeflow Jobs at the end, and the Synapse pools are deleted.

QUESTIONS TO ASK FIRST

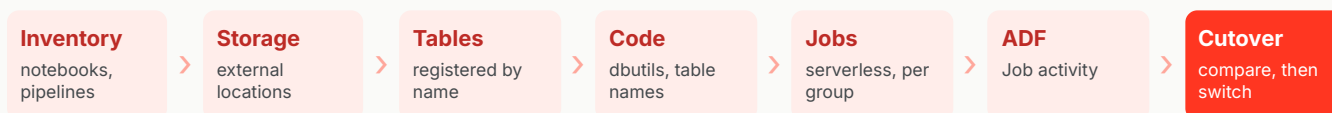
- **What do the ADF pipelines do besides calling notebooks?** Copy activities from on-premises sources may stay in ADF. Pipelines that only chain notebooks can move to Lakeflow Jobs.

- **Where do the notebooks read and write?** Lake database tables, ADLS paths and dedicated SQL pools each need a different replacement.
- **Which Spark version do the Synapse pools run?** Behaviour can change on a newer Databricks Runtime, ANSI mode for example. That sets how much testing each group needs.
- **How does Synapse reach the storage and sources today?** Managed private endpoints in the Synapse managed VNet do nothing for Databricks. Classic compute needs VNet injection and private endpoints, and serverless a network security perimeter rule or NCC private endpoints (S.18).

WHICH TOOL, AND WHY

- **Lakeflow jobs called from ADF, then Lakeflow Jobs alone.** My choice. ADF keeps working during the move, and orchestration moves only once the code is stable.
- **Rewrite everything as pipelines.** Right for some loads later. Changing the logic and the platform at once makes every difference hard to explain.
- **ADF Notebook activity on a new cluster per run.** The older pattern. Each activity starts its own cluster, and the cluster definition lives in ADF instead of next to the code.
- **Lift and shift onto all-purpose clusters.** Quickest to start and the most expensive to run: all-purpose compute is billed at a higher rate than jobs compute.

AT A GLANCE



The data stays where it is, the code moves in groups, and ADF calls Databricks jobs until the schedules move too.

HOW I'D BUILD IT

1 Inventory and group the notebooks

I list every notebook with the tables and paths it reads and writes, the linked services it uses and the ADF pipelines that call it. Notebooks that share tables form a group, usually five to ten. Each group moves together, so no table is written by both platforms at once.

2 Register the storage in Unity Catalog

The ADLS containers become external locations on an access connector. Synapse lake database tables become Unity Catalog external tables on the same folders, so Databricks reads exactly what Synapse wrote. Tables Databricks will own after cutover become managed later, with SET MANAGED. If the accounts deny public access, the network path is opened before the first group moves: private endpoints in the workspace VNet for classic compute, and for serverless a network security perimeter rule for the AzureDatabricksServerless service tag on same-region accounts.

Watch out. A storage firewall that admits only Synapse's managed private endpoints returns 403 to serverless jobs, while the external location looks correct. **8.11**

3 Change the code

Each notebook loses its Synapse specifics: mssparkutils calls become dbutils, linked service tokens become Unity Catalog connections or service credentials, and paths become three-part table names. Code that read abfss:// paths with a key or a linked service token changes too, because serverless refuses both. SparkSession setup and executor settings are removed, because Databricks sets them.

Watch out. A Spark config the notebook sets at the top is ignored or refused on serverless, and the job changes behaviour without it. [4.11](#)

4 Move the secrets

Secrets from linked services move to Unity Catalog secrets (Runtime 17.3 LTS or serverless environment 4 and above; check their release status), or disappear: storage goes through external locations, and Azure services through service credentials. Where Key Vault must stay the store, its scope needs the vault on the access policy model, not Azure RBAC. Nothing is printed or pasted into cells.

Watch out. A secret printed in cell output is redacted on a best-effort basis only. A transformed value shows in full. [7.11](#)

5 Run each group as a Lakeflow job

Each group becomes one job with a task per notebook, in the order ADF used, running on serverless compute as a service principal. The run date is a job parameter. Before cutover I compare outputs with Synapse for the same inputs.

Watch out. Databricks Runtime 17.0 and above turn on ANSI mode, so a cast that returned NULL in Synapse now fails the task. [4.10](#)

6 Call the jobs from ADF

In each ADF pipeline, the Synapse notebook activities become one Azure Databricks Job activity that runs the group's job by ID with parameters. The linked service authenticates with ADF's managed identity, which has permission to run those jobs and nothing more.

Watch out. A linked service with an engineer's personal access token stops every pipeline when the token expires or the engineer leaves. [7.13](#)

7 Run both, compare, switch

For two weeks both platforms run each group on the same inputs, writing to separate tables. A comparison job checks row counts and totals per table and day. When a group matches, ADF switches to the Databricks job and the Synapse notebook is disabled, not deleted.

Watch out. A copy of a production job used for the parallel run writes to production tables unless the catalog is a parameter. [5.04](#)

8 Move schedules and retire Synapse

Pipelines that only chained notebooks become Lakeflow job schedules and triggers. ADF keeps its copies from on-premises sources and calls the job when they finish. When no group uses Synapse Spark, the pools are deleted.

Watch out. Without tags on each job, the Databricks bill cannot be split by group, and nobody can show what the move saved. **8.01**

SECURITY IN ONE BREATH

ADF reaches Databricks with its managed identity, and jobs run as service principals. Storage access goes through external locations, so nobody needs a storage key. Synapse identities lose their storage roles once their pools are gone.

HOW I'D KNOW IT WORKS

- Every migrated group matches its Synapse output for two weeks of parallel runs.
- No notebook contains mssparkutils, a storage key or an abfss:// path.
- ADF pipelines call Databricks only through Job activities, and each run links to a Lakeflow job run.
- The Synapse Spark pool cost line is zero in the month after cutover.

IF THEY PUSH FURTHER

What if the notebooks also write to a dedicated SQL pool?

Keep that write until its readers have moved to a Databricks SQL warehouse or to Gold tables in Unity Catalog. The SQL pool is retired last. **C.01**

What if a notebook fails on Databricks with the same error on every retry?

Retries help transient errors only. Fix the cause, then repair the run so only the failed tasks run again. **5.02**

What if the move must cut cost straight away?

Run the groups on serverless in standard mode, where a few minutes of start-up does not matter, and tag each job so the comparison with Synapse is per group. **8.06**

S.07 Move a platform of 150 jobs on classic job clusters to serverless where it pays, and keep classic where it does not.

Short answer. I'd measure first: cost per run, start-up share and runtime for each job, from the billing and Lakeflow system tables plus the Azure VM bill. Short, frequent and spiky jobs go to serverless. Long, steady jobs that keep a cluster busy may stay on classic job clusters with Spot workers. Jobs that need init scripts, Maven libraries, RDDs or Scala notebooks stay classic until rewritten. Each job moves through a pilot and a two-week cost comparison, tagged so its spend stays attributable.

QUESTIONS TO ASK FIRST

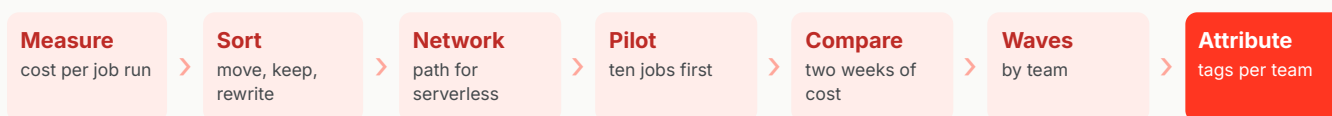
- **What does each job cost today, including the Azure VM bill?** Classic cost is DBUs plus VMs, disks and network on the Azure bill. Serverless DBUs include the compute, but private endpoints, public connectivity and data transfer are billed on top, so compare totals.
- **How long does each job run, and how much of it is start-up?** A five-minute job that waits five minutes for a cluster gains most from serverless.

- **Which jobs reach storage or databases behind a firewall?** Serverless runs outside the workspace VNet and needs its own network path.
- **Which jobs have a deadline, and which can wait to start?** That decides standard mode or **PERFORMANCE_OPTIMIZED** for each job.

WHICH TOOL, AND WHY

- **Serverless jobs, chosen per job.** My choice. No cluster to size or patch, quick start-up, and billing per job. A job moves only when its numbers say so.
- **Classic job clusters with Spot workers.** Still the cheapest for long, steady jobs that keep every worker busy. Spot workers with fallback to on-demand cut the VM bill further.
- **Pools for classic jobs.** Shorter start-up on classic, but idle pool VMs are billed by Azure while they wait.
- **Serverless workspaces.** Generally available on Azure: a workspace with serverless compute only, no VNet to manage. Good for a new team with no classic needs. The jobs that must stay classic cannot live there.
- **Everything serverless at once.** Fast to announce. Jobs that rely on init scripts, custom configs or RDDs fail, and some long jobs cost more.

AT A GLANCE



Every job ends on one of three lists: move, keep on classic, or rewrite first. Cost data decides between the first two.

HOW I'D BUILD IT

1 Measure each job

I join system.billing.usage to system.lakeflow.job_task_run_timeline on job and run ID, and add the VM cost for classic clusters from Azure Cost Management by cluster tag. For each job: runs per day, setup and execution seconds, DBUs and total cost per run. For serverless I add the networking charges Databricks bills: private endpoints per hour each, and public connectivity and data transfer per GB. A network security perimeter rule to same-region storage carries no such charge.

Watch out. Databricks usage shows only DBUs for classic jobs. VMs, disks and network sit on the Azure bill, and a comparison without them favours classic. **8.07**

2 Sort the jobs into three lists

Move: short or frequent jobs, jobs whose start-up is a large share of the run, and spiky jobs. Keep: long jobs that keep a cluster busy for hours, and jobs that need GPUs or specific VM types. Rewrite first: jobs that use init scripts, Maven libraries, RDD APIs, Scala or R notebooks, global temp views or caching APIs, none of which serverless supports.

Watch out. Code that uses sparkContext or RDDs fails on serverless the same way it fails on standard access mode. **4.12**

3 Open the network path

Serverless compute reaches storage and databases from Databricks' network, not from the workspace VNet. Same-region storage accounts get a network security perimeter rule for the AzureDatabricksServerless service tag. Databases and accounts that must deny all public access get private endpoints in a network connectivity configuration, approved on the Azure side, before the first job moves. S.18 covers the full design.

Watch out. A job that reached a firewalled storage account from a VNet-injected cluster gets 403 on serverless. Serverless subnet IDs in the firewall no longer work. **8.11**

4 Replace cluster settings

Libraries move from cluster settings into the job's serverless environment, with versions pinned. Spark configs are dropped unless serverless supports them. Streaming tasks with a processing time trigger change to availableNow on a schedule, or move into Lakeflow Spark Declarative Pipelines because serverless jobs support only availableNow.

Watch out. A job that relied on a Spark config set on its cluster runs without it on serverless. **4.11**

5 Pick the performance mode per job

Jobs with a deadline close to their run time use **PERFORMANCE_OPTIMIZED**. Nightly batches with hours of slack use standard mode, which starts in 4 to 6 minutes and uses fewer DBUs. The mode is a job setting, so it lives with the job in Declarative Automation Bundles.

Watch out. Leaving every job performance optimised pays the higher rate for batches that never needed the speed. **8.06**

6 Pilot, then compare for two weeks

Ten jobs from the move list go first, chosen across teams. Each runs on serverless for two weeks while its classic definition stays in Git. I compare cost per run, duration and failures with the four weeks before.

Watch out. Serverless bills each job on its own. Jobs that were cheap only because they shared a warm cluster can cost more alone. **8.02**

7 Move the rest in waves

Teams move their own jobs in waves of 20 to 30 through the bundle: the job cluster block becomes an environment, and the performance target is set. A job whose cost per run, VMs included, is still more than 20 percent above its classic baseline after two weeks goes back to classic, and the team's owner approves the move back or the exception.

Watch out. Bundle jobs are locked in the UI (UI_LOCKED). Someone who disconnects a job from its source to switch it by hand makes it drift, and the next deploy puts it back on classic. **9.02**

8

Attribute serverless spend

Every job carries team and cost centre tags. Serverless usage policies, in Public Preview, attach those tags to serverless usage. A daily query over `system.billing.usage` shows serverless spend per team and flags any team whose spend jumps week on week.

Watch out. Without tags, serverless usage shows the job but not the team, and one team's spike is found at month end. **8.04**

SECURITY IN ONE BREATH

Serverless jobs run as service principals with Unity Catalog access only. Network security perimeter rules and network connectivity configurations decide which storage accounts and databases serverless can reach. Classic clusters that stay keep a cluster policy with fixed limits.

HOW I'D KNOW IT WORKS

- Every job sits on one of the three lists with a reason.
- Each moved job costs no more than 20 percent above its classic baseline per run, VMs included, after two weeks, and any job above that is back on classic.
- No moved job fails with a network or configuration error after its first week.
- Every serverless DBU in `system.billing.usage` carries a team tag.

A WORKED EXAMPLE

Situation. For example, 150 jobs ran on classic job clusters. 90 ran under 15 minutes, 40 of them hourly or more often, and start-up took 4 to 7 minutes of each run. 25 jobs ran over two hours a night on large Spot clusters.

Task. Cut cost and start-up waits without missing any deadline.

Action. We moved 95 jobs to serverless: a pilot of 10, then three waves of about 28, 70 of them in standard mode. 30 long jobs stayed on classic, and 25 went to the rewrite list for RDD code and init scripts. Each wave had a two-week comparison, and reverting a job was one bundle change and a deploy.

Result. In this example compute cost for the moved jobs fell by about a third, VMs included, and hourly jobs finished 5 minutes sooner on average. Three jobs went back to classic because they cost more.

IF THEY PUSH FURTHER

What if the long classic jobs fail at random on Spot?

Keep the driver on demand, use Spot with fallback to on-demand for workers, and retry only on lost-instance errors. **4.05**

What if finance asks why the Databricks line rose while the Azure VM line fell?

Serverless DBUs include compute that used to appear as VMs. Show both lines together, per job. **8.07**

What if the monitoring queries on system tables get expensive?

Run them on a small serverless warehouse, aggregate per day into a summary table, and point the dashboard at that. **8.05**

S.08 Retire an on-premises Hadoop cluster with Hive tables, Oozie workflows and 60 TB of HDFS data.

Short answer. I'd move the bulk of the 60 TB to ADLS with a Data Box, then keep it in step with DistCp over ExpressRoute until cutover. Each copied folder is registered as a Unity Catalog table and converted: Parquet to Delta in place, ORC and text rewritten into managed Delta tables with liquid clustering. Oozie workflows become Lakeflow jobs in bundles, and HiveQL moves to Databricks SQL with tests that compare outputs. HDFS goes off after a month of matching results.

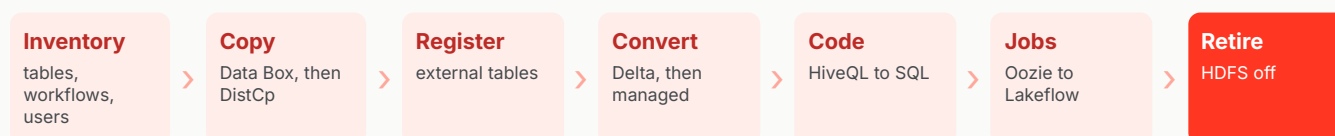
QUESTIONS TO ASK FIRST

- **What formats are the Hive tables?** Parquet converts in place. ORC, Avro and text are rewritten, which sets the compute and time for the move.
- **How much of the 60 TB is still read?** Cold history moves once and can sit in a cooler tier. Only active data needs repeated syncs.
- **What does the network to Azure look like?** 60 TB over a shared 1 Gbps link takes about a week at full speed. A Data Box can be quicker and leaves the link alone.
- **What runs besides Hive: Spark jobs, Sqoop, HBase, Ranger, Kerberos?** Each needs a replacement or a decision to drop it. Kerberised sources need a service account and a network path from Azure first.

WHICH TOOL, AND WHY

- **Data Box for the bulk, DistCp for changes, Delta in Unity Catalog.** My choice. The bulk moves offline, changes move online, and every table ends up governed in Unity Catalog.
- **DistCp over the network only.** Simplest when the link is fast and free. It competes with production traffic for days.
- **Keep the tables as external Parquet and ORC.** Least work on day one. No Delta transactions, no Predictive Optimization, and every Hive habit comes along.
- **Lift the cluster onto Azure VMs.** Moves Hadoop without changing it. Licences, patching and tuning stay, so I would use it only as a short bridge.

AT A GLANCE



Data first, then tables, then code and schedules. Each stage is checked against Hadoop before the next starts.

HOW I'D BUILD IT

1 Inventory the cluster

From the Hive metastore database I export every table with its format, location, partitions and size, and from HDFS the folder sizes and last access times. From Oozie I export each workflow with its actions, schedule and dependencies. Ranger policies become a list of who may read what. I flag Hive ACID (transactional) tables, custom SerDes and Java Hive UDFs, because none of them copies as plain files. The inventory decides the waves.

2 Copy the data to ADLS

A Data Box takes the bulk: DistCp copies from HDFS to the device, and Azure loads it into an ADLS container. After that, DistCp with the update option copies what changed, nightly, over ExpressRoute. The storage account sits in the same region as the workspaces.

Watch out. A storage account in another region slows every scan and adds transfer charges for as long as the data lives there. **1.08**

3 Register the copied folders

Each copied folder becomes an external table in a migration catalog, created from the Hive DDL with the location rewritten to abfss://. Queries can now compare the Hadoop and Azure copies table by table. HDFS ACLs are not copied: access is granted again in Unity Catalog, to groups.

Watch out. Hive tables partitioned by day and hour arrive as thousands of small folders. Do not carry that partitioning into Delta. **3.01**

4 Convert to Delta

Parquet tables are converted in place with **CONVERT TO DELTA**, which writes a Delta log and leaves the files where they are. They keep their Hive partitions, so on Runtime 18.1 and above I follow with **REPLACE PARTITIONED BY WITH CLUSTER BY** to move them to liquid clustering in place. ORC, Avro and text tables are rewritten with CTAS into managed Delta tables, clustered on their main filter columns. ACID tables are first read through Hive into plain Parquet or ORC, since their delta files need compaction and a copy of the folder is not consistent. A custom SerDe table is rewritten from a copy read with its own reader. Converted tables become managed with SET MANAGED once their readers have switched.

Watch out. Converted tables stay external until you change them, and external tables get no Predictive Optimization. **1.01**

5 Port the HiveQL

Most HiveQL runs on Databricks SQL with small changes. Java Hive UDFs are rewritten as Unity Catalog SQL or Python functions, or kept as a separate decision about where their jar can run. I go through the differences that change results, then give each script a test that compares its output with Hadoop's on the same day's data.

Watch out. A query that relied on Hive returning NULL for a bad cast fails under ANSI mode, the default from Databricks Runtime 17.0. **4.10**

6 Rebuild Oozie workflows as Lakeflow jobs

Each Oozie workflow becomes a Lakeflow job: actions become tasks, decision nodes become if/else condition tasks, and coordinators become schedules or file arrival triggers. Jobs live in Declarative Automation Bundles run as service principals and use serverless or job compute.

Watch out. An Oozie coordinator catches up missed runs. A paused or failed Lakeflow schedule does not, so build the catch-up where it matters.

7 Run both platforms and compare

For a month the Lakeflow jobs run on Azure data alongside Oozie on Hadoop. A comparison job checks row counts and key totals per table and day, and differences go to the script owner.

Watch out. A task that runs although its upstream was skipped produces a plausible result on yesterday's data, and the comparison passes for the wrong reason. **5.10**

8 Cut over and retire Hadoop

Each wave switches its consumers to the Databricks tables and stops its Oozie workflows. Sqoop imports are replaced by Lakeflow Connect or jobs. When no workflow has run on Hadoop for a month, HDFS is archived and the cluster switched off.

Watch out. Service accounts that read the ADLS copy directly during the move keep their Azure roles. Remove them at cutover so every read goes through Unity Catalog. **7.05**

SECURITY IN ONE BREATH

Ranger policies are rewritten as Unity Catalog grants to Entra ID groups, not copied as file ACLs. Only the access connector holds a data role on the ADLS containers, and jobs run as service principals. The Data Box is wiped by Microsoft's process when it returns.

HOW I'D KNOW IT WORKS

- Every Hive table has an ADLS copy whose row count matches Hadoop at cutover.
- Every Oozie workflow maps to a Lakeflow job, or to a decision to drop it signed off by its owner.
- A month of comparison shows matching totals for each migrated table.
- No identity other than the access connector has a data role on the ADLS containers.

IF THEY PUSH FURTHER

What if HBase tables are part of the estate?

Analytical reads move to Delta tables. Point lookups go to Lakebase, the managed Postgres database in Databricks, generally available on Azure, fed from those Delta tables. Azure Cosmos DB stays an option for teams already on it. **10.05**

What if some tables hold millions of small files?

Do not convert them in place. Rewrite them into managed tables with liquid clustering, so file counts fall and Predictive Optimization keeps them compact. **3.03**

What if the business wants lineage from day one?

Lineage covers reads and writes through Unity Catalog by table name. Path-based writes and the copy itself do not appear. **7.09**

S.09 Keep a sales table no more than ten minutes behind an Azure Event Hubs stream of 5,000 events per second.

Short answer. I'd read Event Hubs through its Kafka endpoint into a streaming table in a pipeline in Lakeflow Spark Declarative Pipelines on serverless compute, triggered every three minutes by a Lakeflow job. Bronze keeps the raw message with its partition, offset and enqueued time. Silver parses and deduplicates it into the sales table, which stays about five minutes behind. A separate check every five minutes compares the newest enqueued time in Silver with the clock and alerts at eight minutes. I switch the pipeline to continuous only if the triggered runs miss that at peak.

QUESTIONS TO ASK FIRST

- **Which tier is the namespace, and how many partitions does the event hub have?** Partitions cap read parallelism. On Standard, a throughput unit takes 1,000 events or 1 MB a second in, so I size on the larger of the two: five units at 1 KB an event, ten at 2 KB.
- **Is ten minutes measured from the sale or from Event Hubs?** Tills that queue events offline make the sale time far older than the enqueued time. The promise has to name one.
- **Can events arrive twice?** Event Hubs delivers at least once, so Silver needs a key to remove duplicates.
- **Is the Event Hubs namespace private?** Serverless runs outside the workspace VNet. A namespace that denies public access needs a private endpoint rule in a network connectivity configuration, approved on the namespace (S.18).

WHICH TOOL, AND WHY

- **Triggered pipeline every three minutes on serverless.** My choice. Ten minutes leaves room for a run every three to five minutes, and the pipeline handles checkpoints, retries and scaling. Start-up and processing must fit inside the interval, peaks included.
- **Pipeline in continuous mode.** Lowest latency with micro-batches, and compute runs around the clock. I keep it for the day the triggered runs miss ten minutes at peak.
- **Structured Streaming job on a classic cluster.** Full control of triggers and tuning. I would size, patch and watch the cluster myself.
- **Event Hubs Capture plus Auto Loader.** Simple and cheap. Capture writes files on a time window, which adds minutes before Auto Loader sees them.

AT A GLANCE



One triggered pipeline from Event Hubs to Silver every three minutes, and a separate check that measures freshness from outside it.

HOW I'D BUILD IT

1 Size Event Hubs for the peak

Each Standard throughput unit takes 1,000 events or 1 MB a second in, whichever limit comes first. I assume 2 KB a message, which is 10 MB a second, so ten units at the average, not five; I replace the assumption with the real payload size from a day of metrics. I turn on auto-inflate with a ceiling well above the peak. Partitions are set at creation for the parallelism the pipeline needs, up to 32 on Standard.

2 Authenticate without a key

The pipeline connects to the namespace on port 9093 with a Unity Catalog service credential, supported from Databricks Runtime 16.1. The access connector behind the credential holds the Azure Event Hubs Data Receiver role on the event hub. No shared access key sits in a secret or in the code. If the namespace denies public access, serverless reaches it through a private endpoint rule in a network connectivity configuration, which stays PENDING until someone approves it on the namespace.

Watch out. A shared access key in a secret scope breaks the stream the day someone rotates it in Azure. **7.12**

3 Land raw messages in Bronze

The first streaming table stores the message body as a string with the partition, offset, enqueued time and load time. Nothing is parsed here, so a bad message never stops the stream.

`maxOffsetsPerTrigger` caps each micro-batch, so a restart after a backlog catches up in steady steps.

Watch out. Each micro-batch writes small files. The pipeline compacts its tables, and a separate OPTIMIZE job beside it only adds conflicts. **2.11**

4 Protect Bronze from a full refresh

Event Hubs Standard keeps events for up to 7 days, so Bronze is the only full history, and an outage longer than that loses data no restart recovers. I set `pipelines.reset.allowed` to false on Bronze, so a full refresh rebuilds Silver from Bronze and never goes back to Event Hubs.

Watch out. A reset Bronze checkpoint starts from the earliest or the latest offset, so days of events are replayed or skipped. **2.06**

5 Parse and deduplicate into Silver

Silver parses the body with a declared schema and keeps sale ID, store, amount and sale time. It drops duplicates on sale ID within a one-hour watermark on enqueued time. Expectations send rows without a sale ID to a quarantine table.

Watch out. Changing the dedupe key or the watermark changes the stream's state, and the restart can fail on the old checkpoint. Plan a full refresh of Silver. **2.08**

6 Start triggered, go continuous only if needed

A Lakeflow job runs the pipeline update every three minutes on serverless compute, which scales with the backlog. In the first weeks I watch the lag at the daily peak. Only if triggered runs miss ten minutes there does the pipeline move to continuous mode, and I compare the daily DBUs of both.

Watch out. Continuous mode keeps compute up around the clock, including the quiet hours of the night. **5.07**

7 Measure freshness from outside

A small job every five minutes computes the gap between now and the newest enqueued time in Silver, and compares Bronze row counts with incoming messages in Azure Monitor. It alerts at eight minutes. The pipeline's own status says succeeded even when it has fallen behind. This check is the primary guard. Anomaly detection in data quality monitoring, in Public Preview, adds a schema-wide net for stale tables in the sales schema, and I would not rely on it alone.

Watch out. When each run takes longer than the interval, the stream falls further behind every hour while every run reports success. **2.10**

8 Serve the table

Reports read Silver, or Gold views over it, through a serverless SQL warehouse. Hourly summaries are materialized views with TRIGGER ON UPDATE, so they refresh when Silver changes without a separate schedule.

Watch out. A summary view that fails to refresh keeps serving its last result, and the report looks fine. **6.05**

SECURITY IN ONE BREATH

The pipeline reads Event Hubs through a service credential with the receive role only, and nothing holds a key. It runs as a service principal that owns Bronze and Silver. Analysts read Silver and Gold, never Bronze, which holds raw payloads.

HOW I'D KNOW IT WORKS

- The freshness check stays under seven minutes through a full trading day, peaks included.
- Hourly incoming message counts in Azure Monitor match Bronze row counts.
- Silver has no duplicate sale IDs.
- A forced 15-minute stop raises the alert, and the backlog clears within ten minutes of the restart.

A WORKED EXAMPLE

Situation. For example, a classic Structured Streaming job read Event Hubs every 30 seconds with a shared access key from a secret scope. Its cluster autoscaled to its maximum of 16 workers and stayed there. Twice a month the key was rotated and the stream stopped until someone noticed.

Task. Keep the sales table under ten minutes behind, without the key incidents, at lower cost.

Action. We moved ingestion to a serverless pipeline triggered every three minutes, with a service credential, Bronze and Silver as streaming tables, and an outside freshness check with an alert at eight minutes. We ran it beside the old job for two weeks and compared hourly counts.

Result. In this example Silver ran four to six minutes behind Event Hubs at peak, and the key incidents stopped. Daily compute cost fell by about 60 percent against the cluster that never scaled down.

IF THEY PUSH FURTHER

What if the target dropped to a few seconds?

Real-time mode for pipelines, in Public Preview, gets there, but it reads and writes Kafka and Event Hubs only, not Delta, and does not support dropDuplicatesWithinWatermark. I'd use it for an Event Hubs to Event Hubs path, an alert feed for example, and keep the sales table on this design.

What if the stream had to stay on a classic cluster?

Classic autoscaling is not built for streaming and tends to sit at its maximum. Use a fixed size for peak, or move to a pipeline. **4.04**

What if a fraud model also needs the same events?

A second query reads Bronze with its own checkpoint. Two queries never share one checkpoint. **2.07**

S.10 Bring 200 SQL Server tables into the lakehouse with changes applied every 15 minutes, including deletes.

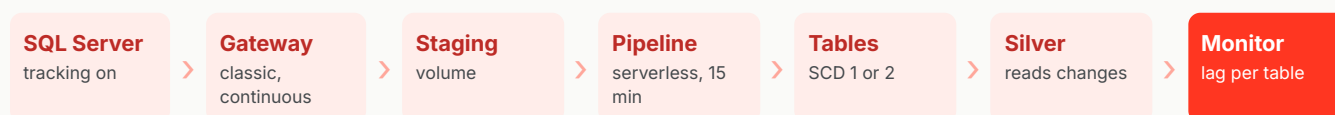
Short answer. I'd use the Lakeflow Connect SQL Server connector, with change tracking on keyed tables that need only the current row and CDC on tables that need history, because SCD type 2 requires CDC. If the source is an availability group, every table uses CDC through the listener, because change tracking forces a full refresh after a failover. One ingestion gateway on classic compute reads changes continuously into a staging volume. One ingestion pipeline on serverless applies them to streaming tables every 15 minutes, deletes included. 200 tables fit within the recommended 250 per pipeline, and tables with clashing names get a destination_table override. Silver reads the connector's tables through change data feed or incremental materialized views.

QUESTIONS TO ASK FIRST

- **Which tables have primary keys, and is the server in an availability group?** Change tracking needs a key and is lighter on the server, but its data does not reach secondaries, so a failover means a full refresh of every table. On an availability group I use CDC through the listener.
- **Where does SQL Server run, and how will Azure reach it?** The gateway runs on classic compute in the workspace network and needs a route: a private endpoint for Azure SQL, ExpressRoute or VPN for on premises.
- **Do any table names repeat across schemas?** One pipeline cannot ingest two tables with the same name, unless each gets its own destination_table name.
- **Do consumers need history, or only the current row?** SCD type 1 keeps the latest row and removes deleted ones. SCD type 2 keeps every version.

WHICH TOOL, AND WHY

- **Lakeflow Connect SQL Server connector.** My choice. Managed change capture with deletes, schema changes and restarts, on a gateway and a serverless pipeline that Databricks runs.
- **Query-based connector.** Generally available since May 2026 and needs no gateway. It reads by cursor column and queries the tables directly, and hard delete tracking is still in Beta.
- **ADF copy with watermarks.** Familiar and flexible. Deletes and schema changes are yours to handle, for 200 tables.
- **Lakehouse Federation.** Queries SQL Server live without copying. Right for a few lookups, wrong for 200 tables read every 15 minutes.

AT A GLANCE

The gateway never stops. The ingestion pipeline runs every 15 minutes and applies inserts, updates and deletes to streaming tables.

HOW I'D BUILD IT

1 Prepare SQL Server

A dedicated login gets the permissions the connector needs, and Databricks' utility objects script prepares the database. I turn on change tracking for keyed tables kept as the current row, CDC for tables that need SCD type 2, or CDC for every table on an availability group, and check the retention: the gateway must read changes before SQL Server cleans them up.

Watch out. If changes are cleaned up before the gateway reads them, every affected table needs a full refresh.

2 Open the network path

Only the gateway talks to SQL Server. It runs on classic compute in the workspace's VNet, so that VNet needs a route to the primary instance, through the availability group listener where there is one: a private endpoint for Azure SQL Database, or ExpressRoute for a server on premises. Read replicas and secondaries are not supported.

Watch out. A firewall rule for the workspace subnet does not cover serverless. Keep the source connection on the gateway. **S.18**

3 Create the gateway and the pipeline

I create a Unity Catalog connection to SQL Server, one ingestion gateway and one ingestion pipeline for the 200 tables, defined with Declarative Automation Bundles. Tables whose names repeat across schemas stay in the same pipeline, each with its own `destination_table` name, for example `customers_sales`. The gateway is classic compute billed around the clock, so I put it under a cluster policy and budget it as a standing cost.

Watch out. Each extra pipeline brings its own gateway running all day. Split only where the 250 table limit forces it. **2.14**

4 Keep the gateway running

The gateway runs in continuous mode and is never stopped between runs: a restart provisions a new VM and risks missing changes. It runs as a service principal, under a cluster policy.

Watch out. A gateway owned by an engineer's account fails when that account is disabled, and changes pile up until retention drops them. **7.13**

5 Apply changes every 15 minutes

A Lakeflow job runs the ingestion pipeline every 15 minutes. Databricks recommends at least 5 minutes between runs, and the ingestion pipeline cannot run continuously. Each run applies inserts, updates and deletes. Most tables use SCD type 1, and tables the business audits use SCD type 2, which needs CDC on those tables.

Watch out. A run that takes longer than 15 minutes delays the next one, and freshness drifts to 30 minutes without a failure. **5.13**

6 Build Silver from the changes

The connector's tables receive updates and deletes, so a plain stream over them fails on the first change. Silver reads them through change data feed, or as materialized views that refresh incrementally. On Databricks Runtime 19 and above, change data feed also works automatically on tables with row tracking, with no table property to set; on 18 LTS the property must be on. Deletes reach Silver as delete rows, not as rows that quietly stay.

Watch out. A change data feed reader that falls behind longer than the table's retention misses those changes for good. **3.10**

7 Handle schema changes

New source columns arrive on the next run, and dropped columns stay in the table, marked inactive. A column rename or a type change needs a full refresh of that table, so the source team announces them and I schedule the refresh outside business hours.

Watch out. A rename handled like a new column leaves the old column empty from that day on, and reports read nulls. **3.09**

8 Monitor lag and failures

A query over the pipeline event log gives the last applied change per table, and an alert fires when any table is more than 30 minutes behind. Gateway and pipeline failures notify the platform group through a notification destination.

Watch out. A failure notification that goes to one person's inbox is read four hours later. **9.07**

SECURITY IN ONE BREATH

The SQL Server login has the permissions the connector needs and nothing more. Its password sits in the Unity Catalog connection, which only the platform group can use. The gateway and the pipeline run as a service principal, and analysts read Silver and Gold.

HOW I'D KNOW IT WORKS

- Every table's latest applied change is under 30 minutes old during the day.
- A row deleted in SQL Server is gone from its SCD type 1 table after the next run.
- Row counts match between SQL Server and the lakehouse for a sample of tables each night.
- The gateway has run without a restart since the last planned change.

IF THEY PUSH FURTHER

What if the source fails over to another node?

With CDC through the availability group listener, the gateway resumes on the new primary without data loss. Change tracking data does not reach secondaries, so with it every table in the pipeline needs a full refresh. That is why I use CDC there.

What if two Silver jobs MERGE into the same table?

They can conflict on the same files. Keep one writer per table, or split the work by key range. **3.07**

What if a few tables should be read live instead?

Lakehouse Federation queries SQL Server directly. Good for small lookups, but every query reaches the source. **6.04**

S.11 Design daily aggregates over IoT readings that can arrive up to 30 days late.

Short answer. I'd separate arrival from aggregation. Readings land in an append-only Bronze table, and Silver keeps one row per device and reading time, so a late or repeated reading changes exactly one row. Daily aggregates are a materialized view over Silver that refreshes incrementally, so a reading 30 days late recomputes only its own day and device. I would not hold 30 days of streaming state in a watermark.

QUESTIONS TO ASK FIRST

- **How late is late, and how often?** If most readings arrive within an hour and a few up to 30 days, the design must be cheap for the common case and correct for the rare one.
- **Can a device send the same reading twice?** Then Silver needs a key, device and reading time, and keeps one row per key.
- **May yesterday's numbers change after they are reported?** If not, late data goes into a correction record instead of silently changing history.
- **How long must raw readings be kept?** Silver is the source for every recompute, so it keeps at least the 30-day window, usually far more.

WHICH TOOL, AND WHY

- **Silver keyed on device and time, aggregates as a materialized view.** My choice. A late reading updates one row, and the view recomputes only the groups that changed.
- **Streaming aggregation with a 30-day watermark.** Correct in theory. The state store holds every open window for 30 days, and every restart and code change carries that state.
- **Nightly full recompute.** Simple and always correct. The cost grows with history, and the aggregates are a day old.
- **Recompute changed days with REPLACE WHERE.** My fallback when the view cannot refresh incrementally. A job finds the changed days and rewrites only those.

AT A GLANCE



A late reading touches one Silver row and one day of aggregates. Nothing waits 30 days for it.

HOW I'D BUILD IT

1 Keep Bronze append-only

Readings arrive from Event Hubs or as files and land in Bronze unchanged, with the arrival time added. Bronze is never updated, so every later step can be rebuilt from it.

2 Upsert into Silver on device and reading time

A streaming job reads new Bronze rows and, in `foreachBatch`, first keeps the latest arrival per device ID and reading time inside the batch, then MERGEs the result into Silver on that key. Silver has liquid clustering on reading date and device.

Watch out. Two Bronze rows for one reading in the same batch make the MERGE fail, so the dedupe comes first. A retried batch then changes nothing; an append would double the rows. [2.12](#)

3 Keep the MERGE cheap for old days

Most batches touch today; a late reading adds an old date. The MERGE condition lists the reading dates in the batch, so with clustering and deletion vectors it reads only files for those dates and marks changed rows instead of rewriting files.

Watch out. A MERGE on device and time alone reads all of Silver to find matches, and every batch gets slower as history grows. **3.08**

4 Build the aggregates as a materialized view

Daily aggregates by device, site and day are a materialized view over Silver. Silver has row tracking, deletion vectors and change data feed on, so the refresh can find the groups that changed. On Databricks Runtime 19 and above, change data feed comes automatically with row tracking; on 18 LTS I set the property. Sums use DECIMAL, because floating-point SUM and AVG are not refreshed incrementally.

Watch out. A view that cannot refresh incrementally recomputes all history on every refresh, and still succeeds. **5.08**

5 Check that refreshes stay incremental

Before go-live I run `EXPLAIN CREATE MATERIALIZED VIEW` to see the planned technique. After each refresh, the event log's planning_information row shows GROUP_AGGREGATE or FULL_RECOMPUTE. With REFRESH POLICY INCREMENTAL STRICT, a refresh that cannot be incremental fails instead of recomputing everything.

Watch out. A refresh that fails leaves the view at its last result. Reports keep showing it, with no sign it is stale. **6.05**

6 Refresh on change, with a ceiling

The view refreshes with `TRIGGER ON UPDATE AT MOST EVERY INTERVAL 15 MINUTES`, so late data flows in without a separate schedule. A small table records each day whose totals changed after it was reported, with old and new values.

Watch out. Without AT MOST EVERY, a busy Silver table wakes the refresh every minute and the bill follows. **5.07**

7 Keep retention for recovery, not for lateness

The 30-day lateness is handled by keys in Silver, not by table history. Time travel only needs to cover a few days for recovery, so retention stays at its defaults. Time travel and RESTORE do not apply to the materialized view, and fail on any table with a row filter, column mask or ABAC policy unless the running identity is exempt (7.17): a bad aggregate is fixed by correcting Silver or the code and refreshing.

Watch out. Raising retention to 30 days to replay late data keeps every rewritten file for 30 days and gains nothing. **3.04**

SECURITY IN ONE BREATH

Devices write to Event Hubs or to their landing folder only. The streaming job and the view run as a service principal that owns Bronze, Silver and the aggregates. Report users read the aggregates, and only engineers read Silver.

HOW I'D KNOW IT WORKS

- A test reading sent 29 days late appears in Silver once and changes only its own day in the aggregate.
- Sending the same reading twice leaves one row in Silver.
- Planning information shows an incremental technique on normal days.
- The aggregate matches a full recompute from Silver for a sample of days each week.

A WORKED EXAMPLE

Situation. For example, 40,000 devices sent about 600 million readings a day. Around 2 percent arrived more than a day late, from sites with poor connectivity, and a few up to 28 days late. The old design used a streaming aggregation with a 7-day watermark and dropped anything later.

Task. Count every reading in its own day, up to 30 days late, without recomputing history every night.

Action. We changed Silver to a MERGE on device and reading time, built the daily aggregate as a materialized view with decimal sums, and turned on row tracking and change data feed on Silver. For two weeks we checked planning information after every refresh.

Result. In this example refreshes were incremental on all but two days, both after a bulk backfill. The share of readings missing from the aggregates fell from about 0.3 percent to zero.

IF THEY PUSH FURTHER**What if the view cannot refresh incrementally?**

Fall back to a job: read Silver's change data feed since the last run, list the changed days, and rewrite only those with `INSERT INTO ... REPLACE WHERE`. **3.10**

What if late readings arrive as files in bursts?

Auto Loader picks them up like any new file. A file arrival trigger with a wait after the last change starts one run per burst. **5.14**

What if devices also send deletions for bad readings?

The MERGE deletes the matching row in Silver. Deletion vectors mark it, and the view refreshes that day. **3.05**

S.12 Serve an operations dashboard that 300 people watch during trading hours, refreshed every five minutes.

Short answer. I'd build the dashboard on a few small summary tables, materialized views that refresh with TRIGGER ON UPDATE at most every five minutes. The AI/BI dashboard is published with shared data permissions on its own serverless SQL warehouse, so 300 viewers share one cache instead of each running queries. A dashboard schedule every five minutes in trading hours keeps that cache current, and the warehouse stops outside them. Every page shows when its data was last updated.

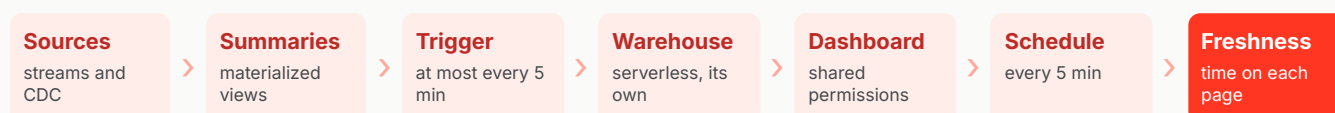
QUESTIONS TO ASK FIRST

- **What does refreshed every five minutes mean to the users?** Data at most five minutes old, or a page that reloads every five minutes. The first sets the pipelines, the second only the dashboard.
- **Does everyone see the same numbers?** Then one shared cache serves all 300. Per-person filters on sensitive data change that.
- **How fresh are the sources?** A dashboard cannot be fresher than the tables under it.
- **What are trading hours, and in which time zones?** They set the schedule and the hours when the warehouse needs to run.

WHICH TOOL, AND WHY

- **AI/BI dashboard on materialized views.** My choice. Summary tables keep queries small, shared data permissions give viewers one cache, and everything stays in Databricks.
- **Power BI in DirectQuery.** Right if the business already lives in Power BI. 300 users on five-minute data send many queries, so heavy visuals need aggregations or Import.
- **Dashboard queries on detail tables.** Nothing to build. Every refresh scans detail data for every visual, and cost grows with the viewers.
- **A job that rebuilds summaries every five minutes.** Easy to reason about. One schedule per table is more to run than views that refresh when their sources change.

AT A GLANCE



Small tables, one warehouse, one cache. The schedule fills the cache, and viewers read from it.

HOW I'D BUILD IT

1 Agree what fresh means

With the operations lead I write down the promise: data on the dashboard is at most ten minutes old in trading hours, and each page shows when it was last updated. The source tables, fed by streaming and CDC pipelines, must already be within a few minutes.

Watch out. A promise made about the dashboard alone breaks the first time an upstream pipeline falls behind. **S.09**

2 Build small summary tables

Each tile reads a summary of at most a few thousand rows: sales by store and hour, open orders by status, stock alerts. Each is a materialized view over Silver with **TRIGGER ON UPDATE AT MOST EVERY INTERVAL 5 MINUTES**, so it refreshes when its sources change and no more often. A view can watch up to 10 upstream tables.

Watch out. A view that fails to refresh keeps serving its last result, and the tile looks normal. **6.05**

3 Give the dashboard its own warehouse

A serverless SQL warehouse serves only this dashboard, small in size with a maximum cluster count set for the peak. Ad hoc analysts use another warehouse, so their heavy queries never queue in front of the tiles.

Watch out. One warehouse for everyone lets one heavy query slow every dashboard. **6.03**

4 Size by queueing, not by size

In the first week I watch peak queued queries on the warehouse's monitoring page. Queueing means more clusters; spilling to disk in the query profile means a larger size.

Watch out. Moving to a larger size does nothing for queueing, and every query costs more. **6.01**

5 Publish with shared data permissions

The dashboard is published with shared data permissions, so all viewers share one cache and see the same numbers. Viewers get access to the published dashboard, not to the tables.

Watch out. Every viewer sees what the publisher can read. The publisher should hold SELECT on the summary schema and nothing more. **6.07**

6 Refresh the cache on a schedule

The dashboard cache is best effort and can serve results up to 24 hours old, and new data does not invalidate it. A dashboard schedule every five minutes in trading hours, written in Quartz cron, reruns the datasets and refreshes the cache. Serving from the cache does not start the warehouse.

Watch out. A pipeline that refreshes the tables does not refresh the dashboard. Without the schedule, viewers see the cache. **6.12**

7 Stop paying outside trading hours

The warehouse auto-stops after a few idle minutes, and the schedule runs only in trading hours, so nothing keeps it awake at night. The view triggers still fire when sources change, on their own serverless compute.

Watch out. A stopped warehouse makes the first viewer wait at opening. A schedule run just before trading starts warms it. **6.02**

8 Alert on stale data

A SQL alert checks the newest update time across the summary views every five minutes and notifies the operations support group when it is more than ten minutes old. It runs on its own schedule, so it fires even when nobody opens the dashboard. This explicit alert is the primary guard. Anomaly detection in data quality monitoring, in Public Preview, adds a net over the source schemas for tables nobody wrote an alert for, and I would not rely on it alone.

Watch out. An alert is evaluated only when its schedule runs. A daily schedule finds the problem the next day. **6.08**

SECURITY IN ONE BREATH

Viewers see the published dashboard and nothing else. The publisher holds SELECT on the summary schema only, so shared permissions expose no detail data. If store managers must see only their stores, the dashboard switches to individual data permissions with a row filter, and the cache becomes per viewer.

HOW I'D KNOW IT WORKS

- The last updated time on every page stays within ten minutes of the clock through a trading day.
- Peak queued queries stay near zero, and tiles load in a few seconds at the busiest hour.
- The warehouse shows no running time outside trading hours.
- Stopping one source pipeline for 15 minutes fires the stale data alert.

IF THEY PUSH FURTHER

What if the same numbers must appear in Power BI?

Point Power BI at the same summary views, in Import mode or with heavy visuals aggregated, and keep it off the detail tables. **6.09**

What if each store manager may see only their own stores?

Add a row filter to the summary views and publish with individual data permissions. Each viewer then runs their own queries, so size the warehouse for that. **7.08**

What if someone runs a heavy query on the dashboard warehouse?

Give analysts no access to it, and set STATEMENT_TIMEOUT so a runaway query ends on its own. **6.06**

S.13 Serve Gold tables to 2,000 Power BI users with row-level security and reports that open in under five seconds.

Short answer. I'd keep the security rule in one place: a Unity Catalog row filter on the Gold fact tables, with one exempt group for the platform jobs that must still copy and restore them, reached by each viewer's own identity through Power BI single sign-on. Facts stay in DirectQuery, small dimensions go to Dual, and the heaviest visuals read pre-aggregated Gold tables. Revenue and margin are defined once in a metric view, and the Power BI measures are checked against it. A serverless SQL warehouse sized on peak queued queries serves the reports, through a gateway if the workspace is private. I prove the five seconds with Performance Analyzer and query history before go-live, not after.

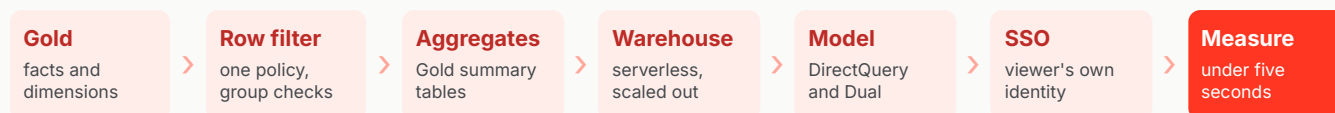
QUESTIONS TO ASK FIRST

- **What decides which rows a user may see?** A region, a cost centre or a customer list. If it already lives in an HR or entitlement table, a job turns it into group membership instead of a list I maintain by hand.
- **How fresh must the reports be?** Daily data can be imported and served from memory. Data that changes during the day pushes the facts towards DirectQuery.
- **How many people open reports at the same moment?** 2,000 named users is rarely 2,000 at once. The Monday 09:00 peak sizes the warehouse, not the head count.
- **Which Power BI licence and capacity do we have, and is the workspace private?** Publishing from Databricks needs Premium, Premium Per User or a Fabric capacity, with the XMLA endpoint set to Read Write. A workspace or storage account behind Private Link or a firewall puts a data gateway in the path of every report query.

WHICH TOOL, AND WHY

- **Composite model with single sign-on.** My choice. Facts in DirectQuery, dimensions in Dual, viewers' own identities sent to Databricks. The row filter in Unity Catalog applies in every tool that reads the table.
- **Import with Power BI roles.** Fastest to open and cheapest on the warehouse. The security rule then lives in the semantic model as well, and two copies of a rule drift apart.
- **DirectQuery for every table.** Simple, but every slicer click sends queries for every visual, including dimension lookups that never change.
- **AI/BI dashboards in Databricks.** Good for users who already work in Databricks. This organisation reports in Power BI, so it is a second front end to support.

AT A GLANCE



One rule in Unity Catalog, one warehouse for reports, and a model that asks for as little as it can.

HOW I'D BUILD IT

1 Write the row rule once

Each region has an account group, such as `sales-emea`. I write one SQL function that checks the viewer's group membership, and attach it with an ABAC row filter policy on the Gold schema, so every tagged fact table gets it. Dimensions that are not sensitive stay unfiltered. The policy has an EXCEPT group that holds the platform service principal for clone, restore and sharing jobs: a filtered table cannot be cloned or shared by anyone else, and time travel on tables under ABAC policies is in Beta. 7.17 covers who owns that list and how it is audited.

Watch out. A filter that calls `current_user()` returns a different answer when a job or a shared identity runs the query. **7.07**

2 Keep the filter cheap

The function is plain SQL: a CASE on `is_account_group_member`, one column argument, no Python and no table lookup. A job keeps the region groups in step with the entitlement source.

Watch out. A filter that looks up a mapping table adds that lookup to every query on the table, for every user. **7.08**

3 Pre-aggregate and define the measures once

The landing page shows totals by day, region and product. I build those as Gold summary tables or materialized views, carrying the same region column so the same policy covers them. Revenue, margin and order count live in a Unity Catalog metric view, which Genie and AI/BI dashboards use directly. Power BI reads a metric view only through a native query with `MEASURE()` in DirectQuery, since the BI compatibility mode was removed in April 2026, so I keep the model's DAX measures and test them against the metric view each release.

Watch out. Two definitions of revenue give two answers for the same month. **10.07**

4 Build the model

A Power BI admin first grants tenant-wide consent to the Databricks Dataset Publishing Integration app in Entra ID. Then I publish the Gold schema from Catalog Explorer with Publish to Power BI workspace, which carries foreign keys as relationships and column comments as descriptions. Facts and summaries are DirectQuery. Small, non-sensitive dimensions are Dual, so slicers do not query the warehouse. I turn on Assume referential integrity where Gold keys are enforced upstream, and add Apply buttons to the slicer panes.

Watch out. Without query reduction, one report page sends dozens of queries every time a user moves a slicer. **6.09**

5 Turn on single sign-on

In the semantic model's data source credentials, I select the option for report viewers to use their own Power BI identities in DirectQuery. Every viewer must then exist in the Databricks account, hold SELECT through a group and have CAN USE on the warehouse.

Watch out. Dual tables are imported with the model's credential, so a dimension that needs filtering must stay DirectQuery.

6 Size the warehouse and the path to it

Reports get their own serverless SQL warehouse, separate from analysts and jobs. I set the maximum clusters from the Peak Queued Queries metric in a load test, and keep the cluster size where queries do not spill to disk. If the workspace or the workspace storage account is private, every query goes through a virtual network or on-premises data gateway: I run the gateway as a cluster of at least two members, size it for the peak, and load test through it. Large results come back by Cloud Fetch straight from the workspace storage account, so a storage firewall must admit the gateway too.

Watch out. One warehouse for everyone lets one analyst's heavy query slow every report. **6.03**

7 Measure, then tune

I open the main pages with Performance Analyzer as three test users from different regions. Each slow visual maps to a statement in query history, where I check read bytes, files and time waiting for compute. Most fixes are in the model or in Gold, not in the warehouse size.

Watch out. A bigger warehouse does not speed up a query that scans the whole table. **6.01**

SECURITY IN ONE BREATH

Viewers reach data only through the report, with their own identity, so the audit log names the person and Unity Catalog applies the same row filter as everywhere else. Report users get CAN USE on the report warehouse only. Region groups are changed only through the entitlement job, which is reviewed like code.

HOW I'D KNOW IT WORKS

- Three test users from different regions see only their own rows, in Power BI and in the SQL editor.
- The landing page opens in under five seconds at the Monday peak, measured with Performance Analyzer.
- Peak Queued Queries on the report warehouse stays at or near zero during the busiest hour, and the gateway shows no queued requests.
- Query history shows report queries under each viewer's name, not one shared account.

A WORKED EXAMPLE

Situation. For example, a retailer had 2,000 Power BI users on an imported model with Power BI roles. Refresh took 70 minutes, and the region list in the model differed from the one in Databricks.

Task. Keep one row rule, keep reports under five seconds, and stop the long refresh.

Action. We moved the rule into an ABAC row filter on region groups, built two Gold summary tables for the landing page, set facts to DirectQuery and dimensions to Dual, and gave reports their own serverless warehouse with a maximum of 4 clusters.

Result. In this example the landing page opened in about 3 seconds at peak, the import refresh disappeared, and access reviews covered one rule instead of two.

IF THEY PUSH FURTHER

What if the security team will not allow single sign-on?

Then the model connects as a service principal and the warehouse sees one identity. The rule has to move to Power BI roles, and the audit trail for each viewer comes from Power BI, not Databricks. **6.07**

What if the model must refresh straight after the nightly load?

The Power BI task in Lakeflow Jobs, in Public Preview, updates and refreshes the semantic model as the last task of the job, run as a service principal.

What if some reports still miss five seconds?

Add user-defined aggregations in Power BI over the summary tables, or move those pages to Import with incremental refresh, and accept a second copy of the rule for that model. **6.09**

S.14 Share a subset of Gold with three external partners, one on Databricks and two without it.

Short answer. I'd use OpenSharing with one share and three recipients. The partner on Databricks gets a Databricks-to-Databricks recipient, so no token ever leaves us. The other two use Databricks-to-Open sharing, preferably with OIDC federation, otherwise with bearer tokens that expire. Each recipient carries a partner_id property, and a dynamic view filters rows with CURRENT_RECIPIENT(), so one share serves all three, nobody sees another partner's rows, and Gold keeps its liquid clustering.

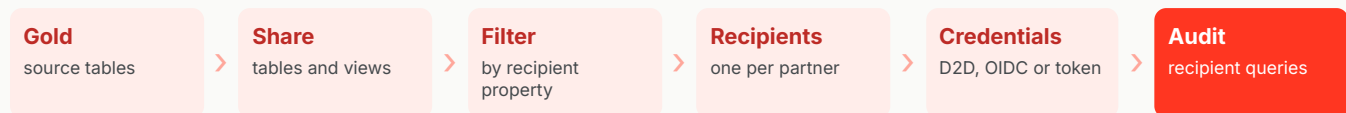
QUESTIONS TO ASK FIRST

- **Which rows and columns may each partner see?** If the answer is the same columns filtered by partner, one share with recipient properties works. Different columns per partner means a view per partner.
- **What will the two non-Databricks partners read with?** Spark, pandas, Power BI and other clients speak the open protocol. Their identity provider decides whether OIDC federation is possible.
- **Do partners need history or a stream of changes?** Tables with deletion vectors or column mapping, which most new Gold tables have, must be shared WITH HISTORY, the default on Runtime 16.2 and above. History lets a recipient time travel and stream, so whatever must stay current only reaches partners through a view.
- **Where are the partners?** Sharing within a region has no egress cost. Another region or cloud brings the cloud provider's transfer charges.

WHICH TOOL, AND WHY

- **OpenSharing with recipient properties.** My choice. Read-only, live, no copies to manage, and every recipient query is in our audit log. One share, filtered per recipient.
- **Extract files to partner storage.** Works with any partner, but creates copies we cannot revoke, a job per partner, and no record of what they read.
- **Partner logins in our workspace.** Gives partners our workspace, our compute and our bill. Hard to justify for read-only data.
- **Clean rooms.** For joint analysis where neither side may see the other's raw rows. Here we publish data one way, so a share is enough.

AT A GLANCE



One share, three recipients, and a filter that reads who is asking.

HOW I'D BUILD IT

1 Turn on sharing and set token lifetime

A metastore admin enables OpenSharing for the metastore and sets the default recipient token lifetime. Tokens can live at most one year, and I set them shorter. I create a group that owns shares and recipients, so no single engineer does.

2 Shape the shared data

Partners get a narrow Gold schema built for them, not our internal tables. Columns they must not see never enter it. Every table carries a `partner_id` column, and I keep the liquid clustering it already has. I do not repartition Gold by `partner_id` for the share: with three partners that gains nothing and brings back the small files and rewrites that clustering removed.

Watch out. A table under ABAC policies can be shared only by an owner who is exempt from them, and recipients then read it in full, so the filter has to live in the share (7.17). [C.12](#)

3 Create the share with a partner filter

For each table I create a dynamic view with `WHERE partner_id = CURRENT_RECIPIENT('partner_id')`, on Runtime 14.2 or above, and add the views to the share. A recipient without the property sees nothing. Tables that every partner may read in full are added directly, `WITH HISTORY`, which also exposes old versions and rows deleted since, within retention, so I check that is acceptable. A partner that needs `table_changes` also needs change data feed switched on before the share. To test a view I share it with a recipient for our own metastore, because a provider cannot query a dynamic view directly.

Watch out. A view shared to an open recipient is fully materialised on our side for each query, with no predicate pushdown and no `LIMIT`, and we pay for the compute.

4 Add the Databricks partner

The partner runs `SELECT CURRENT_METASTORE()` and sends us the sharing identifier. I create the recipient with `USING ID` and a `partner_id` property, then grant `SELECT` on the share. They mount it as a catalog and grant it to their own groups.

5 Add the two open partners

If a partner's identity provider supports it, I create the recipient with `OIDC` federation, so they sign in with short-lived tokens from their own provider. Otherwise the recipient gets a bearer token: the activation link works once, so I send it over a secure channel and add an IP access list.

Watch out. An open recipient's reader must support the table features we use, such as deletion vectors and column mapping. [3.06](#)

6 Plan rotation

Each token's expiry goes into the runbook a month ahead. Rotation gives the recipient a new token while the old one keeps working for the seconds I choose, and a recipient holds at most two tokens at a time.

7 Watch usage

Recipient queries appear in the audit log under the recipient's name. A monthly query lists what each partner read and when, and recipients that read nothing for 90 days are reviewed.

Watch out. Audit logs in the system table are kept for a year, so evidence for older periods has to be copied out in time. **7.10**

8 Onboard and offboard partners

A new partner needs a signed agreement, the data owner's approval of the columns and rows, a recipient with its `partner_id` property, a test as that recipient and only then the grant. At the end of a contract I revoke the share grant, drop the recipient, which ends its tokens, remove its rows from any partner-specific table and check the audit log shows no further queries. Data the partner already pulled cannot be recalled, so the agreement says what they delete.

SECURITY IN ONE BREATH

Partners read through the share only and cannot write. Each recipient sees rows for its own `partner_id` through a dynamic view, set by us as a recipient property it cannot change. Bearer tokens expire, are bound to an IP access list and are rotated on a schedule; the Databricks partner has no token at all.

HOW I'D KNOW IT WORKS

- Querying the share as each recipient returns only that partner's rows, and the counts match Gold.
- A recipient without a property, or with a wrong one, sees no rows.
- Every token's expiry date is in the runbook, with a rotation date before it.
- The audit log shows each partner's queries under its own recipient name.

IF THEY PUSH FURTHER**What if a fourth partner is in another region?**

The same share works. Egress between regions is charged by the cloud provider, so for heavy readers I check whether a scheduled copy in their region costs less. **1.08**

What if a partner needs every change, including deletes?

Views cannot be streamed. Build a Gold table for that partner alone, share it WITH HISTORY to that recipient only, and keep enough history for a partner that falls behind. **3.10**

What if a partner leaks a token?

Rotate it with the existing token set to expire at once, check the audit log for queries from unexpected addresses, and tighten the IP access list. **7.13**

S.15 Let the Microsoft Fabric team build reports on Databricks Gold without copying it.

Short answer. I'd give Fabric a mirrored Azure Databricks catalog over a Gold schema built for it. Mirroring copies only metadata: Fabric gets shortcuts to the Delta tables, a SQL analytics endpoint and Direct Lake, and the files stay in our storage. Unity Catalog grants, row filters and ABAC policies do not travel, so the schema is a governed derived copy of Gold that holds only data the Fabric team may see. That is the one copy I accept, and Fabric's own roles take over from there.

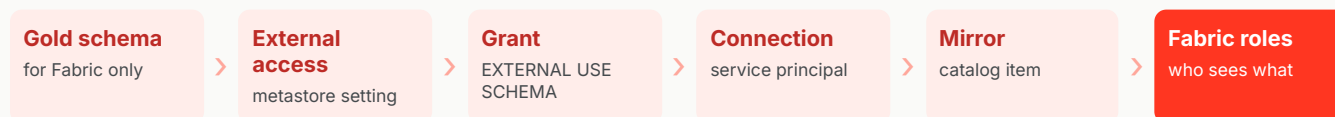
QUESTIONS TO ASK FIRST

- **Which Gold tables does the Fabric team need?** Mirroring skips views, materialized views, streaming tables and tables with row filters or column masks, including those reached by an ABAC policy set on the catalog or schema. Whatever they need must be a plain Delta table.
- **Who in Fabric may see what?** Unity Catalog permissions are not mirrored. If different Fabric users need different rows, that is set again in Fabric, or the data is split into separate schemas.
- **Is our storage behind a firewall, and what Fabric capacity do we have?** A storage firewall needs trusted workspace access for the Fabric workspace identity, which works only on a purchased F SKU capacity, not on a P SKU or a trial. A workspace on Private Link needs a virtual network data gateway.
- **How fresh must the Fabric reports be?** Data changes reach Fabric within seconds to minutes. Reports that need the exact state of a nightly load should refresh after it.

WHICH TOOL, AND WHY

- **Mirrored Azure Databricks catalog.** My choice. No copy, no pipeline, Direct Lake reports on our tables, and tables the data owner adds to the schema follow once approved.
- **Power BI DirectQuery to a SQL warehouse.** Keeps Unity Catalog security per user. Every report query runs on our warehouse, and the Fabric team cannot use Fabric's other engines on the data.
- **OneLake shortcut to the storage path.** Reads the folder, not the table. Path access to managed tables is not supported, and a folder read ignores the Delta log.
- **Copy into a Fabric lakehouse.** Fabric then owns a copy, with its own refresh to run and its own version of the truth.
- **Publish to Fabric from Databricks.** In Public Preview: a catalog owner publishes a Unity Catalog catalog to a Fabric workspace from the Databricks side, as read-only mirrored tables. Worth trying once it is GA.

AT A GLANCE



Databricks decides what Fabric may read. Fabric decides who in Fabric reads it.

HOW I'D BUILD IT

1 Build a governed derived copy for Fabric

I create a schema such as gold_fabric with managed Delta tables built from Gold by a job. It is a copy, so it has an owner, a refresh schedule, lineage back to Gold and a place in the data catalog. It holds only the columns and rows the Fabric audience may see, with no row filters, masks or ABAC policies reaching it, since such tables are not mirrored. I check that no catalog-level policy applies to the schema. The cost sits in the refresh job and storage for the copy on our side, in storage reads when Fabric scans, and in the Fabric capacity that runs the reports.

Watch out. Pointing Fabric at the folders behind managed tables gives numbers that drift from ours. **1.07**

2 Allow external engines to read

A metastore admin turns on External data access for the metastore. The catalog owner grants EXTERNAL USE SCHEMA on gold_fabric to a service principal created for Fabric, together with USE CATALOG, USE SCHEMA and SELECT.

Watch out. EXTERNAL USE SCHEMA is not part of ALL PRIVILEGES, so a principal with every other right still cannot connect. **7.01**

3 Open the network path

If the storage account has a firewall, I add trusted workspace access for the Fabric workspace identity: a resource instance rule for the Fabric workspace, which can only be created with an ARM template or PowerShell, not in the Azure portal. The Fabric workspace must sit on a purchased F SKU. Fabric reads the files with that identity even when the connection uses a service principal. If the Databricks workspace is on Private Link, the Fabric admin creates a virtual network data gateway and the connection in advance.

Watch out. Opening the storage account to Fabric reads below Unity Catalog, so any row filter you rely on is not enforced there. **7.05**

4 Create the mirrored catalog

In the Fabric workspace, New item, Mirrored Azure Databricks catalog, with the service principal connection. I select the gold_fabric schema. Automatic sync of future changes sends any table added to that schema to Fabric, so adding a table to gold_fabric is a reviewed change that the data owner approves. If that gate is not wanted, I turn automatic sync off and pick tables one by one.

5 Set Fabric permissions

The credential on the connection is used for every query, so Unity Catalog sees one identity. Fabric workspace roles and item permissions decide who reads which table. If two audiences need different rows, I publish two schemas and two mirrored items.

Watch out. A shared report can show its viewers data they could not query themselves. **6.07**

6 Keep the tables healthy for both engines

The tables are managed, so Predictive Optimization compacts and vacuums them. Before I turn on a new Delta table feature on a mirrored table, I check that Fabric still reads it.

Watch out. A table feature one engine writes can make another reader fail or return old rows. **3.06**

SECURITY IN ONE BREATH

Fabric reaches only gold_fabric, through one service principal with read rights and EXTERNAL USE SCHEMA. The storage firewall admits the Fabric workspace identity and nothing broader. Inside Fabric, workspace roles decide access, and Unity Catalog's audit log shows every read by the service principal.

HOW I'D KNOW IT WORKS

- Every table in gold_fabric appears in the mirrored item, and a table added after approval appears without further steps.
- Row counts in the SQL analytics endpoint match Databricks for the same table.
- Tables outside gold_fabric are not visible in Fabric, even to Fabric workspace admins, and the storage account lists one resource instance rule for the Fabric workspace.
- No storage path outside the gold_fabric tables is reachable from Fabric.

IF THEY PUSH FURTHER**What if Fabric users need per-user row security?**

Either use Power BI DirectQuery with single sign-on to a SQL warehouse, where Unity Catalog filters apply, or rebuild the rule in Fabric. Mirroring alone carries no row security. **7.06**

What if the Fabric team wants to write back?

Mirroring is read-only. They write to their own Fabric lakehouse, and we ingest from there like any other source. **C.13**

What if Gold lives in another region?

Fabric reads our storage across regions and pays the transfer on every scan. Keep the Fabric workspace and the Gold storage in the same region. **1.08**

S.16 Open SQL to 150 analysts without one query or one team running away with the budget.

Short answer. I'd give analysts serverless SQL warehouses that admins create and size, one per group of teams, each tagged with its cost centre. Analysts get CAN USE and nothing more, so the maximum cluster count is the spending ceiling. A workspace statement timeout stops the accidental runaway query, though any user can raise it in their session. Budgets filtered on each team's tag send the monthly warning, and a daily query on billing and query history names the team and the query before the month ends.

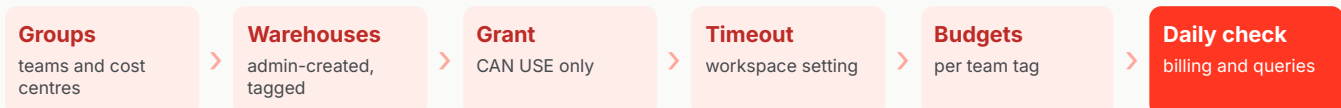
QUESTIONS TO ASK FIRST

- **How are the 150 analysts grouped?** By team or cost centre. That grouping becomes the warehouse split, the tag and the budget line.
- **What is the monthly budget, and who owns it?** A number per team turns cost into an alert threshold. Without an owner, an alert reaches nobody who can act.
- **How long may a single query run?** Most analyst queries finish in minutes. The default timeout is two days, which nobody chooses on purpose, and the right value comes from query history, not a guess.
- **Which data may analysts query?** Grants decide what they read. Warehouses decide only how fast and at what cost.

WHICH TOOL, AND WHY

- **Serverless SQL warehouses per team group.** My choice. Starts in seconds, stops when idle, and each warehouse carries tags that land in system.billing.usage.
- **One large shared warehouse.** Easy to run, but one team's heavy work slows everyone and the bill cannot be split by team.
- **A warehouse for every analyst.** Perfect attribution, many idle warehouses and nobody watching their sizes.
- **All-purpose clusters.** Priced and tuned for notebooks, not for many short SQL queries, and users can change their own settings.

AT A GLANCE



The size and maximum clusters cap what a warehouse can spend. The tags and the daily check say who spent it.

HOW I'D BUILD IT

1 Take away the controls that spend money

Analysts do not get unrestricted cluster creation, so they cannot create warehouses. Admins create them, own them and keep CAN MANAGE. Analyst groups get CAN USE, which runs queries but does not change size or scaling.

Watch out. Cluster policies do not govern SQL warehouses at all; the only limits on a warehouse are who may create it and who holds CAN MANAGE on it. **4.07**

2 Split by team group and tag

I create three or four serverless warehouses, one per group of teams with similar work, each tagged with cost_centre and team. Maximum clusters is set from Peak Queued Queries in the first weeks, then held there.

Watch out. Moving to a larger size does nothing for queueing, and every query costs more. **6.01**

3 Stop the runaway query

I pull 30 days of query history for every statement type, SELECT, INSERT, MERGE and CREATE TABLE AS, and set the workspace STATEMENT_TIMEOUT just above the longest legitimate run, for example the p99 plus a margin, here two hours. The platform lead proposes the value and the owners of the jobs and dashboards that would have been cut off approve it. I first publish the list as a report only, then set the value in a quiet week, and announce it. To revert, I set it back to the default of two days. A team with a real need for longer queries files a request with the query, owner and end date, and gets its own tagged warehouse; per-warehouse timeouts are in Beta.

Watch out. Any user can SET STATEMENT_TIMEOUT higher in a session, so the timeout stops accidents, not intent. **6.06**

4 Keep auto stop short

Serverless warehouses stop after 10 minutes idle by default, and I keep that or lower it to 5 in the UI. Report warehouses that must be warm at 08:00 are handled separately (6.02).

5 Set budgets

In the account console I create a budget per team, scoped to its team tag rather than to a workspace, with alerts at 50, 80 and 100 percent of its monthly figure and the team lead on the email list. Budgets warn; they do not stop spend, and their emails can arrive up to 24 hours late, which is why the daily check exists.

Watch out. Finding one team's share of the bill at month end is too late to change it. **8.04**

6 Run a daily cost and query check

A job joins `system.billing.usage` by warehouse and tag with list prices, and `system.query.history` for the longest and largest queries per user. `system.query.history` covers SQL warehouses and serverless compute but not classic clusters, so any classic compute is costed from billing by cluster tag. A SQL alert fires when a team passes its daily share, and a second one lists statements that ran longer than the workspace timeout, which can only happen through a session override.

Watch out. A monitoring query that scans all of history every hour can cost more than the warehouses it watches. **8.05**

7 Give analysts a way to see their own cost

A dashboard on the daily check shows each team its spend and its ten heaviest queries. `statement_text` in query history is masked by default, so the job that builds it runs as an identity in the `databricks_pii_access` group, and the dashboard shows only each team's own rows.

SECURITY IN ONE BREATH

Analysts can run queries on their team's warehouse and nothing else on compute. Grants on catalogs decide what they read. Cost data and full query text sit with admins, and teams see only their own part through a dashboard.

HOW I'D KNOW IT WORKS

- No analyst holds CAN MANAGE on any warehouse, and no analyst-created warehouse exists.
- A test query without a session override is cancelled at two hours, and the daily alert lists any statement that ran longer than that.
- Every warehouse's usage in `system.billing.usage` carries `cost_centre` and team tags.
- The daily alert fires in a test where one team's spend doubles.

A WORKED EXAMPLE

Situation. For example, 150 analysts shared one X-Large warehouse with a maximum of 10 clusters. One month, a data science team's exploratory joins took 40 percent of the bill.

Task. Keep SQL open to everyone and make the next spike visible within a day.

Action. We split into four tagged serverless warehouses with CAN USE grants, set a two hour timeout and four-cluster maximums, created a budget per team tag, and built a daily job over billing and query history with a SQL alert.

Result. In this example the next spike was flagged the following morning, traced to one scheduled query that had lost its date filter, and fixed the same day.

IF THEY PUSH FURTHER

What if one team needs a much bigger warehouse for a month?

Create it for that team, tagged and with an end date in the runbook, rather than resizing the shared one. **6.03**

What if finance wants the total to match the Azure invoice?

System tables price at list rates. The invoice reflects your discounts, so reconcile the two once and keep the ratio. **8.07**

What if analysts also use serverless notebooks?

SQL warehouses take tags directly. Serverless notebooks and jobs take tags through serverless usage policies, which are in Public Preview. **8.01**

S.17 Build a governed lakehouse for patient records from four hospital systems, with masking and an audit trail.

Short answer. I'd run it in workspaces with the compliance security profile on, and keep raw patient data in a catalog that only the ingestion identity can read. Silver replaces direct identifiers with a stable patient key, which is pseudonymisation, so Silver is still patient data. Data classification tags sensitive columns, and ABAC policies on those tags mask them and filter rows by hospital. Classification reaches a new table within about a day, so new tables wait in a staging schema until they are tagged. The audit trail is the system tables, copied to our own table and to immutable storage before they age out.

QUESTIONS TO ASK FIRST

- **Which regulations and contracts apply?** HIPAA and similar rules decide the compliance security profile, key management and how long audit evidence is kept.
- **Who may see identifiable data, and why?** Usually a small clinical team. Everyone else works on masked or pseudonymised data, and that split drives every policy.
- **Can clinicians from one hospital see another hospital's patients?** If not, every table needs a hospital column and a row filter on it.
- **How do the four systems deliver data?** Database replication, HL7 or FHIR files and exports each need a different ingestion path, and some need a private network route.

WHICH TOOL, AND WHY

- **Unity Catalog ABAC with data classification.** My choice. Classification tags sensitive columns, and one policy per tag masks them across every table in the schema.
- **Row filters and masks table by table.** Precise, but each new table is unprotected until someone remembers to add the function.
- **Dynamic views over hidden tables.** Works, but users and tools must be kept away from the base tables, and every new column needs a view change.
- **Separate copies per audience.** Easy to explain, and each copy is another place patient data lives and must be secured.

AT A GLANCE



Identifiers stop at Bronze, tags drive the rules, and every read leaves a record.

HOW I'D BUILD IT

1 Prepare the workspaces

Production workspaces get the compliance security profile, which adds a hardened image, enhanced security monitoring and automatic cluster updates. It is an add-on with its own charge. Storage uses customer-managed keys where the contract requires them.

2 Land raw data in a locked catalog

Each system lands in its own schema of a health_raw catalog. Only the ingestion service principal and a break-glass group can read it. Every row keeps the source system and load time.

Watch out. Code that prints sample rows or secrets puts them into notebook output and job logs. **7.11**

3 Pseudonymise in Silver

Silver replaces names, addresses and record numbers with a patient key from a lookup table that sits in its own restricted schema. Clinical data stays, with a hospital_id on every row. Whoever holds the lookup table can reverse the key, so Silver is protected like Bronze. Data counts as de-identified only after the privacy officer has signed off a method such as HIPAA Safe Harbor or expert determination.

Watch out. A CTAS copy of a protected table starts with no filter or mask. **7.06**

4 Classify and tag

I enable data classification on the Silver and Gold catalogs. It scans tables, detects personal and health data and applies governed tags, and I review its suggestions before turning on automatic tagging. Classification and ABAC row filter and column mask policies are GA; scanning views and policies on views are Beta, so I protect tables and keep views out of the policy path. Pipelines that know their identifier columns set the tag when they create the table. New tables land in a staging schema that only the pipeline can read, and a promotion step moves them on only when their sensitive columns carry tags.

Watch out. New tables are typically scanned within 24 hours, and automatic tagging does not backfill existing columns before the next scan, so a table read before its scan has no policy on it.

5 Write the policies once

An ABAC column mask policy on the catalog masks every column tagged as a direct identifier, except for the exempt groups: the clinical group, and the platform service principal that runs recovery, clone and sharing jobs, since those fail on protected tables for anyone else (7.17). A row filter policy limits rows to the user's own hospital through group membership. Each hospital has one account group, synced from an Entra group that the hospital's access owner manages and reviews every quarter, and removals reach Databricks with a delay (7.14).

Watch out. Complex filter functions slow every query on the table. **7.08**

6 Serve each audience

Gold has a schema per audience: clinical, operations and research. Research gets aggregates and de-identified extracts only. Views for an audience are owned by a group, and their owners hold no more access than the audience.

Watch out. A view reads its source tables as its owner, so policies on those tables are judged against the owner and readers see what the owner sees. An owner in an exempt group passes data through unmasked.

7.04

7 Keep the audit trail

A daily job copies `system.access.audit`, `system.query.history` and `system.access.table_lineage` rows for the health catalogs into a table in a restricted schema, and as plain Parquet files into a storage container with a time-based immutability policy, not as Delta, because Delta maintenance deletes files. The table is for querying and is append-only by convention only, since anyone with MODIFY or ownership can change it, so the immutable copy is the evidence. The audit table is in Public Preview. `system.query.history` leaves out classic compute, so the health catalogs are bound to workspaces where policy allows SQL warehouses and serverless compute only. The system tables keep a year; the copy keeps what the regulator asks.

Watch out. Evidence older than the system table retention is gone unless you kept it. 7.10

SECURITY IN ONE BREATH

Raw identifiers live in one catalog that people cannot read. Everywhere else, tags and ABAC policies mask identifiers and filter by hospital, with exceptions granted to named groups. Every read is recorded in the audit system table and copied to a table that only the compliance team can query.

HOW I'D KNOW IT WORKS

- A user outside the clinical group sees masked identifiers in every Gold and Silver table, including one created two days ago, while a table created today is still in staging.
- A clinician from one hospital sees no rows from another hospital.
- Data classification results list no untagged column of a sensitive class in Silver or Gold.
- The audit copy holds yesterday's reads of the health catalogs, with user, time and source address.

A WORKED EXAMPLE

Situation. For example, four hospitals sent data to one team, and access was managed with 60 dynamic views. Two new tables had been readable unmasked for weeks before anyone noticed.

Task. Make protection automatic for new tables and give compliance a full year of read evidence on demand.

Action. We moved raw data into a locked catalog, added a patient key in Silver, enabled data classification, wrote two ABAC policies on governed tags and set up a daily audit copy job.

Result. In this example the 60 views became two policies. A table added the next month waited a day in staging, was tagged by the next scan and was masked when it moved to Silver, and the compliance team answered an access request in an hour.

IF THEY PUSH FURTHER

What if researchers need record-level data?

Give them a de-identified extract in a separate catalog, approved per study once the privacy officer has accepted the de-identification method, and bind that catalog to a research workspace only. **1.04**

What if a patient asks to be erased?

I work through a checklist: raw files and volumes, Bronze, the patient key lookup table, Silver and Gold. Then REORG TABLE ... APPLY (PURGE) and VACUUM remove old files, storage soft delete and any clones or snapshots are cleared, and copies in audit and query history exports are handled. The data protection owner signs off. **7.16**

What if we need to restore a protected table?

Time travel is not supported on tables with table-level row filters or column masks, and with ABAC policies it is only in Beta. Clones and sharing on ABAC tables work only for an exempt identity, so the recovery service principal sits in the exempt group (7.17) and its use is audited. Otherwise the recovery plan rebuilds the table from Silver. **3.04**

S.18 Connect workspaces and serverless compute to storage accounts and databases that allow no public network access.

Short answer. I'd give classic compute a NAT gateway for outbound traffic and private endpoints in the workspace's own virtual network. Serverless reaches same-region ADLS through a network security perimeter in Transition mode, which costs nothing, and gets private endpoints through a network connectivity configuration only for databases and for accounts that must have public access Disabled. The workspace storage account goes through its own firewall procedure. Databases on premises sit behind a forwarder tier in Azure, because a load balancer cannot point at them directly. Secrets move off Key Vault-backed scopes where the vault's firewall gets in the way.

QUESTIONS TO ASK FIRST

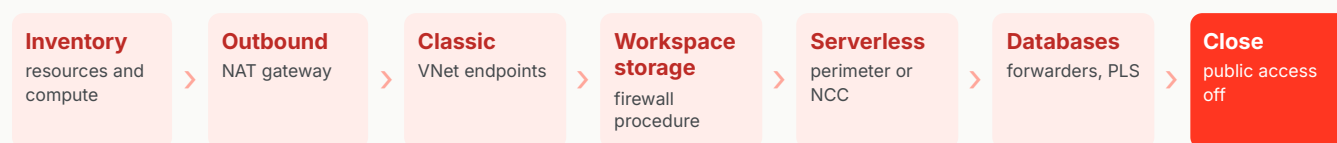
- **Which compute will reach these resources?** Classic clusters run in our virtual network. Serverless runs in Databricks' network and needs either a network security perimeter or its own private endpoints.
- **Is the workspace VNet-injected with secure cluster connectivity?** Both are required for classic Private Link and for the workspace storage firewall. A workspace without them must be redeployed or migrated first.
- **Must every account show public network access Disabled, or is a perimeter enough?** A perimeter keeps the account on selected networks and is free within the region. Disabled means private endpoints, billed per hour each.
- **Where do the databases sit?** An Azure SQL database takes a private endpoint directly. A database on a VM or on premises needs a Private Link service and, for on premises, forwarders in front of it.

WHICH TOOL, AND WHY

- **Perimeter for ADLS, private endpoints for the rest.** My choice. Classic gets endpoints in the workspace VNet; serverless reaches same-region storage through a perimeter with the AzureDatabricksServerless service tag, and databases and locked-down accounts through NCC private endpoints.
- **Private endpoints for everything.** Lets every resource show Disabled, at an hourly charge per endpoint and against a limit of 100 serverless endpoints per region.
- **Classic compute only.** Avoids the serverless setup and gives up serverless SQL, jobs and pipelines.

- **IP allowlists on each resource.** Serverless outbound addresses change, with updates as often as every 30 days, and storage accounts cannot use the serverless subnet IDs any more.

AT A GLANCE



Two compute planes, the cheapest private path for each resource, then the public door closes.

HOW I'D BUILD IT

1 List every resource and who reaches it

A table lists each storage account, database, Key Vault and API, with the sub-resource it needs (dfs, blob, sqlServer, vault), the region, whether classic, serverless or both use it, and whether policy demands public access Disabled. The workspace storage account and the landing account for uploads are on the list too. Each row also gets an approver, a change window and the rollback, which is the setting as it was before. I price the endpoints first: endpoints x hours x hourly rate, plus GB processed x per-GB rate, from the Azure price list. The perimeter path is free, and Databricks bills serverless private networking on top.

2 Give classic compute a way out

Virtual networks created after 31 March 2026 have no outbound internet by default, so a new workspace's clusters cannot reach PyPI, Maven or Azure services. I put a NAT gateway on both workspace subnets, or route both through our firewall, for a stable egress IP. No egress load balancer with secure cluster connectivity.

Watch out. A new workspace fails where the old one worked, because only the new VNet lost default outbound access. **8.10**

3 Connect classic compute

In the workspace VNet I create private endpoints for each storage account's dfs and blob sub-resources and for each database, with the private DNS zones linked to the VNet. Each storage account gets a resource instance rule for the Azure Databricks access connector, so external locations work. Back-end Private Link keeps cluster traffic to the control plane off the internet.

Watch out. An endpoint without a linked private DNS zone leaves traffic on the public path, or failing. **8.13**

4 Lock the workspace storage account

The workspace's own storage account has its own procedure, Enable firewall support: Premium plan, VNet injection, secure cluster connectivity, a separate /28 subnet for its dfs and blob endpoints, and an access connector created outside the managed resource group. Before I run it, I list the external locations that use the managed-group connector. I rehearse the procedure on a non-production workspace first, since the connector deletion cannot be undone, and the platform lead approves the production run in a change window.

Watch out. Enabling it with the Azure CLI or PowerShell deletes the access connector in the managed resource group, and every storage credential on it stops working.

5 Connect serverless

Same-region ADLS accounts join a network security perimeter in Transition mode, where the perimeter rules are checked first and the account's own firewall still applies, with an inbound rule for the regional AzureDatabricksServerless service tag. Public network access stays on Enabled from selected networks, and Databricks does not recommend Enforced mode here. This path has no charge. Accounts that must be Disabled, and databases, get private endpoint rules in a network connectivity configuration, approved by each resource owner within a two-day service level I agree in advance, and billed per hour. Limits: 10 NCCs and 100 private endpoints per region.

Watch out. Setting an account to Secured by perimeter blocks serverless with PERMISSION_DENIED, and an endpoint nobody approves stays PENDING. **8.11**

6 Reach private databases

A load balancer's backend pool must sit in its own VNet, so it cannot point at a server on premises. For SQL Server on premises I add a forwarder tier, a small scale set of NAT or HAProxy VMs in a VNet with ExpressRoute or VPN, behind an internal standard load balancer and a Private Link service added to the NCC. The NCC rule lists the instance-specific host names jobs use, with no wildcards. The network team owns the forwarders' high availability and patching, and is on call for them. Private network gateway, in Beta since September 2026, may replace this tier later.

Watch out. A federated query over a private link still pulls rows across it, so a filter that does not push down is slow. **6.04**

7 Fix secrets behind a vault firewall

Key Vault-backed scopes work only with the vault access policy model and need Allow trusted Microsoft services to bypass this firewall; a vault in a perimeter set to Secured by perimeter breaks them. If the landing zone forces Azure RBAC on vaults, I move secrets to Unity Catalog secrets, GA since August 2026, and use service credentials for Azure services.

Watch out. A rotated or unreachable Key Vault secret breaks every job that reads it at run time. **7.12**

8 Close public access and egress

One resource at a time, in its change window with the owner present, once both compute planes pass a read and write test, I set public network access to Disabled or keep the perimeter rule as the only way in. The rollback is the recorded setting. A serverless network policy in restricted mode, tried first in dry-run, limits outbound traffic; denials appear in system.access.outbound_network, in Public Preview.

Watch out. Code that ran on a dedicated cluster may call addresses that standard access mode and serverless block. **4.12**

SECURITY IN ONE BREATH

No resource accepts traffic from the open internet. Classic compute reaches each one through a private endpoint in our VNet, serverless through the perimeter or an endpoint we approved, and it can call out only to destinations on our list. Unity Catalog still decides which user may read what.

HOW I'D KNOW IT WORKS

- Every resource in the inventory shows Disabled, or selected networks with only the perimeter and access connector rules, and an approved endpoint where one is needed.

- A test job reads and writes each storage account from a classic cluster and from serverless, and a new cluster installs a PyPI package through the NAT gateway.
- A federated query to the on-premises SQL Server succeeds from a serverless SQL warehouse, with one forwarder stopped.
- A call to an address outside the egress policy is denied and logged.

IF THEY PUSH FURTHER

What if we have more than 100 resources in one region?

Put same-region storage on the perimeter, which does not use private endpoints, and keep endpoints for databases and Disabled accounts. **8.12**

What if Power BI must reach the workspace?

With public access off, the Power BI service needs a virtual network or on-premises data gateway, and Cloud Fetch needs that gateway to reach the workspace storage account. **S.13**

What if a resource owner refuses to approve endpoints?

Then serverless cannot reach it through the NCC. Run that workload on classic compute until they do, and keep it in the inventory as an exception.

S.19 Answer an auditor's question: who read the salary table last quarter, from where, and what did they take out.

Short answer. I'd answer from three system tables. The audit log, in Public Preview, gives each access with the user, the time and the source IP address. Query history gives the statement, the rows read and the rows returned for SQL warehouses and serverless compute. Lineage ties reads to the job, dashboard or notebook that made them. Download events show what left the platform, and I close with the gaps: shared identities and access below Unity Catalog.

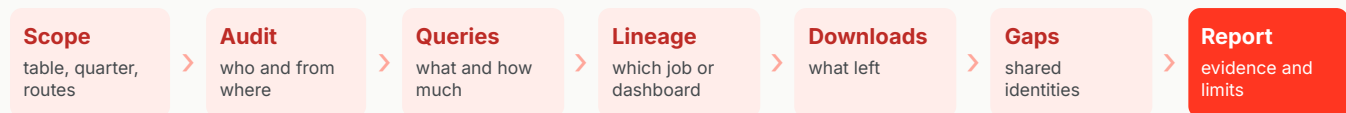
QUESTIONS TO ASK FIRST

- **What exactly counts as a read?** A query, a dashboard refresh, a job, a Power BI report or a download. The auditor may mean all of them, and each leaves a different trace.
- **Is last quarter inside a year?** The audit, query history and lineage system tables keep 365 days, so last quarter is covered without a backup.
- **Which identities reach the table?** Named users are easy. A service principal behind Power BI or a published dashboard hides the people behind it.
- **Can anyone reach the files directly?** If the table's folder is reachable by storage keys or a mount, reads outside Unity Catalog leave no trace in its logs.

WHICH TOOL, AND WHY

- **System tables in SQL.** My choice. Audit, query history and lineage join on user and time, in one notebook I can hand to the auditor.
- **Diagnostic settings to Log Analytics.** Generally available and long-lived if already set up. It does not carry every service, and account-level events are only in the system table.
- **Catalog Explorer lineage.** Good for a picture of who uses the table, not for a list of reads by person and time.

AT A GLANCE



Each table answers one part of the question. Together they answer it, and the gaps go in the report too.

HOW I'D BUILD IT

1 Get the right access

I ask an account admin for SELECT on `system.access` and `system.query`. `statement_text` is masked unless I am an account admin or in the `databricks_pii_access` group, and the audit question needs the full text. I work in a notebook the auditor can read later.

Watch out. A query that scans the whole audit table for every question costs more than it needs to; I filter on `event_date` first. **8.05**

2 Who touched the table, and from where

From `system.access.audit` I take `unityCatalog` events for the table's full name in the quarter: `action_name`, `user_identity.email`, `source_ip_address`, `event_time` and the response status code, including denied attempts.

Watch out. Removing a user from a group does not end their access at once, so check reads for a while after each removal. **7.14**

3 Check what query history covers

`system.query.history` includes SQL warehouses, serverless compute for notebooks and jobs, and pipelines, but not classic compute clusters. From lineage and the audit log I list the classic clusters and users that touched the table. For those reads I can show who and when, but not the statement or the rows, and the report says so.

Watch out. A table read from a classic cluster has no row in query history, so an empty result there is not an empty answer. **7.09**

4 What did they run

`system.query.history` gives each statement against the table, with `executed_by`, `client_application`, `read_rows` and `produced_rows`. `produced_rows` is the closest measure of what a person took out of a query.

5 Which tool made the read

`system.access.table_lineage` rows with the salary table as the source and no target are reads. `entity_type` and `entity_metadata` say whether a job, dashboard, notebook or query made them, and `statement_id` joins back to query history.

Watch out. Lineage records only a subset of reads and writes, so a missing row is not proof that no read happened. **7.09**

6 What left the platform

The audit log records result downloads: downloadQueryResult from the SQL editor, downloadPreviewResults and downloadLargeResults from notebooks. I list each one for the quarter, by user, and match it to the statement that produced it.

7 Name the shared identities

Reads by a service principal are expanded where I can. A dashboard published with shared data permissions runs every viewer's query as the publisher, so the dashboard's own access events name the viewers. A Power BI model without single sign-on shows only its service principal.

Watch out. A shared dashboard shows viewers data they could not query themselves. **6.07**

8 Close with the gaps

I check for mounts, storage keys and external locations over the table's folder. Any path that bypasses Unity Catalog is reported as a gap, together with Azure Storage logs if they were enabled.

Watch out. A revoked user can keep reading from a cluster that still holds a key or a mount. **7.05**

SECURITY IN ONE BREATH

The answer itself is sensitive: it names people and shows salary queries. I keep the notebook and its output in a restricted schema, share it with the auditor only, and keep the audit data inside the platform.

HOW I'D KNOW IT WORKS

- Every user in the audit result has a matching statement in query history, or a documented reason why not, such as classic compute.
- Every download event is matched to a statement and a row count.
- Every service principal read is traced to a job, dashboard or model, with its viewers where known.
- The report lists each route that bypasses Unity Catalog, or states there is none.

IF THEY PUSH FURTHER**What if the auditor asks about two years ago?**

System tables keep a year. Without an earlier copy or diagnostic logs, say so, and start a daily copy of the audit table today. **7.10**

What if they want to know which columns were read?

system.access.column_lineage helps for writes into other tables. For plain reads, statement_text in query history is the evidence. **7.09**

What if salary data was in a share?

Recipient queries appear in the audit log under the recipient's name. What the partner did with it after that is outside our logs. **C.13**

S.20 Move 200 production jobs off personal accounts and tokens onto identities that do not leave.

Short answer. I'd start with an inventory from `system.lakeflow.jobs` and the token report, so every job's owner, run-as identity and token is known. Then I create service principals, one per domain, Databricks-managed unless the jobs must also reach Azure resources as that identity, and grant them Unity Catalog rights through groups before any job moves. Jobs move one at a time first, then a domain at a time: run as the service principal, owned by it, tested on a full run, with the old identity recorded for rollback. Outside callers get scoped OAuth secrets. Last, personal tokens are capped, the exceptions listed, then switched off.

QUESTIONS TO ASK FIRST

- **Which jobs run as people today?** `system.lakeflow.jobs` has `run_as` for every job. `run_as_user_name` is empty for rows written before early December 2025, so I use `coalesce(run_as_user_name, run_as)` or old jobs drop out of the list.
- **Which outside systems call Databricks with a personal token?** Schedulers, CI and scripts. The token report in the account console shows tokens by owner and last use.
- **How should domains be split?** One service principal per domain limits what one leaked secret can open. One for everything is simpler and riskier. A domain whose jobs also read Key Vault or storage as that identity gets an Entra service principal.
- **Who may change jobs once they run as a service principal?** Changing run-as needs CAN MANAGE or IS OWNER on the job and the Service Principal User role on the principal. That role decides who can act as the identity.

WHICH TOOL, AND WHY

- **Databricks-managed service principals per domain.** My choice, and what Databricks recommends for Databricks automation. OAuth secrets can be scoped to the APIs a caller needs and last at most 730 days, up to five per principal. Rotation is ours to run.
- **Microsoft Entra service principals.** Right when the same identity must also reach Azure resources, or the identity team must govern it. They sign in with Entra tokens, not Databricks OAuth secrets, so scoped OAuth secrets are not available for them.
- **Shared service accounts.** A user with a password is still a person to Entra, with sign-in policies and licences, and someone always knows the password.

AT A GLANCE



Find every personal dependency, move it to a service principal, then remove the old door.

HOW I'D BUILD IT

1 Build the inventory

I take the latest row per job from `system.lakeflow.jobs`, where `delete_time` is null, with `coalesce(run_as_user_name, run_as)`, `creator_id` and `tags`, and keep every job whose identity is a person. The account token report adds every token, its owner, scopes and last use. A tracking table holds job, current identity, target principal and status.

Watch out. A personal token behind production jobs stops them on the day the account is disabled. **7.13**

2 Create the principals

I create one Databricks-managed service principal per domain. Where a domain's jobs must also use Azure resources as that identity, I ask the identity team for an Entra service principal and add it to the account instead. I assign each to the workspaces it needs and grant the platform group the Service Principal User role on it. No person keeps a secret for it: the secret goes to CI or to Unity Catalog secrets.

3 Grant first, through groups

Each service principal joins a domain group that holds `USE CATALOG`, `USE SCHEMA`, `SELECT` and `MODIFY` where needed. From lineage and the job code I list what each job reads and writes, and compare that with the principal's effective rights before any job moves.

Watch out. The grants look right, and the job still fails on one table that was granted to the old user directly. **7.02**

4 Move one job, then the rest

I switch one job first and run it end to end. Then, for each job in the domain, someone with `CAN MANAGE` and the Service Principal User role sets Run as to the domain's service principal and makes it the owner. Pipelines built with Lakeflow Spark Declarative Pipelines also run as their owner by default, so they get a run-as service principal too. I trigger one full run and compare row counts with the previous run. The tracking table keeps each job's previous run-as and owner, so a rollback is one setting per job, and I leave the old identity enabled until the domain has run clean for a week.

Watch out. A notebook that worked as a person can fail as a job task when it relies on that person's home folder or settings. **5.15**

5 Fix the compute

Job clusters in dedicated access mode that name the old user are changed to the service principal or to standard access mode. Secrets move to Unity Catalog secrets (GA), in a schema where the domain group holds `READ SECRET`, and can stay backed by Key Vault where the value must live there. Key Vault-backed scopes remain for code that still uses `dbutils.secrets`, if the vault admits trusted services. On classic compute, Unity Catalog secrets need standard access mode on Runtime 17.3 LTS or above, so I test each job's cluster.

Watch out. A rotated Key Vault secret breaks every job that copied it into a config. **7.12**

6 Move the outside callers

Schedulers and scripts on a Databricks-managed principal use OAuth with its client ID and a secret scoped to the APIs they call, such as jobs, so a leaked secret cannot reach the rest. Callers on an Entra principal use Entra tokens. CI uses workload identity federation and holds no Databricks secret. Each caller's personal token is revoked once it runs a week on OAuth.

Watch out. An OAuth secret expires at its set lifetime, so a caller that runs rarely needs its expiry date in the runbook. [7.13](#)

7 Close the door

I set the maximum lifetime for new tokens to 90 days, and any token that must stay is scoped to the APIs it needs. Before switching personal access tokens off, I list every remaining token user from the token report. Each exception, such as a partner integration that still needs a PAT, has an owner, a scope, an expiry and security's approval. The platform lead and security approve the switch, and I make it on a test workspace first, then in production in a quiet window. To roll back, I re-enable PATs: disabling does not delete tokens, and unexpired ones work again. Tokens unused for 90 days are already revoked automatically.

Watch out. Removing someone from an Entra group does not cut their access in the same minute. [7.14](#)

SECURITY IN ONE BREATH

Production jobs run as domain service principals whose rights come from groups. No person holds their secrets, and only the platform group may act as them. Personal tokens are off in production apart from listed exceptions, and the token report is reviewed monthly in the other workspaces.

HOW I'D KNOW IT WORKS

- The latest row for every production job in `system.lakeflow.jobs` shows a service principal as `run_as`.
- The token report lists no active personal token in the production workspace beyond approved exceptions.
- Every job's first run after the move matches the row counts of its last run before it.
- Disabling a test user who used to own jobs causes no failed run in the following week.

A WORKED EXAMPLE

Situation. For example, 200 jobs ran as 23 people, 9 of them no longer in the team, and three schedulers called the Jobs API with personal tokens.

Task. Move every job and caller to service principals within six weeks, without a missed nightly load.

Action. We created five domain service principals, all Databricks-managed except one Entra principal for a domain that reads Key Vault, moved one domain a week with a full test run per job, and switched the schedulers to OAuth. In week six we switched personal tokens off in production.

Result. In this example 4 jobs failed their test run on a direct grant to a former owner and were fixed before cutover. No nightly load was missed.

IF THEY PUSH FURTHER

What if a job needs a user's own permissions?

Then it is not a production job for a shared purpose. Keep it in that user's area, or give the service principal the specific right. [7.01](#)

What if a service principal's secret leaks?

Remove the secret, issue a new one, and review that principal's access in the audit log. One principal per domain limits what the leak reached. **7.11**

What if CI cannot use workload identity federation?

Use OAuth machine-to-machine with a secret stored in the CI system's vault, and put its expiry in the runbook. **9.03**

S.21 Set up dev, test and prod with Declarative Automation Bundles and CI for three teams that share one metastore.

Short answer. I'd give each team a repository with one bundle, built with Declarative Automation Bundles, and three targets. Dev deploys in development mode under each engineer's name. Test and prod deploy only from CI, in production mode, running as a service principal per team and environment. Catalogs are split by environment and team and bound to their workspaces, so a dev deployment cannot write prod even if its config is wrong. CI authenticates with workload identity federation and deploys the same commit to test, then prod.

QUESTIONS TO ASK FIRST

- **Do the three teams share any tables?** Shared tables need one owner. The other teams read them through grants, never through their own bundles.
- **Are there separate workspaces per environment?** Separate workspaces let catalog bindings enforce the line between dev and prod.
- **Which CI system do we use?** GitHub Actions and Azure DevOps both support workload identity federation, so no long-lived Databricks secret sits in CI.
- **Who approves a production release?** A named approver per team, normally the team lead, plus a platform approver when a change touches shared tables or bindings. The approval is set on the prod environment in CI. Nobody deploys prod from a laptop.

WHICH TOOL, AND WHY

- **Declarative Automation Bundles in CI.** My choice. Jobs, pipelines and their settings in YAML in Git, with targets per environment and the CLI to validate and deploy.
- **Terraform for everything.** Strong for workspaces, networking and Unity Catalog objects. Heavier for teams who change job definitions every week.
- **Manual promotion in the UI.** Quick for one job. With three teams it guarantees drift between environments.

AT A GLANCE

Code moves through targets. Data stays fenced by catalog bindings.

HOW I'D BUILD IT

1 Fence the data first

Each team gets dev_, test_ and prod_ catalogs, for example prod_sales, so three teams give nine catalogs, each bound to its environment's workspaces. Dev workspaces get read-only bindings on prod where they need real data.

Watch out. Dev and prod sharing one metastore lets a dev workspace drop a prod table unless catalogs are bound. **1.04**

2 Create the identities

Each team gets a service principal for test and one for prod, six in all, with rights only on its own catalogs. A federation policy on each one trusts only its repository and CI environment.

3 Write the bundle

databricks.yml declares the jobs and pipelines once, with the catalog name as a variable. The dev target uses mode: development, which prefixes resource names and pauses schedules. Test and prod use mode: production, run_as the team's service principal, and git branch main.

Watch out. A target that inherits the dev cluster block ships dev sizes to production. **9.01**

4 Validate on pull request, deploy to test

CI runs databricks bundle validate against test with the test service principal, plus unit tests. CI uses DATABRICKS_AUTH_TYPE set to github-oidc or an Azure DevOps federated service connection. On merge, CI deploys to test, runs the jobs on a test data set and checks row counts and expectations. The same commit is then held for approval.

Watch out. A bundle that validates on a laptop can fail in CI, where variables, auth and the CLI version differ. **9.03**

5 Promote the same commit to prod

After approval, CI deploys the commit to prod with the prod service principal. Jobs deployed by a bundle are UI_LOCKED, and engineers hold no CAN MANAGE on prod jobs, so the bundle is the only way in. A weekly check lists prod jobs whose edit_mode is EDITABLE.

Watch out. Anyone with CAN MANAGE can press Disconnect from source and edit the job by hand; that, not a failed deploy, is how prod drifts. **9.02**

6 Handle what existed before

Jobs created by hand before the bundle are bound to it with databricks bundle deployment bind, or deleted. I never run destroy on a target that shares a workspace with work the bundle did not create.

Watch out. Destroying a bundle can delete a job it adopted but did not create. **9.04**

7 Roll back a bad deploy

A bundle deploy changes job and pipeline definitions, not data. A revert commit goes through the same CI path to test and then prod, with the approver on call for incidents, and the bundle carries any pause state so the revert does not resume schedules. If the release wrote bad data, S.23 is the recovery. I rehearse one revert in test.

Watch out. Reverting the code does not restore tables the release changed.

8 Write down who does what

A short RACI covers each release. Team engineers are responsible for the bundle and tests. The team lead is accountable and approves prod. The platform team is responsible for CI templates, catalogs, bindings, principals and federation policies, and is consulted on every change to them. The owner of a shared table is consulted before a change touches it, and security is informed of new principals and grants. Prod deploys happen in a window agreed in advance.

SECURITY IN ONE BREATH

Prod jobs run as team service principals that CI reaches through federation, with no stored secret. Engineers hold read access in prod and full access in their own dev catalogs. Catalog bindings stop one environment's workspaces from writing another's data.

HOW I'D KNOW IT WORKS

- Every prod job lists the team's prod service principal as run_as and is still UI_LOCKED.
- A dev deployment pointed at a prod catalog fails on the binding, not on review.
- CI holds no Databricks token or client secret for test or prod.
- Test and prod job definitions differ only in the values the targets set.

IF THEY PUSH FURTHER

What if teams want a shared library?

Build it as a versioned wheel in its own repository, and pin the version in each bundle so a change reaches teams when they choose.

What if staging and prod have already drifted?

Export the prod jobs, bind them to the bundle, and let the next deploy set both from Git. **9.06**

What if a hotfix is needed at night?

The same pipeline with a shorter approval. Pausing a job at night goes through the break-glass procedure, and the next deploy must carry the pause in the bundle, or it resumes the schedule. **9.11**

S.22 The Databricks bill rose 60 percent in a quarter. Find out why and bring it down without slowing the business.

Short answer. I'd first confirm the rise in system.billing.usage, priced with list prices, and split it by product, SKU and workspace, then by job and warehouse. The rise is almost always a short list of causes: jobs on all-purpose compute, compute that never stops, warehouses too big or never idle, serverless in performance mode where it is not needed, and new work nobody tagged. I fix the top five by cost, measure each one, and add tags and budgets so the next rise is visible within a week.

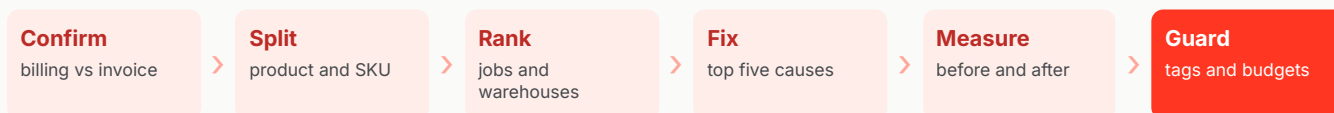
QUESTIONS TO ASK FIRST

- **Is the rise in Databricks units or in Azure VM charges?** System tables show Databricks usage. Classic VM and storage costs are on the Azure bill, so I check both.
- **Did the business grow in the same quarter?** More data, more users or new products explain some of the rise. That part is the cost of growth.
- **Which deadlines must not move?** Reports due at 07:00 keep their speed. Overnight batches with slack can run slower and cheaper.
- **Are resources tagged by team?** Without tags, the split stops at job and warehouse, and someone has to map those to teams.

WHICH TOOL, AND WHY

- **System tables and the usage dashboard.** My choice. system.billing.usage with list_prices answers by product, SKU, job, warehouse and run-as identity.
- **Azure Cost Management.** Shows the invoice and classic VM costs. It does not see inside Databricks to the job or query.
- **Cutting sizes across the board.** Fast to do, and it slows the reports that mattered along with the waste.

AT A GLANCE



Find where the money went, fix the biggest lines, and make the next rise visible early.

HOW I'D BUILD IT

1 Confirm the rise

I sum usage_quantity by month and SKU from system.billing.usage, join list prices on SKU and price period, and compare with the Azure invoice. Rows with negative quantities are corrections, so I always sum, never pick rows.

Watch out. The Databricks usage figure and the Azure bill never match exactly, and the gap is expected. **8.07**

2 Split by product, then rank

I compare quarter on quarter by billing_origin_product, is_serverless and workspace. The account usage dashboard gives the same view for people who prefer a chart. Grouping by usage_metadata.job_id, warehouse_id and identity_metadata.run_as then gives the twenty biggest lines, each compared with the one before.

Watch out. When the bill doubles and nobody can say which jobs caused it, tags were missing long before. **8.01**

3 Fix compute that runs jobs the wrong way

Jobs on all-purpose clusters move to job compute or serverless. Clusters without auto-termination get it, and long-idle clusters are found from their usage with no job runs.

Watch out. Running jobs on an all-purpose cluster to skip start-up time can triple their cost. **4.02**

4 Fix warehouses

Warehouses with high cost and low Peak Queued Queries are made smaller or given fewer clusters. Warehouses that never reach auto stop are traced to the dashboard or alert that keeps them awake, and that alert's owner signs off any new schedule before I change it.

Watch out. A warehouse that never goes idle is often kept awake by one scheduled query. **6.02**

5 Match serverless to deadlines

Nightly serverless jobs with slack turn performance mode off and run in standard mode: same SKU, fewer units, and a startup of 4 to 6 minutes. Time-critical jobs keep performance mode.

Watch out. Paying for speed on a nightly batch that has three hours of slack buys nothing. **8.06**

6 Check Photon and interactive serverless

For the top jobs, I compare cost per run with and without Photon on a test run. Short, bursty notebook work on serverless is compared with a small shared cluster.

Watch out. Photon speeds up many jobs and costs more per hour, so a job that does not speed up costs more. **8.03**

7 Commit only to what remains

Once the fixes have held for a month, I price the stable baseline against an Azure Databricks pre-purchase commitment, and VM reservations or savings plans for classic compute. I commit to part of that baseline, never to the pre-fix figure, and finance approves the amount.

Watch out. A commitment sized on the inflated quarter pays for waste after it is removed.

8

Guard the next quarter

Every cluster, job and warehouse carries a team tag, with serverless notebooks and jobs tagged through serverless usage policies, in Public Preview. Budgets scoped to each team tag, not to a workspace, send alerts at fixed thresholds, up to 24 hours late, and a weekly query flags any job whose cost rose by half. A named FinOps owner, a platform lead working with a finance partner, runs the weekly flag and a monthly review and owns the budget alerts.

Watch out. One team using most of the spend is found at month end unless someone watches weekly. **8.04**

SECURITY IN ONE BREATH

Billing tables show who ran what, so access goes to the platform team and finance through a group. Team leads see their own spend through a dashboard filtered to their tags.

HOW I'D KNOW IT WORKS

- The quarter's rise is explained line by line, with the part due to growth named separately.
- Each fix has a before and after cost per run from `system.billing.usage`.
- No job runs on all-purpose compute, and every cluster has auto-termination.
- Report deadlines are met in the month after the changes.

A WORKED EXAMPLE

Situation. For example, a platform's monthly Databricks cost rose from 40,000 to 64,000 USD at list prices over one quarter.

Task. Explain the rise and remove the waste without delaying the 07:00 reports.

Action. The split showed three causes: 14 jobs moved to an all-purpose cluster during an incident and never moved back, an X-Large warehouse kept awake by a five-minute alert, and new serverless jobs in performance mode overnight. We moved the jobs back, set the alert to hourly after its owner agreed, and turned off performance mode on nightly jobs.

Result. In this example the three causes were worth about 9,000, 5,000 and 4,000 USD a month in that order, so monthly cost fell by 18,000 to about 46,000 USD. The remaining 6,000 matched a new product line, and the 07:00 reports were unchanged.

IF THEY PUSH FURTHER

What if serverless costs more than the classic clusters we had?

Compare cost per run, not cost per hour. Short interactive work often costs more on serverless; batch jobs often cost less. **8.02**

What if the monitoring itself is expensive?

Queries over system tables that scan all of history every few minutes add up. Filter on dates and run them hourly or daily. **8.05**

What if a cluster ran for a week unnoticed?

Alert on clusters running longer than a set time with no job attached, from usage and compute events, every hour. **4.06**

COMPARE

STORAGE

INGESTION

DELTA

COMPUTE

JOBS

SQL

SECURITY

PLATFORM

CI/CD

AI

SCENARIOS

S.23 A Monday release wrote bad data into six Gold tables and dropped one. Recover by lunchtime.

Short answer. I'd stop the damage first: pause the jobs from the release through the break-glass procedure and roll the code back. Then I bring the dropped table back with UNDROP, and restore the plain Delta tables to the version before the release with RESTORE, after checking each one in DESCRIBE HISTORY. Materialized views and tables with row filters or masks cannot be restored: views are fixed by correcting the code and refreshing, filtered tables are rebuilt from Silver. Downstream streams and reports are handled next, and last I rerun the fixed jobs so Monday's good data is not lost.

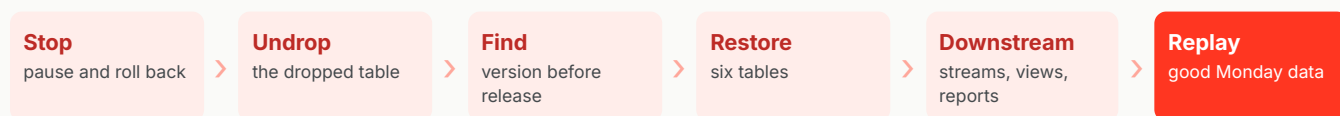
QUESTIONS TO ASK FIRST

- **When did the release first write?** The last version before that commit, per table, is the restore point.
- **Did anything else write to those tables since?** Restoring removes every write after the restore point, good or bad. Those writes have to be replayed.
- **Who reads these tables downstream?** Streams, Change Data Feed readers, materialized views and Power BI each react differently to a restore.
- **Which of the six are materialized views or filtered tables?** Time travel and RESTORE do not work on materialized views, nor on tables with table-level row filters or column masks; with ABAC policies time travel is only in Beta. Those are refreshed or rebuilt, not restored.

WHICH TOOL, AND WHY

- **RESTORE and UNDROP.** My choice. Both work on the table in place, take minutes, and keep the history of what happened.
- **Rebuild from Silver.** Always possible if Silver is intact. Slower for large tables, and the only route for filtered or masked tables and materialized views.
- **Clone of an old version.** A shallow clone at the target version is a cheap way to inspect the old data beside the bad. It reads the source's old files, so a VACUUM on the source can break it; a deep clone keeps its own files and costs a full copy.

AT A GLANCE



Stop the writer, bring each table back to the version before the release, then tell everything downstream.

HOW I'D BUILD IT

1 Name roles and start the clock

The incident lead decides and approves, a restorer runs the reviewed notebook, and a communicator updates the business every hour. Timeboxes: writers stopped within 15 minutes, restore points agreed by 10:00, a restore or rebuild decision at 10:30, and anything not restored by 12:00 moves to rebuild from Silver. The incident lead approves each RESTORE on a prod table, a second person reads its DESCRIBE HISTORY first, and the data owner is told.

Watch out. Without a named lead, two people can run RESTORE on the same table. **9.11**

2 Stop the damage

On-call holds only CAN MANAGE RUN, which can cancel runs but not pause a schedule, and the jobs are UI_LOCKED by the bundle. So I cancel the running runs, then use the break-glass route: time-limited CAN MANAGE through a group, or the runbook job that pauses schedules as a service principal, both audited. The revert commit sets the schedules to PAUSED in the bundle, so the next deploy does not resume them. Business users are told which reports are wrong and when they will be right.

Watch out. A deploy of the old commit resumes every schedule that was paused by hand. [9.11](#)

3 Bring back the dropped table

SHOW TABLES DROPPED IN the schema lists it with its table ID. UNDROP TABLE brings it back by name, or WITH ID if a new table already took the name. Primary and foreign key constraints are not recovered, so I recreate them with `ALTER TABLE ... ADD CONSTRAINT`.

Watch out. Recovery is possible only within the retention window, 7 days by default, and not for every table type. [1.02](#)

4 Find each restore point

DESCRIBE HISTORY on each plain table shows the version, time, operation and job that wrote it. The last version before the release's first write is the target, and I note it per table.

Watch out. Time travel needs the old files, so a short retention or an early VACUUM leaves nothing to restore. [3.04](#)

5 Check before restoring

A shallow clone at the target version lets me compare row counts and a few totals with the bad version, and I drop it afterwards, or make it a deep clone if it must outlive the source's retention. On a table under ABAC policies the clone runs as an identity exempt from the policy (7.17). If the old version is right, I restore.

6 Restore, refresh or rebuild

Plain Delta tables get RESTORE TABLE ... TO VERSION AS OF the target version, one at a time, with MODIFY on each. Gold materialized views have no time travel, so I fix their code and run a full refresh once the sources are right. Tables with table-level row filters or column masks are rebuilt from Silver, with the filter and mask functions still attached. A table under ABAC policies is restored only by an exempt identity, and time travel on it is in Beta, so the recovery service principal sits in the exempt group (7.17).

Watch out. A stream that reads a restored table fails on the restore commit, because the commit rewrites files. [3.11](#)

7 Handle downstream readers

Each stream that reads these tables has failed on the restore commit. For each one I rebuild or restore its target to the same point and start it on a new checkpoint at the restore version, since its old checkpoint no longer matches its target, and I never delete a checkpoint on its own (2.06). Change Data Feed consumers get a note that the restore shows as changes. Materialized views and Power BI models refresh once.

Watch out. A Change Data Feed reader can miss or double changes around a restore. **3.10**

8 Replay Monday's good data

The fixed jobs rerun for Monday's window from Silver. I compare row counts and key totals with the previous Monday and a known report.

Watch out. A materialized view that stopped refreshing keeps showing the bad data after the source is fixed. **6.05**

SECURITY IN ONE BREATH

Restore needs MODIFY on each table, and undrop needs ownership or MANAGE plus CREATE TABLE, so the recovery runs as the platform service principal through a reviewed notebook, approved by the incident lead. The break-glass grant expires the same day. Every pause, RESTORE and UNDROP stays in the table history and the audit log.

HOW I'D KNOW IT WORKS

- The dropped table is back, with its grants, row count and constraints.
- Each restored table shows a RESTORE to the noted version, and each view or filtered table a refresh or rebuild after the fix.
- Monday's totals match the source after the replay.
- Every downstream stream, view and model shows correct data by lunchtime.

IF THEY PUSH FURTHER

What if the bad data was found two weeks later?

Time travel and RESTORE stop at the deleted file retention, 7 days by default, and VACUUM may have removed the files. Rebuild from Silver, and raise retention for Gold if late discovery is common. **3.04**

What if Silver was damaged too?

Restore Silver first, then rebuild Gold from it. Keeping Bronze as append-only raw data is what makes this possible.

How do we stop a release doing this again?

Deploy to test with real-sized data and checks on row counts before prod, and keep drops out of release code. **9.06**

S.24 Plan the handover of a Databricks platform whose only engineer leaves in four weeks. Their account owns the jobs, tokens and dashboards.

Short answer. I'd spend the first week on an inventory of everything tied to that account: jobs, pipelines, tokens, dashboards, warehouses, Unity Catalog objects, Azure resources and CI. Then production moves to service principals, Unity Catalog objects move to a group, and jobs and dashboards are re-owned. Alerts go to a group, and the engineer writes the runbook while a colleague shadows them. Two weeks before the last day, I remove their access and watch what breaks while they can still help.

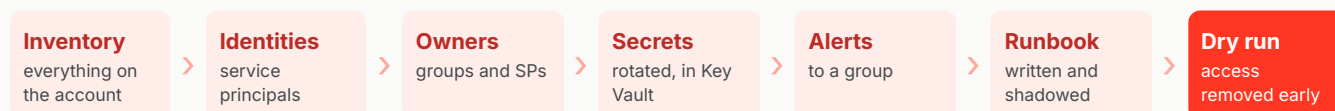
QUESTIONS TO ASK FIRST

- **Who owns the platform from next month?** A named lead and an Entra group. Every owner field I change points at one of them.
- **Is everything deployed from Git?** If jobs were built in the UI, binding them to a bundle in Git, with Declarative Automation Bundles, is the first task, because it is the only full record we will have.
- **Which Azure resources did the engineer create?** Access connectors, Key Vaults, storage accounts and app registrations may have them as the only owner.
- **Is the engineer an account admin?** Then someone else must be one before they leave, or nobody can change account-level settings.

WHICH TOOL, AND WHY

- **System tables, the token report and the bundle.** My choice. They list ownership and run-as for every job, every token and every deployed resource, and the bundle puts the rest in Git.
- **Interviews and a wiki page.** Necessary for the reasons behind decisions, and never complete on their own.
- **Disable the account on the last day and fix what breaks.** Quick to plan and painful to live through, with nobody left to ask.

AT A GLANCE



In four weeks I find everything, move it to identities that stay, write it down, then prove it runs without the person.

HOW I'D BUILD IT

1 Inventory what hangs on the account

Before anything else, two other people become account admins and workspace admins and get Owner or Contributor on the Azure resource group, so the plan does not depend on the leaver. From `system.lakeflow.jobs` I list every job with the engineer as creator, owner or run-as, using `coalesce(run_as_user_name, run_as)` because the name column is empty for rows written before early December 2025. The token report lists their tokens. In SQL I list catalogs, schemas, external locations and storage credentials they own. I add dashboards, warehouses, pipelines, Git credentials and Azure resources to a handover table with columns for item, new owner and status.

Watch out. A personal token behind production jobs stops them on the day the account is disabled. **7.13**

2 Create identities that stay

One service principal per domain, Databricks-managed unless its jobs need Azure access as that identity (S.20), with rights through groups and no secret held by a person. The platform group gets the Service Principal User role on each, so its members can set run-as.

3 Re-own Unity Catalog objects

Catalogs, schemas, external locations and storage credentials move to the platform group with ALTER ... OWNER TO. Unity Catalog objects can be owned by a group, so no single person holds them again.

Watch out. Owners have rights nobody granted explicitly, so removing an owner can change access in ways grants do not show. **7.01**

4 Move jobs, pipelines and dashboards

A workspace admin sets each job's owner and run-as to a service principal, since jobs cannot be owned by a group. Pipelines get a run-as service principal. Dashboards published with shared data permissions are republished as a service principal.

Watch out. A bundle deployed to prod from a laptop runs every job as the person who deployed it, unless it sets run_as. **9.05**

5 Re-own Azure and Entra objects

Unity Catalog does not hold everything. I list what the engineer owns in Azure and Entra: role assignments with az role assignment list, access connectors, Key Vaults, storage accounts, app registrations and their secrets, Entra groups, and Git or CI service connections. Each gets two owners from the platform group, and any Owner role held by a person moves to a group. An Entra administrator takes part, because I cannot change those objects myself.

Watch out. An access connector or app registration with a single owner is stuck once that person is disabled. **8.15**

6 Rotate secrets

Every secret the engineer has seen is rotated, including service principal secrets. Where code can use them, secrets move to Unity Catalog secrets, with READ SECRET granted to the domain groups. The rest stay in Key Vault, whose access policies or role assignments list the platform group, not the person. Key Vault-backed secret scopes are checked the same way.

Watch out. A secret that once appeared in notebook output or Git history must be rotated as well as moved. **7.11**

7 Send alerts to a group

Job failure notifications and SQL alerts go to a group mailbox and a channel. A single job health dashboard over job_run_timeline replaces the engineer's habit of checking runs each morning.

Watch out. A failure that reaches only the leaver's inbox is found by a business user hours later. **9.07**

8 Write, shadow, then run a dry week

The engineer writes a runbook for daily checks, reruns, backfills, releases and cost reviews, and a colleague follows it while they watch. Two weeks before the last day I remove their workspace and account roles but keep the account. Anything that breaks is fixed while they can still explain it.

Watch out. Logs that lived only on terminated clusters leave no trail for the next person. [9.10](#)

SECURITY IN ONE BREATH

No production job, pipeline, dashboard or Unity Catalog object stays on a personal account. Service principal secrets live in Key Vault with rotation dates in the runbook. At least two people hold account admin, and an access review on the platform groups confirms who keeps access on the last day.

HOW I'D KNOW IT WORKS

- The handover table shows a new owner and a Done status for every row.
- No job, pipeline, dashboard or Unity Catalog object lists the engineer as owner or run-as.
- During the dry week, every scheduled run succeeds without the engineer's access.
- A forced test failure reaches the group mailbox and the channel.

A WORKED EXAMPLE

Situation. For example, one engineer built a platform of 3 workspaces with 85 jobs. The inventory found 85 jobs running as them, 6 tokens used by schedulers, 11 catalogs and external locations they owned, and 9 dashboards published with their credentials.

Task. Move everything off the personal account within four weeks and show the platform runs without them.

Action. In week 1 we made two colleagues account admins, built the handover table and bound the jobs to a bundle in Git. In week 2 we created four service principals and moved the jobs. In week 3 we re-owned catalogs and Azure objects, republished dashboards and rotated secrets. In week 4 we removed their roles for the dry run.

Result. In this example the dry run surfaced two failures, a scheduler still on a personal token and an external location owned by the engineer. Both were fixed the same day. A Slack webhook that only the engineer knew about expired five weeks later and silenced alerts for a day, so the inventory still had one gap. Effort was about 30 person-days over four weeks, a third of it the engineer's own time.

IF THEY PUSH FURTHER

What if the engineer has already left?

A workspace admin re-owns jobs and dashboards, and a metastore admin re-owns Unity Catalog objects. Start with the jobs behind the most important reports. [7.13](#)

What if nobody knows why a job exists?

Check its last runs, what it writes and who reads that in lineage. If nothing reads its output for a month, pause it, and delete it a month later. [9.09](#)

What if old run history is needed later?

Job run history in the UI covers 60 days. `system.lakeflow.job_run_timeline` keeps a year, so export what the runbook needs. [5.16](#)

S.25 Serve a churn model and customer features to a customer-facing app that needs answers in under 50 milliseconds.

Short answer. I'd register the model in Unity Catalog, publish the customer features from a Gold feature table to an online feature store backed by Lakebase, and serve the model on a Model Serving endpoint with route optimization on and scale to zero off. The endpoint looks up the features itself, so the app sends only a customer ID. Screens that can show yesterday's score read it from a Lakebase synced table instead, which is cheaper. I prove the 50 milliseconds at p99 from the app's own region before launch.

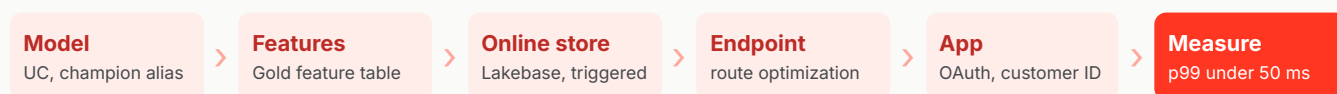
QUESTIONS TO ASK FIRST

- **Must the score reflect what the customer did in this session?** If a nightly score is good enough, batch scoring into a synced table answers in a few milliseconds with no endpoint. Only in-session signals need real-time inference.
- **Is 50 milliseconds the median or the p99, and measured where?** Platform overhead, the feature lookup, the model and the network path from the app all count against it, so I measure each rather than assume a split.
- **What is the peak request rate?** It sets the provisioned concurrency. Without route optimization, endpoints in a workspace are limited to 200 queries per second in total.
- **How fresh must the features be?** Hourly features publish in triggered mode after each feature job. Features that change within minutes need continuous mode, which costs more.

WHICH TOOL, AND WHY

- **Model Serving with online feature lookup.** My choice. An endpoint with route optimization and provisioned concurrency, and features read from a Lakebase online store by key at request time.
- **Nightly scores in a Lakebase synced table.** Fastest and cheapest for scores that may be a day old: the app runs one Postgres lookup by customer ID.
- **SQL warehouse lookup.** Built for scans, not for thousands of single-row reads a second; a second per answer is normal.
- **Model inside the app.** Fast, but the app team then owns model versions, feature code and lineage outside Unity Catalog.

AT A GLANCE



The app sends a key, the endpoint fetches the features and scores, and every piece stays in Unity Catalog.

HOW I'D BUILD IT

1 Register the model and build the feature table

The churn model is trained with features from the feature table through the feature engineering client, logged with MLflow and registered in Unity Catalog, and the approved version gets the champion alias. A job writes customer_features with customer_id as a non-null primary key and change data feed on, from the same Silver tables the training set used. Features that depend on the request, such as minutes since login, are feature functions in the FeatureSpec, not columns.

Watch out. Rows with a null key never reach the online table, and nothing fails to tell you. **10.11**

2 Publish to the online store

I create an online store on Lakebase and publish the table with `publish_table` in TRIGGERED mode, run as the last task of the feature job. CONTINUOUS mode is for features that must change within minutes.

Watch out. An online store does not scale to zero; it bills all month, so size it from the read rate. **10.05**

3 Create the endpoint for latency

I create the endpoint with route optimization on, which can only be set at creation, scale to zero off and a minimum provisioned concurrency from the peak. Concurrency is roughly queries per second times model run time, so 400 requests a second at 30 milliseconds is 12, in multiples of 4, with a maximum above it for headroom. The served entity is the version behind the champion alias. A small model, such as gradient-boosted trees, keeps model time to a few milliseconds.

Watch out. Scale to zero means a cold start on the first request after a quiet spell, and capacity is not guaranteed. **10.03**

4 Connect the app

The app's backend calls the endpoint's route optimization URL with an OAuth token from a service principal that holds CAN QUERY on the endpoint, and caches the token until it expires. Its OAuth secret is scoped to the APIs it needs. The app runs in the workspace's region. I switch on AI Gateway rate limits and usage tracking, both GA for custom model endpoints, so one client cannot take all the concurrency and spend is attributed, and I check on staging that they work with route optimization.

Watch out. Endpoints with route optimization reject personal access tokens. **10.08**

5 Add the fallback

A nightly batch job scores every customer and writes the result to Gold, synced to a Lakebase table in triggered mode. If the endpoint call takes longer than the app's budget, the app shows yesterday's score from that table.

6 Load test, then watch

Before launch I replay twice the expected peak from the app's region and read p50 and p99 from the app side and from the endpoint metrics. Inference tables, on for the endpoint, record requests and responses so a drop in answer quality shows up later. After launch, an alert fires when p99 passes 40 milliseconds or the online table falls behind the feature job.

Watch out. An endpoint that meets the target at noon can miss it at the evening peak if concurrency was sized on the average. **10.09**

7 Release new versions safely

A new version is served beside the champion with a small traffic share, compared on live requests, then moved to 100 percent and given the alias. Traffic splitting is documented for custom model endpoints, but the docs do not say it works when route optimization is on, so test it on staging first; if it fails, serve the new version on a second endpoint with route optimization and split in the app. The endpoint serves a version number, so moving the alias alone changes nothing that is served.

Watch out. Model, alias and endpoint can disagree about which version is in production. **10.02**

8

Price it and name the owners

Monthly cost is the provisioned concurrency minimum, billed while the endpoint is up, plus the online store's capacity units, billed all month, plus the feature job, the nightly batch and the fallback table. I put the figures from the pricing pages beside the request volume. The data team owns the feature job and its freshness alert, the ML team the model and alias, the platform team the endpoint, its concurrency and tags, and the app team the latency budget and the fallback.

Watch out. An endpoint nobody owns stays at peak size long after the campaign that needed it.

SECURITY IN ONE BREATH

The app holds no Databricks token of a person: a service principal with CAN QUERY on one endpoint and a secret scoped to serving. Feature tables expose a customer ID and derived signals, not names or contact details. Unity Catalog grants decide who can read the feature table, the online table and the model, and the endpoint's usage is attributed to its tags.

HOW I'D KNOW IT WORKS

- At twice the expected peak, p99 measured in the app stays under 50 milliseconds.
- The endpoint's served version matches the version behind the champion alias.
- The online table's latest update follows each feature job within the agreed minutes.
- With the endpoint stopped in a test, the app shows yesterday's score and no error.

A WORKED EXAMPLE

Situation. For example, a telecom's customer app chose a retention offer from a churn score for 3 million customers, with a peak of 400 requests a second. The first build queried a SQL warehouse and took about 900 milliseconds per answer.

Task. Bring answers under 50 milliseconds at p99 without the app team handling features.

Action. We published 40 customer features to a Lakebase online store in triggered mode after the hourly feature job, created an endpoint with route optimization, scale to zero off and a minimum concurrency of 12 (400 requests a second at about 30 milliseconds in the model), and added a nightly score in a synced table as the fallback.

Result. In this example p99 in the app was about 35 milliseconds at peak, and the fallback served 0.1 percent of requests during one endpoint update.

IF THEY PUSH FURTHER

What if traffic is ten requests an hour?

Then an endpoint with provisioned concurrency costs more than it earns. Score in batch and serve from a synced table. **10.03**

What if the app also needs to write data back?

Lakebase is Postgres: the app writes to its own tables in the same database. Synced tables stay read-only, owned by the sync. **C.22**

What if the model needs a feature only the request knows?

Pass it in the request and define the logic as a feature function in the FeatureSpec, so training and serving compute it the same way. **10.11**

S.26 Design how the platform recovers if West Europe is lost for two days: Gold back within four hours, everything else within a day.

Short answer. I'd run active-passive into North Europe, with a second workspace and metastore that exist today, deployed by the same Terraform and bundles. Gold is shared from the primary metastore to the secondary over OpenSharing, and an hourly job copies it into local managed tables there, so Gold has an RPO of an hour once the first full copy has run. Silver and Bronze copy nightly. Materialized views, streaming tables and tables with table-level filters or masks do not clone, so their pipelines rebuild them after failover. Geo-redundant storage is not the plan, and a test twice a year proves the four hours.

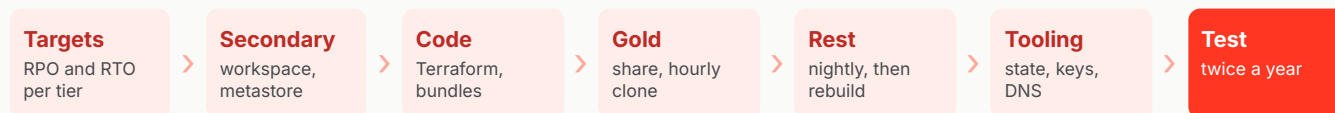
QUESTIONS TO ASK FIRST

- **Which tables are Gold, and how much data may Gold lose?** Four hours is the RTO. The RPO, often an hour, sets how often Gold copies, and the list sets what the copy costs.
- **Where do the sources live?** If the source systems are in West Europe too, the secondary region has nothing new to load until they return. Recovery then means serving the last good Gold.
- **Can we use managed disaster recovery?** Only if the account team accepts us: it is gated. It needs Premium workspaces with the Mission Critical add-on and serverless compute in both regions.
- **Who declares the disaster, and who fails back?** A named role with a deputy, reachable without the primary region. Without one, four hours go on deciding.

WHICH TOOL, AND WHY

- **Warm standby with OpenSharing and deep clone.** My choice. Generally available parts, an RPO set by the copy schedule, and a secondary that can serve read-only work every day.
- **Managed disaster recovery.** Gated: apply through the account team. It replicates managed tables, grants and workspace assets and gives a stable URL, but not materialized views, streaming tables, pipelines, secrets, models, serving endpoints, vector indexes, shares, published dashboards, tables with row filters, masks or ABAC policies, or managed-table writes from outside engines.
- **Geo-redundant storage and rebuild.** Microsoft says not to rely on it for Databricks: it copies files, not the workspace, the metastore, jobs or grants.
- **Active-active.** Both regions write; twice the compute and conflicts to resolve. More than a four-hour RTO needs.

AT A GLANCE



Everything that defines the platform is code, Gold is copied hourly, and the rest can be a day behind.

HOW I'D BUILD IT

1 Set the targets per tier

With the business I agree two tiers: Gold, RPO 1 hour and RTO 4 hours; everything else, RPO and RTO 24 hours. Each Gold table, report and job is listed under its tier, with an owner.

Watch out. Geo-redundant storage brings back files, not the workspace, the metastore, the jobs or the grants. **8.08**

2 Build the secondary now

Terraform deploys a workspace in North Europe with its own metastore, storage, access connector, NCC and perimeter rules. Identities come from the same Entra tenant through the account. I request vCPU quota there today and give its VNet a NAT gateway. Idle, the secondary costs its storage, NAT gateway and private endpoints; compute runs only on the copy schedule and at failover.

Watch out. A new region has its own vCPU quota, and clusters fail to start at the worst moment. **8.14**

3 Deploy code to both regions

Every bundle gets a dr target that deploys the same jobs to the secondary with schedules paused. CI deploys to dr on each release, so the code there is never older than prod.

Watch out. A dr target that inherits prod values can point the secondary at West Europe storage. **9.01**

4 Copy Gold every hour

The primary metastore shares the Gold schemas to the secondary metastore over Databricks-to-Databricks OpenSharing, WITH HISTORY and with change data feed on, which tables with deletion vectors or column mapping need anyway. Tables with table-level row filters or masks cannot be shared, and for tables under ABAC policies the share owner and the copy identity must be exempt (7.17). An hourly job in North Europe copies each shared table into a local managed table in the secondary metastore, by DEEP CLONE or by a MERGE from the change feed. The docs do not say that deep clone from a shared table works, so prove one of the two in week one. The first copy moves every table across regions: I run and time it during the build, outside the RTO clock, and count the RPO from the moment it finishes. After that only changes move. Cross-region reads are billed by Azure, roughly GB copied x the egress rate, on the first copy and on every change, plus the clone compute and a second copy of Gold in storage. Grants, tags and ABAC policies come from Terraform, since a clone does not carry them.

Watch out. Materialized views and streaming tables cannot be cloned, and nothing the failover needs may exist only in the primary metastore. **C.18**

5 Copy the rest, rebuild what does not copy

Silver is copied the same way nightly. Landing containers with raw files use storage replication, which is fine for files Databricks does not manage. Delta streams in the secondary start from new checkpoints with startingVersion, because checkpoints do not travel. Auto Loader streams also start on a new checkpoint, which would ingest every replicated file again, so I start them with modifiedAfter set to the last replicated ingestion time and load the gap by hand, and I prove this on game day. After failover, pipelines run a full refresh of their materialized views and streaming tables from the copied tables. Tables with table-level filters or masks are rebuilt from copied Silver, and tables under ABAC policies are copied by an exempt identity, with the policies and filter functions from Terraform.

Watch out. A stream restarted without its checkpoint reprocesses everything it can see. **2.06**

6 Make the tooling survive the region

Failover needs more than Databricks. Terraform state sits in a storage account that survives a West Europe loss, and CI runners are not all in West Europe. Key Vault values are copied to a vault in North Europe by CI. Databricks and Unity Catalog secrets, which managed DR does not replicate either, are created in both workspaces by the same code. Entra is global, but I test conditional access and group sync from the secondary. The private DNS zones, and the records that point apps and Power BI at a workspace, are in the runbook with the person who changes them.

Watch out. A runbook that starts with run Terraform fails if the state sat in the lost region.

7 Write and own the failover runbook

The runbook names who declares, then lists the steps: stop the clone jobs, unpause the dr jobs, point Power BI and apps at the secondary warehouse, tell users. Two account admins who can sign in without the primary region are named in it.

Watch out. If the only account admin is unreachable, nobody can change account settings during the outage. **8.15**

8 Fail back, and test twice a year

Failback reverses the share: once West Europe is healthy, the secondary metastore shares to a recipient in the primary metastore, the same copy job moves the changes back, and counts are compared before schedules and apps move. The first pass is a full copy again if the primary lost data, and it includes the writes made in North Europe during the outage. A game day fails Gold over on a weekend, times each step against the four hours, records the real RPO from the secondary side and then fails back.

Watch out. A plan nobody has run takes twice as long as written. **8.09**

SECURITY IN ONE BREATH

The share goes to one recipient, our own secondary metastore. The secondary has the same private networking, the same groups and the same grants from Terraform, so failover does not open anything. The clone jobs run as a service principal, and access to the secondary is reviewed with the primary.

HOW I'D KNOW IT WORKS

- Each copy job writes the source version and time it copied to a control table in North Europe, and an alert fires from that table when any Gold table is more than 90 minutes behind, with no read of the primary.
- A grants comparison over information_schema shows the same privileges on Gold in both regions.
- The last game day restored every Gold report in under four hours, with the measured RPO recorded.
- Every release deploys to the dr target, with schedules paused, in the same CI run.

IF THEY PUSH FURTHER

What if the account team enables managed disaster recovery?

Use it for managed tables, grants and workspace assets, and keep the rebuild and tooling steps for what it excludes. It bills both workspaces at the Mission Critical rate. **C.18**

What if four hours becomes one?

Clone Gold every 15 minutes, keep a small warehouse running in the secondary and rehearse quarterly. Below that, active-active is the answer, with its cost. **8.09**

What if only one storage account in West Europe fails?

That is not a region outage. Recover the tables in place with time travel or RESTORE, except materialized views and filtered or masked tables, which are rebuilt. Tables under ABAC policies need an exempt identity (7.17). **C.11**

Glossary.

The terms and abbreviations the book uses, and the products that changed name. Each pattern spells out what it needs; this page is for the rest.

ABAC. Attribute-based access control: policies apply filters, masks or grants where a governed tag matches.

Access connector. An Azure managed identity that Unity Catalog uses to reach ADLS without a secret.

AI Gateway. Rate limits, usage tracking, guardrails and payload logging for serving endpoints.

AI Search. Formerly Vector Search. Keeps embeddings of a Delta table in sync for similarity search.

Auto compaction. Merges small files after a write. Optimized writes shuffle first for fewer, larger files.

Auto Loader. The cloudFiles source. Finds new files in cloud storage and tracks them in a checkpoint.

Break-glass account. Emergency identity or group with time-limited high rights. Every use is audited.

Change data feed (CDF). Row-level changes of a Delta table. Automatic CDF needs Runtime 19 and row tracking.

Compliance security profile. Hardened compute and stricter monitoring and encryption. HIPAA and PCI-DSS need it; extra cost.

Data classification. An LLM scan that tags Unity Catalog columns holding sensitive data, for ABAC to act on.

Data profiling. Formerly Lakehouse Monitoring. Column statistics, drift and custom metrics over time.

DBU. Databricks unit: the billed unit of processing, per second, at a rate set by compute type.

Declarative Automation Bundles. Formerly Databricks Asset Bundles. YAML that defines and deploys resources per target.

Deletion vectors. Mark rows deleted without rewriting the Parquet file. REORG ... PURGE removes them physically.

External location. A storage path plus the credential that may access it; Unity Catalog checks grants on it.

EXTERNAL USE SCHEMA. Lets a non-Databricks engine read a schema, once the metastore allows external access.

Genie Agents. Formerly Genie spaces. Natural-language questions over tables and metric views you curate.

Governed tag. A tag with centrally set keys, values and applicators. ABAC policies match on it.

Hive metastore (HMS). The legacy per-workspace metastore (hive_metastore catalog). Unity Catalog replaces it.

Inference table. A Delta table where AI Gateway logs a serving endpoint's requests and responses.

Lakebase. Managed PostgreSQL for low-latency reads and writes. Synced tables copy Delta tables in.

Lakeflow Connect. Managed connectors from databases and SaaS applications into Unity Catalog tables.

Lakeflow Jobs. Formerly Workflows. Tasks with dependencies, triggers, retries and notifications.

Lakeflow Spark Declarative Pipelines. Formerly DLT. Declare streaming tables and materialized views; it plans the rest.

Liquid clustering. Clustering keys you can change without a rewrite. CLUSTER BY AUTO picks them.

LTS. Long Term Support runtime. Runtime 18 LTS (Spark 4.1) is current in October 2026.

Metric view. Measures and dimensions defined once, for dashboards, Genie and SQL.

NCC. Network connectivity configuration: private endpoint rules for serverless compute.

Network policy (serverless). Account rules limiting where serverless compute may reach on the internet; dry-run mode.

NSP. Network security perimeter. A rule for the AzureDatabricksServerless tag admits serverless.

OAuth M2M. Short-lived tokens for service principals from a client ID and secret, not personal tokens.

OpenSharing. Formerly Delta Sharing. Shares tables across metastores and organisations without copying.

PAT. Personal access token. Acts as its creator, with their rights, until it expires or is revoked.

Photon. The vectorised engine. Operators it cannot run, such as Python UDFs, fall back to Spark.

Predictive Optimization. Runs OPTIMIZE, VACUUM and ANALYZE on managed tables when it judges they pay off.

Provisioned concurrency. Requests a custom model endpoint serves at once; set minimum and maximum in multiples of 4.

Route optimization. Endpoint setting for lower latency and higher QPS. Set at creation; needs OAuth.

RPO and RTO. Recovery point objective: data you can lose. Recovery time objective: time to be back.

SCC. Secure cluster connectivity. Classic nodes have no public IPs and call out to the control plane.

Serverless usage policies. Formerly serverless budget policies, Public Preview. Tags that attribute serverless spend.

Service principal. A non-human identity for jobs, pipelines and CI.

Streaming table. A Delta table updated incrementally from streams or change feeds, including AUTO CDC targets.

UI_LOCKED and EDITABLE. Job edit_mode values. A bundle-deployed job is UI_LOCKED: read-only in the UI.

Unity Catalog. The governance layer: one metastore per region for catalogs, tables, volumes, models and grants.

VNet injection. Classic compute in your own virtual network: you control routing, DNS and egress.

Workload identity federation. An outside workload, such as CI, swaps its own token for a short-lived Databricks one.

From seven books to one.

The seven Azure Databricks Engineering Patterns books held 350 scenarios, 50 each. The Azure Databricks Practitioner's Handbook 4.0 keeps 104 of them, rewrites each one, adds 30 new patterns, and regroups all 134 by layer into ten parts.

SEVEN BOOKS · 350 SCENARIOS

AZURE DATABRICKS PRACTITIONER'S HANDBOOK 4.0 · 134 PATTERNS

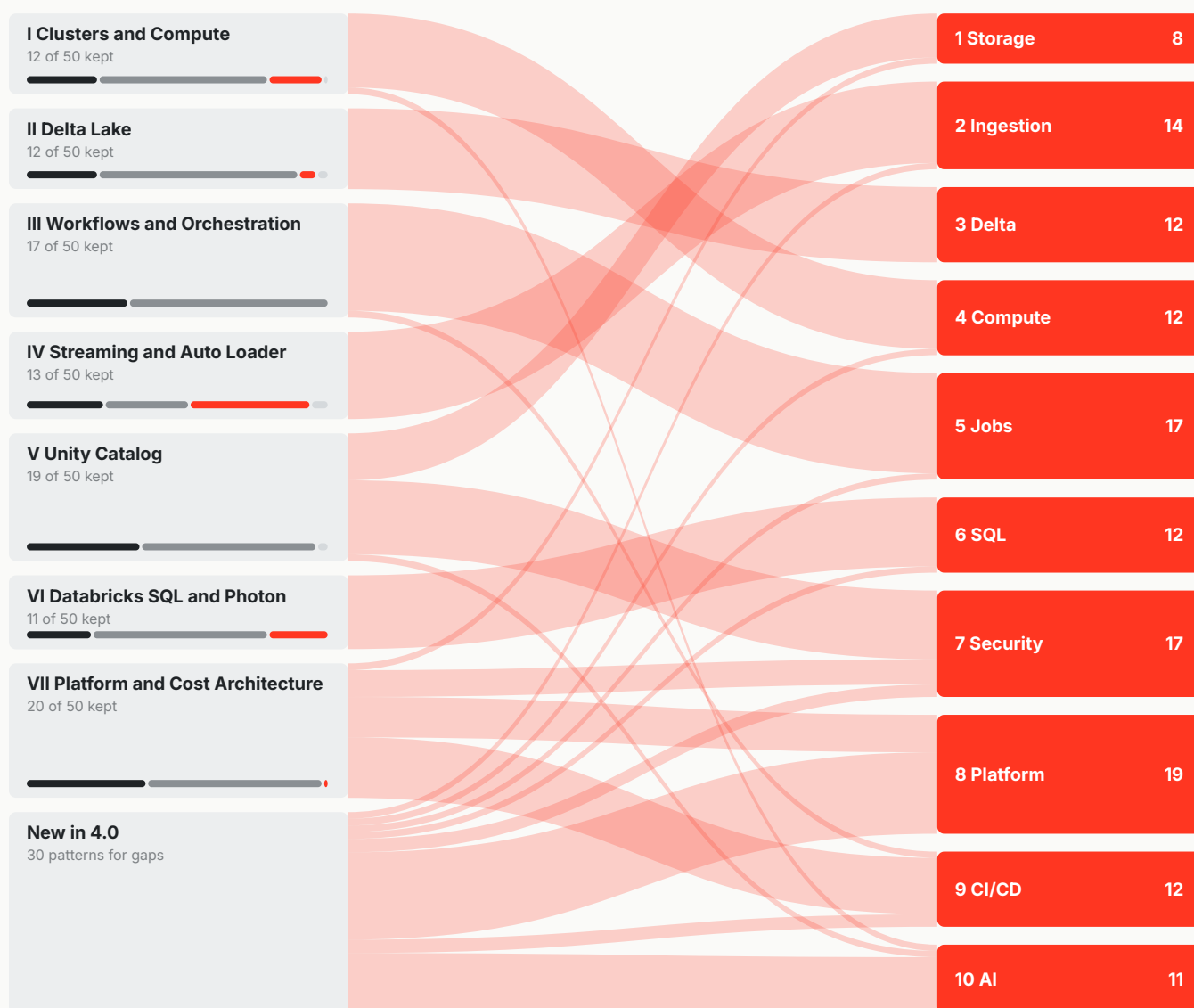


FIG. Ribbons show where each book's kept scenarios landed. The bar under each book splits its 50 scenarios.

Kept and rewritten
 Merged into another pattern
 Moved to the PySpark Handbook 4.0 or the SQL Handbook 4.0
 Cut: not Databricks-specific, out of date, or wrong

Of the 350: 104 kept, 195 merged, 43 moved to the Handbooks and 8 cut. Merged scenarios live on inside the pattern that absorbed them.